Politecnico di Torino

Master of Science in Computer Engineering

Department of Control and Computer Engineering (DAUIN)

Database and Data Mining Group (DBDMG)

Large Language Models for Software Engineering

Theoretical foundations, methodologies, and practical architectures

Comprehensive master's course study guide

Authored for graduate students, researchers, and practitioners

Course Instructors:
Prof. Flavio Giobergia
Prof. Riccardo Coppola
Politecnico di Torino

Academic Year:
2025 / 2026

Target Program:
M.Sc. Computer Engineering

Compiled and Typeset with LaTeX and `tcolorbox`
Torino, Italy — Academic Year 2025/2026

Contents

Preface iii

Notation and conventions v

I Foundations of large language models 1

1 Introduction to language models 2
- 1.1 The sequence prediction problem and probabilistic language modeling . . . 3
- 1.2 Markov assumptions and n-gram language models . . . 6
- 1.3 Transition count matrix construction walkthrough . . . 8
- 1.4 Autoregressive estimation and generation . . . 10
- 1.5 Fundamental limitations of statistical n-gram models: sparsity and zero-frequency collapse . . . 12
- 1.6 Statistical smoothing and discounting algorithms . . . 15
- 1.7 Evaluation metrics: cross-entropy and perplexity . . . 19
- 1.8 Historical evolution of language models . . . 23

2 Deep learning foundations for NLP 29
- 2.1 From biological neurons to the artificial perceptron . . . 30
- 2.2 Stacking linear layers and the proof of linear stacking collapse . . . 34
- 2.3 Taxonomy of neural activation functions . . . 39
- 2.4 Architectural task formulations and software engineering case studies . . . 42
- 2.5 Loss function formulations and optimization landscapes . . . 44
- 2.6 Computational graphs and exact adjoint backpropagation . . . 48
- 2.7 The sequential computation impasse: bridge to recurrent neural networks . . . 53

3 Recurrent neural networks and sequence modeling 55
- 3.1 Sequential data and constraints of feed-forward neural networks . . . 56
- 3.2 The recurrent neural network cell and weight sharing . . . 59
- 3.3 Backpropagation through time and the proof of vanishing gradients . . . 62
- 3.4 Case correction demonstration: Sherlock Holmes training dynamics . . . 65
- 3.5 Gated architectures: LSTM and GRU . . . 69
- 3.6 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines . . . 73
- 3.7 Persistent bottlenecks of gated RNNs and the road to transformers . . . 76

4 Distributed word representations and Word2Vec 81
- 4.1 The representation problem: why continuous embeddings are mandatory . . . 82
- 4.2 The baseline approach: local one-hot encodings and why they fail . . . 83
- 4.3 Distributed representations and the distributional hypothesis . . . 87
- 4.4 The embedding training loop and semantic geometry evolution . . . 88
- 4.5 Word2Vec continuous bag-of-words (CBOW) . . . 91

4.6 Word2Vec continuous skip-gram and the softmax bottleneck . . . 94
4.7 Scaling softmax: hierarchical softmax and negative sampling . . . 96
4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier . . . 101
4.9 FastText: subword character n-gram embeddings . . . 103
4.10 Emergent vector space geometry: clustering, metrics and relational analogies . . . 105
4.11 Modern subword tokenization and input pipelines for foundation models . . . 108

5 The Transformer architecture and attention mechanisms 125
5.1 The architectural impasse of recurrent sequence models . . . 126
5.2 The complete Transformer architecture: encoder-decoder breakdown . . . 127
5.3 Attention from first principles: differentiable soft dictionary retrieval . . . 129
5.4 Scaled dot-product attention and variance stabilization . . . 132
5.5 The triad of attention mechanisms . . . 136
5.6 Multi-head attention (MHA) . . . 143
5.7 Sublayer mechanics: residual connections and Layer Normalization . . . 145
5.8 Relative positional representations . . . 149
5.9 Parallelization advantages and computational complexity . . . 150
5.10 The full Encoder-Decoder paradigm: T5 . . . 151
5.11 The Encoder-Only paradigm: BERT (Bidirectional Encoder Representations from Transformers) . . . 153
5.12 Autoregressive decoding and sampling strategies . . . 164
5.13 Comparative paradigm synthesis and bridge to scaling laws . . . 176

6 LLM pre-training, scaling laws, and open models 178
6.1 Architectural paradigms and pre-training objectives . . . 178
6.2 GPT-1: Generative pre-training . . . 182
6.3 GPT-2: Zero-shot multitask learning . . . 185
6.4 Kaplan scaling laws . . . 190
6.5 GPT-3 and in-context learning . . . 193
6.6 The model undertraining crisis . . . 197
6.7 Compute-optimal scaling: Chinchilla . . . 201
6.8 Open foundation models . . . 207

7 Language model evaluation, metrics, and benchmarks 212
7.1 Evaluation metrics for language models . . . 213

Preface

Pedagogical vision

The emergence of Large Language Models (LLMs) represents a fundamental paradigm shift in computer science and software engineering. Moving beyond rigid syntax-directed rules, modern software engineering increasingly leverages deep probabilistic models capable of synthesizing code, repairing complex bugs, formalizing requirements, and orchestrating multi-agent systems.

This study guide accompanies the Master of Science course *Large Language Models for Software Engineering* at Politecnico di Torino, taught by Prof. Flavio Giobergia and Prof. Riccardo Coppola. The curriculum is intentionally structured into two complementary pillars:

1. **Part I: Foundations of large language models** — Traces the mathematical and architectural lineage of language modeling, beginning with n-gram Markov chains, proceeding through feedforward networks, distributed word embeddings (Word2Vec, FastText), recurrent networks and vanishing gradient proofs, and culminating in the self-attention Transformer architecture, pre-training scaling laws, instruction tuning (RLHF, DPO), and parameter-efficient fine-tuning (LoRA, QLoRA).

2. **Part II: Large language models for software engineering** — Bridges foundational model capabilities with the Software Development Life Cycle (SDLC), exploring prompt engineering patterns, autonomous agent frameworks (ReAct, multi-agent collaboration), automated test generation, software maintenance and refactoring, rigorous evaluation methodologies (Pass@k, CodeBLEU), and security, intellectual property, and ethical implications.

Visual callout structure

To maximize learning effectiveness and exam readiness, the guide embeds three specialized pedagogical environments:

Exam insights: Recurrent exam patterns and pitfalls

Highlighted in terracotta/amber, these boxes isolate specific questions frequently asked in oral and written examinations, point out common mathematical misconceptions (e.g., mixing logarithm bases when computing perplexity), and underscore key grading criteria.

Intuition & analogy: Geometric and conceptual analogies

Highlighted in royal purple, these boxes provide conceptual, visual, and intuitive interpretations of complex mathematical machinery (e.g., attention as a soft differentiable lookup table, high-dimensional vector space geometry).

External deep dive: Seminal research and authoritative multimedia

Highlighted in pine teal, these boxes contextualize textbook theory with seminal academic papers [1–3] and breakdowns by leading researchers and educators.

Examination framework and course organization

As detailed by the instructors in the introductory lecture, assessment in this course is divided into two distinct components:

1. **Written Examination (18 Points)**:
    - Duration: 75 minutes.
    - Format: Typically around 12 questions (combining closed-ended multiple-choice items and open-ended technical/mathematical derivations).
    - Scope: Comprehensive coverage across all foundational and applied software engineering topics.
    - Threshold: A minimum score of 10/18 is strictly required to achieve sufficiency.
2. **Practical Course Project (14 Points)**:
    - Implementation: Development of an applied LLM-for-software-engineering system using Python and PyTorch.
    - Modality: Undertaken individually or in small teams.
    - Scoring: Total points across the written test (18) and project (14) sum to 32, with scores exceeding 30 earning *30 e Lode* (honors).
3. **Lecture and Lab Schedule**:
    - Monday: 08:30 – 11:30 (Room 8) — Core theoretical foundations.
    - Wednesday: 11:30 – 13:00 (Room R1) — Practical workshops, interactive coding, and PyTorch laboratories.

How to use this study guide

Each chapter is fully self-contained and begins with precise learning objectives, followed by exhaustive mathematical derivations, algorithmic walkthroughs, and practical software engineering implications. Students are encouraged to independently verify all mathematical derivations and implement the core algorithms to develop deep mechanical empathy for large language models.

Notation and conventions

Throughout this study guide, we adopt consistent mathematical, algorithmic, and typographic conventions aligned with standard deep learning and software engineering literature.

Linear algebra and tensor operations

Symbol	Type	Description
x, y, z	Scalar	Real-valued scalars or scalar components ($x \in \mathbb{R}$).
$\mathbf{x}, \mathbf{y}, \mathbf{h}$	Vector	Column vectors ($\mathbf{x} \in \mathbb{R}^d$). Lowercase bold font.
$\mathbf{x}^\top$	Transpose	Transpose of vector $\mathbf{x}$ (row vector $\mathbf{x}^\top \in \mathbb{R}^{1\times d}$).
$\mathbf{A}, \mathbf{W}, \mathbf{X}$	Matrix	Real matrices ($\mathbf{A} \in \mathbb{R}^{m\times n}$). Uppercase bold font.
$\mathbf{A}_{i,j}$ or a_{ij}	Scalar	Entry at row i and column j of matrix $\mathbf{A}$.
$\mathbf{I}$ or $\mathbf{I}_d$	Matrix	Identity matrix of dimension $d \times d$.
$\|\mathbf{x}\|_2$ or $\|\mathbf{x}\|$	Scalar	Euclidean (L_2) norm of vector $\mathbf{x}$, $\sqrt{\sum_i x_i^2}$.
$\|\mathbf{A}\|_F$	Scalar	Frobenius norm of matrix $\mathbf{A}$, $\sqrt{\sum_{i,j} A_{i,j}^2}$.
$\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u}^\top \mathbf{v}$	Scalar	Standard dot (inner) product between vectors $\mathbf{u}$ and $\mathbf{v}$.
$\odot$	Operator	Element-wise (Hadamard) product between tensors of identical shape.
$\otimes$	Operator	Outer product or Kronecker tensor product.

Table 1: Linear algebra and tensor notation summary.

Probability and information theory

Neural networks, transformers, and software engineering

Symbol	Type	Description
$\mathbb{P}(A)$	Scalar	Probability of event A ($\mathbb{P} : \mathcal{F} \to [0, 1]$).
$\mathbb{P}(X = x)$	Scalar	Probability mass function for discrete random variable X.
$\mathbb{P}(Y \mid X)$	Distribution	Conditional probability distribution of Y given X.
$\mathbb{E}_{x\sim P}[f(x)]$	Scalar	Expected value of function $f(x)$ under distribution P.
$\mathbb{V}\text{ar}(X)$	Scalar	Variance of random variable X, $\mathbb{E}[(X - \mathbb{E}[X])^2]$.
$\mathcal{H}(P)$	Scalar	Shannon entropy of distribution P, $-\sum_x P(x) \log P(x)$.
$\mathcal{H}(P, Q)$	Scalar	Cross-entropy between target distribution P and model Q.
$D_{\mathrm{KL}}(P \parallel Q)$	Scalar	Kullback–Leibler (KL) divergence from Q to P.
$\mathrm{PPL}(W)$	Scalar	Perplexity of word sequence W under a language model.

Table 2: Probability and information theoretic notation.

Symbol	Type	Description
$\mathcal{V}$	Set	Discrete vocabulary of tokens; $\lvert\mathcal{V}\rvert = V$ denotes vocabulary size.
T or T	Integer	Sequence length (number of input or output tokens).
d_{model}	Integer	Hidden embedding dimensionality of the Transformer model.
d_{ff}	Integer	Dimensionality of the intermediate Feed-Forward Network layer.
d_k, d_v	Integer	Dimensionality of query/key and value projections per head.
h	Integer	Number of parallel attention heads ($h \cdot d_k = d_{\text{model}}$).
$\sigma(z)$	Function	Standard logistic sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$.
$\mathrm{softmax}(\mathbf{z})$	Function	Softmax normalized exponential vector operator.
$\mathrm{ReLU}(z)$	Function	Rectified Linear Unit activation: $\max(0, z)$.
$\mathrm{GELU}(z)$	Function	Gaussian Error Linear Unit activation.
$\mathcal{L}$ or $\mathcal{L}$	Scalar	Scalar objective / loss function minimized during optimization.
Pass@k	Metric	Probability that at least one of k generated code samples passes unit tests.
CodeBLEU	Metric	Multi-aspect code evaluation metric combining BLEU, AST, and data flow.

Table 3: Model architecture parameters and SE metrics.

Part I

Foundations of large language models

CHAPTER 1

Introduction to language models

Human language and software engineering source code represent discrete symbolic systems governed by intricate, multi-level structures spanning phonology, syntax, semantics, and pragmatics. For decades, computational linguistics attempted to capture these phenomena through hand-crafted rule bases and formal context-free grammars. However, natural language is inherently ambiguous, productive, and elastic: speakers continuously coin new idioms, tolerate grammatical ellipses, and convey complex connotations that elude rigid deterministic parsers.

To build computational systems capable of understanding, completing, and generating coherent text, modern computer science reframes language processing from deterministic rule parsing into a discrete stochastic process. By treating words and subwords as discrete random variables, we formulate language modeling as the task of quantifying the joint probability distribution over arbitrary token sequences. Through the unconditional factorization of the chain rule of probability, this joint estimation problem collapses into sequential next-token conditional prediction. This deceptively simple objective—predicting the identity of the next token given preceding context—constitutes the foundational golden thread uniting the earliest twentieth-century statistical Markov models with modern multi-billion-parameter foundation models. In this chapter, we establish the mathematical foundations of probabilistic language modeling, formalize the trade-offs of classical n-gram representations, analyze the pathological breakdown of discrete frequency counting, derive statistical smoothing algorithms, and establish information-theoretic metrics for sequence evaluation.

1.1 The sequence prediction problem and probabilistic language modeling

At its core, a language model formalizes the intuitive mechanism by which humans evaluate the fluency, plausibility, and likelihood of word sequences. In Natural Language Processing (NLP) and modern AI-assisted Software Engineering, we model human language and source code not through rigid, brittle rule systems, but as a discrete stochastic process governed by probability distributions.

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens (e.g., words, subwords, or punctuation symbols). An arbitrary natural language sentence, code snippet, or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \ldots, w_N) \in \mathcal{V}^N$.

Definition 1.1: *Statistical language model*

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \ldots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \tag{1.1}$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval:

- Fluent, grammatically coherent sentences receive high probability mass:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.1$$

- Syntactically valid but semantically unusual or rare sentences receive noticeably lower probability:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“purple”}, \text{“sentence”}) = 0.01$$

- Ungrammatical, corrupted, or disjoint sequences receive negligible probability:

$$\mathbb{P}(\text{“this”}, \text{“are”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.001$$

Evaluating the joint probability $\mathbb{P}(w_1, \ldots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is intractable because observing every conceivable multi-word sequence in a training dataset is impossible. To structure this estimation problem, we invoke the fundamental *chain rule of probability*.

Definition 1.2: *Chain rule decomposition of language models*

For any sequence of random variables $(w_1, w_2, \ldots, w_N)$, the joint probability factors identi-

cally and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned}\mathbb{P}(w_1, w_2, \ldots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 \mid w_1) \cdot \mathbb{P}(w_3 \mid w_1, w_2) \cdots \mathbb{P}(w_N \mid w_1, \ldots, w_{N-1}) \\ &= \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}),\end{aligned} \tag{1.2}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ represents the prefix history preceding timestep t. For the initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, giving $\mathbb{P}(w_1 \mid w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (1.2) to our running example yields the concrete factorization:

$$\begin{aligned}\mathbb{P}(\text{this, is, a, reasonable, sentence}) = \mathbb{P}(&\text{this}) \\ &\cdot \mathbb{P}(\text{is} \mid \text{this}) \\ &\cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ &\cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ &\cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}).\end{aligned}$$

Reframing language modeling as conditional prediction. This mathematical decomposition marks a pivotal conceptual shift in Natural Language Processing. Instead of attempting to score complete, static documents simultaneously, the modeling focus shifts entirely to sequential, conditional next-word estimation:

Given history $w_{<t}$, estimate the categorical probability distribution over all words in $\mathcal{V}$: $\mathbb{P}(w_t = w \mid w_{<t})$.

This formulation aligns directly with the classic psychological *cloze test* (fill-in-the-blank):

$$\text{“}\textit{This is a reasonable}\ _____\text{”} \implies \mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$$

Every modern autoregressive language model, including GPT-4 and Claude, relies fundamentally on this conditional next-token formulation.

Next-word prediction as generative modeling.

This observation reveals a profound equivalence in statistical NLP: estimating the local conditional probability $\mathbb{P}(w_t \mid w_{<t})$ across a vocabulary is mathematically sufficient to compute the global probability $\mathbb{P}(W)$ of any arbitrary document via Equation (1.2). Conversely, generating novel text reduces to iteratively sampling from this local conditional distribution and appending each sampled token to the conditioning history. Thus, mastering conditional next-token prediction equates to mastering generative language modeling in its entirety.

Path traversal across the probability tree.

Sequence generation can be understood geometrically as walking down a directed decision tree (Figure 1.1). At the root node $\langle s \rangle$, the model confronts the entire vocabulary $\mathcal{V}$. Choosing a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ and lands at a new conditioning state. From this state, another set of branches opens, weighted by $\mathbb{P}(w_2 \mid w_1)$. The joint probability of an entire sentence is simply the product of edge weights along that unique trajectory, and text generation corresponds to sampling a valid path from the root to a designated terminal token $\langle /s \rangle$.

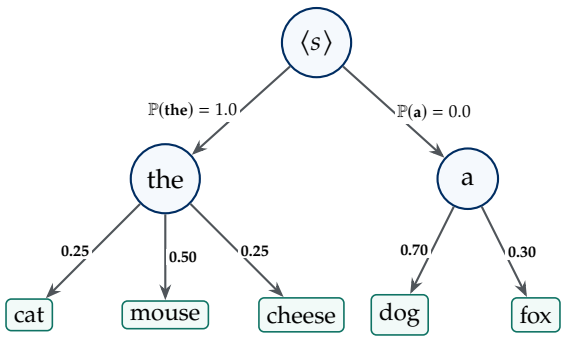


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

1.2 Markov assumptions and n-gram language models

Although Equation (1.2) is mathematically exact and preserves all syntactic and semantic dependencies, computing $\mathbb{P}(w_t \mid w_{<t})$ directly in empirical applications presents an insurmountable combinatorial bottleneck. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \ldots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \tag{1.3}$$

Even for a modest vocabulary of $V = 10{,}000$ words, tracking histories of length $t = 4$ requires estimating probabilities over $10{,}000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs exactly zero times.

To make estimation mathematically tractable, the principle of conditional independence formalizes that future tokens depend only on a bounded historical window. In the landmark 1948 treatise on information theory [4], it was demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern pedagogical treatments [5] formalize this simplification as the foundation of classical statistical natural language processing.

Definition 1.3: *Markov assumption of order $n-1$*

An n-gram language model assumes that the conditional probability of the current word w_t depends exclusively on the immediately preceding $n-1$ words, rendering it conditionally independent of all earlier discourse history:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.4}$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0)**: Completely memoryless. Each word is treated as statistically independent of all past and future tokens:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1)**: First-order Markov chain. Conditioning depends exclusively on the single preceding token w_{t-1}:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2)**: Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.2.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially growing state space into a fixed, stationary cardinality. Under an n-gram model, the conditioning context is always restricted to $n-1$ words, regardless of the absolute sequence position t:

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \tag{1.5}$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n-1)$-dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V-1) \approx V^n \tag{1.6}$$

free parameters. Table 1.1 contrasts the unconstrained historical state space with fixed n-gram models for realistic vocabulary sizes.

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n-gram Markov orders for vocabularies $V = 10{,}000$ and $V = 50{,}000$.

Model / Order	**Effective Context**	**Formula**	**States ($V = 10^4$)**	**States ($V = 5 \times 10^4$)**
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

While the unconstrained state space exceeds the total number of atoms in the observable universe ($\approx 10^{80}$) by step $t = 20$, a bigram model maintains a manageable state space of 50,000 rows. However, this drastic dimensional reduction comes at the cost of complete amnesia for any context preceding the immediate $n-1$ tokens.

1.2.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [5]. For a generic n-gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{C(w_{t-n+1}, \ldots, w_{t-1}, w_t)}{C(w_{t-n+1}, \ldots, w_{t-1})}, \tag{1.7}$$

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (1.7) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \tag{1.8}$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.3 Transition count matrix construction walkthrough

To understand exactly how statistical n-gram language models operate, we trace the step-by-step extraction of bigram frequencies and transition probabilities from the authoritative lecture corpus:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Vocabulary extraction. Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|V| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

Sequential bigram decomposition. We iterate through both training sentences token by token, recording each observed transition $(w_{t-1} \to w_t)$:

- From Sentence 1 (*"The cat chased the mouse happily"*):
  1. $(\text{The}, \text{cat}) \implies C(\text{The}, \text{Cat}) \leftarrow 1$
  2. $(\text{cat}, \text{chased}) \implies C(\text{Cat}, \text{Chased}) \leftarrow 1$
  3. $(\text{chased}, \text{the}) \implies C(\text{Chased}, \text{The}) \leftarrow 1$
  4. $(\text{the}, \text{mouse}) \implies C(\text{The}, \text{Mouse}) \leftarrow 1$
  5. $(\text{mouse}, \text{happily}) \implies C(\text{Mouse}, \text{Happily}) \leftarrow 1$
- From Sentence 2 (*"The mouse ate the cheese"*):
  6. $(\text{The}, \text{mouse}) \implies C(\text{The}, \text{Mouse}) \leftarrow 2$
  7. $(\text{mouse}, \text{ate}) \implies C(\text{Mouse}, \text{Ate}) \leftarrow 1$
  8. $(\text{ate}, \text{the}) \implies C(\text{Ate}, \text{The}) \leftarrow 1$
  9. $(\text{the}, \text{cheese}) \implies C(\text{The}, \text{Cheese}) \leftarrow 1$

Unigram normalization counts. Summing across rows provides the total unigram occurrences $C(w_{t-1})$:

$$\begin{aligned}
C(\text{The}) &= 4 \quad (\text{cat} : 1, \text{mouse} : 2, \text{cheese} : 1) \\
C(\text{Cat}) &= 1 \quad (\text{chased} : 1) \\
C(\text{Chased}) &= 1 \quad (\text{the} : 1) \\
C(\text{Mouse}) &= 2 \quad (\text{happily} : 1, \text{ate} : 1) \\
C(\text{Ate}) &= 1 \quad (\text{the} : 1) \\
C(\text{Cheese}) &= 0 \quad (\text{terminal word, no outgoing transitions}) \\
C(\text{Happily}) &= 0 \quad (\text{terminal word, no outgoing transitions})
\end{aligned}$$

Full empirical transition probability matrix. Applying Equation (1.8) yields the complete 7×7 transition matrix $\mathbf{M} \in \mathbb{R}^{7\times7}$ where row i represents condition w_{t-1} and column j represents next token w_t. Table 1.2 presents both raw counts and normalized MLE probabilities.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t \mid w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

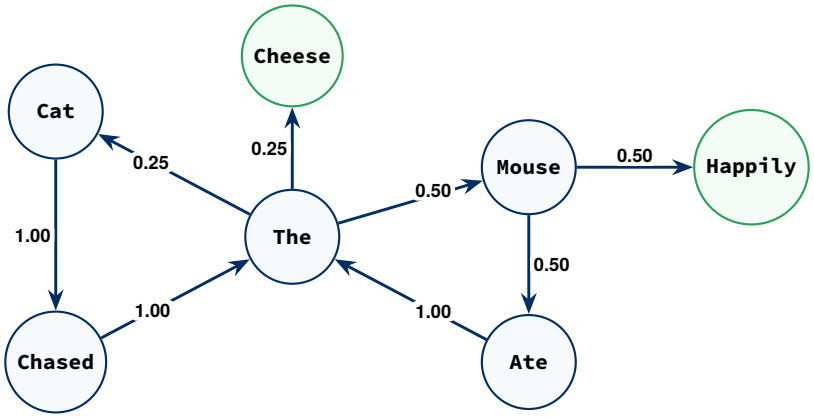


Figure 1.2: Markov state transition graph for the 7-token cat/mouse corpus. Directed edges indicate bigram transition probabilities $\mathbb{P}(w_t \mid w_{t-1})$. States `Cheese` and `Happily` (green) are terminal absorbing states with zero outgoing probability mass.

1.4 Autoregressive estimation and generation

With the transition matrix $\mathbf{M}$ constructed (Table 1.2) and its state graph mapped (Figure 1.2), the language model can execute two distinct operational tasks:

1. **Sequence Probability Estimation**: Assign a likelihood score to any proposed test sentence.
2. **Autoregressive Generation**: Incrementally sample tokens to produce novel synthetic text.

1.4.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

"the cat chased the cheese"

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the})\cdot\mathbb{P}(\text{cat} \mid \text{the})\cdot\mathbb{P}(\text{chased} \mid \text{cat})\cdot\mathbb{P}(\text{the} \mid \text{chased})\cdot\mathbb{P}(\text{cheese} \mid \text{the}). \tag{1.9}$$

Assuming our generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table 1.2:

$$\begin{aligned}\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25\\ \mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0\\ \mathbb{P}(\text{the} \mid \text{chased}) &= 1.0\\ \mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25\end{aligned}$$

Multiplying these values step by step:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \tag{1.10}$$

Despite the sequence *"the cat chased the cheese"* never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n-gram models exhibit rudimentary combinatorial generalization.

1.4.2 5-step autoregressive generation trace

Now consider using the bigram model for open-ended text generation. Autoregressive sampling begins with a seed prefix, samples the next token from the resulting conditional probability distribution, appends the sampled token to the history, and repeats.

We trace a concrete 5-step generation sequence starting from the seed token *"the"*:

Step 1: Condition on prefix: history = ("the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- Generated prefix: *"the mouse"*.

Step 2: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"ate"**.
- Generated prefix: *"the mouse ate"*.

Step 3: Condition on prefix: history = (. . . , "ate").

- Active row in transition matrix: Row "Ate".
- Candidate distribution: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$ (deterministic transition).
- Emitted token: **"the"**.
- Generated prefix: *"the mouse ate the"*.

Step 4: Condition on prefix: history = (. . . , "the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- *Pathological state*: A repetitive structural loop emerges because the first-order Markov model cannot retain the memory that it already emitted "mouse" two steps earlier.
- Generated prefix: *"the mouse ate the mouse"*.

Step 5: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"happily"**.
- *Pathological state*: While syntactically licensed by local transitions, an unnatural semantic tautology is generated due to the absence of long-range semantic grounding.
- Generated sequence: ***"The mouse ate the mouse happily"***.

Because the word *"happily"* has zero outgoing transitions in Table 1.2, the process reaches an absorbing terminal state and generation halts naturally.

Analysis of absorbing terminal states and infinite loops. During the sequence generation analysis, two fundamental edge cases emerge:

1. **Absorbing terminal states after sentence-ending tokens**: In Table 1.2, terminal sentence-ending words have a row count of zero ($C(\text{Happily}) = 0$), making the conditional probability division $0/0$ mathematically undefined. In standard production language modeling, this is formalized by appending an explicit end-of-sequence token $\langle /s \rangle$ (or [EOS]) to every training sentence, guaranteeing that every word possesses a non-zero outgoing transition to either another word or $\langle /s \rangle$.

2. **Infinite cyclic loops under greedy decoding**: Because bigram models condition exclusively on the single preceding token w_{t-1}, once the model transitions from *"ate"* to *"the"*, it retains zero memory that it already described the mouse eating cheese two steps earlier. If greedy decoding is used (arg max), the transition path becomes fully deterministic and falls into a stationary limit cycle: *"the → mouse → ate → the → mouse → . . . "*. Modern LLMs resolve this through long-context self-attention horizons, temperature-controlled stochastic sampling, and repetition penalties.

1.5 Fundamental limitations of statistical n-gram models: sparsity and zero-frequency collapse

While n-gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from fatal structural limitations that ultimately motivated the deep learning revolution.

1.5.1 Catastrophic data sparsity and parameter explosion

To condition on $n-1$ words, an n-gram model requires storing a discrete probability table of dimensionality $|\mathcal{V}|^n$. For realistic natural language and software applications:

- Vocabulary size: $|\mathcal{V}| \approx 50{,}000$ to $100{,}000$ unique words or code identifiers.
- Bigram table ($n = 2$): $|\mathcal{V}|^2 = 50{,}000^2 = 2.5 \times 10^9$ parameters ($\approx$ 10 GB at 32-bit floating point).
- Trigram table ($n = 3$): $|\mathcal{V}|^3 = 50{,}000^3 = 1.25 \times 10^{14}$ parameters (over 500 Terabytes).
- 4-gram table ($n = 4$): $|\mathcal{V}|^4 = 50{,}000^4 = 6.25 \times 10^{18}$ parameters (Exabyte scale).

Even if storage were free, the number of distinct 4-grams in all digitized human literature is dwarfed by 10^{18}. Consequently, the overwhelming majority of valid phrases never appear in training corpora.

1.5.2 The zero-frequency catastrophe: probability and perplexity collapse

The most lethal consequence of combinatorial sparsity is the *zero-frequency catastrophe*. Under Maximum Likelihood Estimation, any n-gram that never appeared verbatim in the training text receives count $C = 0$, yielding:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^{t})}{C(w_{t-n+1}^{t-1})} = 0. \tag{1.11}$$

By the probabilistic chain rule (Equation (1.2)), evaluating a test sequence $W_{\text{test}} = (w_1, \ldots, w_N)$ requires taking the product across all constituent conditional transitions:

$$\mathbb{P}(W_{\text{test}}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}) = \mathbb{P}(w_1) \cdots 0 \cdots \mathbb{P}(w_N \mid w_{<N}) = 0. \tag{1.12}$$

A single unobserved token transition collapses the estimated joint probability of an entire document, article, or codebase to absolute zero.

Furthermore, when evaluating the model via Empirical Cross-Entropy $\mathcal{H}(W_{\text{test}})$ or Perplexity $\text{PPL}(W_{\text{test}})$:

$$\mathcal{H}(W_{\text{test}}) = -\frac{1}{N} \sum_{t=1}^{N} \log_2 \mathbb{P}(w_t \mid w_{<t}) \longrightarrow \infty, \quad \text{PPL}(W_{\text{test}}) = 2^{\mathcal{H}(W_{\text{test}})} \longrightarrow \infty. \tag{1.13}$$

An unobserved test n-gram results in infinite perplexity ($\text{PPL} = \infty$), rendering the entire language model statistically useless for evaluation or comparison.

1.5.3 Finite context horizon limitations

An n-gram model is strictly blind to any token that occurred more than $n-1$ steps in the past:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.14}$$

Because the conditioning window is rigidly restricted to the preceding $n-1$ tokens, any semantic constraint, topic coherence, or syntactic agreement established earlier in the discourse is discarded. This finite context horizon causes severe memory amnesia across two critical domains:

Linguistic agreement breakdown in natural language. Consider long-range syntactic agreements where an arbitrary number of modifiers, prepositional phrases, or subordinate relative clauses separate the grammatical subject from its governing predicate:

*"The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent."*

If $n = 4$ (a 4-gram model), the model predicts the verb by conditioning exclusively on the three preceding tokens:

$$\mathbb{P}(\text{verb} \mid \text{compiler}, \text{infrastructure}, [\ldots]).$$

Because the singular noun *"infrastructure"* is immediately adjacent to the verb position, the local 4-gram transition statistics strongly favor the singular verb *"was"*. The model has completely forgotten the true plural antecedent subject *"engineers"*, producing an ungrammatical agreement violation.

Structural and scoping breakdown in software engineering code. In programming languages, code validity requires strict, long-range determinism governed by context-free grammars and scope resolution rules:

1. **Lexical Scoping and Variable Lifecycles**: A variable declared in a function header:

   ```
   void process_batch(std::vector<TransactionRequest>& transactions) { ...}
   ```

   must be invoked using valid `std::vector` member methods (e.g., `.push_back()`, `.size()`) thirty lines later. An n-gram model sees only `transactions.` without access to the declaration, leading to syntactically invalid completions.

2. **Bracket and Block Matching**: Program code requires exact hierarchical pairing of opening and closing delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow constructs (`try ...catch ...finally`). Because n-gram models lack internal state stacks, they frequently emit mismatched braces or premature block terminations.

1.5.4 Discrete orthogonal symbols and lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*"The **cat** chased the mouse happily"*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*"The **feline** chased the mouse happily"*

Because *"cat"* and *"feline"* are completely separate row indices in Table 1.2, observations of one do not inform the other. There is no concept of semantic distance, synonymy, or conceptual overlap.

The causal imperative. While long-range memory amnesia and semantic blindness demand continuous neural architectures (introduced in Chapters 2 and 4), the immediate, fatal crisis in statistical language modeling is the **zero-frequency catastrophe** ($C = 0 \implies P = 0 \implies \text{PPL} = \infty$). Before we can even evaluate an n-gram model, we must establish rigorous mathematical algorithms that steal probability mass from frequent events and reallocate it to unobserved events: this is the domain of **statistical smoothing**.

1.6 Statistical smoothing and discounting algorithms

To prevent probability collapse ($P = 0$) and infinite perplexity (PPL $= \infty$) on unseen test sequences, statistical language modeling relies on *smoothing* (or discounting). The governing principle of smoothing is conservative probability reallocation: we shave off a fraction of probability mass from observed events and redistribute it across unseen events, ensuring that every valid sequence receives a strictly positive probability:

$$\forall w \in \mathcal{V}, \quad \mathbb{P}(w \mid \text{context}) > 0.$$

1.6.1 Laplace add-one and add-α smoothing

The simplest smoothing formulation, originated by Pierre-Simon Laplace, adds a uniform pseudo-count $\alpha > 0$ to every possible transition.

Mathematical formulation. For an unconditioned unigram model, Laplace add-α smoothing assigns:

$$\mathbb{P}_{\text{Laplace}}(w) = \frac{C(w) + \alpha}{N + \alpha|\mathcal{V}|}, \tag{1.15}$$

where $N = \sum_{w \in \mathcal{V}} C(w)$ is the total token count. For a bigram model conditioned on w_{t-1}, the estimator modifies Equation (1.8) as:

$$\mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{1.16}$$

When $\alpha = 1$, the technique is known as *Laplace smoothing* (or Add-1); when $0 < \alpha < 1$, it is termed *Lidstone smoothing*.

Proof of proper probability distribution. We verify that Equation (1.16) satisfies the Kolmogorov axioms by summing over the entire vocabulary $\mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) &= \sum_{w_t \in \mathcal{V}} \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{\sum_{w_t \in \mathcal{V}} C(w_{t-1}, w_t) + \sum_{w_t \in \mathcal{V}} \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{C(w_{t-1}) + \alpha|\mathcal{V}|}{C(w_{t-1}) + \alpha|\mathcal{V}|} = 1.
\end{aligned} \tag{1.17}$$

Why Add-1 fails in practice. Although mathematically clean, Add-1 smoothing performs terribly in natural language processing. In realistic vocabularies where $|\mathcal{V}| = 50{,}000$, if word w_{t-1} occurred $C(w_{t-1}) = 100$ times, adding $\alpha|\mathcal{V}| = 50{,}000$ inflates the denominator from 100 to 50,100. A frequent observed transition with $C(w_{t-1}, w_t) = 50$ is diluted from an empirical probability of 0.50 to:

$$\frac{50 + 1}{100 + 50{,}000} = \frac{51}{50{,}100} \approx 0.00102.$$

Add-1 smoothing steals more than 99.8% of the probability mass from observed events and reallocates it uniformly to unobserved transitions. Even Lidstone smoothing ($\alpha = 0.05$) assigns the same probability to all unseen words, failing to recognize that some unseen words (e.g., common nouns) are inherently more probable than rare technical jargon.

1.6.2 Good-Turing discounting

Originally formulated in [6] during cryptographic research at Bletchley Park, *Good-Turing discounting* derives the probability of events seen r times directly from the count of events seen $r + 1$ times.

Frequency of frequencies. Let $r = C(w_{t-n+1}^{t}) \geq 0$ denote the empirical frequency of an n-gram. We define the *frequency of frequencies* N_r as the total number of distinct n-grams that appeared exactly r times in the training corpus:

$$N_r = |\{x : C(x) = r\}|. \tag{1.18}$$

The total number of tokens in the corpus is $N = \sum_{r=1}^{\infty} rN_r$, and N_0 represents the enormous number of valid n-grams that never appeared ($r = 0$).

Adjusted count derivation. Good-Turing replaces each raw count r with an effective discounted count r^*:

$$r^* = (r + 1)\frac{N_{r+1}}{N_r}. \tag{1.19}$$

For any observed n-gram with count $r > 0$, its Good-Turing smoothed probability is:

$$\mathbb{P}_{\text{GT}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{r^*}{N} = \frac{(r + 1)N_{r+1}}{N \cdot N_r}. \tag{1.20}$$

The total probability mass reserved for all unobserved n-grams ($r = 0$) is determined by the proportion of *singletons* (N_1, words seen exactly once):

$$P_0 = \frac{N_1}{N}. \tag{1.21}$$

Because there are N_0 unobserved n-grams, each individual unseen n-gram receives probability:

$$\mathbb{P}_{\text{GT}}(\text{unseen}) = \frac{P_0}{N_0} = \frac{N_1}{N \cdot N_0}. \tag{1.22}$$

Smoothing N_r for large r. For large values of r, empirical counts become sparse (N_r fluctuates wildly or drops to zero, making N_{r+1}/N_r undefined). In production systems, *Simple Good-Turing* [7] replaces raw N_r values with smoothed estimates derived from a linear regression fit in log space ($\log N_r = a + b \log r$) for $r > k$.

1.6.3 Katz backoff with α normalization

The recursive framework introduced in [8] extended Good-Turing discounting through a hierarchical backoff mechanism: if an n-gram has been observed, use its discounted probability; if it has never been observed, back off to the $(n - 1)$-gram distribution, scaled by a normalizing factor.

Mathematical formulation. For a bigram model, the Katz backoff estimator is:

$$\mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = \begin{cases} \mathbb{P}^*(w_t \mid w_{t-1}) & \text{if } C(w_{t-1}, w_t) > 0, \\ \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) & \text{if } C(w_{t-1}, w_t) = 0, \end{cases} \tag{1.23}$$

where $\mathbb{P}^*(w_t \mid w_{t-1}) = \frac{C^*(w_{t-1}, w_t)}{C(w_{t-1})}$ represents the discounted probability calculated using Good-Turing adjusted counts for low frequencies ($r \leq k$, typically $k = 5$) and MLE for high counts ($r > k$):

$$C^*(w_{t-1}, w_t) = \frac{\frac{r^*}{r} - \frac{(k+1)N_{k+1}}{N_1}}{1 - \frac{(k+1)N_{k+1}}{N_1}} \cdot r \quad \text{for } 1 \leq r \leq k. \tag{1.24}$$

Derivation of the normalizing backoff weight $\alpha(w_{t-1})$. The leftover probability mass shaved off from observed bigrams is:

$$\beta(w_{t-1}) = 1 - \sum_{w_t : C(w_{t-1}, w_t) > 0} \mathbb{P}^*(w_t \mid w_{t-1}). \tag{1.25}$$

To ensure that $\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = 1$, the backoff weight $\alpha(w_{t-1})$ scales the lower-order unigram distribution over unobserved words:

$$\alpha(w_{t-1}) = \frac{\beta(w_{t-1})}{\sum_{w_t : C(w_{t-1}, w_t) = 0} \mathbb{P}_{\text{Katz}}(w_t)}. \tag{1.26}$$

Summing across the entire vocabulary confirms proper normalization:

$$\begin{aligned} \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) &= \sum_{C>0} \mathbb{P}^*(w_t \mid w_{t-1}) + \sum_{C=0} \alpha(w_{t-1}) \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \alpha(w_{t-1}) \sum_{C=0} \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \beta(w_{t-1}) = 1. \end{aligned} \tag{1.27}$$

1.6.4 Kneser-Ney smoothing and continuation probabilities

The pinnacle of statistical language modeling is *Kneser-Ney smoothing* [9]. Kneser-Ney introduces two transformative innovations: absolute discounting and the continuation probability.

Absolute discounting. Rather than scaling counts proportionally, empirical analysis of corpus frequencies demonstrates that subtracting a fixed constant $d \in (0, 1)$ (typically $d \approx 0.75$) from all positive counts accurately models the true discounted frequency:

$$C_{\text{discounted}} = \max\left(C(w_{t-n+1}^t) - d,\, 0\right).$$

The continuation probability failure mode of standard backoff. Consider predicting the missing word in the sentence:

"I can't see without my reading _____"

The word *"glasses"* should receive a high conditional probability. Now consider the word *"Francisco"*. In any standard text corpus, *"Francisco"* has a high unigram frequency because the proper noun *"San Francisco"* appears frequently. Under Katz backoff, because the bigram *"reading Francisco"* has zero count, the model backs off to the unigram frequency of *"Francisco"*, assigning it a dangerously high probability of completing the sentence!

The breakthrough insight in [9] recognized that the unigram backoff distribution should not reflect *how often* a word appears overall, but *how versatile* the word is across diverse preceding contexts.

Definition 1.4: *Continuation probability*

The *continuation probability* $\mathbb{P}_{\text{cont}}(w_t)$ is the ratio of the number of unique preceding word types that precede w_t to the total number of unique bigram types in the vocabulary:

$$\mathbb{P}_{\text{cont}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|} = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{|\{(w_{t-1}, w_t) : C(w_{t-1}, w_t) > 0\}|}. \tag{1.28}$$

Because *"Francisco"* almost exclusively appears preceded by the single word *"San"*, $|\{w_{t-1} : C(w_{t-1}, \text{Francisco}) > 0\}| \approx 1$, resulting in a tiny continuation probability. In contrast, words like *"glasses"* or *"book"* appear in dozens of distinct contexts, yielding high continuation probabilities.

Interpolated Kneser-Ney formulation. Combining absolute discounting with continuation probabilities yields the *Interpolated Kneser-Ney* estimator:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{\max\left(C(w_{t-n+1}^{t}) - d,\, 0\right)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1})\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+2}^{t-1}), \tag{1.29}$$

where $\lambda(w_{t-n+1}^{t-1})$ is an explicit normalizing interpolation weight:

$$\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot \left|\{w : C(w_{t-n+1}^{t-1}, w) > 0\}\right|. \tag{1.30}$$

For a bigram model ($n = 2$), the base recursion terminates at the unigram continuation probability:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max\left(C(w_{t-1}, w_t) - d,\, 0\right)}{C(w_{t-1})} + \frac{d}{C(w_{t-1})} \cdot |\{w : C(w_{t-1}, w) > 0\}| \cdot \mathbb{P}_{\text{cont}}(w_t). \tag{1.31}$$

Proof of proper normalization. Summing Equation (1.31) over all $w_t \in \mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) &= \sum_{w_t : C>0} \frac{C(w_{t-1}, w_t) - d}{C(w_{t-1})} + \lambda(w_{t-1}) \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{cont}}(w_t) \\
&= \frac{\sum_{C>0} C(w_{t-1}, w_t) - d \cdot |\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \lambda(w_{t-1}) \cdot 1 \\
&= \frac{C(w_{t-1}) - d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \frac{d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} = 1.
\end{aligned} \tag{1.32}$$

Interpolated Kneser-Ney consistently outperforms all other statistical smoothing techniques, serving as the industry standard until the advent of deep learning.

The causal imperative. Now that we have established competing smoothing algorithms (Laplace, Good-Turing, Katz, Kneser-Ney), each reallocating probability mass under different mathematical assumptions, we face a crucial practical question: *How do we quantitatively benchmark and compare these models on held-out evaluation text without running costly downstream applications?* This brings us directly to formal evaluation metrics.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [4, 5].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.5: *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [4]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \tag{1.33}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ. To quantify the statistical discrepancy between the true distribution P and our approximating model Q_θ, we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.6: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \tag{1.34}$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{1.35}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Applying the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \geq 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{1.36}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{1.37}$$

Why optimizing cross-entropy optimizes KL divergence. Equation (1.37) reveals the theoretical justification for cross-entropy training:

- The true language distribution P is determined by human linguistic behavior; therefore, the true entropy $H(P)$ is a fixed, immutable physical constant.
- When we minimize cross-entropy with respect to model parameters θ:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} \left(H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta) \right) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{1.38}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_θ and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{1.39}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using our model Q.

1.7.4 Perplexity (PPL)

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners overwhelmingly report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T}\log_2 Q(w_t|w_{<t})} \tag{1.40}$$

$$= \left(\prod_{t=1}^{T}\frac{1}{Q(w_t \mid w_{<t})}\right)^{\frac{1}{T}}. \tag{1.41}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Rigorous proof: perplexity as the effective branching factor

The fundamental intuition taught across NLP literature is that perplexity quantifies the *effective branching factor* of a language model. We now formalize this property through a rigorous proof.

Theorem 1.2: *Perplexity of a uniform K-ary distribution*

Let a language model Q predict next tokens such that at each timestep t, the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K:

$$\mathrm{PPL}(W) = K. \tag{1.42}$$

Proof. Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula:

$$\mathcal{H}(W) = -\frac{1}{T}\sum_{t=1}^{T}\log_2\left(\frac{1}{K}\right) = -\frac{1}{T}\sum_{t=1}^{T}\left(-\log_2 K\right)$$
$$= \frac{1}{T}\left(T\log_2 K\right) = \log_2 K.$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

Physical interpretation. When a model evaluates a text corpus and achieves PPL = 42, it indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words:

- A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (e.g., PPL = 50,000).
- An optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0.
- State-of-the-art LLMs on natural language benchmarks typically achieve perplexities between 8 and 15.

1.7.6 Base-2 vs. base-e logarithmic conversions and perplexity computation

A critical mathematical nuance arises when connecting information-theoretic formulations (which traditionally employ base-2 logarithms) with deep learning frameworks such as PyTorch (which exclusively compute natural base-e cross-entropy loss):

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$,	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

Worked numerical calculation. Suppose a model trained on code completion reports an evaluation cross-entropy loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats:

1. **Direct Base-e Perplexity**:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$

2. **Conversion to Information Bits**:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token}.$$

3. **Base-2 Perplexity Verification**:

$$\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00.$$

Notice that evaluating $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$ would severely underestimate the true branching factor by more than 57%. One must always ensure consistent logarithmic bases when deriving perplexity from loss.

1.7.7 The glass ceiling of smoothed n-gram models

While Kneser-Ney smoothing dramatically reduces test perplexity by eliminating zero-frequency collapses, smoothed n-gram models remain fundamentally handicapped:

1. **Horizon Truncation**: Conditioning remains locked to $n-1$ tokens; increasing $n \geq 5$ yields negligible gains because higher-order frequencies are almost entirely unobserved, causing the model to constantly back off to lower orders.

2. **Zero Parameter Sharing**: Words remain orthogonal indices. The model cannot generalize that *"The programmer wrote a test"* informs *"The developer authored a test"*.

3. **Combinatorial Storage**: Counting tables for large corpora require gigabytes to terabytes of RAM.

These structural limits motivated researchers to explore new paradigms that replace discrete counting tables with continuous, parameterized functions.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively dismantled the mathematical and computational barriers of discrete symbol counting. Figure 1.3 synthesizes the macro-historical progression of language modeling from Claude Shannon's foundational 1948 information theory to modern autonomous software engineering systems.

1.8.1 1948–1980s: Shannon's information theory and Markov n-grams

The conceptual origin of language modeling stems from the landmark 1948 paper *"A Mathematical Theory of Communication"* [4]. In that work, written human language was treated as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a pedagogical thought experiment where an observer attempts to predict the next letter in an unknown sequence of text given the preceding letters. Shannon demonstrated that as the conditioning context length increases from zero-order (independent uniform character selection) to unigram (individual character frequencies), bigram (pairs), and trigram (triplets) models, the generated text rapidly converges toward natural English orthography and syntax.

In this framework, the conditional probability of token w_t given context $w_1, \dots, w_{t-1}$ is simplified via an order-$(n-1)$ Markov assumption, as illustrated in Figure 1.4 for code syntax.

1.8.2 1990s: statistical decision trees and context partitioning

Throughout the 1990s, as speech recognition and machine translation systems scaled, researchers recognized that flat n-gram tables could not condition on long histories without exponential parameter explosion.

To overcome this limitation, *decision-tree language models* were introduced in [10]. Rather than conditioning probability on a rigid, exact sequence of $n-1$ tokens, a binary decision tree asks a sequence of syntactic and lexical questions about the historical context $\mathcal{H} = (w_{t-1}, w_{t-2}, \dots)$.

As illustrated in Figure 1.5, context histories that share identical predictive characteristics are clustered into the same leaf node $\mathcal{L}_k$, yielding conditional distributions $\mathbb{P}(w_t \mid \mathcal{L}_k)$. While this reduced the number of free parameters compared to full tables, the model remained fundamentally bound to discrete categorical splitting without true semantic interpolation.

1.8.3 2000–2003: The neural probabilistic language model

In the landmark 2003 paper *"A Neural Probabilistic Language Model"* [11], natural language processing was revolutionized by introducing continuous distributed representations to defeat the curse of dimensionality.

The neural probabilistic formulation in [11] replaced discrete count tables with a continuous parameterized architecture (Figure 1.6):

1. **Continuous Embedding Projection**: Each word $w \in \mathcal{V}$ is associated with a real-valued distributed feature vector $\mathbf{c}_w \in \mathbb{R}^d$ ($d \ll |\mathcal{V}|$, e.g., $d = 100$). The vocabulary lookup matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is learned jointly via backpropagation.

2. **Semantic Smoothing via Continuous Space**: If *"cat"* and *"feline"* have similar embedding vectors ($\mathbf{c}_{\text{cat}} \approx \mathbf{c}_{\text{feline}}$), observing *"the cat chased the mouse"* automatically increases the model's confidence in *"the feline chased the mouse"*. Continuous geometry solves the orthogonal symbol limitation.

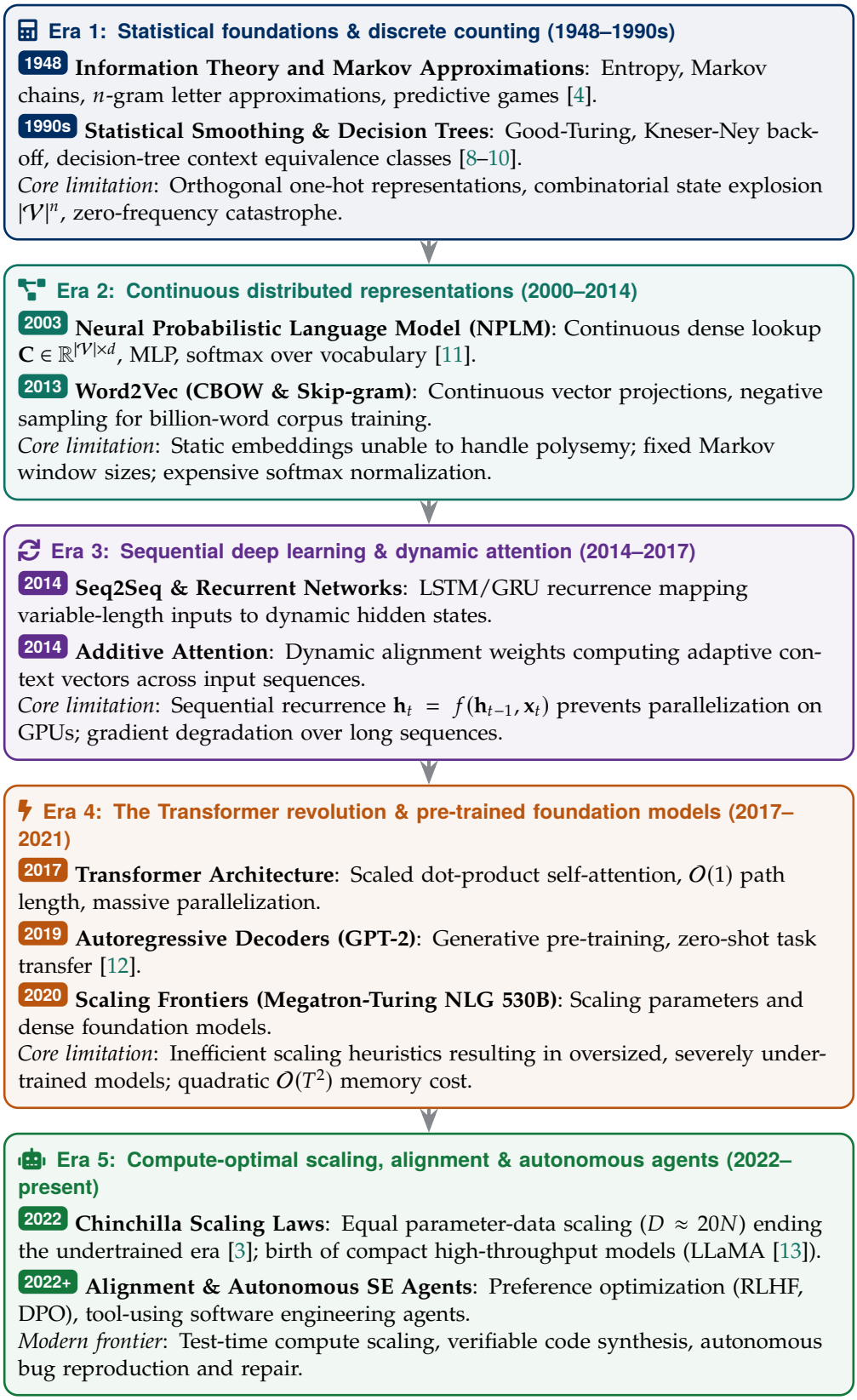


Figure 1.3: Chronological timeline of language model evolution from 1948 to the present.

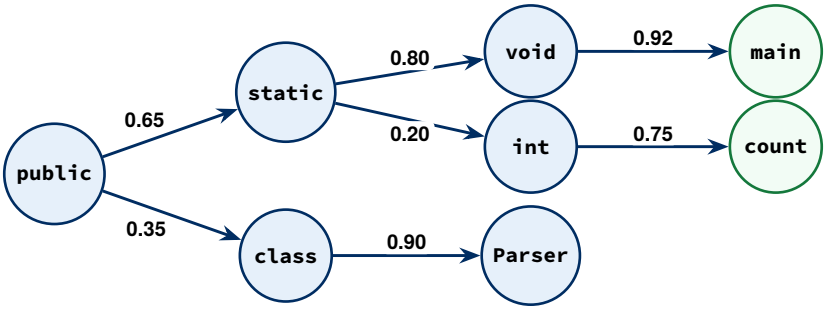


Figure 1.4: Markov chain state transition graph for software tokens.

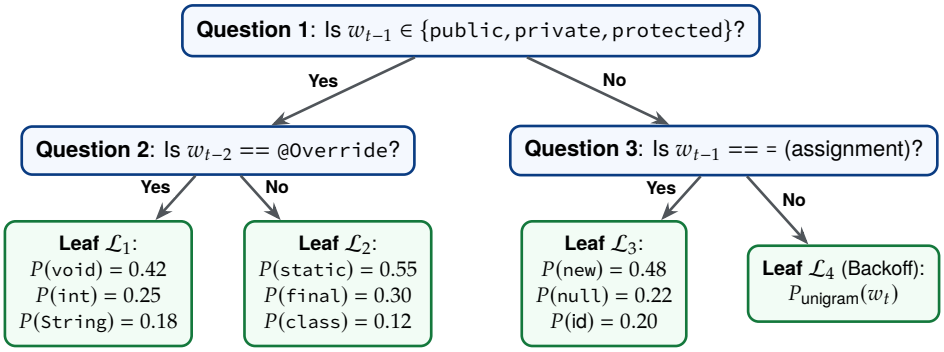


Figure 1.5: Decision tree context partitioning for conditional language modeling [10].

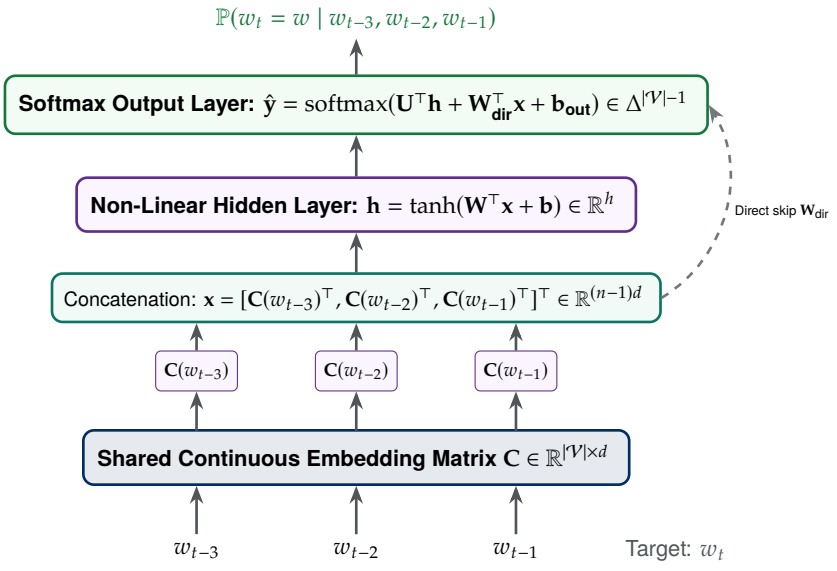


Figure 1.6: The Neural Probabilistic Language Model (NPLM) architecture [11]. Discrete context words are mapped into continuous dense feature vectors via shared matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, concatenated, and processed through a non-linear hidden layer to predict next-token probabilities via softmax.

3. **Softmax Output Normalization**: The network projects hidden activations to $|\mathcal{V}|$ unnormalized logits and applies the softmax function to guarantee a valid probability distribution:

$$\mathbb{P}(w_t = i \mid w_{<t}) = \frac{\exp(z_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j)}. \tag{1.43}$$

1.8.4 2018–2019: Generative pre-training and GPT-2

While continuous representations were established in [11], models throughout the 2000s and 2010s remained restricted in scale and context length. In 2018–2019, the GPT-2 architecture was introduced in [12] (*"Language Models are Unsupervised Multitask Learners"*).

GPT-2 demonstrated that massive, autoregressive decoder-only models trained on hundreds of gigabytes of uncurated internet text (WebText) could perform diverse natural language and software tasks without task-specific supervised training. By predicting next tokens over context windows of 1,024 tokens, the model exhibited emergent zero-shot transfer across reading comprehension, machine translation, and Python code generation.

1.8.5 2020–2022: Megatron-Turing NLG 530B and compute-optimal scaling

Following GPT-2 and GPT-3, the deep learning industry initiated an aggressive scaling arms race, culminating in monolithic models such as the Megatron-Turing NLG 530B [14]. However, early scaling heuristics [15] recommended scaling model parameters faster than training data. Consequently, models like Megatron-Turing NLG 530B and GPT-3 (175B parameters trained on 300B tokens) were severely **undertrained** relative to their parameter capacity.

In 2022, the landmark study *"Training Compute-Optimal Large Language Models"* [3] established the **Chinchilla** scaling laws:

$$D \approx 20 \times N, \tag{1.44}$$

where N is parameter count and D is training tokens. For compute optimality, parameter count and training tokens must scale in equal proportion. This insight prompted the modern era of compact, heavily trained open-weight models (e.g., LLaMA [13] trained on trillions of tokens), dramatically reducing deployment and inference overhead in software engineering environments.

1.8.6 From discrete counting tables to continuous parametric learners: bridge to deep learning

Our exploration of statistical language models in this chapter has revealed a fundamental evolutionary trajectory:

- **The Discrete Counting Limit**: Exact probability factorization via the chain rule (Equation (1.2)) is theoretically exact, but empirical n-gram counting tables suffer from exponential state-space explosion (V^n), context horizon amnesia ($n-1$), and orthogonal semantic blindness.
- **The Smoothing Ceiling**: While smoothing algorithms (Good-Turing, Katz, Kneser-Ney) rescue models from zero-frequency collapse ($P = 0 \implies \text{PPL} = \infty$), they merely redistribute probability mass across discrete table bins. They cannot learn high-dimensional feature interactions or share statistical strength between synonymous phrases.

To transcend the discrete counting paradigm, natural language processing abandoned frequency tables in favor of **continuous, parameterized neural networks**:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}.$$

Instead of counting co-occurrences in text, we parameterize language predictors as continuous functions whose weights $\boldsymbol{\theta}$ are optimized iteratively through gradient descent and backpropagation.

This transition requires establishing the mathematical and computational foundations of deep learning: How does an artificial neuron compute? Why is stacking linear layers without non-linear activations mathematically futile? What allows multi-layer perceptrons to approximate arbitrary complex functions? And how does backpropagation automatically compute gradients across computational graphs?

We address each of these foundational questions in Chapter 2: *Deep Learning Foundations for NLP*.

Chapter 2

Deep learning foundations for NLP

In Chapter 1, we formalized language modeling through the exact decomposition of the chain rule of probability. Yet, our empirical analysis exposed the severe, insurmountable glass ceiling of discrete counting models: table-based frequency estimators suffer from exponential parameter explosion (V^n), context horizon amnesia ($n-1$), and total semantic blindness across distinct vocabulary symbols. Because discrete strings cannot be differentiated and isolated token IDs share zero statistical strength, classical frequency tables cannot scale to the combinatorial complexity of human language or software engineering source code.

To transcend the discrete counting paradigm, modern computational linguistics and artificial intelligence reframe language processing through the lens of *connectionism* and differentiable programming. Instead of tabulating occurrences in discrete bins, we parameterize language predictors as continuous mathematical functions mapping inputs onto high-dimensional metric spaces $\mathbb{R}^d$. In this continuous formulation, semantic similarity corresponds to geometric proximity, syntactic constraints correspond to smooth decision manifolds, and model parameters are learned automatically by propagating error gradients through computational graphs via multivariable calculus.

This chapter establishes the rigorous mathematical and architectural foundations of deep learning required to build modern foundation models. Beginning with the biological inspiration and mathematical formulation of the artificial perceptron, we prove why single layers fail on non-linear tasks (such as XOR) and why multi-layer linear stacking without activations collapses into a single trivial matrix. We then analyze the functional geometry of non-linear activations ($\sigma, \tanh, \text{ReLU}, \text{GELU}$), formalize the Universal Approximation Theorem through localized wavelet bump constructions, derive the convexity of maximum likelihood loss landscapes versus Mean Squared Error, and unpack the exact mechanics of reverse-mode automatic differentiation across directed acyclic graphs.

2.1 From biological neurons to the artificial perceptron

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter 1, we now develop the neural computational foundations that allow language models to scale. In contemporary Natural Language Processing and AI-assisted Software Engineering, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over continuous vector spaces. Rather than storing astronomical tables of discrete counts, we parameterize language predictors as continuous functions whose weights are optimized via gradient descent.

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrochemical signals from upstream cells through branching *dendrites*, integrates these incoming potentials in its central cell body (*soma*), and, when the accumulated electrical voltage exceeds a physiological threshold (the action potential), fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism was first formalized in 1943 [16] as a threshold logic unit and subsequently refined in 1958 [17] into the trainable *Perceptron*.

Definition 2.1: *The artificial perceptron*

Let $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f\left(w_0 + \sum_{i=1}^{n} w_i x_i\right) = f\left(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n\right), \tag{2.1}$$

where $f : \mathbb{R} \to \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \ldots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (2.1) is expressed in compact vector inner-product notation:

$$y = f\left(\sum_{i=0}^{n} w_i x_i\right) = f\left(\mathbf{w}^\top \mathbf{x}\right) \quad \text{with } x_0 = 1,\ w_0 = \text{bias}. \tag{2.2}$$

Figure 2.1 illustrates the structural mechanics of the artificial perceptron: input signals are weighted by synaptic connection strengths, summed at a central junction, and passed through an activation function to generate the output signal.

2.1.1 The perceptron in two dimensions: linear decision hyperplane

To establish geometric intuition, consider a two-dimensional feature space $\mathbf{x} = [x_1, x_2]^\top \in \mathbb{R}^2$ with weights $\mathbf{w} = [w_1, w_2]^\top$ and bias w_0. The pre-activation value is:

$$z(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_0. \tag{2.3}$$

When the perceptron acts as a binary classifier using a step or sign activation ($y = \text{sgn}(z)$), the decision boundary separating the two output classes is precisely the set of points where the pre-activation vanishes:

$$\mathcal{H} = \left\{\mathbf{x} \in \mathbb{R}^2 \mid w_1 x_1 + w_2 x_2 + w_0 = 0\right\}. \tag{2.4}$$

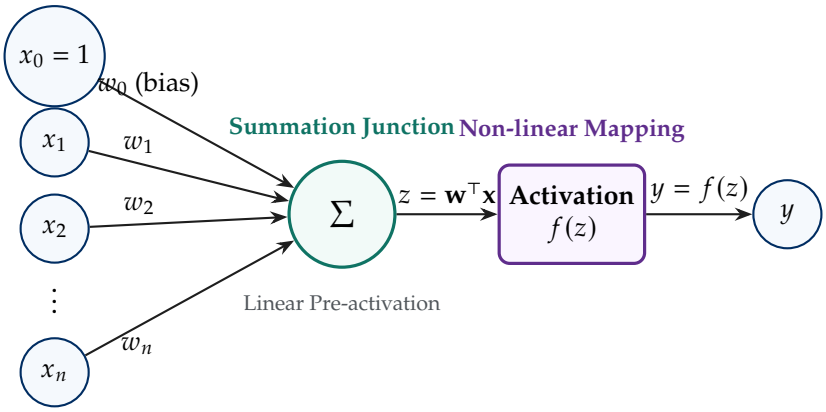


Figure 2.1: Architectural schematic of the artificial perceptron. Inputs x_i are modulated by synaptic weights w_i, accumulated with bias w_0 at the summation junction Σ to yield pre-activation $z = \mathbf{w}^\top \mathbf{x}$, and mapped through activation function $f(z)$ to produce scalar output y.

Geometric interpretation of weights and bias. In two dimensions, Equation (2.4) defines a straight line in the (x_1, x_2) Cartesian plane:

- **Normal Vector**: The weight vector $\mathbf{w} = [w_1, w_2]^\top$ is strictly perpendicular (orthogonal) to the separating line $\mathcal{H}$. For any two points $\mathbf{x}_a, \mathbf{x}_b \in \mathcal{H}$:

$$\mathbf{w}^\top \mathbf{x}_a + w_0 = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_b + w_0 = 0 \implies \mathbf{w}^\top (\mathbf{x}_a - \mathbf{x}_b) = 0.$$

  The vector $\mathbf{w}$ points directly into the positive decision region ($z > 0$).

- **Orthogonal Distance to Origin**: The shortest Euclidean distance from the origin $\mathbf{0} = [0, 0]^\top$ to the separating hyperplane $\mathcal{H}$ is:

$$d = \frac{|w_0|}{\|\mathbf{w}\|_2} = \frac{|w_0|}{\sqrt{w_1^2 + w_2^2}}. \tag{2.5}$$

  The bias w_0 shifts the hyperplane parallel to itself: if $w_0 = 0$, the decision boundary passes exactly through the origin; adjusting w_0 translates the boundary without altering its orientation.

Figure 2.2 diagrams this geometric relationship in $\mathbb{R}^2$.

2.1.2 Connection to classical linear and logistic regression

The artificial perceptron unifies two foundational statistical models studied in classical data analysis:

- **Linear Regression**: If the activation function is the linear identity $f(z) = z$, Equation (2.2) computes:

$$y = \mathbf{w}^\top \mathbf{x},$$

  predicting an unbounded real-valued continuous response variable.

- **Logistic Regression**: If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (2.2) computes:

$$y = \sigma(\mathbf{w}^\top \mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{x}),$$

  predicting the posterior Bernoulli probability of belonging to the positive class.

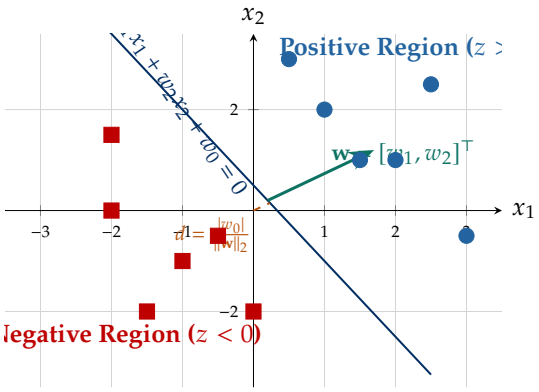


Figure 2.2: Geometric representation of the perceptron decision boundary in $\mathbb{R}^2$. The line $w_1x_1 + w_2x_2 + w_0 = 0$ partitions feature space into positive ($\mathcal{H}^+$) and negative ($\mathcal{H}^-$) half-spaces. The weight vector $\mathbf{w}$ defines the normal vector to the separating hyperplane, and the scalar bias w_0 determines its distance from the origin.

2.1.3 The fatal barrier of single-layer perceptrons: XOR linear inseparability

Despite its mathematical elegance, the single-layer perceptron suffers from a fatal representational barrier that brought artificial intelligence research to a virtual standstill for over a decade. In their seminal 1969 monograph *"Perceptrons"*, Marvin Minsky and Seymour Papert demonstrated that a single-layer perceptron cannot compute the basic *exclusive-or* (XOR) logical function.

The XOR truth table and linear system. Consider the XOR logic function over binary inputs $x_1, x_2 \in \{0, 1\}$:

x_1	x_2	Target $y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

For a single-layer perceptron with weights w_1, w_2 and bias w_0 to compute this function, there must exist parameters such that the linear boundary $w_1x_1 + w_2x_2 + w_0$ outputs positive values ($z \geq 0$) for target 1 and negative values ($z < 0$) for target 0. Substituting the four input pairs yields the system of linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_0 < 0, \tag{2.6}$$
$$(x_1 = 1, x_2 = 0) \implies w_1 + w_0 \geq 0, \tag{2.7}$$
$$(x_1 = 0, x_2 = 1) \implies w_2 + w_0 \geq 0, \tag{2.8}$$
$$(x_1 = 1, x_2 = 1) \implies w_1 + w_2 + w_0 < 0. \tag{2.9}$$

Proof by mathematical contradiction. Summing inequalities (2.7) and (2.8):

$$(w_1 + w_0) + (w_2 + w_0) \geq 0 \implies w_1 + w_2 + 2w_0 \geq 0 \implies w_1 + w_2 \geq -2w_0. \tag{2.10}$$

Now examine inequality (2.9):

$$w_1 + w_2 + w_0 < 0 \implies w_1 + w_2 < -w_0. \tag{2.11}$$

Combining inequalities (2.10) and (2.11) yields:

$$-2w_0 \leq w_1 + w_2 < -w_0 \implies -2w_0 < -w_0.$$

Adding $2w_0$ to both sides yields:

$$0 < w_0 \implies w_0 > 0.$$

However, inequality (2.6) strictly demands that $w_0 < 0$. The assumption that a single linear hyperplane can compute XOR produces a direct mathematical contradiction ($w_0 > 0$ and $w_0 < 0$). ■

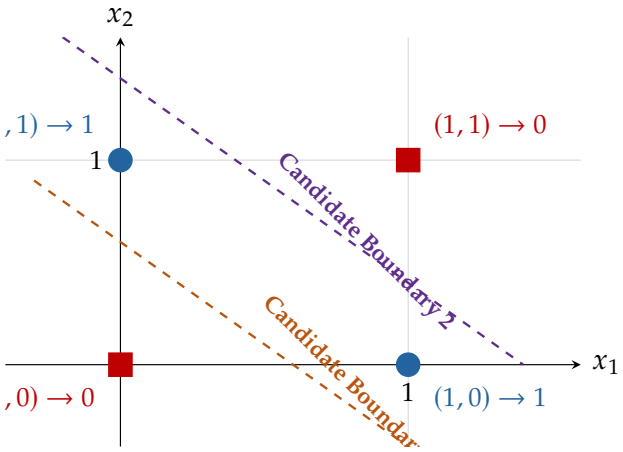


Figure 2.3: The XOR linear inseparability barrier (Minsky & Papert 1969). Class 1 points $(0, 1)$ and $(1, 0)$ (blue circles) cannot be separated from Class 0 points $(0, 0)$ and $(1, 1)$ (red squares) by any single linear hyperplane $w_1x_1 + w_2x_2 + w_0 = 0$. Any single line either isolates $(0, 0)$ or $(1, 1)$, but cannot simultaneously group both off-diagonal points.

As visualized in Figure 2.3, no straight line can separate the diagonal pairs. Because natural language and software engineering phenomena are riddled with non-linear relationships (e.g., mutually dependent programming idioms, nested conditional scopes), single-layer linear perceptrons are fundamentally inadequate.

The causal question. How can we overcome this linear inseparability barrier? An immediate intuitive engineering hypothesis presents itself: *If one linear perceptron can draw one line, can we solve non-linear problems simply by cascading multiple linear perceptrons into a multi-layer network?* We test this hypothesis next.

2.2 Stacking linear layers and the proof of linear stacking collapse

2.2.1 Fully connected linear layers (dense layers)

In multi-layer architectures, individual perceptrons do not operate in isolation; instead, multiple neurons run concurrently across a shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, forming a *dense layer* (or fully connected layer).

Let d_{in} denote the input feature dimensionality and d_{out} denote the number of hidden or output units. Stacking the individual weight vectors $\mathbf{w}_j \in \mathbb{R}^{d_{\text{in}}}$ as columns of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and collecting the scalar biases b_j into a bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the layer computation expresses compactly in matrix notation:

$$\mathbf{y} = f\left(\mathbf{W}^\top \mathbf{x} + \mathbf{b}\right), \tag{2.12}$$

where $f(\cdot)$ is applied elementwise. Figure 2.4 diagrams this fully connected bipartite structure with explicit tensor dimensions and connection weights.

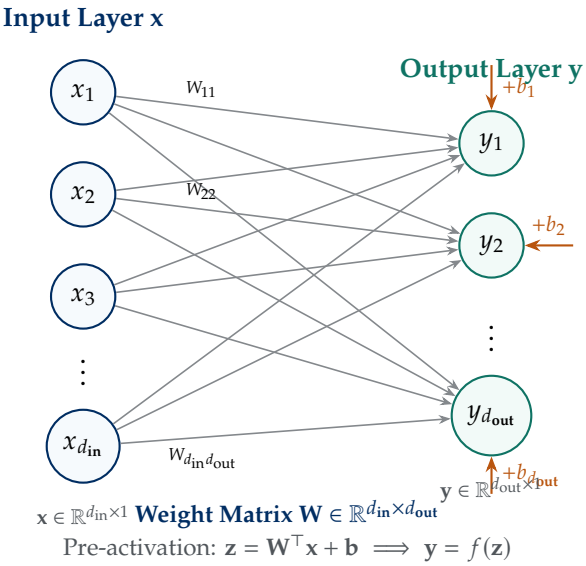


Figure 2.4: Architecture of a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Each directed edge carries an individual scalar weight W_{ij}, and each output neuron incorporates an additive bias b_j.

Exact parameter counting across stacked dense layers. For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$:

$$N_{\text{params}} = \underbrace{d_{\text{in}} \times d_{\text{out}}}_{\text{weight parameters}} + \underbrace{d_{\text{out}}}_{\text{bias parameters}} = d_{\text{out}}(d_{\text{in}} + 1). \tag{2.13}$$

Example Calculation: Consider a two-layer Multi-Layer Perceptron (MLP) mapping an embedding of dimension $d_{\text{in}} = 768$ through a hidden layer $d_{\text{hidden}} = 3072$ to an output projection $d_{\text{out}} = 768$:

1. Layer 1: $3072 \times (768 + 1) = 2{,}359{,}296 + 3072 = 2{,}362{,}368$ parameters.
2. Layer 2: $768 \times (3072 + 1) = 2{,}359{,}296 + 768 = 2{,}360{,}064$ parameters.
3. Total Parameters: $2{,}362{,}368 + 2{,}360{,}064 = 4{,}722{,}432$ parameters.

It is essential to account for the additive bias parameters (d_{out}) alongside the multiplicative weight matrix ($d_{\text{in}} \times d_{\text{out}}$) to obtain exact parameter counts.

2.2.2 The mathematical proof of linear stacking collapse

Now we return to our hypothesis: *Can we solve XOR and non-linear problems simply by cascading multiple linear layers without activation functions?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 2.1: *Linear stacking collapse*

Let $f_1 : \mathbb{R}^n \to \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \to \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n\times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m\times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1, \tag{2.14}$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2. \tag{2.15}$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned} \mathbf{z} &= \mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left(\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2\right) \\ &= \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \end{aligned} \tag{2.16}$$

where $\mathbf{W}_{\text{net}} = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n\times k}$ and $\mathbf{b}_{\text{net}} = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Proof by mathematical induction. Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n\times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m\times k}$ is simply a single matrix $\mathbf{W}_{\text{net}} \in \mathbb{R}^{n\times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}_{\text{net}} \in \mathbb{R}^k$.

By mathematical induction, for any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top \left(\dots \left(\mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) \dots\right) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \tag{2.17}$$

where:

$$\mathbf{W}_{\text{net}} = \prod_{l=1}^{L} \mathbf{W}_l \in \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}, \quad \mathbf{b}_{\text{net}} = \mathbf{b}_L + \sum_{l=1}^{L-1} \left(\prod_{j=l+1}^{L} \mathbf{W}_j\right)^\top \mathbf{b}_l. \tag{2.18}$$

The composite mapping is strictly linear. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. It strictly cannot compute XOR or any non-linear function. ■

The fundamental takeaway. Non-linear activation functions are strictly and mathematically mandatory. Without non-linearity between layers, deep networks collapse into trivial shallow affine mappings. Non-linear activations warp and fold coordinate space, transforming linearly inseparable data manifolds into representations where classes can be cleanly separated.

2.2.3 Universal approximation theorem and bump-function synthesis

Given that non-linear activation functions prevent linear collapse, what family of functions can a non-linear neural network represent? The theoretical foundation validating neural networks as general-purpose function approximators is the *Universal Approximation Theorem*.

Theorem 2.2: *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid) [18]. Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n-dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n. Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^{M} \alpha_j \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right) \tag{2.19}$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} \left|F(\mathbf{x}) - g(\mathbf{x})\right| < \epsilon. \tag{2.20}$$

Originally proved in [18] for sigmoidal activations and generalized in [19] and [20] to arbitrary continuous non-polynomial activations, this theorem guarantees that a feedforward network with a single hidden layer has the mathematical capacity to approximate any continuous function (as discussed in the course theoretical foundations).

Intuitive geometric derivation: bump-function synthesis. To understand intuitively why a single hidden layer can approximate any continuous curve, consider a 1D function $g(x) : [a, b] \to \mathbb{R}$.

Step 1: Constructing a localized rectangular bump. Consider two sigmoid neurons with very high slope $w \gg 0$ located at spatial offsets x_1 and x_2 (with $x_1 < x_2$). As $w \to \infty$, the sigmoid $\sigma(w(x - x_1))$ converges to a Heaviside step function $H(x - x_1)$.

Subtracting the second sigmoid from the first produces a localized *bump function* (or rectangular wavelet):

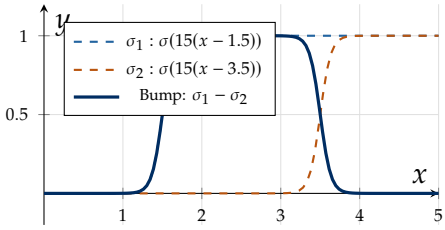
$$B(x) = \sigma(w(x - x_1)) - \sigma(w(x - x_2)) \approx \begin{cases} 1 & \text{if } x_1 \le x \le x_2 \\ 0 & \text{otherwise} \end{cases}. \tag{2.21}$$

Figure 2.5(a) illustrates how two opposing sigmoids subtract to create a localized rectangular pulse.

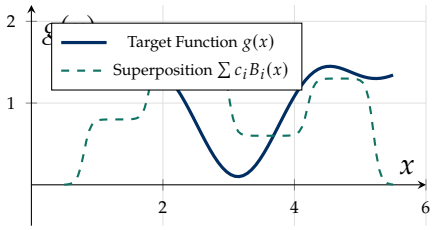
Step 2: Riemann sum function reconstruction. By the definition of continuous functions on compact intervals, $g(x)$ can be approximated to arbitrary accuracy ϵ by a piecewise-constant step function consisting of M uniform sub-intervals $[x_k, x_{k+1}]$ with heights $c_k = g(x_k)$, exactly as in standard Riemann integration:

$$g(x) \approx \sum_{k=1}^{M} c_k B_k(x) = \sum_{k=1}^{M} c_k \left[\sigma(w(x - x_k)) - \sigma(w(x - x_{k+1}))\right]. \tag{2.22}$$

Each bump $B_k(x)$ requires exactly two hidden sigmoid neurons. Consequently, a single hidden layer containing $2M$ neurons can synthesize any continuous function to arbitrary precision, as visualized in Figure 2.5(b).



(a) Bump Synthesis via Opposing Sigmoids



(b) Riemann-style Function Reconstruction

Figure 2.5: Visual derivation of the Universal Approximation Theorem. (a) Subtracting two shifted steep sigmoids produces a localized rectangular bump function $B(x)$; (b) Superimposing multiple scaled bump functions reconstructs any continuous target function to arbitrary accuracy ϵ.

2.2.4 Why deep networks if shallow networks are universal approximators?

If a single hidden layer can approximate any continuous function, why does modern deep learning exclusively build deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

Three foundational theoretical and geometric arguments answer this fundamental question:

1. **Non-Constructive Existence vs. Optimization**: The Cybenko and Hornik theorems guarantee that a set of parameters exists within some arbitrary ϵ-ball, but offer no gradient-based algorithm to discover them. In practice, gradient descent over shallow networks with massive width easily becomes trapped in degenerate saddle points or overfits catastrophic boundary noise.

2. **Exponential Parameter Inefficiency and the Curse of Dimensionality**: Approximating highly oscillatory or compositional functions in d_{in} dimensions with a single hidden layer requires a width M that scales exponentially with input dimensionality ($M \sim O(2^{d_{\text{in}}})$), rendering shallow networks computationally intractable for real-world NLP vocabularies and context windows.

3. **Exponential Linear Region Counting with Depth**: Simon Prince [21] and Montúfar et al. [22] mathematically formalized the expressive advantage of depth through *linear region counting*. A shallow network with 1 hidden layer containing K rectified linear units partitions input space $\mathbb{R}^{d_{\text{in}}}$ into at most:

$$\mathcal{N}_{\text{shallow}} \leq \sum_{j=0}^{d_{\text{in}}} \binom{K}{j} = O(K^{d_{\text{in}}}) \tag{2.23}$$

linear regions. In contrast, cascading D layers of width K allows subsequent layers to compute piecewise linear functions of piecewise linear functions, effectively folding the input domain repeatedly. The maximal number of linear regions generated by a deep ReLU network grows exponentially with depth D:

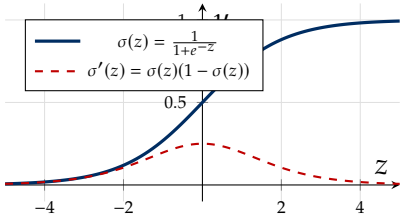
$$\mathcal{N}_{\text{deep}} = O\left(\left(\frac{K}{d_{\text{in}}}\right)^{(D-1)d_{\text{in}}} K^{d_{\text{in}}}\right) \sim O(K^D). \tag{2.24}$$

A deep network of depth D can thus produce an astronomical number of linear regions $O(K^D)$ using only $O(D \cdot K^2)$ total parameters. To match this expressive capacity, a shallow 1-layer network would require an exponential number of neurons $M \sim K^D$, necessitating astronomical parameter storage and memory bandwidth.

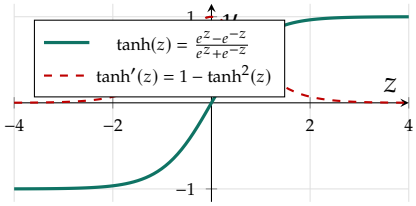
Deep architectures are therefore fundamentally more efficient: they factorize high-dimensional manifolds into hierarchical abstractions, extracting local lexical tokens at early layers, compositional syntactic trees at intermediate layers, and long-range semantic discourse at deep layers.

2.3 Taxonomy of neural activation functions

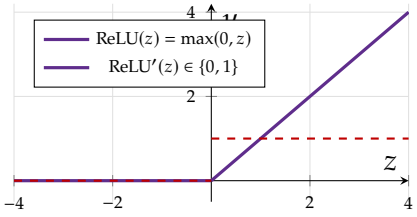
Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads. Figure 2.6 presents exact mathematical plots and derivatives for the primary activation functions.



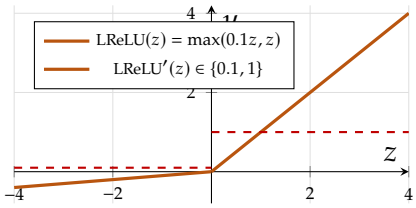
(a) Sigmoid (Logistic)



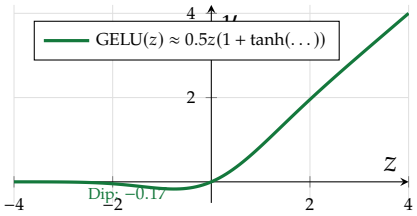
(b) Hyperbolic Tangent (tanh)



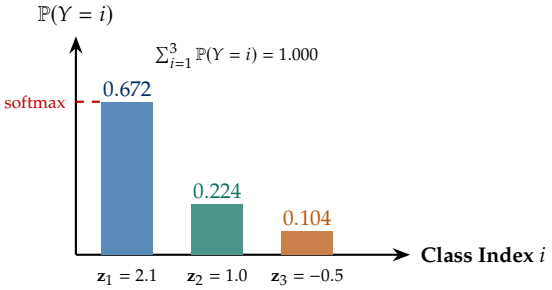
(c) Rectified Linear Unit (ReLU)



(d) Leaky ReLU ($\alpha = 0.1$)



(e) Gaussian Error Linear Unit (GELU)



(f) Softmax Simplex Mapping ($\mathbb{R}^3 \to \Delta^2$)

Figure 2.6: Taxonomy of primary neural activation functions plotted across their operating ranges: (a) Sigmoid $\sigma(z)$ with peak derivative 0.25; (b) Zero-centered $\tanh(z)$ with peak derivative 1.0; (c) Piecewise linear ReLU(z); (d) Leaky ReLU preventing dead units; (e) Non-monotonic GELU(z) used in modern Transformers; (f) Multi-class Softmax mapping logits to the probability simplex.

1. The logistic sigmoid (σ).

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \text{Range: } (0,1), \quad \sigma'(z) = \sigma(z)(1-\sigma(z)). \tag{2.25}$$

The derivative attains its global maximum of $\sigma'(0) = 0.25$ at the origin. For large positive or negative inputs ($|z| \geq 4$), $\sigma'(z) \to 0$, causing severe *vanishing gradients* in deep networks.

2. Hyperbolic tangent (tanh).

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\,\sigma(2z) - 1, \quad \text{Range: } (-1,1), \quad \tanh'(z) = 1 - \tanh^2(z). \tag{2.26}$$

The hyperbolic tangent is strictly zero-centered, possessing average activation values near zero, which accelerates gradient descent compared to non-negative sigmoids. However, it still saturates for $|z| \geq 3$, yielding vanishing gradients.

3. Rectified linear unit (ReLU**).**

$$\text{ReLU}(z) = \max(0, z), \quad \text{Range: } [0, \infty), \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}. \tag{2.27}$$

ReLU completely eliminates gradient vanishing for positive pre-activations ($z > 0$). However, if a large gradient update shifts a neuron's weights such that $z \leq 0$ across all training samples, its activation and derivative permanently collapse to zero ($\text{ReLU}'(z) = 0$), a pathology known as the *dying ReLU problem*.

4. Leaky ReLU (LReLU).

$$\text{LReLU}(z) = \max(\alpha z, z) \quad \text{with } 0 < \alpha \ll 1, \quad \text{Range: } (-\infty, \infty), \quad \text{LReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}. \tag{2.28}$$

By maintaining a small constant slope α (typically $\alpha = 0.01$) in the negative half-plane, Leaky ReLU guarantees that a non-zero gradient always backpropagates, preventing dead neurons.

5. Gaussian error linear unit (GELU**).** Introduced by Dan Hendrycks and Kevin Gimpel, GELU scales the input by its cumulative standard normal probability $\Phi(z)$:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right] \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{2.29}$$

GELU is smooth, differentiable everywhere, and non-monotonic, dipping to a global minimum of ≈ -0.17 at $z \approx -0.75$. This smoothness provides curvature information during optimization, making GELU the default activation in Transformer language models (GPT-2, GPT-3, BERT, RoBERTa).

6. Softmax. For categorical multi-class predictions over C mutually exclusive classes, Softmax normalizes a real-valued logit vector $\mathbf{z} = [z_1, \ldots, z_C]^\top \in \mathbb{R}^C$ onto the unit probability simplex Δ^{C-1}:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \quad \text{such that } \hat{y}_i \in (0, 1) \quad \text{and} \quad \sum_{i=1}^{C} \hat{y}_i = 1. \tag{2.30}$$

Softmax forms the standard vocabulary output head of modern autoregressive language models, computing next-token probability distributions across vocabulary $\mathcal{V}$.

Table 2.1 provides a comparative reference summarizing the mathematical properties of each activation function.

Table 2.1: Comprehensive taxonomy of neural activation functions, mathematical formulations, operating bounds, and architectural domains.

Activation	Domain	Range	Peak Gradient	Primary Use Case in Language Modeling
Sigmoid (σ)	$\mathbb{R}$	$(0, 1)$	0.25 ($z = 0$)	Binary classification output heads; gating mechanisms in LSTM/GRU highways.
Tanh	$\mathbb{R}$	$(-1, 1)$	1.00 ($z = 0$)	Recurrent hidden state candidate generation; zero-centered coordinate representations.
ReLU	$\mathbb{R}$	$[0, \infty)$	1.00 ($z > 0$)	Classical deep feedforward networks; compute-efficient linear regions.
Leaky ReLU	$\mathbb{R}$	$\mathbb{R}$	1.00 ($z > 0$), α ($z \leq 0$)	Prevents dying neuron syndrome; robust discriminator networks.
GELU	$\mathbb{R}$	$[-0.17, \infty)$	≈ 1.08	Canonical Transformer activation (GPT, BERT); smooth, probabilistic gating.
Softmax	$\mathbb{R}^C$	Δ^{C-1}	Matrix Jacobian	Next-token prediction vocabulary head; maps logits to discrete probability simplex.
Identity	$\mathbb{R}$	$\mathbb{R}$	1.00 ($\forall z$)	Regression output head; unbounded linear prediction.

2.4 Architectural task formulations and software engineering case studies

In machine learning applications for software engineering, task formulations map specific software artifacts (commits, ASTs, bug reports, source code tokens) into continuous predictions. Three primary architectural heads dominate:

2.4.1 1. Binary classification: automated vulnerability detection

Consider predicting whether a proposed code commit contains a security vulnerability (e.g., buffer overflow, SQL injection).

- **Input Vector**: $\mathbf{x} \in \mathbb{R}^d$ (extracted code embeddings or static analysis metrics).
- **Output Layer**: A single linear neuron producing logit $z = \mathbf{w}^\top \mathbf{h} + b \in \mathbb{R}$.
- **Activation**: Logistic Sigmoid $\hat{y} = \sigma(z) = \mathbb{P}(\text{Vulnerable} = 1 \mid \mathbf{x}) \in [0, 1]$.
- **Decision Rule**: Flag for manual audit if $\hat{y} \geq \tau$ (where $\tau = 0.5$ by default, tuned to optimize recall).

Figure 2.7(a) illustrates the resulting separating hyperplane in feature space.

2.4.2 2. Multi-class classification: automated AST node type prediction

In program repair and neural compiler optimization, models classify Abstract Syntax Tree (AST) nodes into C discrete syntactic types (e.g., `MethodDeclaration`, `VariableDeclaration`, `IfStatement`, `WhileStatement`).

- **Output Layer**: C parallel linear neurons producing logit vector $\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} \in \mathbb{R}^C$.
- **Activation**: Softmax operator $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$.
- **Decision Rule**: Predicted AST type $\hat{c} = \arg\max_i \hat{y}_i$.

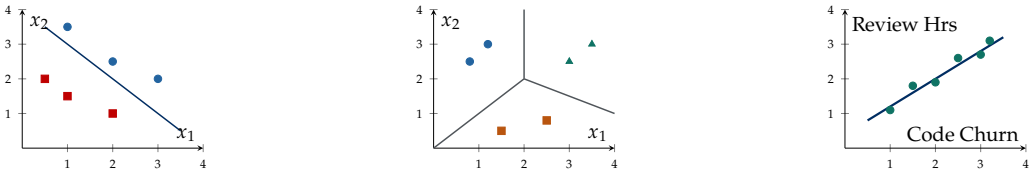
Figure 2.7(b) diagrams the resulting Voronoi-like partitioned multi-class decision space.

2.4.3 3. Regression: pull request review effort estimation

Engineering managers must predict continuous quantities, such as the estimated review time (in hours) required for an incoming pull request based on code churn, changed files, and author experience.

- **Output Layer**: A single linear neuron producing scalar $z = \mathbf{w}^\top \mathbf{h} + b$.
- **Activation**: Linear Identity function $f(z) = z \in \mathbb{R}$.
- **Prediction**: $\hat{y} = z \in \mathbb{R}$ (continuous unbounded prediction).

Figure 2.7(c) illustrates continuous line fitting through software engineering observation points.



(a) Binary: Defect Detection (b) Multi-class: AST Type (c) Regression: PR Review Effort

Figure 2.7: Task formulations across Software Engineering workflows: (a) Binary vulnerability detection with separating hyperplane; (b) Multi-class defect categorization with partitioned decision regions; (c) Continuous PR review effort regression with residual errors.

2.5 Loss function formulations and optimization landscapes

2.5.1 Loss function formulations

Training a neural network requires an objective function $\mathcal{L}(\theta)$ that measures the discrepancy between model predictions $\hat{y}$ and ground-truth targets y.

1. Mean squared error (MSE). For continuous regression targets:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2. \tag{2.31}$$

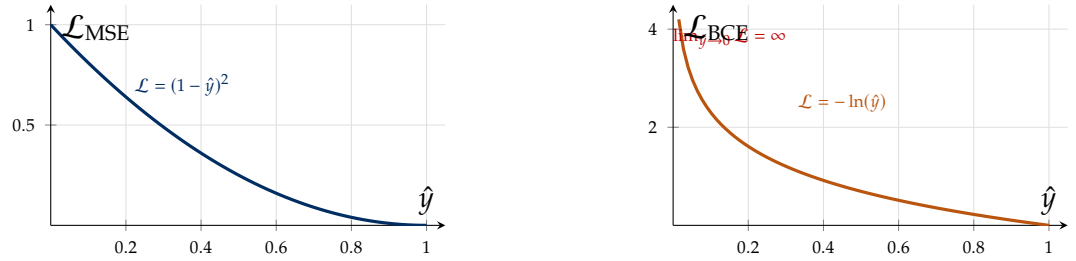
2. Binary cross-entropy (BCE). For binary classification with target $y \in \{0,1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0,1]$:

$$\mathcal{L}_{\text{BCE}}(\theta) = -y\ln(\hat{y}) - (1-y)\ln(1-\hat{y}). \tag{2.32}$$

3. Categorical cross-entropy (CCE). For multi-class classification over C mutually exclusive classes with one-hot target vector $\mathbf{y} \in \{0,1\}^C$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\sum_{i=1}^{C} y_i \ln(\hat{y}_i) = -\ln\left(\hat{y}_{\text{target}}\right) = -\ln\left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{2.33}$$

Figure 2.8 compares the mathematical profiles of MSE and BCE for a positive target $y = 1$.



(a) MSE Loss for $y = 1$ (Bounded: $\mathcal{L} \in [0,1]$)

(b) BCE Loss for $y = 1$ (Asymptotic Penalty)

Figure 2.8: Comparison of loss function behaviors for target $y = 1$. While MSE is bounded quadratically at 1.0, BCE imposes an asymptotic infinite penalty ($\mathcal{L} \to \infty$) as predicted probability $\hat{y} \to 0$, providing steep, non-vanishing corrective gradients.

2.5.2 Rigorous proof: why MSE fails for classification

A fundamental theoretical question is: *Why do we never use Mean Squared Error with sigmoid activation for classification?* The answer lies in gradient saturation and non-convexity.

Theorem 2.3: *Gradient saturation of MSE under sigmoid activation*

Let a binary classification model predict $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ with ground-truth target $y \in \{0,1\}$. Under the quadratic loss $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y-\hat{y})^2$, the gradient with respect to the

pre-activation logit z is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}). \tag{2.34}$$

Conversely, under Binary Cross-Entropy $\mathcal{L}_{\text{BCE}} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, the gradient is:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y. \tag{2.35}$$

Proof. 1. For MSE, applying the calculus chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}. \tag{2.36}$$

The derivative of the loss with respect to $\hat{y}$ is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2}(y - \hat{y})^2 \right] = -(y - \hat{y}).$$

The derivative of the sigmoid is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$. Substituting:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y})\hat{y}(1 - \hat{y}). \tag{2.37}$$

2. For BCE, expanding the loss:

$$\mathcal{L}_{\text{BCE}} = -y \ln(\sigma(z)) - (1 - y) \ln(1 - \sigma(z)). \tag{2.38}$$

Differentiating directly with respect to logit z:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= -y \frac{\sigma'(z)}{\sigma(z)} - (1 - y) \frac{-\sigma'(z)}{1 - \sigma(z)} \\
&= -y \frac{\sigma(z)(1 - \sigma(z))}{\sigma(z)} + (1 - y) \frac{\sigma(z)(1 - \sigma(z))}{1 - \sigma(z)} \\
&= -y(1 - \sigma(z)) + (1 - y)\sigma(z) \\
&= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\
&= \sigma(z) - y = \hat{y} - y. \quad \blacksquare
\end{aligned}$$

Analytical comparison of optimization behavior. The fundamental architectural flaws of MSE for classification are immediately apparent from Equations (2.37) and (2.35):

- **The Worst-Case MSE Pathology (Confident and Wrong)**: Suppose the true label is $y = 1$, but the network is completely mistaken, predicting $z = -10 \implies \hat{y} \approx 0.000045$. The prediction error is maximal: $(y - \hat{y}) \approx 1.0$. However, evaluating the MSE gradient:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(1.0) \times (0.000045) \times (0.999955) \approx -0.000045 \approx 0!$$

  The gradient completely vanishes! Even though the model is entirely wrong, parameter updates halt because the sigmoid derivative $\hat{y}(1 - \hat{y})$ saturates to zero. Optimization gets trapped on a flat plateau.

- **The BCE Remedy**: Under BCE, for the identical scenario ($y = 1, \hat{y} \approx 0$):

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y = 0 - 1 = -1.0.$$

The sigmoid derivative in the numerator cancels perfectly with the denominator in the derivative of the logarithm! The gradient is strictly linear with respect to the error $(y - \hat{y})$ and provides maximum learning signal precisely when the model is most wrong.

- **Loss Surface Geometry and Convexity in Logit Space**: As formalized by Goodfellow et al. [23], the fundamental geometric pathology of combining MSE with sigmoidal classification units is the destruction of convexity.

To see this analytically, evaluate the second derivative of the loss with respect to pre-activation logit z. For MSE with $\hat{y} = \sigma(z)$:

$$\begin{aligned}\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2} &= \frac{\partial}{\partial z}\left[-(y - \sigma(z))\sigma'(z)\right] \\ &= (\sigma'(z))^2 - (y - \sigma(z))\sigma''(z). \end{aligned} \tag{2.39}$$

Recalling that $\sigma''(z) = \sigma'(z)(1 - 2\sigma(z))$, we have:

$$\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2} = \sigma'(z)\left[\sigma'(z) - (y - \sigma(z))(1 - 2\sigma(z))\right]. \tag{2.40}$$

When the prediction is severely wrong (e.g., $y = 1$ and $z \ll 0$ so $\sigma(z) \to 0$), the term $(y - \sigma(z))(1 - 2\sigma(z)) \approx (1)(1) = 1$, which strictly dominates $\sigma'(z) \approx 0$. Consequently, the bracketed quantity becomes negative, causing the second derivative to flip sign or vanish. This sign reversal breaks positive semi-definiteness, creating vast non-convex plateaus, inflection points, and saddle regions where gradient descent stalls indefinitely.

In stark contrast, for Binary Cross-Entropy, the first derivative is $\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \sigma(z) - y$. Differentiating again with respect to z:

$$\frac{\partial^2 \mathcal{L}_{\text{BCE}}}{\partial z^2} = \frac{\partial}{\partial z}(\sigma(z) - y) = \sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0 \quad \forall z \in \mathbb{R}. \tag{2.41}$$

Because $\sigma(z) \in (0, 1)$ for all finite $z \in \mathbb{R}$, the second derivative is strictly positive everywhere. The BCE loss surface is therefore **strictly convex** with respect to the logit z. In logit space, there are no local plateaus, no saddle points, and any stationary point is guaranteed to be the unique global minimum.

2.5.3 Maximum likelihood estimation derivation of cross-entropy

Cross-entropy is not an empirical heuristic; it is the mathematically canonical loss derived from Maximum Likelihood Estimation (MLE).

Bernoulli likelihood derivation (BCE). For binary classification, target $y_i \in \{0, 1\}$ follows a Bernoulli distribution parameterized by probability $p_i = \hat{y}_i = \sigma(z_i)$:

$$\mathbb{P}(Y = y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}. \tag{2.42}$$

For an independent dataset of N samples, the total likelihood is $\mathcal{L}_{\text{data}}(\theta) = \prod_{i=1}^{N} \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$. Taking the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\ln \mathcal{L}_{\text{data}}(\theta) = -\sum_{i=1}^{N}\left[y_i \ln \hat{y}_i + (1 - y_i)\ln(1 - \hat{y}_i)\right] = \sum_{i=1}^{N} \mathcal{L}_{\text{BCE},i}. \tag{2.43}$$

Minimizing BCE is mathematically identical to maximizing the data likelihood under a Bernoulli distribution.

Categorical likelihood derivation (CCE). For multi-class classification over C classes with one-hot targets $\mathbf{y}_i$, the likelihood follows a Categorical distribution:

$$\mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = \prod_{c=1}^{C} \hat{y}_{ic}^{\,y_{ic}} \implies -\ln \mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{c=1}^{C} y_{ic} \ln \hat{y}_{ic} = \mathcal{L}_{\text{CE}}. \tag{2.44}$$

2.5.4 Gradient descent and adaptive optimizers

Parameters θ are updated iteratively along the negative gradient:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)}), \tag{2.45}$$

where $\alpha > 0$ is the learning rate.

Limitations of vanilla gradient descent. In deep neural architectures, vanilla Gradient Descent struggles with:

1. **Ravines and Ill-Conditioned Curvature**: Valleys where gradient magnitude is large across the ravine walls but tiny along the base, causing erratic cross-valley oscillation.
2. **Saddle Points**: Points where $\nabla\mathcal{L} = 0$ while the Hessian possesses both positive and negative eigenvalues.

Adaptive remedies: Momentum and Adam.

- **Momentum**: Incorporates an exponentially decaying velocity vector $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta)\nabla\mathcal{L}(\theta_t)$, accelerating descent through saddle regions.
- **Adam (Adaptive Moment Estimation)** [24]: Tracks running averages of both gradients (first moment $\mathbf{m}_t$) and squared gradients (second uncentered moment $\mathbf{v}_t$), with bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \tag{2.46}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \tag{2.47}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \tag{2.48}$$

Adam scales learning rates adaptively per parameter, making it the standard optimizer across LLM pre-training and fine-tuning pipelines.

2.6 Computational graphs and exact adjoint backpropagation

To compute gradients efficiently across billions of parameters, deep learning frameworks (PyTorch, JAX) represent mathematical workflows as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 2.2: *Computational graph*

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical operation $(+, \times, -, \exp)$.
- Each directed edge $e = (u, v) \in \mathcal{E}$ represents intermediate data flow from variable u into operation v.

2.6.1 Exact scalar adjoint backpropagation on a computational DAG

To build foundational intuition for how reverse-mode automatic differentiation operates on directed acyclic graphs, consider a canonical scalar computational model.

Problem setup. Consider a model with two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on training pair $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction:} \quad \hat{y} = \theta_1 \theta_2 x, \tag{2.49}$$

$$\text{Objective Loss:} \quad \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \tag{2.50}$$

Forward decomposition. The graph decomposes Equation (2.50) into four primitive operations:

$$a = \theta_1 \theta_2 \quad \text{(parameter interaction product)}, \tag{2.51}$$

$$b = ax = \theta_1 \theta_2 x \quad \text{(prediction } \hat{y}\text{)}, \tag{2.52}$$

$$c = b - y = \theta_1 \theta_2 x - y \quad \text{(residual error)}, \tag{2.53}$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad \text{(scalar squared loss)}. \tag{2.54}$$

Backward adjoint chain rule derivation. Backpropagation computes the derivative of scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by traversing the graph in reverse topological order:

$$\text{Step 1:} \quad \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial (c^2)}{\partial c} = 2c. \tag{2.55}$$

$$\text{Step 2:} \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial (b - y)}{\partial b} = 2c \cdot 1 = 2c. \tag{2.56}$$

$$\text{Step 3:} \quad \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial (ax)}{\partial a} = 2c \cdot x = 2cx. \tag{2.57}$$

$$\text{Step 4:} \quad \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1 \theta_2 x - y) x \theta_2. \tag{2.58}$$

$$\text{Step 5:} \quad \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1 \theta_2 x - y) x \theta_1. \tag{2.59}$$

Figure 2.9 illustrates this complete forward and backward adjoint execution.

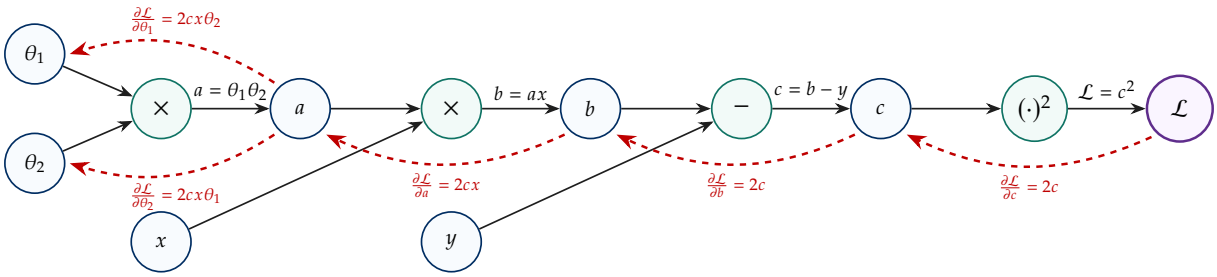


Figure 2.9: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1\theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1\theta_2 x - y)^2$. Solid black arrows trace forward execution; dashed red arrows trace reverse-mode adjoint backpropagation via the chain rule [25].

Geometric perturbation perspective.
Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus algebra, but as a systematic tracking of sensitivity nudges. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ. That nudge ripples through the product node to scale a by $\epsilon\theta_2$. The change in a wiggles b by $\epsilon\theta_2 x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c\epsilon\theta_2 x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

Mechanics of reverse-mode autodiff engines: Micrograd.
Andrej Karpathy's open-source library Micrograd implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure 2.9.

Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial\mathcal{L}/\partial\cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to operand children nodes.
- `._backward`: a closure storing the local multivariate chain rule.

For example, multiplication is implemented as:

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish reverse dependency ordering, sets `loss.grad = 1.0`, and executes `node._backward()` across all parent nodes in reverse topological order.

2.6.2 Reverse-mode automatic differentiation: Vector-Jacobian products (VJPs)

As formalized by Bishop & Bishop [26], the theoretical foundation of backpropagation rests on the duality between *forward-mode* and *reverse-mode* automatic differentiation.

Consider a multi-stage computational sequence where an intermediate vector $\mathbf{u} \in \mathbb{R}^{D_u}$ maps to $\mathbf{v} \in \mathbb{R}^{D_v}$ via differentiable mapping $\mathbf{v} = \mathbf{g}(\mathbf{u})$, eventually yielding a scalar training loss $\mathcal{L} \in \mathbb{R}$.

The sensitivity of the mapping is captured by its Jacobian matrix:

$$\mathbf{J_g} = \frac{\partial \mathbf{v}}{\partial \mathbf{u}} \in \mathbb{R}^{D_v \times D_u}, \quad (\mathbf{J_g})_{jk} = \frac{\partial v_j}{\partial u_k}. \tag{2.60}$$

Forward-mode automatic differentiation (Jacobian-Vector Products). Forward mode propagates directional perturbation vectors $\mathbf{r}_u \in \mathbb{R}^{D_u}$ from inputs toward outputs by evaluating *Jacobian-Vector Products* (JVPs):

$$\mathbf{r}_v = \mathbf{J_g}\mathbf{r}_u \in \mathbb{R}^{D_v}. \tag{2.61}$$

To calculate the complete gradient of loss $\mathcal{L}$ with respect to a parameter vector of dimension P (where $P = D_{\text{in}}$), forward-mode must seed directional vectors along each coordinate basis $\mathbf{e}_1, \ldots, \mathbf{e}_P$ sequentially. Computing the full gradient vector requires P separate forward sweeps, yielding a computational complexity of:

$$\text{Cost}_{\text{forward-mode}}(\nabla_\theta \mathcal{L}) = O(P \times \text{Cost(forward pass)}). \tag{2.62}$$

In modern foundation models where parameter count reaches billions ($P \sim 10^9 - 10^{11}$), forward-mode differentiation is entirely infeasible.

Reverse-mode automatic differentiation (Vector-Jacobian Products). Reverse mode (backpropagation) inverts the direction of differentiation. Beginning at the final scalar loss $\mathcal{L} \in \mathbb{R}$ with initial adjoint seed $\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$, the algorithm propagates adjoint row vectors $\bar{\mathbf{v}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{v}}\right)^\top \in \mathbb{R}^{1 \times D_v}$ backward through the DAG by evaluating *Vector-Jacobian Products* (VJPs):

$$\bar{\mathbf{u}} = \bar{\mathbf{v}}\mathbf{J_g} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{v}}\right)^\top \frac{\partial \mathbf{v}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times D_u}. \tag{2.63}$$

Because the objective $\mathcal{L}$ is a single scalar ($D_{\text{out}} = 1$), a **single reverse traversal** of the computational graph simultaneously evaluates the exact partial derivatives with respect to all P parameters. The computational cost of reverse-mode autodiff scales independently of parameter count P:

$$\text{Cost}_{\text{reverse-mode}}(\nabla_\theta \mathcal{L}) \leq \omega \cdot \text{Cost(forward pass)}, \quad \omega \in [2, 5]. \tag{2.64}$$

Crucially, production deep learning runtimes never materialize the full dense Jacobian matrix $\mathbf{J_g}$ in GPU memory (which would require $O(D_u D_v)$ memory). Instead, each primitive node provides an analytic VJP operator that computes the vector-matrix product directly. For linear projections, this operator computes the matrix-vector products $\mathbf{W}\boldsymbol{\delta}$ and outer products $\mathbf{x}\boldsymbol{\delta}^\top$ derived below.

2.6.3 Vectorized matrix adjoint backpropagation for dense layers

While scalar computational graphs illustrate the chain rule clearly, production deep learning libraries execute backpropagation via *vectorized matrix calculus* across high-dimensional tensors. We now derive the exact vectorized matrix adjoint equations for fully connected layers with explicit tensor dimensions.

Layer formulation and forward variables. Consider a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ through weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and bias $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ to pre-activation $\mathbf{z} \in \mathbb{R}^{d_{\text{out}}}$ and activation $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \tag{2.65}$$

$$\mathbf{y} = f(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{2.66}$$

where scalar component z_j expands as:

$$z_j = \sum_{i=1}^{d_{\text{in}}} W_{ij} x_i + b_j. \tag{2.67}$$

The layer output $\mathbf{y}$ propagates downstream to contribute to a final scalar objective loss $\mathcal{L} \in \mathbb{R}$.

Adjoint definition. Assume downstream backpropagation has computed the loss gradient with respect to layer pre-activation $\mathbf{z}$. We define the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\boldsymbol{\delta} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot f'(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{2.68}$$

where $\odot$ denotes the elementwise Hadamard product.

1. Weight matrix adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$). To compute the derivative of scalar loss $\mathcal{L}$ with respect to individual matrix element W_{ij}, we apply the multivariate chain rule across all output pre-activations z_k:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial W_{ij}}. \tag{2.69}$$

Because $z_k = \sum_{l=1}^{d_{\text{in}}} W_{lk} x_l + b_k$, the partial derivative is non-zero only when $k = j$:

$$\frac{\partial z_k}{\partial W_{ij}} = \begin{cases} x_i & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}. \tag{2.70}$$

Substituting this back:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \frac{\partial \mathcal{L}}{\partial z_j} x_i = \delta_j x_i = x_i \delta_j. \tag{2.71}$$

Collecting all elements W_{ij} into matrix form reveals that the weight gradient is identically the **outer product** of the upstream input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x} \boldsymbol{\delta}^\top \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}. \tag{2.72}$$

2. Bias vector adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$). For each scalar bias component b_j:

$$\frac{\partial \mathcal{L}}{\partial b_j} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial b_j} = \delta_j \cdot 1 = \delta_j. \tag{2.73}$$

In vector notation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}. \tag{2.74}$$

3. Upstream input adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$). To propagate error signals back to previous layers in a deep neural network, we must evaluate the gradient of loss $\mathcal{L}$ with respect to the input feature vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$. Applying the chain rule to input component x_i:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \delta_k W_{ik} = \sum_{k=1}^{d_{\text{out}}} W_{ik} \delta_k. \tag{2.75}$$

Recognizing this sum as the i-th element of the matrix-vector multiplication between weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and adjoint vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{in}}}. \tag{2.76}$$

Figure 2.10 diagrams this vectorized adjoint error propagation across the dense layer.

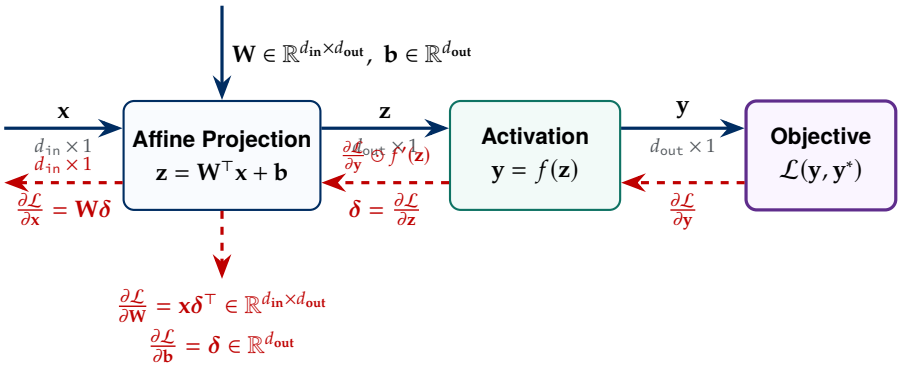


Figure 2.10: Vectorized matrix adjoint backpropagation across a fully connected dense layer. Solid navy arrows trace forward execution mapping input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$; dashed red arrows trace reverse-mode adjoint error propagation ($\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$), yielding parameter updates $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}\boldsymbol{\delta}^\top$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta}$, and upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta}$.

Mini-batch matrix generalization. In GPU implementations, B samples are grouped into a batch data matrix $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$. The layer computes:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{1}_B \mathbf{b}^\top \in \mathbb{R}^{B \times d_{\text{out}}}, \quad \mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{B \times d_{\text{out}}}. \tag{2.77}$$

Defining batch adjoint matrix $\boldsymbol{\Delta} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \in \mathbb{R}^{B \times d_{\text{out}}}$, the gradient equations generalize to highly efficient Basic Linear Algebra Subprograms (BLAS) matrix multiplications:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \boldsymbol{\Delta} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \tag{2.78}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\Delta}^\top \mathbf{1}_B \in \mathbb{R}^{d_{\text{out}}}, \tag{2.79}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \boldsymbol{\Delta}\mathbf{W}^\top \in \mathbb{R}^{B \times d_{\text{in}}}. \tag{2.80}$$

These matrix equations form the computational workhorse executed across billions of parameters in modern GPU training clusters.

2.7 The sequential computation impasse: bridge to recurrent neural networks

In this chapter, we have formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs / Multi-Layer Perceptrons):

1. **Continuous Units**: The artificial perceptron computes weighted inner products over real-valued vectors.
2. **Non-Linearity**: Non-linear activations (σ, tanh, ReLU, GELU) prevent multi-layer linear stacking collapse, folding space to solve linearly inseparable problems like XOR and enabling universal function approximation.
3. **Principled Optimization**: Cross-entropy objective functions provide linear error signals without gradient saturation, and backpropagation calculates matrix adjoints across computational DAGs via vectorized matrix calculus.

The feedforward assumption and its sequential failure. Throughout these formulations, feedforward networks operate under a fundamental structural assumption: the input is a static, fixed-dimensional Euclidean vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, and the network maps this vector through a cascade of static linear projections $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ to produce a fixed-dimensional output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \Theta) = f_L(\mathbf{W}_L \cdots f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_L). \tag{2.81}$$

While this architecture is highly effective for tabular data and fixed-attribute classification, it immediately encounters a computational impasse when confronted with sequential data—such as human natural language, software source code tokens, execution traces, and temporal signals. Sequential data violates the core assumptions of feedforward processing in three fundamental ways:

1. **Rigid Input Dimensionality**: Natural language sentences and source code blocks have elastic, variable lengths T. An FCNN strictly expects an input of dimension d_{in}, forcing artificial truncation or massive zero-padding that wastes computational throughput and memory.
2. **Zero Internal State and Temporal Memory**: Feedforward networks are memoryless. Each forward pass $\mathcal{F}(\mathbf{x})$ is independent and decoupled from prior inputs. They cannot maintain temporal context or resolve long-range grammatical and syntactic dependencies across multiple time steps.
3. **Absence of Parameter Sharing Across Positions**: In an FCNN, each position in the input is governed by separate weight parameters. If a syntactic construct (such as a variable declaration `"let x = 5;"` or a noun phrase *"a large language model"*) appears at token position 1 versus token position 20, the FCNN must learn separate weight matrices for each position independently, destroying sample efficiency.

The path forward. To process variable-length sequences, maintain evolving context over time, and achieve translational invariance across sequence positions, deep learning requires a computational paradigm with internal recurrence and temporal weight sharing.

This architectural necessity leads directly to Chapter 3: *Recurrent Neural Networks and Sequence Modeling*.

Chapter 3

Recurrent neural networks and sequence modeling

In Chapter 2, we developed the continuous mathematical foundation of neural representations, proving how non-linear activations prevent matrix collapse and how reverse-mode automatic differentiation computes parameter sensitivities across complex computational graphs. Yet, our architectural analysis revealed an irreducible structural impasse: feedforward networks are intrinsically static. By operating exclusively over fixed-dimensional Euclidean vectors $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, feedforward networks treat each input in isolation, possess no persistent internal memory, and enforce rigid, position-specific parameterizations that fail to generalize across sequence positions.

Human natural language and software source code, however, are inherently sequential and elastic. Words derive their meaning from preceding and succeeding context, syntactic rules govern grammatical agreement across distant clauses, and execution traces unfold chronologically over time. To process sequential phenomena effectively, deep learning models must encode the underlying physical symmetries of time-series data: *temporal stationarity* and *causal recurrence*.

This chapter develops the theoretical and practical foundations of recurrent sequence modeling. We begin by examining the failure modes of feedforward networks and formalizing the inductive biases of convolutional and recurrent architectures through the twin symmetries of translation equivariance and temporal parameter sharing. We then derive the mechanics of the simple recurrent cell, prove the mathematical pathology of vanishing and exploding gradients in Backpropagation Through Time (BPTT) via Pascanu's spectral radius theorem, analyze character-level training dynamics on a classic literary corpus, unpack the Constant Error Carousel (CEC) of gated LSTM and GRU architectures, and evaluate Sequence-to-Sequence pipelines with Bahdanau additive attention.

3.1 Sequential data and constraints of feed-forward neural networks

In Chapter 2, we formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs) and backpropagation across computational DAGs. As established at the conclusion of that chapter, however, feedforward architectures operate under a rigid, memoryless paradigm: they assume fixed-dimensional Euclidean inputs $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, possess no internal recurrence or persistent temporal state across sequential time steps, and cannot enforce grammatical agreement across temporal horizons.

In standard deep learning, a Fully-Connected Neural Network (FCNN) or Multi-Layer Perceptron is represented as a cascade of weight matrices interleaved with non-linear activation functions:

$$f(\mathbf{x}) = \sigma\left(\mathbf{W}_{\text{out}}\,\text{ReLU}\left(\mathbf{W}_2\,\text{ReLU}\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) + \mathbf{b}_{\text{out}}\right). \tag{3.1}$$

While FCNNs excel on tabular datasets with fixed, predefined attributes, they exhibit three critical architectural constraints when applied to natural language and temporal sequences:

Rigid input dimensionality constraints.

The input dimensionality is rigidly fixed by the number of columns in the first weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_{\text{in}}}$. For structured tabular data, this constraint is natural because every sample contains the same predefined attributes. However, for unstructured sequential data, fixed input sizing is deeply problematic:

- Natural language sentences come in all lengths: from 2 words (*"Run now"*) to 80 words in complex legal contracts.
- Audio recordings exhibit variable sampling frequencies and durations.
- Images come in varying resolutions and aspect ratios.

Flattening a variable-length text sequence into a rigid vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ requires setting an arbitrary maximum length $T_{\max}$ and applying zero-padding to shorter sentences while truncating longer ones. This wastes vast memory on empty pad tokens and destroys the elastic temporal nature of human language.

Lack of parameter sharing and translation invariance.

In an FCNN, each position in the flattened input vector is associated with its own distinct set of weight parameters. Consequently, if the same linguistic pattern occurs in different portions of the input, the network must learn that pattern multiple times from scratch:

- In computer vision, a dog is a dog regardless of whether it appears on the left or the right side of an image. An FCNN must learn a separate "dog-detector" for the left pixels and the right pixels.
- In NLP, the noun phrase *"a dog"* possesses the exact same grammatical and semantic structure whether it appears at token position 1 (*"A dog barked loudly"*) or token position 10 (*"After a long walk in the park, a dog barked"*).

Because an FCNN does not share weights across sequence positions, it is wildly inefficient and requires massive datasets to observe every linguistic pattern at every conceivable spatial index.

3.1.1 Inductive biases and the twin physical symmetries: CNNs vs. RNNs

As articulated in modern deep learning theory [23, 26], unconstrained multi-layer perceptrons possess minimal *inductive bias*: they treat every input dimension as an arbitrary coordinate, ignoring any topological or temporal relationships between adjacent dimensions. To scale deep learning to complex domains, architectures must incorporate inductive biases that exploit the intrinsic **physical symmetries** of the underlying data:

1. Spatial translation equivariance in Convolutional Neural Networks (CNNs). In spatial domains (such as 2D images), the data-generating distribution is characterized by two fundamental symmetries:

- **Locality**: Pixels that are spatially adjacent exhibit strong mutual statistical correlations, whereas pixels separated by large Euclidean distances are approximately independent.
- **Translation Invariance / Equivariance**: Shifting an input feature map $\mathbf{x}$ by spatial displacement vector $\mathbf{g}$ should produce a correspondingly shifted feature representation $f(g(\mathbf{x})) = g(f(\mathbf{x}))$.

Imposing locality and translation equivariance forces the dense weight matrix $\mathbf{W} \in \mathbb{R}^{D_{\text{out}} \times D_{\text{in}}}$ to collapse into a localized discrete convolution operator $\mathbf{K} \in \mathbb{R}^{k \times k}$ ($k \ll \sqrt{D_{\text{in}}}$). By sliding this compact kernel across all spatial coordinates, the parameter footprint is reduced by several orders of magnitude, enabling sample-efficient visual recognition.

2. Temporal stationarity and parameter sharing in Recurrent Neural Networks (RNNs). In temporal and linguistic domains, the governing symmetry is **temporal stationarity**:

$$\mathbb{P}(X_t \mid X_{t-1}, \dots, X_1) \approx \mathcal{T}(X_t, X_{t-1}, \dots; \Theta), \tag{3.2}$$

meaning that the fundamental grammatical rules, syntactic patterns, and conditional transition probabilities of a language do not alter their mathematical structure simply because an utterance takes place at token index $t = 1$ versus index $t = 50$.

While an FCNN requires separate, independent weight matrices $\mathbf{W}_t \in \mathbb{R}^{d \times d}$ for each position $t \in \{1, \dots, T\}$—causing parameter count to scale as $O(T \cdot d^2)$—a recurrent architecture models sequence progression through a stationary state-transition mapping $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \Theta)$. By sharing the parameter set $\Theta = \{\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h\}$ across all time steps, the parameter footprint remains completely invariant to sequence length: $O(d_h^2 + d_h d_{\text{in}})$.

3. 1D Temporal Convolutions (TCNs) vs. Recurrence. Simon Prince [21] highlights an alternative formulation: 1D Temporal Convolutions (e.g., WaveNet, Temporal Convolutional Networks / TCNs). Instead of maintaining an evolving recurrent state $\mathbf{h}_t$, a 1D TCN applies causal dilated 1D convolutions over time. While 1D CNNs achieve $O(1)$ sequential parallelization during training, their effective memory horizon is strictly bounded by their receptive field, scaling exponentially with network depth as $O(K^L)$. In contrast, recurrent networks maintain a theoretically unbounded memory horizon within a single compact state $\mathbf{h}_t$, albeit at the expense of strictly sequential training.

Table 3.1 formalizes the architectural trade-offs across the four foundational deep learning paradigms.

Rigid output dimensionality constraints.
The output dimensionality is rigidly determined by the number of rows in the final projection matrix $\mathbf{W}_{\text{out}}$. In many sequential tasks (such as machine translation, text summarization, and dialogue generation), the output sequence length is unknown a priori and varies dynamically based on the input:

English (5 words): *"It is indeed sunny today"* $\implies$ Italian (4 words): *"Effettivamente oggi è soleggiato"*.

An FCNN cannot dynamically adjust its output layer to emit variable-length token sequences.

Table 3.1: Architectural comparison across deep learning paradigms: Inductive biases, computational complexity, and sequence handling.

Dimension	Feedforward (FCNN)	1D Temporal CNN (TCN)	Recurrent (RNN/LSTM)	Transformer
Input Elasticity	Rigid ($T_{\max}$ fixed)	Elastic (up to receptive field)	Elastic (unbounded variable length)	Elastic (bounded by context window $L_{\max}$)
Parameter Sharing	None (position-specific)	1D sliding causal kernel	Replay over time (temporal recurrence)	Shared projection weights across tokens
Training Latency	$O(1)$ parallel matrix pass	$O(1)$ parallel causal convolutions	$O(T)$ strictly sequential	$O(1)$ parallel self-attention
Receptive Field	Memoryless (fixed window)	Bounded by depth $O(K^L)$	Theoretically unbounded ($\mathbf{h}_t$)	Direct $O(1)$ all-to-all token routing
Inference Cost / Step	Re-evaluates entire window	Causal FIFO buffer rolling	$O(1)$ update step $f(\mathbf{h}_{t-1}, \mathbf{x}_t)$	KV Cache lookup $O(T)$ operations

3.2 The recurrent neural network cell and weight sharing

To overcome the constraints of FCNNs, **Recurrent Neural Networks (RNNs)**—first formalized as the Simple Recurrent Network in [27]—process sequential data element by element, maintaining an internal continuous *hidden state* $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as a recurrent memory summarizing the sequence history observed up to time step t.

3.2.1 Recurrent cell architecture

At each discrete time step $t \in \{1, 2, \ldots, T\}$, the recurrent cell is characterized by two inputs and two outputs:

- **Input 1**: The current sequence element $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ (e.g., a one-hot vector or continuous embedding).
- **Input 2**: The hidden state produced in the previous time step $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$.
- **Output 1**: The model's prediction for the current time step $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_{\text{out}}}$.
- **Output 2**: The updated hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$, which is recursively forwarded to time step $t+1$.

3.2.2 Mathematical formulation with explicit tensor dimensions

Internally, the vanilla RNN cell comprises an input projection layer, a recurrent state transition layer, and an output projection layer:

$$\underbrace{\mathbf{h}_t}_{[d_h \times 1]} = \tanh\left(\underbrace{\mathbf{W}_{hh}}_{[d_h \times d_h]}\underbrace{\mathbf{h}_{t-1}}_{[d_h \times 1]} + \underbrace{\mathbf{W}_{xh}}_{[d_h \times d_{\text{in}}]}\underbrace{\mathbf{x}_t}_{[d_{\text{in}} \times 1]} + \underbrace{\mathbf{b}_h}_{[d_h \times 1]}\right), \tag{3.3}$$

$$\underbrace{\hat{\mathbf{y}}_t}_{[V \times 1]} = \text{softmax}\left(\underbrace{\mathbf{W}_{hy}}_{[V \times d_h]}\underbrace{\mathbf{h}_t}_{[d_h \times 1]} + \underbrace{\mathbf{b}_y}_{[V \times 1]}\right). \tag{3.4}$$

3.2.3 Weight sharing across time

The defining feature of RNNs is **temporal weight sharing**. The parameter matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are *strictly identical* across all time steps $t \in \{1, \ldots, T\}$. Because the same weights are applied regardless of time step, the model develops **translation invariance**: if the same context occurs at step $t = 2$ or step $t = 50$, the recurrent cell behaves identically. The network learns grammatical rules (such as capitalizing the first letter after a quotation mark) once, and applies that rule everywhere in the text.

Recurrent dynamics as a trajectory through high-dimensional phase space.
As visualized in geometric visual analyses [28], the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ can be understood geometrically as the position of a particle traveling through a continuous d_h-dimensional phase space:

- At rest ($t = 0$), the system sits at initial coordinate $\mathbf{h}_{-1} = \mathbf{0}$.
- When each discrete input token $\mathbf{x}_t$ arrives, the linear projection $\mathbf{W}_{xh}\mathbf{x}_t$ applies an external directional impulse or "vector nudge" to the particle.

- Simultaneously, the recurrent transformation $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ rotates, scales, and folds the previous position, while the non-linear squashing activation tanh confines the entire state space within the hypercube $(-1, 1)^{d_h}$.
- The unfolding sequence of text traces a smooth continuous trajectory through state space. Regions of this phase space function as **attractor basins**: for instance, reading an opening quote mark (") knocks the state into a specific sub-manifold where the emission projection $\mathbf{W}_{hy}$ naturally maps coordinates into uppercase character distributions. The state remains trapped in this basin until an exit trigger (such as a whitespace or subsequent character) forces the trajectory out into the lowercase manifold.

3.2.4 Unrolling the recurrent cell across time

To trace how an RNN processes a sequence, we *unroll* the recursive feedback loop across discrete time steps $t = 0, 1, \ldots, T$.

Consider the character casing task:

- Input sequence: `"you are very kind, mr.  holmes, but i cannot do that"`
- Target sequence: `"You are very kind, Mr.  Holmes, but I cannot do that"`

Each ASCII character is represented as a 256-dimensional one-hot vector $\mathbf{x}_t \in \{0, 1\}^{256}$:

- **Step** $t = 0$: Receives input $s_0 =$ `"` with initial state $\mathbf{h}_{-1} = \mathbf{0}$. The cell produces state $\mathbf{h}_0$. The network realizes: *"We just encountered an opening quote ("), which signals the beginning of dialogue; the following character should be capitalized."*
- **Step** $t = 1$: Receives input $s_1 =$ `y` and prior state $\mathbf{h}_0$. The network emits output $\hat{y}_1 =$ `Y` and produces updated state $\mathbf{h}_1$. The state records: *"We just emitted a capitalized letter at the start of dialogue; following letters in this word should probably be lowercase."*
- **Step** $t = 2$: Receives input $s_2 =$ `o` and prior state $\mathbf{h}_1$. The network emits output $\hat{y}_2 =$ `o` and produces state $\mathbf{h}_2$, continuing lowercase word progression.

Figure 3.1 illustrates this unrolled computational structure and the backward flow of gradients.

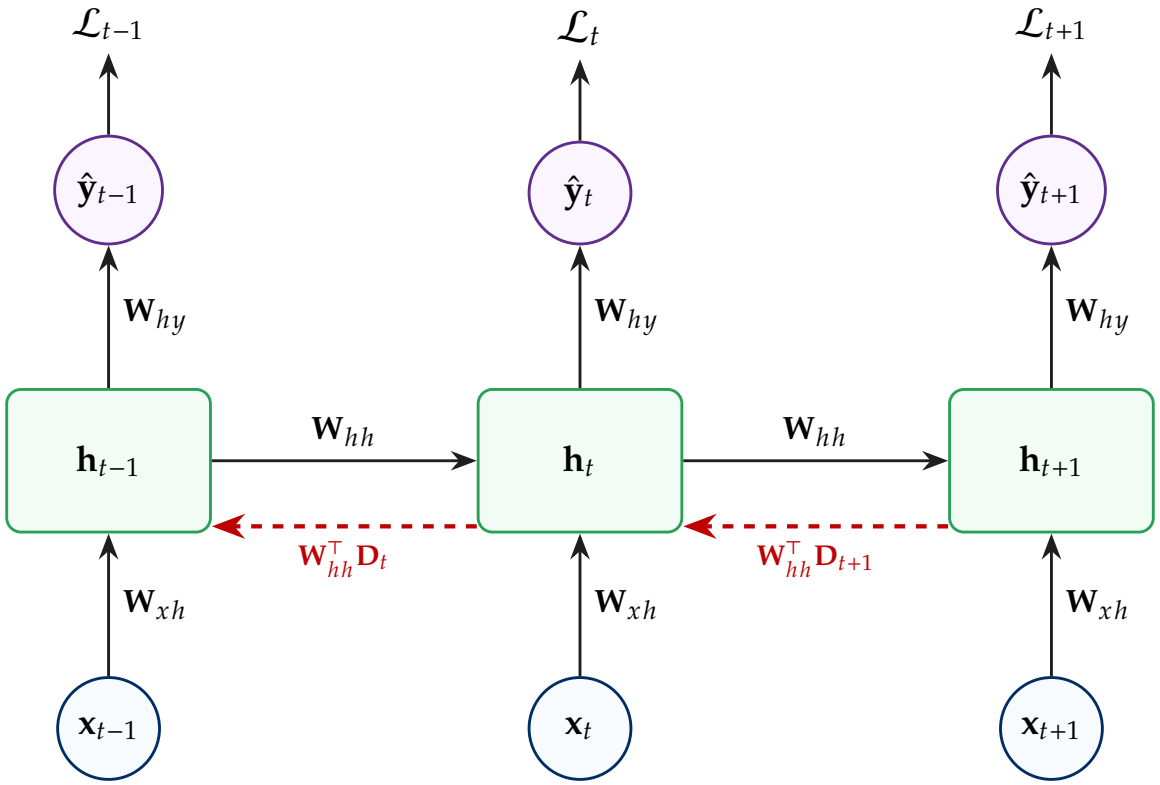


Figure 3.1: Unrolling an RNN cell across time steps $t-1, t, t+1$. Solid black arrows show the forward propagation of inputs $\mathbf{x}_t$ and recurrent states $\mathbf{h}_t$. Dashed red arrows trace Backpropagation Through Time (BPTT), where gradients accumulate across time via repeated multiplications by the Jacobian $\mathbf{W}_{hh}^\top \mathbf{D}_t$.

3.3 Backpropagation through time and the proof of vanishing gradients

When an RNN processes a sequence of length T, the computational graph is unrolled T times. The total sequence loss is the sum of cross-entropy losses incurred at each time step:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t. \tag{3.5}$$

To optimize the shared recurrent weight matrix $\mathbf{W}_{hh}$ via Gradient Descent, we apply **Backpropagation Through Time (BPTT)**, first formalized for recurrent structures in [29]. The fundamental difficulty of learning long-term dependencies through gradient descent was rigorously analyzed in [30] and [31]. The gradient of the total loss with respect to $\mathbf{W}_{hh}$ accumulates the contributions across all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{3.6}$$

where $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denotes the direct derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$ treating $\mathbf{h}_{k-1}$ as constant.

The temporal transmission term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the sensitivity of state $\mathbf{h}_t$ to an earlier state $\mathbf{h}_k$ ($t > k$). By the multivariate chain rule, this sensitivity decomposes into a product of single-step Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{3.7}$$

3.3.1 Rigorous mathematical proof of gradient pathology

We now derive the explicit single-step Jacobian and prove why vanilla RNNs suffer from vanishing and exploding gradients.

Theorem 3.1: *Jacobian product and spectral radius gradient decay*

Let the recurrent state transition be $\mathbf{h}_j = \tanh(\mathbf{z}_j)$ where $\mathbf{z}_j = \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h$ with $\mathbf{h}_j \in \mathbb{R}^{d_h}$ and $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$. Under standard column-vector convention, the single-step forward temporal Jacobian $\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \in \mathbb{R}^{d_h \times d_h}$ is:

$$\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh} = \mathbf{D}_j \mathbf{W}_{hh}, \tag{3.8}$$

where $\mathbf{D}_j = \operatorname{diag}(1 - \tanh^2(\mathbf{z}_j)) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the local derivatives of the hyperbolic tangent function. Consequently, the multi-step temporal transmission evaluates to:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}. \tag{3.9}$$

When transmitting the gradient of a scalar loss $\mathcal{L}_t$ backward through time, the column-vector gradient $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t \in \mathbb{R}^{d_h}$ satisfies the adjoint recurrence $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t = \mathbf{W}_{hh}^\top \mathbf{D}_j \nabla_{\mathbf{h}_j} \mathcal{L}_t$. Let $\lambda_{\max} = \|\mathbf{W}_{hh}\|_2 = \sigma_{\max}(\mathbf{W}_{hh})$ denote the spectral norm (maximum singular value) of $\mathbf{W}_{hh}$, and let $\rho(\mathbf{W}_{hh}) = \max_i |\lambda_i|$ denote its spectral radius. The Euclidean norm of the transmitted gradient is bounded by:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| (\lambda_{\max})^{t-k}. \tag{3.10}$$

Proof. Differentiating the i-th component of $\mathbf{h}_j$ with respect to the m-th component of $\mathbf{h}_{j-1}$:

$$\frac{\partial h_{j,i}}{\partial h_{j-1,m}} = \frac{\partial \tanh(z_{j,i})}{\partial z_{j,i}} \frac{\partial z_{j,i}}{\partial h_{j-1,m}} = (1 - h_{j,i}^2) W_{hh,i,m}.$$

In matrix notation, the entry in row i and column m of $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ is $(1 - h_{j,i}^2) W_{hh,i,m} = (\mathbf{D}_j)_{ii} W_{hh,i,m}$, which corresponds exactly to the matrix product $\mathbf{D}_j \mathbf{W}_{hh}$. Taking the transpose for backward column-vector gradient propagation yields $(\mathbf{D}_j \mathbf{W}_{hh})^\top = \mathbf{W}_{hh}^\top \mathbf{D}_j$, matching the backward adjoint arrows in Figure 3.1. Because $|\tanh'(z)| = 1 - \tanh^2(z) \in [0, 1]$, the operator 2-norm of each diagonal matrix satisfies $\|\mathbf{D}_j\|_2 \leq 1$. Applying the sub-multiplicative property of matrix norms to Equation (3.9):

$$\begin{aligned} \|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}\|_2 = \| \prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh} \|_2 &\leq \prod_{j=k+1}^{t} \|\mathbf{D}_j\|_2 \|\mathbf{W}_{hh}\|_2 \\ &\leq \prod_{j=k+1}^{t} 1 \cdot \lambda_{\max} = (\lambda_{\max})^{t-k}. \end{aligned}$$

Multiplying by $\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\|_2$ yields Equation (3.10). □

3.3.2 Two pathological regimes: vanishing and exploding gradients

As proved by Pascanu et al. [31], the asymptotic behavior of the temporal Jacobian product $\prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}$ bifurcates into two destructive regimes dictated by the spectrum of $\mathbf{W}_{hh}$:

1. **Vanishing Gradient ($\lambda_{\max} < 1$ or $\rho(\mathbf{W}_{hh}) < 1$)**: If the largest singular value or spectral radius of $\mathbf{W}_{hh}$ is strictly less than 1 (e.g., $\lambda_{\max} = 0.9$), over a temporal distance of 50 steps:

   $$0.9^{50} \approx 0.00515 \quad \text{(over 99.5\% of the gradient signal is extinguished).}$$

   Over 100 steps, $0.9^{100} \approx 0.000026$. By Gelfand's formula ($\lim_{n\to\infty} \|\mathbf{W}^n\|^{1/n} = \rho(\mathbf{W})$), the gradient decays exponentially to zero. Parameter updates from distal tokens vanish completely, meaning the network cannot learn long-range temporal dependencies. Saturated activations ($\tanh'(z) \to 0$) accelerate this decay exponentially.

2. **Exploding Gradient ($\lambda_{\max} > 1$ or $\rho(\mathbf{W}_{hh}) > 1$)**: If the singular values exceed 1 (e.g., $\lambda_{\max} = 1.1$), over 50 steps:

   $$1.1^{50} \approx 117.39.$$

   Over 100 steps, $1.1^{100} \approx 13{,}780$. Gradients grow exponentially along the dominant eigenvectors, destabilizing training, inducing severe numerical overflow, and producing `NaN` parameter corruptions.

Global norm gradient clipping. To combat exploding gradients, Pascanu et al. [31] introduced the technique of *global norm clipping*. Instead of clipping individual scalar gradient elements independently (which alters the geometric direction of the descent vector), global norm clipping rescales the collective parameter gradient vector $\mathbf{g} = \nabla_\Theta \mathcal{L}$ whenever its total Euclidean norm exceeds a predefined threshold $\gamma > 0$:

$$\mathbf{g} \leftarrow \min\left(1, \frac{\gamma}{\|\mathbf{g}\|_2}\right) \mathbf{g}. \tag{3.11}$$

Geometrically, Equation (3.11) leaves the gradient direction strictly unaltered ($\frac{\mathbf{g}}{\|\mathbf{g}\|}$ is preserved) while projecting the step magnitude onto the hypersphere of radius γ. This prevents gradient descent from taking catastrophic, destructive leaps across steep loss ravines and cliff-like bifurcations.

The long-term dependency degradation. Because hidden states are rewritten at every time step without persistent memory protection, vanilla RNNs suffer from catastrophic recency bias: they remember only the most recent tokens. When tested on long-range word prediction:

*"The **mouse** was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the _____"*

the target word is unambiguously *"mouse"*. However, vanilla RNNs fail completely because the hidden state has overwritten the subject *"mouse"* across the intervening 20 tokens.

3.4 Case correction demonstration: Sherlock Holmes training dynamics

To demonstrate how an RNN learns sequential syntax in practice, we examine an empirical case study trained on Arthur Conan Doyle's *The Adventures of Sherlock Holmes*:

Experimental configuration.

- **Corpus**: Raw text from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*.
- **Sequence Length**: Subsequences sliced between 10 and 100 ASCII characters.
- **Token Representation**: 256-dimensional one-hot vectors ($d_{\text{in}} = 256$).
- **Architecture**: Single-layer vanilla RNN with hidden dimension $d_h = 32$.
- **Optimization**: Adam optimizer (lr = 0.001), batch size 32, trained for 10 epochs.
- **Padding Mechanism**: Null bytes (`0x00`, rendered visually as `X`) used to pad variable-length sequences to 100 characters.

Fixed evaluation test sentence. The model is evaluated after each epoch on a fixed unseen test string:

- **Input Test Sequence**: `upon his pale face.  "it may be so, or it may not, mr. holmes," said he, "but if you are so very sharp`
- **Target Test Sequence**: `upon his pale face.  "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp`

Qualitative phase transitions across 10 epochs. Table 3.2 records the loss trajectory and qualitative phase transitions across training:

3.4.1 Character-level modeling, temperature sampling, and hidden state interpretability

The Sherlock Holmes casing experiment exemplifies a broader paradigm analyzed in the landmark study *"The Unreasonable Effectiveness of Recurrent Neural Networks"* [32]: **character-level sequence modeling**.

The elegance of character-level modeling.
Unlike word-level models that require complex tokenizers, morphological analyzers, and massive vocabularies ($V \geq 50{,}000$), character-level RNNs operate over an extremely compact alphabet (e.g., standard 256-dimensional ASCII).

- There are **zero Out-of-Vocabulary (OOV) tokens**: any arbitrary string, code snippet, mathematical formula, or neologism can be represented as an sequence of bytes.
- The model is forced to discover the hierarchical structure of language entirely from scratch: it must learn how characters form words, that words are separated by spaces, how punctuation delimits clauses, and how grammatical agreement spans across clauses.

Table 3.2: Epoch-by-epoch loss progression and qualitative phase transitions on the Sherlock Holmes casing task.

Epoch	Loss	Observed Generation & Qualitative Linguistic Transition
01	3.1681	`XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 1 (Null-byte frequency)*: The model discovers that null padding bytes (`0x00` / `X`) appear with high statistical frequency due to padding imbalance, predicting all `X`'s.
02	2.3334	`eXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 2 (Vowel emergence)*: The model predicts `e` (the most frequent character in English orthography) alongside padding characters.
03	1.7436	`tst X s te e eee t e e e te se ee se e e ee eee XXXee` *Phase 3 (Whitespace and unigrams)*: The model learns that whitespace characters occur regularly, predicting spaces, `e`, `t`, and `s`.
04	0.9519	`upon his pale face.  cit may be sos or it may nots mr.  holresst` *Phase 4 (The Identity Breakthrough)*: A massive drop in loss occurs when the network learns to forward lowercase letters directly from input to output (`W.grad` captures the identity diagonal).
05–06	0.3579	`upon his pale face.  "it may be so, or it may not, mr.  holmes,"` *Phase 5 (Punctuation and Syntax)*: The model captures quotation marks (`"`), commas, and spaces accurately.
07–08	0.1009	`upon his pale face.  "It may be so, or it may not, mr.  holmes,"` *Phase 6 (Quote Capitalization)*: The model learns that characters immediately following opening quotes (`"`) must be capitalized (`"It`).
09–10	0.0515	`upon his pale face.  "It may be so, or it may not, mr.  Holmes,"` *Phase 7 (Period Capitalization and Causal Blindness)*: The model capitalizes `Holmes` because it follows a period (`.`). Crucially, the model **fails to capitalize** `mr.` because it cannot see into the future! It does not know that `m` begins the title `Mr.` until the `r.` arrives.

Autoregressive sampling and temperature dynamics.
During text generation, the RNN emits an unnormalized logit vector $\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^V$. Rather than taking the deterministic arg max, we convert logits into a sampling distribution modulated by a scalar hyperparameter: the **temperature** $T > 0$:

$$\mathbb{P}(y_t = c \mid \mathbf{h}_t) = \frac{\exp\left(\frac{z_{t,c}}{T}\right)}{\sum_{j=1}^{V} \exp\left(\frac{z_{t,j}}{T}\right)}. \tag{3.12}$$

The temperature controls the entropy and randomness of the emitted sequence:

- **Low Temperature ($T \to 0$ / Greedy Decoding)**: Exaggerates differences between logits. The largest logit absorbs almost all probability mass ($p_{\max} \to 1$). The model makes confident, conservative predictions, but frequently falls into pathological mode collapse—getting trapped in infinite repetitive loops (e.g., `"the the the the"` or repeating identical sentence fragments).
- **Balanced Temperature ($T \in [0.6, 0.8]$)**: Preserves the syntactic and morphological structure discovered by the network while introducing sufficient stochastic diversity to produce coherent, rich generations. In empirical experiments demonstrated in [32], character-level LSTMs trained at $T = 0.7$ generated structurally sound Shakespearean plays (including character names, stage directions, and poetic meter), syntactically compilable C code from the Linux kernel, and valid algebraic geometry LaTeX documents.
- **High Temperature ($T > 1.2$)**: Flattens the probability distribution toward a uniform distribution ($p_c \to 1/V$). The network makes chaotic, exploratory predictions, generating misspelled words, hallucinated punctuation, and unpronounceable character noise.

Interpretable hidden state neurons.
By inspecting the internal cell activations $\mathbf{h}_t$ of a trained LSTM as it reads character sequences, the empirical investigation in [32] revealed that individual hidden units act as interpretable linguistic finite-state machines:

1. **The Quote Detection Neuron**: A single scalar unit in $\mathbf{h}_t$ that fires strongly positive (+1) the moment an opening quotation mark (") is parsed, remains persistently active throughout the enclosed dialogue, and resets immediately to −1 when the matching closing quote mark is encountered.
2. **The Code Indentation / Scope Depth Neuron**: In code generation models, a specific neuron's activation ramps up linearly with every tab or whitespace indentation inside nested loops or conditionals, tracking variable scope depth and resetting when closing brackets (}) appear.
3. **The Line Length Tracker**: A unit whose activation monotonically increases across 80 characters, anticipating when to emit the carriage return (`\n`) character to conform to standard formatting conventions.

Causal lookahead blindness and overfitting dynamics.
In practical character sequence modeling, two essential takeaways emerge from the Sherlock Holmes casing demonstration:

- **Causal Lookahead Blindness**: Why did the one-to-one RNN fail to capitalize `"mr."` into `"Mr."` even after 10 epochs? The model operates strictly causally from left to right. When processing the character `'m'`, the model only sees past history. It cannot see into the future to know whether `'m'` begins the proper title `"Mr."` or common words like `"me"`, `"my"`, or

`"much"`. Without lookahead, capitalizing `'m'` would be an unsupported guess. Conversely, the model successfully capitalizes `"Holmes"` because it has already observed the preceding period and space (`". h"`), triggering capitalization.

- **Overfitting vs. Generalization Dynamics**: Extending training for hundreds of epochs on a finite corpus (such as Doyle's novel) causes the model to memorize specific character n-grams verbatim. While training loss continues to drop toward zero, the network loses its ability to generalize to novel sentence structures, reinforcing the necessity of early stopping and validation monitoring.

3.5 Gated architectures: LSTM and GRU

To solve the vanishing gradient problem and allow neural networks to retain dependencies over hundreds of time steps, researchers introduced **Gated RNNs**:

3.5.1 Long short-term memory (LSTM)

Introduced in [33], the **Long Short-Term Memory (LSTM)** network partitions memory into two separate states:

1. **Cell State $\mathbf{c}_t \in \mathbb{R}^{d_h}$**: The *long-term memory* highway, which runs straight down the entire chain with minimal linear interactions.
2. **Hidden State $\mathbf{h}_t \in \mathbb{R}^{d_h}$**: The *short-term memory*, which acts as the visible output emitted at each step.

Definition 3.1: *LSTM gating equations*

At time step t, given input $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, the LSTM evaluates three sigmoid gates and a candidate state:

$$\text{Forget Gate:} \quad \mathbf{f}_t = \sigma\left(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right) \in (0,1)^{d_h}, \tag{3.13}$$

$$\text{Input Gate:} \quad \mathbf{i}_t = \sigma\left(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right) \in (0,1)^{d_h}, \tag{3.14}$$

$$\text{Candidate Cell:} \quad \tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{3.15}$$

$$\text{Cell State Update:} \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}, \tag{3.16}$$

$$\text{Output Gate:} \quad \mathbf{o}_t = \sigma\left(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right) \in (0,1)^{d_h}, \tag{3.17}$$

$$\text{Hidden State Update:} \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \in (-1,1)^{d_h}, \tag{3.18}$$

where $\odot$ denotes the element-wise Hadamard product, and $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_{\text{in}}}$ denotes vector concatenation.

Semantic roles of the gates. Each gate squashes pre-activations through a sigmoid function $\sigma \in [0,1]$, functioning as a continuous valve:

- Multiplying by $0 \implies$ drop/forget everything.
- Multiplying by $1 \implies$ preserve/pass everything.
- **Forget Gate ($\mathbf{f}_t$)**: Regulates what proportion of the past cell state $\mathbf{c}_{t-1}$ to discard. (e.g., when a closing quote or period arrives, the forget gate purges past sentence subjects).
- **Input Gate ($\mathbf{i}_t$)**: Regulates what proportion of the new candidate information $\tilde{\mathbf{c}}_t$ to write into the long-term cell state.
- **Output Gate ($\mathbf{o}_t$)**: Regulates which portions of the internal cell state $\mathbf{c}_t$ should be revealed as visible hidden state $\mathbf{h}_t$ for current output predictions.

Theorem 3.2: *The Constant Error Carousel proof*

In an LSTM, the gradient of the loss with respect to past cell state $\mathbf{c}_{t-1}$ contains an additive

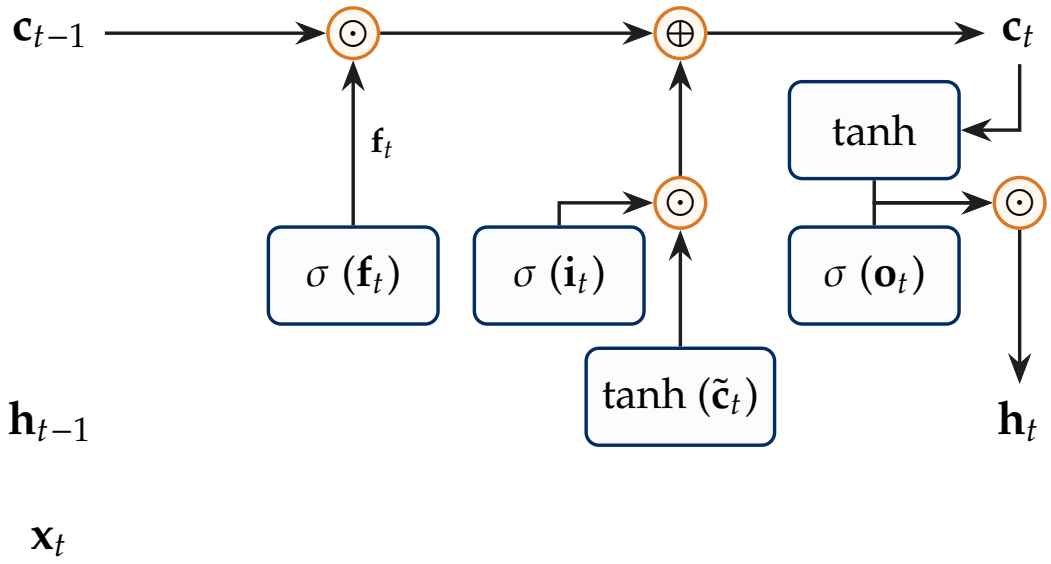


Figure 3.2: The LSTM memory cell architecture. The top horizontal path forms the uninterrupted additive cell state highway $\mathbf{c}_t$ (the Constant Error Carousel). Forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ modulate information flow using multiplicative valves.

direct path:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}}\mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}}\tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}\mathbf{i}_t. \tag{3.19}$$

When the network sets forget gate $\mathbf{f}_t \approx \mathbf{1}$, the leading derivative collapses to $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$. Consequently, over any arbitrary temporal horizon $T - k$:

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{T} \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \prod_{j=k+1}^{T} \mathbf{I} = \mathbf{I}. \tag{3.20}$$

Gradients flow backward across hundreds of time steps without exponential decay. This linear additive gradient highway is the **Constant Error Carousel (CEC)**.

The conveyor belt intuition: additive persistence of memory.
To build physical intuition for the Constant Error Carousel, think of the cell state $\mathbf{c}_t$ as a continuous factory conveyor belt running from the start of the sequence to the end. The standard recurrent transition in a vanilla RNN continually picks up the items, squashes them with weights and tanh, and puts a degraded version back on the belt. Over 50 time steps, the original items are unrecognizable.

In an LSTM, items on the conveyor belt are untouched by default. As the belt passes each station:

1. The forget gate $\mathbf{f}_t$ acts as a discard mechanism: if $\mathbf{f}_t \approx 1$, the past item rides past undisturbed; if $\mathbf{f}_t \approx 0$, it is swept off the belt.
2. The input gate $\mathbf{i}_t$ adds new relevant cargo $\tilde{\mathbf{c}}_t$ onto the belt via simple vector addition ($\oplus$).
3. The output gate $\mathbf{o}_t$ inspects the cargo on the belt, reads off a copy, and displays it to the outside world as hidden state $\mathbf{h}_t$, leaving the underlying cell state intact on the highway.

Because writing is additive rather than multiplicative, information can ride the conveyor belt across hundreds of words without experiencing exponential decay.

3.5.2 Gated recurrent unit (GRU)

Introduced in [34], the **Gated Recurrent Unit (GRU)** simplifies the LSTM architecture by merging the cell state and hidden state into a single recurrent state $\mathbf{h}_t$ and coupling the forget and input gates into a single *update gate* $\mathbf{z}_t$:

Definition 3.2: *GRU equations*

At time step t, given input $\mathbf{x}_t$ and prior state $\mathbf{h}_{t-1}$:

$$\text{Update Gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z\right), \tag{3.21}$$

$$\text{Reset Gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r\right), \tag{3.22}$$

$$\text{Candidate Hidden State:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h\right), \tag{3.23}$$

$$\text{Hidden State Interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{3.24}$$

Physical gate routing and dynamic memory modulation. The GRU cell operates through two coupled mechanisms illustrated in Figure 3.3:

1. **Reset Gate Modulation ($\mathbf{r}_t$):** The reset gate vector $\mathbf{r}_t \in (0, 1)^{d_h}$ acts as a dynamic filter applied to the previous hidden state $\mathbf{h}_{t-1}$ via Hadamard multiplication ($\mathbf{r}_t \odot \mathbf{h}_{t-1}$). If $\mathbf{r}_t \approx \mathbf{0}$, the candidate state $\tilde{\mathbf{h}}_t$ completely ignores past memory, resetting the representation to depend exclusively on the current token $\mathbf{x}_t$ (e.g., at sentence boundaries or clause resets). If $\mathbf{r}_t \approx \mathbf{1}$, the full history flows into the candidate calculation.

2. **Update Gate Convex Interpolation ($\mathbf{z}_t$):** Rather than using distinct input and forget gates like the LSTM, the GRU couples memory retention and candidate addition into a single convex combination:
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$
When $\mathbf{z}_t \approx \mathbf{0}$, the unit passes the previous state $\mathbf{h}_{t-1}$ forward unchanged ($1 \odot \mathbf{h}_{t-1} + 0 \odot \tilde{\mathbf{h}}_t = \mathbf{h}_{t-1}$), establishing an unobstructed linear identity shortcut similar to the LSTM Constant Error Carousel. When $\mathbf{z}_t \approx \mathbf{1}$, the unit overwrites the state entirely with the newly proposed candidate $\tilde{\mathbf{h}}_t$.

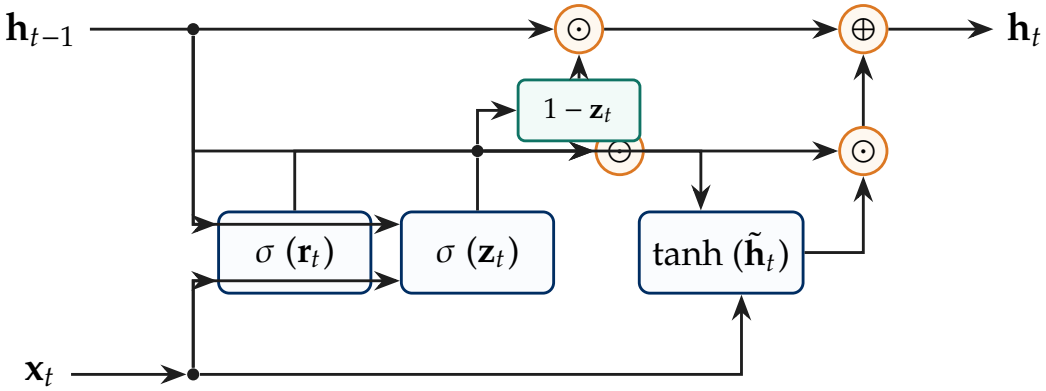


Figure 3.3: The Gated Recurrent Unit (GRU) cell architecture. Reset gate $\mathbf{r}_t$ dynamically filters previous hidden state $\mathbf{h}_{t-1}$ before constructing candidate state $\tilde{\mathbf{h}}_t$. Update gate $\mathbf{z}_t$ controls the convex interpolation $(1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$, unifying long- and short-term memory into a single state vector $\mathbf{h}_t$.

Parameter count comparison: vanilla RNN vs. LSTM vs. GRU. Comparing the exact parameter counts of recurrent cells for hidden dimension d and input dimension m clarifies the architectural trade-offs across sequence models:

- **Vanilla RNN**: Contains 1 transition set $(\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h) \implies d^2 + dm + d = d(d + m + 1)$.
- **GRU**: Contains 3 gate sets $(\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t) \implies 3 \times d(d + m + 1)$.
- **LSTM**: Contains 4 gate sets $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t, \mathbf{o}_t) \implies 4 \times d(d + m + 1)$.

The GRU reduces parameters by 25% compared to the LSTM while exhibiting comparable empirical performance across diverse sequence modeling benchmarks.

3.6 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines

3.6.1 Limitations of one-to-one sequence mapping

Thus far, we have considered models where each input step produces exactly one output step ($|X| = |Y|$), such as character-level case correction. However, the majority of core NLP tasks (machine translation, question answering, code generation) violate this assumption:

1. **Unequal Sequence Lengths**: In machine translation, source and target languages rarely match in word count:

   English (5 words): *[it] [is] [indeed] [sunny] [today]* $\implies$ Italian (4 words): *[effettivamente] [oggi] [è] [soleggiato]*.

   A one-to-one recurrent model would process 5 tokens and be forced to produce 5 output tokens.

2. **Non-Monotonic Alignment and Causal Blindness**: Natural languages exhibit different word orders. In the example above, the Italian word *"effettivamente"* must be emitted at step 1, but its English counterpart *"indeed"* does not appear until step 3! In a standard one-to-one model, the cell cannot see the future, rendering early word generation impossible.

3.6.2 The encoder-decoder architecture

To overcome these limitations, the **Encoder-Decoder (Seq2Seq)** architecture was introduced in [35] and [34]:

1. **Encoder**: An LSTM processes the entire variable-length source sequence $(x_1, \ldots, x_N)$ step by step, updating its internal state. The final hidden state $\mathbf{h}_N$ acts as a dense, compressed summary vector encoding the semantics of the whole input sentence.

2. **Decoder**: A separate LSTM initialized with state $\mathbf{h}_N$. The decoder autoregressively emits target tokens step by step. The initial input token is the special Beginning of Sequence symbol $\langle$BOS$\rangle$. At each subsequent step t, the input is the token emitted at step $t-1$. Generation continues until the model emits the special End of Sequence symbol $\langle$EOS$\rangle$.

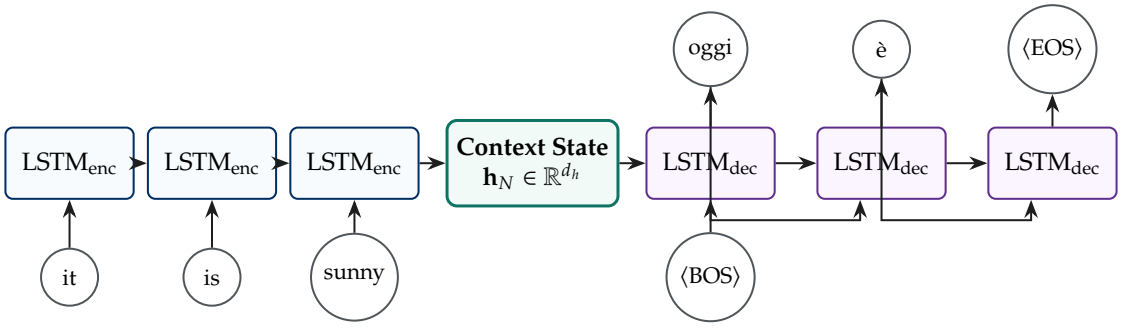


Figure 3.4: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder LSTM compresses the variable-length source sentence into a single context state $\mathbf{h}_N$. The Decoder LSTM is initialized with $\mathbf{h}_N$ and autoregressively emits target tokens from $\langle$BOS$\rangle$ until emitting $\langle$EOS$\rangle$.

3.6.3 Teacher forcing vs. inference exposure bias

During training, the decoder's predicted token $\hat{y}_{t-1}$ is often incorrect in early epochs. If the model feeds its own faulty prediction into step t, errors compound catastrophically down the sequence.

To stabilize and accelerate training, practitioners use **Teacher Forcing**:

$$\mathbf{x}_{\text{dec},t} = y^*_{t-1} \quad \text{(pass true ground-truth target token instead of predicted } \hat{y}_{t-1}\text{)}. \tag{3.25}$$

Exposure Bias Warning: While Teacher Forcing provides rapid convergence during training, it creates a discrepancy at inference time when ground-truth tokens are unavailable, causing models to be fragile when recovering from early generation mistakes.

3.6.4 Sherlock Holmes case correction revised: one-to-one vs. encoder-decoder

To empirically contrast the representational power of one-to-one recurrent mapping against the Sequence-to-Sequence Encoder-Decoder architecture, we evaluate both models on an identical, challenging character casing sentence:

Input Test Sequence: `"indeed i am sherlock holmes,"`
Target Test Sequence: `"Indeed I am Sherlock Holmes,"`

Theoretical expectations from architectural constraints. This evaluation sentence contains four distinct capitalization targets, each testing a specific causal and temporal capacity:

1. **`"I"` of `"Indeed"` (beginning of quote)**: Should be capitalized because it is immediately preceded by an opening quotation mark (`"`). A one-to-one model can easily learn this local transition, as the trigger character is already present in its past recurrent state $\mathbf{h}_{t-1}$.

2. **`"I"` of `"I"` (standalone subject pronoun)**: Must be capitalized because it functions as the first-person singular pronoun. A one-to-one model is structurally blind here: when reading `'i'`, the network only knows that a word is beginning with `'i'`. It cannot determine whether this character is an isolated pronoun or the initial letter of common words like `"in"`, `"it"`, or `"into"` until the subsequent whitespace arrives at step $t+1$. Without future lookahead, capitalizing `'i'` at step t is mathematically unsupported.

3. **`"S"` of `"Sherlock"` (proper noun first name)**: Must be capitalized because it begins the proper noun *Sherlock*. When the one-to-one cell receives `'s'`, it only observes an arbitrary lowercase word beginning with `'s'`. It cannot anticipate that the subsequent characters will form the name `"herlock"`.

4. **`"H"` of `"Holmes"` (proper noun last name)**: After the model parses the preceding token sequence `"sherlock "`, its recurrent state $\mathbf{h}_{t-1}$ retains the memory of the detective's first name. When an `'h'` arrives, the transition matrix $\mathbf{W}_{hh}$ leverages the learned bigram association $\mathbb{P}(\texttt{Holmes} \mid \texttt{Sherlock})$ to correctly capitalize the last name.

Empirical results across 1070 training epochs. Table 3.3 documents the verbatim loss trajectories and generated outputs across training checkpoints from Epoch 1 to Epoch 1070.

Analysis of qualitative phase transitions and convergence trade-offs. The comparative trajectories in Table 3.3 reveal three fundamental pedagogical principles:

Table 3.3: Empirical comparison of character casing dynamics on the Sherlock Holmes corpus: One-to-One Recurrent Mapping vs. Sequence-to-Sequence Encoder-Decoder across training epochs 1 to 1070.

	One-to-One Mapping		Encoder-Decoder Architecture	
Epoch	**Loss**	**Generated Output**	**Loss**	**Generated Output**
0001	5.3212	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	5.4591	wwxxxxxxxxxxxxxxxxxxxxxxxxxxx
0002	3.7157	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	4.0161	axxxxxxxxxxxxxxxxxxxxxxxxxxxx
0003	2.7964	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	2.8268	xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
0004	2.7981	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	2.7671	xxxxxxxxxxxxxxxxxx
0030	0.2264	"indeed i am sherlock holmes,"	0.2315	"indeed i am sherlock holmes,"
0050	0.0802	"indeed i am sherlock holmes,"	0.0705	"indeed i am sherlock holmes,"
0160	0.0301	"Indeed i am sherlock Holmes,"	0.0259	"Indeed i am sherlock Holmes,"
0530	0.0224	"Indeed i am sherlock Holmes,"	0.0197	"Indeed I am sherlock Holmes,"
1070	0.0232	"Indeed i am sherlock Holmes,"	0.0156	"Indeed I am Sherlock Holmes,"

1. **The Causal Barrier of One-to-One Recurrence**: The one-to-one model plateaus at Epoch 160 with loss 0.0301 and cannot improve further through Epoch 1070 (loss 0.0232). As predicted, it successfully capitalizes `"Indeed"` (from the quote mark) and `"Holmes"` (from the preceding `"sherlock"`), but remains permanently trapped producing lowercase `"i"` and `"sherlock"`. Because the lockstep recurrent cell must emit output $\hat{y}_t$ simultaneously with reading x_t, it suffers from irreversible causal lookahead blindness.

2. **Sequence-Level Information Integration in Seq2Seq**: The Encoder-Decoder model overcomes causal blindness by decoupling sequence ingestion from sequence emission. The encoder reads the entire string before generation begins. Consequently, the context vector $\mathbf{h}_N$ encodes global contextual features: it "remembers" that `"i"` is bounded by whitespace on both sides (making it the isolated pronoun `"I"`) and that `"s"` begins the full proper name `"sherlock"`. By Epoch 530, the decoder resolves `"Indeed I am sherlock Holmes,"` (loss 0.0197), and by Epoch 1070 it fully resolves all four capitalization targets: `"Indeed I am Sherlock Holmes,"` with superior loss 0.0156.

3. **The Sample Complexity Trade-off (**100× **Training Epochs)**: Why does the Encoder-Decoder require 1070 epochs when the simple character model converged in 10 epochs? In one-to-one mapping, temporal alignment is strictly hardcoded: character t at the input maps directly to character t at the output. In Seq2Seq, the model is given zero alignment hints; it must simultaneously discover: (i) how to compress the entire sentence into vector $\mathbf{h}_N$, (ii) how to decode variable-length autoregressive tokens, and (iii) how to align output positions with latent source semantics. Learning this unconstrained non-monotonic mapping from scratch drastically increases optimization difficulty and sample complexity.

3.7 Persistent bottlenecks of gated RNNs and the road to transformers

While LSTMs and Encoder-Decoder architectures dominated sequence modeling from 2014 to 2017, three irreducible physical and computational bottlenecks emerged that ultimately necessitated a revolutionary paradigm shift:

The Shannon channel capacity and vector bottleneck [35].
In the canonical Seq2Seq framework, an input sequence $(x_1, \dots, x_{T_x})$ of arbitrary length and linguistic entropy is forced to compress into a single continuous vector of fixed dimensionality:

$$\mathbf{h}_{T_x} = \text{Encoder}(x_1, \dots, x_{T_x}) \in \mathbb{R}^{d_h}. \tag{3.26}$$

By Shannon's source coding and rate-distortion theory, a finite-dimensional vector in floating-point precision has bounded information capacity. As sequence length increases beyond $T_x \geq 30$ tokens, the recurrent state suffers from catastrophic information dilution: earlier tokens (often containing essential syntactic subjects and conditioning context) are progressively overwritten by later inputs.

The sequential computation wall ($O(T)$ training latency).
Modern deep learning hardware accelerators (e.g., NVIDIA Hopper/Blackwell GPUs, Google TPUs) derive their performance from massive SIMD (Single Instruction, Multiple Data) parallelism across thousands of tensor cores. Recurrent architectures impose an unbreakable temporal recurrence dependency:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t). \tag{3.27}$$

Computing hidden state $\mathbf{h}_t$ strictly requires the completion of state $\mathbf{h}_{t-1}$. Consequently, an RNN cannot parallelize training across the temporal dimension T. While convolutional and attention layers process the entire sequence in $O(1)$ parallel sequential steps via batched matrix multiplications, RNNs require $O(T)$ sequential operations, rendering pre-training on multi-trillion-token web-scale corpora computationally intractable.

Temporal path length ($O(T)$ vs. $O(1)$ direct routing).
In an unrolled recurrent graph, the shortest path between token x_1 and prediction y_T scales linearly with sequence length: $O(T)$. Every additional step introduces an intermediate Jacobian multiplication $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Even with the additive Constant Error Carousel of LSTMs, error signals become noisy and diffuse over horizons exceeding 100 to 200 tokens.

The two-stage historical leap to foundation models: These structural ceilings precipitated a two-stage intellectual revolution:

1. **Stage 1: Soft Alignment [36]**: Additive Attention shattered the information vector bottleneck by maintaining direct memory access to all encoder annotations $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$. However, because the underlying backbone remained recurrent, training was still bound to the sequential execution wall.

2. **Stage 2: Attention Is All You Need [1]**: The landmark study demonstrated that if attention provides direct $O(1)$ information flow between any pair of tokens regardless of temporal distance, **recurrence is completely redundant**. Discarding recurrent cells entirely in favor of multi-head self-attention unlocked full GPU sequence parallelization and dynamic contextualization, establishing the modern Transformer architecture.

3.7.1 Breaking the vector bottleneck: Bahdanau additive attention

In the standard Seq2Seq architecture (Section 3.6), the encoder must squeeze all grammatical, lexical, and structural information of a source sentence of arbitrary length ($T_x = 40, 80$) into the

single final state $\mathbf{h}_{T_x} \in \mathbb{R}^{d_h}$. As sentence length increases, empirical translation performance drops precipitously: the fixed-size hidden vector acts as an extreme information bottleneck.

To shatter this bottleneck, **Additive (Soft) Attention** was introduced in [36], allowing the decoder to dynamically search and focus across all encoder hidden states at every generation step.

Mathematical formulation with explicit tensor dimensions. Let the source sequence $(x_1, \ldots, x_{T_x})$ be encoded by a Bidirectional RNN, producing a sequence of source annotation vectors:

$$\mathbf{h}_j = \begin{bmatrix} \overrightarrow{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{bmatrix} \in \mathbb{R}^{2d_h}, \quad \forall j \in \{1, 2, \ldots, T_x\}. \tag{3.28}$$

At decoder generation step i, let $\mathbf{s}_{i-1} \in \mathbb{R}^{d_{\text{dec}}}$ denote the previous decoder hidden state.

1. **Alignment Energy Score**: The relevance of source token j to target emission i is evaluated by a small feed-forward network parameterized by $\mathbf{W}_a \in \mathbb{R}^{d_a \times d_{\text{dec}}}$, $\mathbf{U}_a \in \mathbb{R}^{d_a \times 2d_h}$, and $\mathbf{v}_a \in \mathbb{R}^{d_a}$:
$$e_{ij} = \mathbf{v}_a^\top \tanh\left(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j\right) \in \mathbb{R}. \tag{3.29}$$

2. **Softmax Attention Weights**: Exponentiating and normalizing across all T_x source positions yields a valid categorical distribution over input tokens:
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad \text{where } \alpha_{ij} \in (0, 1) \text{ and } \sum_{j=1}^{T_x} \alpha_{ij} = 1. \tag{3.30}$$
The scalar α_{ij} represents the degree to which the decoder should attend to source word x_j when generating target word y_i.

3. **Dynamic Context Vector**: The context vector $\mathbf{c}_i$ is computed as the expected annotation under distribution $\boldsymbol{\alpha}_i$:
$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^{2d_h}. \tag{3.31}$$
Unlike standard Seq2Seq where context $\mathbf{h}_{T_x}$ is static, $\mathbf{c}_i$ is dynamically recomputed at every decoder step i, steering the model's focus to the relevant source phrase.

4. **Decoder State and Emission Update**: The decoder hidden state updates by conditioning on the previous token y_{i-1} and dynamic context $\mathbf{c}_i$:
$$\mathbf{s}_i = \text{RNN}_{\text{dec}}\left(\mathbf{s}_{i-1},\ [\mathbf{y}_{i-1}, \mathbf{c}_i]\right), \tag{3.32}$$
and target token probabilities are emitted via $\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_s[\mathbf{s}_i, \mathbf{y}_{i-1}, \mathbf{c}_i] + \mathbf{b}_s)$.

Figure 3.5 illustrates the Bahdanau Additive Attention architecture.

Historical milestone: from Schmidhuber's CEC to Sutskever's Seq2Seq.
The lineage of sequence modeling represents one of the most intellectually cohesive arcs in artificial intelligence:

- In 1997, the milestone paper *"Long Short-Term Memory"* [33] solved the vanishing gradient problem via the Constant Error Carousel, proving that additive highways preserve error signals indefinitely.
- In 2014, the Gated Recurrent Unit (GRU) and RNN encoder-decoder architectures were formulated in [34] for statistical machine translation.

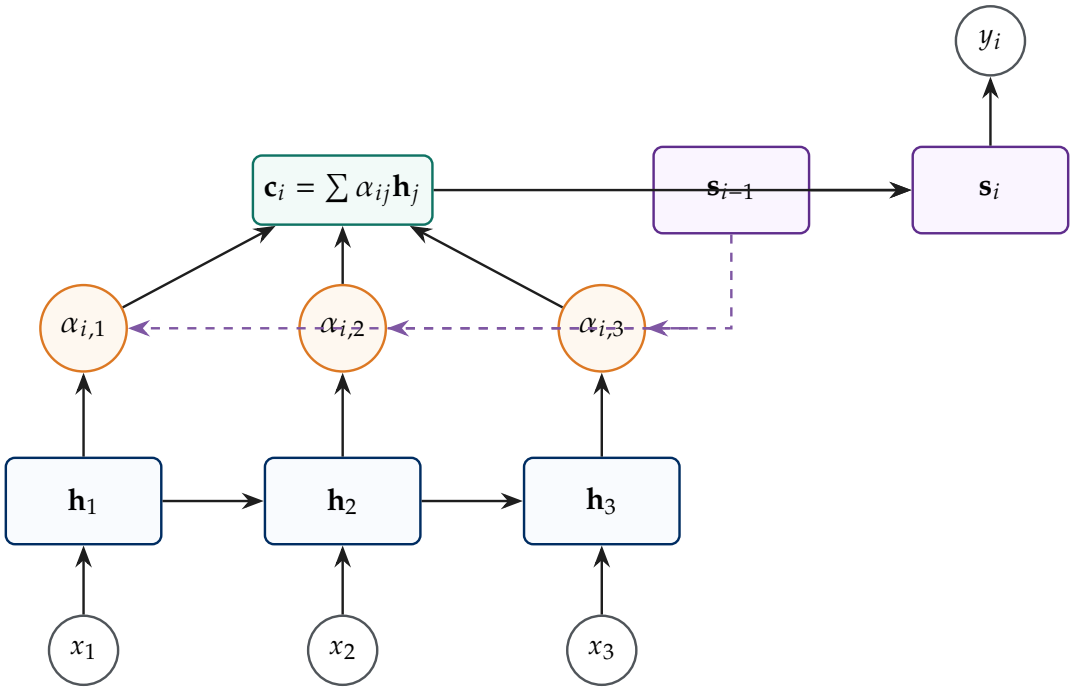


Figure 3.5: The Bahdanau Additive Attention mechanism. At decoder step i, previous decoder state $\mathbf{s}_{i-1}$ is compared against all encoder annotations $\mathbf{h}_j$ via Equation (3.29), producing attention distribution α_{ij}. The weighted sum synthesizes dynamic context vector $\mathbf{c}_i \in \mathbb{R}^{2d_h}$, directly feeding relevant source features into decoder state $\mathbf{s}_i$.

- Simultaneously in 2014, the landmark study *"Sequence to Sequence Learning with Neural Networks"* [35] demonstrated that multi-layer deep LSTMs could map sentences across languages end-to-end. Crucially, it was discovered that reversing the source sentence (*"A B C → C B A"*) dramatically improved performance by shortening the initial temporal gradient distance between source and target tokens.

3.7.2 Causal bridge to Chapter 4: the representation problem and continuous word embeddings

Throughout this chapter, we have traced the linear evolutionary arc of sequential neural architectures:

1. **Vanilla RNNs** introduced temporal recurrence and weight sharing to eliminate the fixed-dimensional constraints of feedforward networks, processing elastic sequences via an evolving hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$.

2. **LSTMs and GRUs** resolved exponential vanishing gradients along the temporal chain via the Constant Error Carousel ($\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$) and multiplicative gating highways, enabling gradient propagation across extended sequences.

3. **Seq2Seq Encoder-Decoder Pipelines** overcame the causal lookahead blindness of one-to-one recurrent cells by mapping variable-length input sequences into dynamic variable-length output sequences.

4. **Bahdanau Additive Attention** shattered the single-vector bottleneck ($\mathbf{h}_N \in \mathbb{R}^{d_h}$) by granting the decoder dynamic soft addressing across all encoder annotations ($\mathbf{c}_i = \sum_j \alpha_{ij}\mathbf{h}_j$).

The unresolved input bottleneck: what is $\mathbf{x}_t$? Despite these mathematical triumphs in temporal modeling, every recurrent transition $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$ rests upon a fundamental, unaddressed prerequisite: *the input $\mathbf{x}_t$ must be a numeric vector in $\mathbb{R}^{d_{in}}$*. In our pedagogical character casing example, we used a toy ASCII one-hot vector $\mathbf{x}_t \in \{0,1\}^{256}$. However, scaling sequence models to real-world natural language or software engineering source code reveals three catastrophic failure modes of local one-hot representations:

1. **Dimensionality Explosion**: Realistic vocabularies contain $V = |\mathcal{V}| \geq 50{,}000$ to $1{,}000{,}000$ unique words. Representing tokens as one-hot vectors $\mathbf{e}_i \in \{0,1\}^V$ forces the recurrent input projection matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times V}$ to balloon into millions of parameters, causing severe memory bloat and extreme sample inefficiency.

2. **Orthogonal Metric Collapse**: Standard basis vectors are strictly mutually orthogonal ($\mathbf{e}_i^\top \mathbf{e}_j = 0$ for all $i \neq j$) and uniformly equidistant ($d_E(\mathbf{e}_i, \mathbf{e}_j) = \sqrt{2}$). The recurrent cell receives zero geometric signal that *"dog"* and *"puppy"*, or keywords `"while"` and `"for"`, share syntactic or semantic roles.

3. **Zero Gradient Parameter Sharing**: During backpropagation through time (BPTT), updating weights for one-hot vector $\mathbf{e}_i$ provides absolutely zero learning transfer to semantically similar words $\mathbf{e}_j$, requiring astronomical training data to observe every word in every temporal context.

The path forward. While recurrent networks solved the *temporal dynamics* problem across sequences, they are crippled without a dense, low-dimensional, continuous metric space $\phi : \mathcal{V} \to \mathbb{R}^d$ ($d \ll V$) where geometric proximity directly encodes syntactic and semantic similarity.

This fundamental representational challenge leads directly to Chapter 4: *Distributed Word Representations and Word2Vec*.

CHAPTER 4

Distributed word representations and Word2Vec

In Chapter 1, we examined statistical language modeling through n-gram Markov chains and smoothed frequency distributions, demonstrating how purely discrete combinatorics collapses beneath the curse of dimensionality. In Chapter 2, we established the mathematical machinery of continuous deep neural networks, proving how non-linear activations prevent rank collapse and how multivariable gradient descent optimizes parameters across continuous manifolds. Furthermore, in Chapter 3, we explored temporal dynamics and gating mechanisms over time-varying hidden states.

Yet, a fundamental representation barrier governs every neural approach to natural language and source code: while human communication is articulated through discrete symbolic tokens, deep neural networks are continuous differentiable matrix operators that can neither compute distances nor evaluate gradients on raw string literals. Treating words as atomic indices via one-hot standard basis vectors incurs extreme sparsity, geometric orthogonality, and zero parameter sharing during backpropagation.

This chapter constructs the theoretical and mathematical bridge from discrete symbols to dense continuous metric spaces. Grounded in Harris and Firth's distributional hypothesis, we formulate self-supervised objective functions that organize latent spaces through context prediction. We dissect the Word2Vec suite—analyzing Continuous Bag-of-Words (CBOW) and Continuous Skip-Gram—and solve the full softmax computational bottleneck via Hierarchical Softmax binary trees and Noise Contrastive Negative Sampling (SGNS). We then address the out-of-vocabulary barrier and morphological blindness through FastText subword character n-gram compositions, analyze the emergent geometric symmetries of semantic clustering and linear vector analogies, and formalize modern subword tokenization and positional encoding pipelines that underpin contemporary foundation models.

4.1 The representation problem: why continuous embeddings are mandatory

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as `"compiler"`, `"optimizer"`, `"gradient"`, and keywords such as `"class"`, `"def"`, `"return"`.

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An L-layer deep neural network $\mathcal{F}(\mathbf{x};\Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x};\Theta) = \sigma_L\Big(\mathbf{W}_L\,\sigma_{L-1}\big(\cdots\sigma_1(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)\cdots\big)+\mathbf{b}_L\Big), \tag{4.1}$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l\times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta\,\nabla_\Theta\mathcal{L}(\Theta). \tag{4.2}$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial\mathcal{L}}{\partial\texttt{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add `"cat"` to `"dog"`, nor compute an inner product between `"while"` and `"for"`.

To bridge this chasm between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

Definition 4.1: *Continuous word embedding*

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \tag{4.3}$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle\cdot,\cdot\rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30{,}000$ to upwards of $V = 1{,}000{,}000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

4.2 The baseline approach: local one-hot encodings and why they fail

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a V-dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0, \dots, 0}_{i-1 \text{ zeros}}, 1, \underbrace{0, \dots, 0}_{V-i \text{ zeros}}]^\top \in \{0,1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \tag{4.4}$$

For example, consider a canonical toy vocabulary of $V = 5$ tokens:

$$\mathcal{V} = \{\texttt{"dog"}, \texttt{"cat"}, \texttt{"fish"}, \texttt{"pen"}, \texttt{"pencil"}\}. \tag{4.5}$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \tag{4.6}$$

4.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In the foundational guide *Build a Large Language Model from Scratch* [37], it is proven that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

Theorem 4.1: *Embedding lookup equivalence*

Let $\mathcal{V}$ be a vocabulary of size V, and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0,1\}^V$.

Case 1: Column-Vector Orientation (Standard Basis Representation). Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ be the embedding matrix whose k-th column corresponds to the d-dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & & | \end{bmatrix}. \tag{4.7}$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the i-th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}}\mathbf{e}_i = \sum_{k=1}^{V} (\mathbf{e}_i)_k (\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k \neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \tag{4.8}$$

Case 2: Row-Vector Orientation (PyTorch and Modern Deep Learning Convention). Let $\mathbf{W} \in \mathbb{R}^{V \times d}$ be an embedding table whose k-th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1 \times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the i-th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top \mathbf{W} = \sum_{k=1}^{V} (\mathbf{e}_i)_k \mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k \neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1 \times d}. \tag{4.9}$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}}\mathbf{e}_i \equiv \texttt{lookup}(\mathbf{W}_{\text{in}}, i). \tag{4.10}$$

4.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ and table indexing $\texttt{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs**: Evaluating the matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V-1) \cdot d$ additions. In a standard language model with vocabulary $V = 50{,}257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:

$$50{,}257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \tag{4.11}$$

   Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \ldots, V-1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:

$$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \texttt{sizeof}(\text{float32}), \tag{4.12}$$

   retrieving the d-dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction)**: Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1{,}024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1{,}024, 50{,}257)$. Storing this tensor in single-precision floating point (`float32`, 4 bytes per element) would require:

$$\text{VRAM}_{\text{one-hot}} = 64 \times 1{,}024 \times 50{,}257 \times 4 \text{ bytes} \approx 13{,}175{,}095{,}296 \text{ bytes} \approx 13.18\,\text{GB of VRAM,} \tag{4.13}$$

   simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1{,}024)$, requiring:

$$\text{VRAM}_{\text{indices}} = 64 \times 1{,}024 \times 8 \text{ bytes} = 524{,}288 \text{ bytes} \approx 524\,\text{KB of VRAM.} \tag{4.14}$$

   This constitutes a memory bandwidth reduction factor of $\frac{13.18\,\text{GB}}{524\,\text{KB}} \approx \mathbf{25{,}128}\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation**: Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \tag{4.15}$$

   This gradient matrix is massive ($V \times d$) and strictly sparse: only row i contains non-zero values, while the remaining $V-1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the i-th row of $\mathbf{W}$.

4.2.3 The 5 structural failure modes of one-hot representations

While the lookup table implementation rescues computational throughput, local one-hot representations fail catastrophically as semantic representations. They suffer from five fatal pathologies:

1. Extreme Sparsity and Curse of Dimensionality. As vocabulary size scales to web scale ($V \geq 50{,}000$ to 10^6), one-hot vectors become astronomical sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

2. Metric Collapse and Semantic Orthogonality ($\cos = 0, d_E = \sqrt{2}$). Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

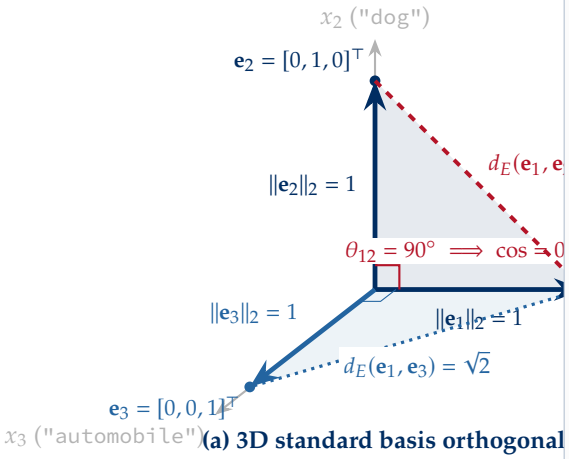
$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \tag{4.16}$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i,j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \tag{4.17}$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\texttt{"pen"}, \texttt{"pencil"}) = d_E(\texttt{"pen"}, \texttt{"crocodile"}) = \sqrt{2}. \tag{4.18}$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.



(a) 3D standard basis orthogonal

Pairwise metric equality matrices:

• **Cosine similarity matrix** (Gramian $\mathbf{E}^\top\mathbf{E} = \mathbf{I}_V$):

$$\mathbf{S}_{\cos} = \begin{array}{c} \\ \text{cat} \\ \text{dog} \\ \text{car} \end{array} \begin{array}{c} \begin{array}{ccc} \text{cat} & \text{dog} & \text{car} \end{array} \\ \begin{pmatrix} 1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & 1 \end{pmatrix} \end{array}$$

• **Euclidean distance matrix** ($\mathbf{D}_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$\mathbf{D}_{L_2} = \begin{array}{c} \\ \text{cat} \\ \text{dog} \\ \text{car} \end{array} \begin{array}{c} \begin{array}{ccc} \text{cat} & \text{dog} & \text{car} \end{array} \\ \begin{pmatrix} 0 & \sqrt{\mathbf{2}} & \sqrt{\mathbf{2}} \\ \sqrt{\mathbf{2}} & 0 & \sqrt{\mathbf{2}} \\ \sqrt{\mathbf{2}} & \sqrt{\mathbf{2}} & 0 \end{pmatrix} \end{array}$$

The geometric pathology:
dist("cat", "dog") = dist("cat", "car") = $\sqrt{2}$
cos("cat", "dog") = cos("cat", "car") = 0
All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.

(b) Semantic metric collapse

Figure 4.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In V-dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|\mathbf{e}_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin $\mathbf{0}$, yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90°$). (b) The resulting Gramian matrix is the identity $\mathbf{I}_V$, treating related concepts like "cat" and "dog" with the exact same geometric distance as totally unrelated concepts like "cat" and "automobile".

3. Zero Gradient Parameter Sharing During Backpropagation. Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token w_i produces gradients exclusively for the parameters associated with index i:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = \mathbf{0}, \quad \forall k \neq i. \tag{4.19}$$

If a model updates its weights upon observing the sentence *"The feline climbed the tree"*, the parameters associated with `"feline"` receive gradient updates, but the parameters for `"cat"`, `"kitten"`, or `"leopard"` remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

4. Simplex Confinement and Absence of Latent Manifolds. One-hot vectors are confined strictly to the V extreme vertices of the standard $(V-1)$-dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^{V} x_i = 1,\ x_i \geq 0 \right\}. \tag{4.20}$$

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

5. Software Engineering Identifier Polymorphism. In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants**: `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms**: `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents**: Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like `";"`.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\texttt{get_user_by_id}}, \mathbf{e}_{\texttt{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{LinkedList}}) = d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{";"}}) = \sqrt{2}. \tag{4.21}$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

4.3 Distributed representations and the distributional hypothesis

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept formalized in connectionist neural networks in 1986 [25].

Definition 4.2: *Local vs. distributed representations*

- **Local Representation**: Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation**: Each concept is represented by a continuous pattern of activation across all d dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

4.3.1 The distributional hypothesis: context as meaning

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics in 1954 [38] and popularized in 1957 [39]:

> *"You shall know a word by the company it keeps."*
> — Distributional Hypothesis [39]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: "I used a **pencil** to write the essay."
Sentence 2: "You used my **pen** to write a letter."

Even if a machine has never encountered a dictionary definition of `"pencil"` or `"pen"`, it observes that both tokens co-occur with identical context words: `"used"`, `"write"`, `"essay"`, `"letter"`. Conversely, an unrelated word such as `"crocodile"` or `"refrigerator"` almost never occurs in the context of `"used a [...] to write"`.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

4.4 The embedding training loop and semantic geometry evolution

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

4.4.1 The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of T running words $C = (w_1, w_2, \ldots, w_T)$, we shift a sliding context window of half-width C across the text. At each step t, the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}). \tag{4.22}$$

This self-supervised prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices**: Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.

2. **Context Lookup and Aggregation**: For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).

3. **Output Candidate Projection**: Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \tag{4.23}$$

4. **Probability Normalization**: Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^{V} \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.24}$$

5. **Loss Evaluation**: Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left(\frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \tag{4.25}$$

6. **Backpropagation Error Adjoints**: Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h}\, \boldsymbol{\delta}^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \boldsymbol{\delta} \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L})\, \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \tag{4.26}$$

7. **Stochastic Gradient Descent Parameter Update**: Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \qquad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \tag{4.27}$$

Figure 4.2 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

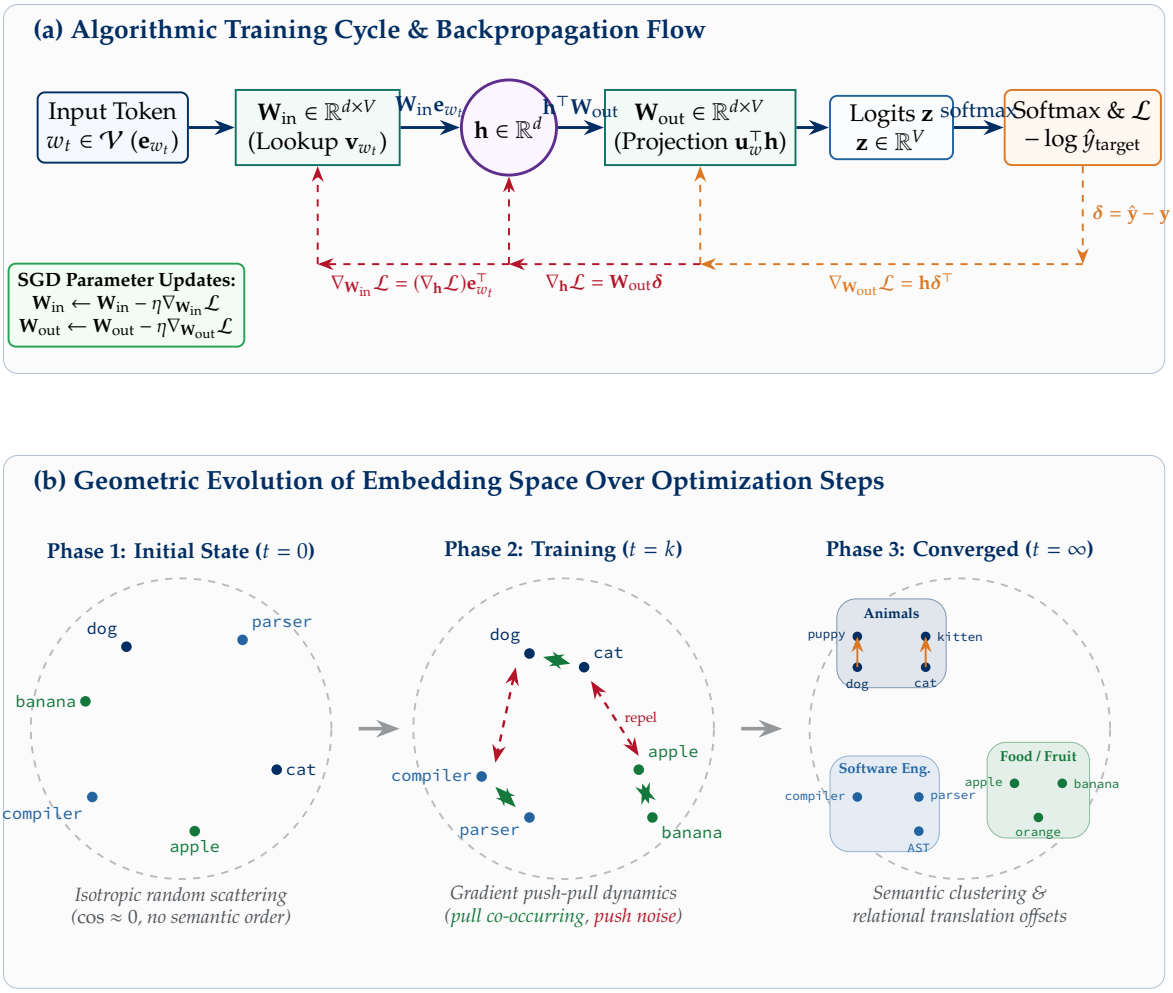


Figure 4.2: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

4.4.2 Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 4.2, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—`"dog"`, `"compiler"`, `"apple"`—are scattered isotropically across the unit sphere with zero structural order.
- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., *"The compiler parsed the syntax tree"*), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

4.4.3 Initialization dynamics and the hockey-stick loss curve

In educational analyses of neural language models and lookup tables [40], it is demonstrated that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \tag{4.28}$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \tag{4.29}$$

For a vocabulary of $V = 50{,}000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50{,}000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological "hockey-stick" loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

4.5 Word2Vec continuous bag-of-words (CBOW)

In 2013, **Word2Vec** was introduced by Mikolov et al. [41, 42] as a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token w_t given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}\} \setminus \{w_t\}. \tag{4.30}$$

4.5.1 Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token k when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token k when acting as the target.

Let $\mathbf{e} \in \{0,1\}^V$ denote the multi-hot context vector indicating the presence of words in C_t:

$$e_k = \sum_{j \in \{-C,\dots,C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j} = w_k\}}. \tag{4.31}$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C} \mathbf{W}_{\text{in}} \mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \tag{4.32}$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \tag{4.33}$$

As emphasized in geometric visual analyses [28], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos\theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.34}$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h}). \tag{4.35}$$

Figure 4.3 illustrates this computational flow.

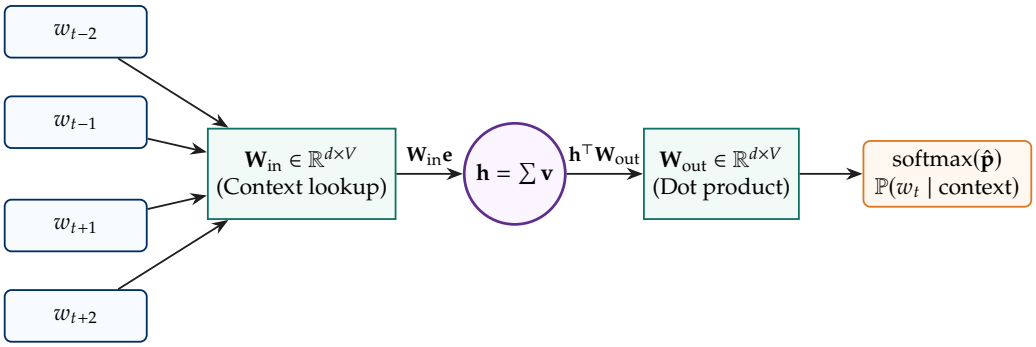


Figure 4.3: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \ldots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t.

4.5.2 Comparison with earlier neural language models: the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior in 2003 [11]. However, that architecture had two major differences:

1. **Strict Causal Prediction**: In [11], language was modeled autoregressively, predicting w_t strictly from past tokens $w_{t-n+1}, \ldots, w_{t-1}$. In Word2Vec, the window was opened symmetrically to both left and right context tokens $(w_{t-C}, \ldots, w_{t+C})$, doubling the statistical signal.

2. **Elimination of the Non-Linear Hidden Layer**: In [11], concatenated context embeddings were passed through a non-linear dense hidden layer $\tanh(\mathbf{W}_h \mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($O(C \cdot d \cdot d_h)$). In Word2Vec, the non-linear activation was stripped away entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

4.5.3 Strengths and representational bottlenecks of CBOW

Strengths:

- **Rapid Training Convergence**: Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.

- **Robust Representations for Frequent Words**: For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

Representational Bottlenecks:

- **Loss of Sequential Word Order (Syntactic Blindness)**: Vector summation in Equation (4.32) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing *"the dog bit the man"* from *"the man bit the dog"*.

- **Rare Word Dilution**: Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., *"the **polymorphism** in the"*), the rare token's embedding is diluted by the dominant stopwords: $\frac{1}{2C}\mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic

word order. To learn high-fidelity representations for infrequent words, inverting the predictive formulation yields the Continuous Skip-Gram architecture.

4.6 Word2Vec continuous skip-gram and the softmax bottleneck

To preserve the distinct gradient signal of rare words, the **Continuous Skip-Gram** architecture was introduced in [41]. Instead of aggregating context words to predict a single center token, Skip-Gram **inverts the predictive direction**: given a single center target word w_t, the model independently predicts each surrounding context word within window radius C.

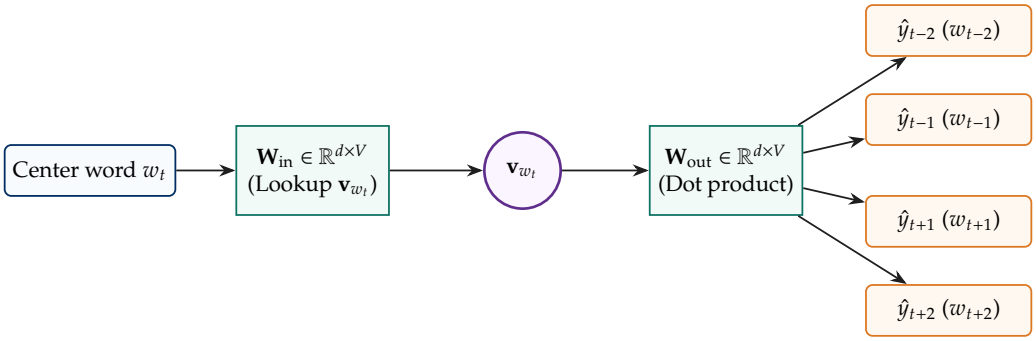


Figure 4.4: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

Definition 4.3: *Skip-gram objective function*

Given a training sequence of T words $w_1, w_2, \ldots, w_T$ and a context window of half-width C, the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} \mid w_t). \tag{4.36}$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O \mid w_I) = \frac{\exp\left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right)}, \tag{4.37}$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

4.6.1 Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence (w_I, w_O) generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as `"refactoring"` or `"polymorphism"`) appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left(\mathbf{u}_{w_O} - \sum_{k=1}^{V} \mathbb{P}(w_k \mid w_I) \mathbf{u}_k \right). \tag{4.38}$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and

nuanced semantic distinctions than CBOW.

4.6.2 The shared computational wall: the full softmax bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an insurmountable computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (4.37):

$$Z(w_I) = \sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right). \tag{4.39}$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token w in the entire vocabulary $\mathcal{V}$**.

- **Computational Complexity per Token**: Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost**: For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \tag{4.40}$$

- **Gradient Synchronization Bottleneck**: In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \ldots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

4.7 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, two distinct scaling methods were formulated in [42]: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

4.7.1 Hierarchical softmax ($O(\log_2 V)$)

First developed in 2005 [43] and adapted in [42], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology**: The V vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification**: For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) =$ root to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters**: Each internal node $j \in \{1, \ldots, L(w)-1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.
- **Binary Branching Probability**: At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \frac{1}{1 + \exp\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right)}. \tag{4.41}$$

  The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1 - \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \sigma\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right). \tag{4.42}$$

- **Path Probability Product**: The conditional probability of reaching leaf word w is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma\left(\text{sign}(w, j) \cdot \mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right), \tag{4.43}$$

  where $\text{sign}(w, j) = +1$ if the path branches left at node j, and -1 if it branches right.

Proof of Exact Normalization. Because probabilities sum to 1 at each binary branching step ($\sigma(z) + \sigma(-z) = 1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w \in \mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \tag{4.44}$$

No global partition function $Z(w_I)$ ever needs to be computed!

Complexity Reduction. Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w) - 1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $O(\log_2 V)$. For a vocabulary of $V = 100{,}000$:

$$\log_2(100{,}000) \approx 16.6 \implies \frac{100{,}000}{17} \approx \mathbf{5{,}880}\times \text{ acceleration}. \tag{4.45}$$

4.7.2 Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $O(\log_2 V)$, an even more efficient and empirically superior alternative was formulated in [42]: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over V mutually exclusive categories, SGNS reformulates the problem as $1 + K$ **independent binary logistic classifications**:

1. **Positive Sample ($y = 1$)**: The true center-context pair (w_c, w_o) observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples ($y = 0$)**: K noise words $\{w_{n_1}, w_{n_2}, \ldots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

Definition 4.4: *Skip-gram negative sampling objective*

For an observed center-context token pair (w_c, w_o) and K negative samples $\{w_{n_1}, \ldots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma\left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}\right), \tag{4.46}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

4.7.3 Step-by-step display math analytical gradient derivations

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{4.47}$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \tag{4.48}$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \tag{4.49}$$

1. Gradient with respect to Positive Target Vector $\mathbf{u}_{w_o}$. The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (4.46):

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\
&= -\left[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})\right] \frac{\partial (\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\
&= \left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{v}_{w_c}.
\end{aligned} \tag{4.50}$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1)\mathbf{v}_{w_c}. \tag{4.51}$$

2. Gradient with respect to Negative Noise Vector $\mathbf{u}_{w_{n_k}}$. Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (4.46):

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= -\left[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\ &= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{v}_{w_c}. \end{aligned} \tag{4.52}$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k}\, \mathbf{v}_{w_c}. \tag{4.53}$$

3. Gradient with respect to Center Word Vector $\mathbf{v}_{w_c}$. The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all K negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^{K} \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^{K} \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}}. \end{aligned} \tag{4.54}$$

4.7.4 Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta \left(1 - \hat{y}_{\text{pos}}\right) \mathbf{u}_{w_o} - \eta \sum_{k=1}^{K} \hat{y}_{\text{neg},k}\, \mathbf{u}_{w_{n_k}}. \tag{4.55}$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force**: Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force**: For every negative sample k, $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (4.55) is $O(K \cdot d)$, which is **entirely independent of vocabulary size** V. In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

4.7.5 Theoretical justification of the $3/4$ unigram noise exponent

How should negative samples be drawn? In [42], the noise distribution $P_n(w)$ is defined as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \tag{4.56}$$

where $f(w)$ denotes the raw unigram occurrence frequency of token w in the corpus.

Why $\alpha = 0.75$ instead of $\alpha = 1.0$ (Raw Frequency) or $\alpha = 0.0$ (Uniform)?

1. **Failure of Raw Frequency ($\alpha = 1.0$)**: Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords (`"the"`, `"of"`, `"and"`, `"to"`) account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from `"the"`, while rare content words (`"polymorphism"`, `"compiler"`) have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling ($\alpha = 0.0$)**: Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.

3. **The Sub-linear Compromise ($\alpha = 0.75$)**: The exponent 3/4 acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Numerical Demonstration of the Rare-Word Boost. Consider a frequent stopword w_{freq} with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., 10^6 occurrences in 10^8 words) and a rare technical term w_{rare} with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in 10^8 words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10{,}000. \tag{4.57}$$

  The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}}\right)^{0.75} = (10{,}000)^{0.75} = (10^4)^{3/4} = 10^3 = 1{,}000. \tag{4.58}$$

By applying the 3/4 power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10{,}000}{1{,}000} = \mathbf{10}\times. \tag{4.59}$$

This 10× boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

4.7.6 Global count matrix factorization: GloVe

Around the same time, **GloVe (Global Vectors for Word Representation)** was introduced in [44]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where X_{ij} counts the number of times word j appears in the context window of word i.

GloVe's core insight is that the semantic distinction between two words i and j relative to a probe word k is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k \mid i)}{P(k \mid j)}. \tag{4.60}$$

Table 4.1 reproduces the seminal demonstration from [44] comparing i = `"ice"` and j = `"steam"` across probe words k.

Table 4.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [44].

Probability & Ratio	k = `"solid"`	k = `"gas"`	k = `"water"`	k = `"fashion"`
$P(k \mid$ `"ice"`$)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid$ `"steam"`$)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid$ `"ice"`$)/P(k \mid$ `"steam"`$)$	**8.9**	**0.085**	1.36	0.96

When k = `"solid"` is strongly related to `"ice"` but not `"steam"`, the ratio is large ($8.9 \gg 1$). When k = `"gas"` is related to `"steam"` but not `"ice"`, the ratio is tiny ($0.085 \ll 1$). When k = `"water"` is equally related to both (or k = `"fashion"` is related to neither), the ratio hovers near 1.0.

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{4.61}$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

4.8.1 The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \ldots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time**: In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., `"recusive"`), and inflected stems.
- **Mapping to `<UNK>`**: When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{oov}} \longmapsto \texttt{<UNK>} \implies \mathbf{v}_{w_{\text{oov}}} = \mathbf{v}_{\texttt{<UNK>}}. \tag{4.62}$$

  This discards all semantic identity. To Word2Vec, a typo like `"algorithmm"` is as completely devoid of meaning as an alien string.

4.8.2 Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words share sub-morphemic stems, prefixes, and suffixes:

Natural language: `"develop"`, `"developer"`, `"developing"`, `"undeveloped"`,
Software identifiers: `"parseFile"`, `"parseTokens"`, `"parseTree"`.

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{\text{in}}$. The model possesses zero architectural mechanism to share parameters across words containing the common root `"develop"` or `"parse"`. If `"undeveloped"` occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token `"develop"`.

4.8.3 Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector $\mathbf{v}_w \in \mathbb{R}^d$ to each word string** w, regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

Definition 4.5: *Polysemy collapse*

Let word w possess M distinct semantic senses $\mathcal{S}_1, \ldots, \mathcal{S}_M$ with respective corpus mixture proportions $\pi_1, \ldots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^{M} \pi_m \mathbf{v}_{\mathcal{S}_m}. \tag{4.63}$$

For example, consider the polysemous word `"bat"`:

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."
Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{\text{bat}}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- `"Python"`: A biological serpent vs. a high-level dynamic programming language.
- `"Spring"`: A season of the year vs. a popular Java enterprise application framework.
- `"rust"`: Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps `"Rust"` to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character n-grams, FastText establishes a morphologically aware embedding paradigm.

4.9 FastText: subword character n-gram embeddings

To conquer the OOV barrier and enable morphological parameter sharing, **FastText** was introduced by Bojanowski et al. [45].

4.9.1 Character n-gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character n-grams**.

1. **Boundary Enclosure**: To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word w is bracketed by special boundary symbols "<" and ">":
$$\tilde{w} = \texttt{"<"} \circ w \circ \texttt{">"}. \tag{4.64}$$
This ensures that the prefix sequence "<re" (as in "<refactor>") is mathematically distinct from the internal substring "re" (as in "<where>").
2. **Subword Extraction**: All character n-grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

Worked Example: Subword Decomposition of ⟨where⟩. Consider the word "where" decomposed into character trigrams ($n = 3$):
$$\texttt{"<where>"} \implies \mathcal{G}_{\texttt{where}} = \Big\{\texttt{"<wh"}, \texttt{"whe"}, \texttt{"her"}, \texttt{"ere"}, \texttt{"re>"}, \texttt{"<where>"}\Big\}. \tag{4.65}$$

4.9.2 Vector sum composition

Each subword n-gram g is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word w is defined as the **sum of its constituent subword vectors**:
$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \tag{4.66}$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word w and context word c becomes:
$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \tag{4.67}$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

4.9.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens**: When an unseen word w_{oov} appears at test time (e.g., a compound identifier like "refactorable" or a code typo like "recusive"), even if the full token was never seen during training, its constituent character n-grams ("<re", "ref", "fact", "able>") were updated across other vocabulary words. FastText synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:
$$\mathbf{v}_{w_{\text{oov}}} = \sum_{g \in \mathcal{G}_{w_{\text{oov}}}} \mathbf{z}_g. \tag{4.68}$$
The OOV barrier is completely eliminated!

2. **Morphological Parameter Sharing**: In natural language, prefixes like `"un-"`, suffixes like `"-tion"`, and inflections like `"-ing"` accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\texttt{"parseXmlDocument"} \quad \text{and} \quad \texttt{"parseJsonDocument"} \tag{4.69}$$

   share the character subwords for `"parse"` and `"Document"`. The model immediately recognizes that both identifiers describe parsing operations on structured documents.

Memory Bounding via the Fowler-Noll-Vo (FNV-1a) Hash Trick. Because the universe of all character n-grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \tag{4.70}$$

Colliding n-grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

4.9.4 Connection to modern subword tokenizers: BPE and WordPiece

As highlighted in pedagogical breakdowns of language models [46], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** [12, 47]: Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** [48]: Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character n-gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character n-grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

4.10 Emergent vector space geometry: clustering, metrics and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

4.10.1 Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items**: `"table"`, `"chair"`, `"desk"`, `"sofa"`.
- **Mammals**: `"dog"`, `"cat"`, `"tiger"`, `"elephant"`.
- **Birds**: `"eagle"`, `"sparrow"`, `"parrot"`, `"pigeon"`.

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

4.10.2 Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \tag{4.71}$$

Country-to-Capital Analogy. For instance, subtracting the vector for `"Berlin"` from `"Germany"` produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \tag{4.72}$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \tag{4.73}$$

The Gender-Royalty Parallelogram. The canonical demonstration established in [42] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{4.74}$$

Figure 4.5 illustrates this parallelogram in two dimensions.

Software Engineering Analogy Equivalents. In code representation learning (e.g., CodeBERT [49], OpenAI Codex embeddings [50]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \tag{4.75}$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \tag{4.76}$$

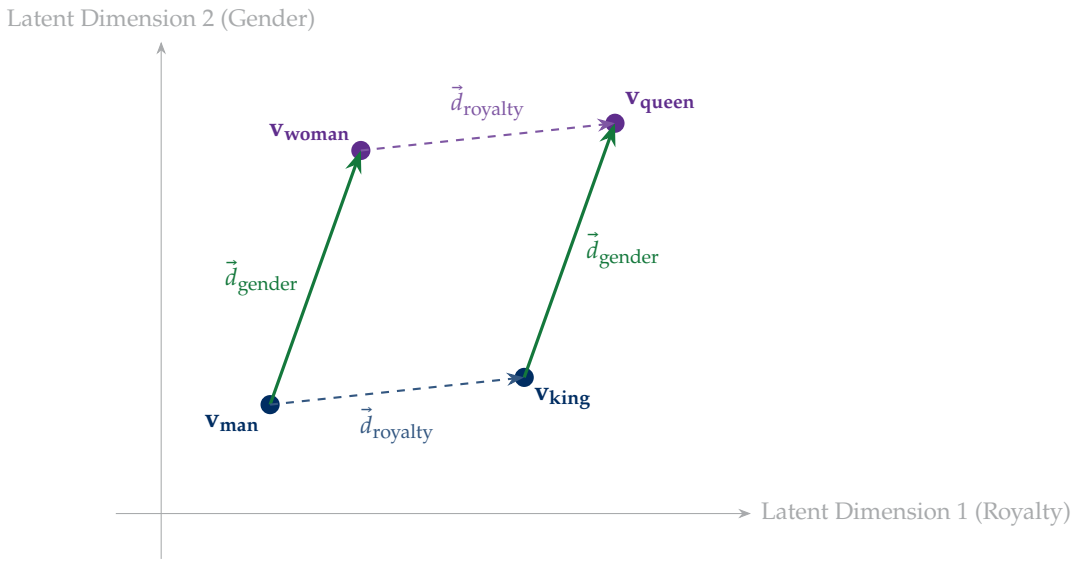


Figure 4.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

4.10.3 Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

1. **Euclidean (L_2) Distance**:

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}. \tag{4.77}$$

2. **Dot Product (Inner Product)**:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta. \tag{4.78}$$

3. **Cosine Distance**:

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \tag{4.79}$$

Theorem 4.2: *Monotonic equivalence under L_2 normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \tag{4.80}$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

Why Cosine Distance Dominates Natural Language Processing. As demonstrated in educational analyses [46], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

4.10.4 Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization ($d < 50$)**: Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization ($d \gg 10{,}000$)**: The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

Theorem 4.3: *Johnson-Lindenstrauss Lemma*

Given $0 < \epsilon < 1$, any set of N points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \tag{4.81}$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1-\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1+\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2. \tag{4.82}$$

For a vocabulary of $N = 100{,}000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

4.11 Modern subword tokenization and input pipelines for foundation models

In Sections 4.1 through 4.10, we formulated the mathematical foundations of classical distributed word representations (Word2Vec, GloVe, FastText). While continuous embeddings solved the differentiability barrier and geometric similarity, classical architectures operated under a fragmented, static paradigm: input text was naively pre-split on whitespace into word-level units, vocabulary sizes ballooned into hundreds of thousands of entries, rare or misspelled tokens collapsed into the uninformative `<UNK>` placeholder, and input sequences were treated as unordered bags or static lookup tables.

Modern Large Language Models (LLMs) such as GPT-2, GPT-3, GPT-4, LLaMA, and Claude discard this isolated design in favor of an integrated, end-to-end **data preprocessing, tokenization, and input embedding pipeline**. As formalized in *Build a Large Language Model from Scratch* [37], preparing raw text for foundation model pretraining requires an indivisible four-stage architectural pipeline:

1. **Subword Tokenization**: Segmenting raw text into atomic subword units using Byte Pair Encoding (BPE), resolving punctuation and whitespace losslessly.
2. **Token ID Mapping**: Converting discrete subwords into numerical indices via an invertible vocabulary lookup table, enriched with special operational context markers.
3. **Continuous Embedding Projection**: Retrieving dense vector representations for each token index from a learned embedding table $\mathbf{W}_{\text{tok}} \in \mathbb{R}^{V \times d}$.
4. **Composite Positional Synthesis**: Injecting sequential order into permutation-invariant neural layers by additively combining continuous token embeddings with positional embeddings: $\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}}$.

Figure 4.6 illustrates this end-to-end data pipeline from raw textual strings to the final continuous tensor ingested by Transformer architectures.

4.11.1 The tokenization spectrum: character, word, and subword trade-offs

Converting raw natural language or software code into discrete units reveals a fundamental trade-off across three representational granularities: character-level, word-level, and subword tokenization.

Why whitespace splitting fails. A naive baseline approach to text tokenization splits raw strings on whitespace characters (`'\s'`). However, whitespace splitting fails catastrophically on punctuation:

```
"Hello, world!  This is a test."  ⟹ (whitespace split)  ['Hello,', 'world!', 'This', 'is',
                                                          'a', 'test.']
```

The comma is fused to `"Hello"`, and periods are fused to `"world"` and `"test"`. Consequently, the model assigns distinct vocabulary indices to `"world"`, `"world!"`, `"world,"`, and `"world?"`. This fragments training counts, bloats the vocabulary with redundant punctuation variants, and destroys syntactic generalization.

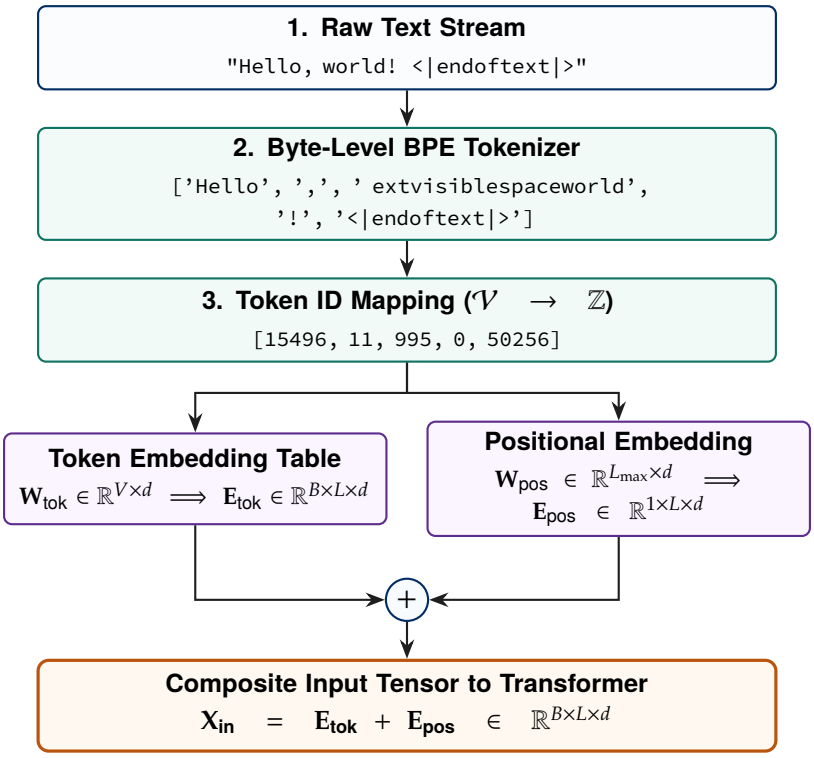


Figure 4.6: The modern foundation model tokenization and input preparation pipeline. Raw text is segmented into byte-level BPE subwords, mapped to unique integer token IDs, and converted into continuous latent vectors where token embeddings and positional embeddings combine additively.

Table 4.2: Comprehensive taxonomy and trade-off analysis of tokenization paradigms in natural language processing and modern foundation models.

Paradigm	Vocab (V)	Length (L)	OOV	Linguistic & Systems Limitations	Representative Models
Character-Level	100 – 256	Massive (4 – 6×)	Zero	High sequence length explodes computational complexity $O(L^2)$ in self-attention; lacks word-level semantic priors.	Char-RNN, Canine, ByT5
Word-Level	50k – 1M+	Minimal (1×)	Severe	Out-of-vocabulary words collapse to `<UNK>`; morphologically blind; massive embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$.	Word2Vec, GloVe, N-gram LMs
Subword-Level	32k – 100k	Moderate (1.2 – 1.4×)	Zero	Requires statistical offline pre-tokenization merge training; optimal Pareto frontier of length and vocabulary.	BPE (GPT-2/3/4, LLaMA), WordPiece (BERT)

Regex-based tokenization with punctuation isolation. To isolate punctuation while preserving them as distinct semantic tokens, a regular expression splitting rule can be formulated [37] that captures punctuation symbols, em-dashes, and whitespaces:

```python
import re

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        # Split on whitespace, punctuation marks, and special dash patterns
        tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        token_ids = [self.str_to_int[t] for t in tokens]
        return token_ids

    def decode(self, token_ids):
        text = " ".join([self.int_to_str[i] for i in token_ids])
        # Clean up spaces before punctuation marks
        text = re.sub(r'\s+([,.?!])', r'\1', text)
        return text

# Building the vocabulary from unique preprocessed tokens
text = "Hello, world. Is this-- a test?"
raw_tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
tokens = [t.strip() for t in raw_tokens if t.strip()]
vocab = {token: idx for idx, token in enumerate(sorted(set(tokens)))}
tokenizer = SimpleTokenizerV1(vocab)
encoded = tokenizer.encode("Hello, world.")
print("Encoded IDs:", encoded)
print("Decoded text:", tokenizer.decode(encoded))
```

Listing 4.1: Basic regular expression text tokenization with punctuation isolation and vocabulary construction [37].

4.11.2 Special context tokens: document boundaries, padding, and masked batching

In practical machine learning and software engineering workflows, a vocabulary cannot consist solely of natural language words. The model requires **special context tokens** to encode structural and operational metadata:

1. **Unknown Word Token** (`<|unk|>`): In fixed-vocabulary systems, words unseen during training cannot be mapped to an integer index. Such tokens are mapped to `<|unk|>`. While this prevents code execution errors, it irreversibly destroys semantic information: a technical identifier or typo is flattened into a generic void.

2. **Document Boundary Delimiter** (`<|endoftext|>`): In large-scale autoregressive pretraining, billions of tokens spanning Wikipedia articles, books, and GitHub repositories are concatenated into continuous sequential streams. Without an explicit separator, the causal attention mechanism would assume that the first sentence of an article is the causal continuation of an unrelated previous document! The token `<|endoftext|>` signals that all preceding context is decoupled from subsequent tokens, allowing clean document demarcation.

3. **Padding Token** (<|pad|>): Mini-batch processing on GPU hardware requires rectangular tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$. When batching sequences of varying lengths ($L_1 \neq L_2$), shorter sequences are padded with <|pad|> tokens up to maximum length L. Crucially, the model's self-attention mechanism is configured with an **attention mask**:

$$M_{ij} = \begin{cases} 0 & \text{if token } j \text{ is a genuine text token,} \\ -\infty & \text{if token } j \text{ is a <|pad|> token.} \end{cases} \tag{4.83}$$

Adding M_{ij} to attention logits before Softmax drives pad attention weights to $\frac{e^{-\infty}}{\sum e^z} = 0$, completely preventing padded positions from corrupting token representations.

4.11.3 Byte Pair Encoding (BPE): algorithmic mechanics and merge trees

To conquer the Out-of-Vocabulary barrier without blowing up sequence length, modern LLMs deploy **Byte Pair Encoding (BPE)**. Originally invented for data compression in [51], BPE was adapted to natural language processing in [47].

Algorithmic formalization. BPE constructs a subword vocabulary through an iterative bottom-up statistical merge procedure partitioned into distinct training and inference phases:

1. **Base Alphabet Initialization**: Initialize base vocabulary $\mathcal{V}_0$ with all individual characters occurring across the training corpus C, augmenting each word with an explicit end-of-word boundary marker (e.g., '</w>'):

$$\mathcal{V}_0 = \{c \mid c \in w,\ w \in C\} \cup \{\texttt{</w>}\}. \tag{4.84}$$

Every word in C is initially split into a sequence of individual character symbols.

2. **Adjacent Pair Frequency Tally**: At iteration k, construct the co-occurrence frequency histogram over all adjacent symbol pairs (p_1, p_2) across all tokenized words in C:

$$(p_1^*, p_2^*) = \arg\max_{(p_1,p_2)} \sum_{w \in C} \text{count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{4.85}$$

3. **Greedy Merge and Priority Rule Logging**: Merge the globally most frequent pair (p_1^*, p_2^*) into an atomic new subword unit $p_{\text{new}} = p_1^* \circ p_2^*$. Crucially, record this merge operation in an ordered priority dictionary $\mathcal{M}$:

$$\mathcal{M}[k] = ((p_1^*, p_2^*) \mapsto p_{\text{new}}). \tag{4.86}$$

4. **In-Place Corpus Contraction**: Substitute every contiguous occurrence of (p_1^*, p_2^*) across corpus C with the merged symbol p_{new}, and expand the vocabulary:

$$\mathcal{V}_k = \mathcal{V}_{k-1} \cup \{p_{\text{new}}\}. \tag{4.87}$$

5. **Termination and Inference Application**: Repeat Steps 2–4 until reaching the target vocabulary budget $|\mathcal{V}_k| = V_{\text{target}}$. At inference time, any arbitrary input word is first split into its constituent base characters, and the learned merge rules in $\mathcal{M}$ are applied sequentially according to their training rank.

A concrete clarification example: training merges and inference on unseen words. To illustrate how BPE resolves out-of-vocabulary tokens without explosion, consider a small transparent corpus consisting of four words with their occurrence frequencies:

$$C = \{\texttt{"hug"} : 10,\ \texttt{"pug"} : 5,\ \texttt{"pun"} : 12,\ \texttt{"hugs"} : 5\}. \tag{4.88}$$

The total corpus count is $10 + 5 + 12 + 5 = 32$ word instances. The initial base vocabulary consists of 7 symbols: $\mathcal{V}_0 = \{\texttt{'h'}, \texttt{'u'}, \texttt{'g'}, \texttt{'p'}, \texttt{'n'}, \texttt{'s'}, \texttt{'</w>'}\}$. The segmented corpus begins as:

```
h u g </w> :  10    p u g </w> :  5    p u n </w> :  12    h u g s </w> :  5
```

- **Iteration 1**: We evaluate the frequency of all adjacent bigrams across the corpus:
  - ('u','g') : 10(hug) + 5(pug) + 5(hugs) = 20.
  - ('p','u') : 5(pug) + 12(pun) = 17.
  - ('h','u') : 10(hug) + 5(hugs) = 15.
  - ('g','</w>') : 10(hug) + 5(pug) = 15.
  - ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('g','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The most frequent pair is ('u','g') with frequency 20. We log Merge Rule 1: ('u','g') → 'ug' and update $\mathcal{V}_1 = \mathcal{V}_0 \cup \{\texttt{'ug'}\}$. The corpus contracts to:

```
h ug </w> :  10    p ug </w> :  5    p u n </w> :  12    h ug s </w> :  5
```

- **Iteration 2**: We recompute adjacent pair frequencies with the merged symbol 'ug':
  - ('h','ug') : 10 + 5 = 15.
  - ('ug','</w>') : 10 + 5 = 15.
  - ('p','u') : 12(pun), ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('p','ug') : 5(pug), ('ug','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The highest frequency is 15. Selecting ('h','ug'), we log Merge Rule 2: ('h','ug') → 'hug' and update $\mathcal{V}_2 = \mathcal{V}_1 \cup \{\texttt{'hug'}\}$. The corpus becomes:

```
hug </w> :  10    p ug </w> :  5    p u n </w> :  12    hug s </w> :  5
```

Inference walkthrough on the unseen word "pugs". Now consider what happens when a model deployed in production encounters the word "pugs" (the plural of pug). Notice that "pugs" **never appeared in the training corpus as a complete word** (the training data only contained "pug" and "hugs").

1. Under a traditional word-level tokenizer, "pugs" is completely absent from vocabulary $\mathcal{V}$ and collapses into the information-destroying token <UNK>.
2. Under Byte Pair Encoding:
   - The word is initialized into base characters: ['p','u','g','s'].
   - The tokenizer queries the learned merge rules in priority order. Merge Rule 1 ('u','g') → 'ug' matches immediately! The sequence is rewritten as:

$$[\texttt{'p'}, \texttt{'u'}, \texttt{'g'}, \texttt{'s'}] \xrightarrow{\text{Merge 1}} [\texttt{'p'}, \texttt{'ug'}, \texttt{'s'}].$$

- Merge Rule 2 (`'h', 'ug'`) → `'hug'` is queried next. It does not match because the preceding symbol is `'p'`, not `'h'`.
- No further merge rules match adjacent symbols.

3. The final emitted token sequence is:

$$\texttt{encode("pugs")} = \texttt{['p', 'ug', 's']}. \tag{4.89}$$

The Out-of-Vocabulary barrier is overcome: the model represents the novel word via known morphological primitives—the initial consonant `'p'`, the shared stem `'ug'`, and the plural suffix `'s'`.

Figure 4.7 visualizes both the bottom-up hierarchical merge tree constructed during training and the top-down decomposition of unseen words at inference time.

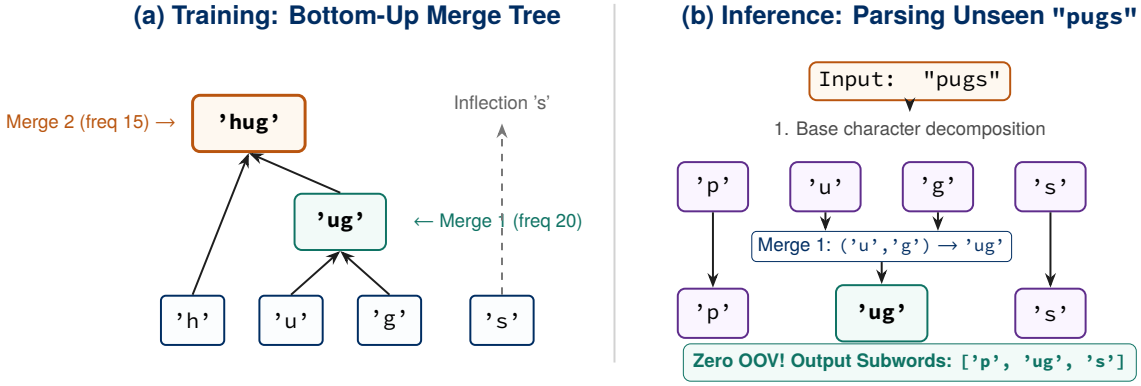


Figure 4.7: Byte Pair Encoding (BPE) mechanics. (a) During training, adjacent high-frequency symbol pairs are greedily merged into subwords. (b) At inference time, unseen words (e.g., `"pugs"`) are parsed by applying learned merge rules in priority order, resolving morphology without out-of-vocabulary collapse.

4.11.4 Byte-level BPE: the 256 UTF-8 byte foundation and elimination of unknown tokens

Despite the success of character-level BPE, applying it directly to Unicode strings faces a severe scaling barrier: the Unicode standard contains over 150,000 unique characters spanning hundreds of languages and symbol sets. Initializing $\mathcal{V}_0$ with every conceivable character creates an unmanageably large base alphabet, yet still fails when a user inputs rare historical glyphs or newly released emojis.

The 256 UTF-8 base byte breakthrough. In GPT-2, this dilemma was solved in [12] by introducing **Byte-Level Byte Pair Encoding (BBPE)**. Instead of characters, the base alphabet $\mathcal{V}_0$ is initialized strictly with the **256 distinct byte values** ($0x00$ through $0xFF$) of the UTF-8 encoding standard:

$$\mathcal{V}_0 = \{0x00, 0x01, \ldots, 0xFF\}, \quad |\mathcal{V}_0| = 256. \tag{4.90}$$

Theorem 4.4: *Total elimination of unknown tokens in byte-level BPE*

Under Byte-Level BPE with base alphabet $\mathcal{V}_0 = \{0x00, \ldots, 0xFF\}$, the probability of an Out-of-Vocabulary (OOV) token is strictly zero:

$$\mathbb{P}(w_{\text{oov}}) = 0 \quad \forall S \in \Sigma^*. \tag{4.91}$$

Proof. By definition of the Unicode transformation format (UTF-8), any arbitrary character or sequence of symbols in the Unicode code space ($U+0000$ to $U+10FFFF$) encodes deterministically into a sequence of between 1 and 4 raw bytes in $\{0x00, \dots, 0xFF\}$. Because the base alphabet $\mathcal{V}_0$ contains all 256 possible 8-bit byte values, every byte sequence can be parsed. If an input string contains an unseen compound word, a programming syntax construct, a severe typo, or an unknown emoji, the tokenizer attempts to apply merged subword rules; if no higher-level merge rule applies, the tokenizer simply falls back to emitting individual byte tokens from $\mathcal{V}_0$. Because no input byte can ever fall outside $\{0x00, \dots, 0xFF\}$, no token can ever be unmapped. The token `<|unk|>` is completely eliminated. ■

Lossless whitespace preservation. Standard natural language processing pipelines strip whitespace or split on spaces, making exact detokenization ambiguous. In Byte-Level BPE, whitespaces are treated as regular characters/bytes. In GPT-2 and Hugging Face tokenizers, a leading whitespace is mapped to a special character (e.g., `' extvisiblespace'` or byte $0x20$), binding the space directly to the succeeding word:

$$\texttt{"Hello world"} \overset{\text{BBPE}}{\Longrightarrow} \texttt{['Hello', ' extvisiblespaceworld']}$$

This guarantees that tokenization is **strictly lossless and fully invertible**:

$$\texttt{decode}(\texttt{encode}(S)) \equiv S \quad \forall S \in \Sigma^*. \tag{4.92}$$

Production implementations: OpenAI's `tiktoken`. In modern production LLM engineering, tokenization must process terabytes of training data without creating CPU bottlenecks. OpenAI's open-source library `tiktoken` implements byte-level BPE in pure Rust, executing tokenization orders of magnitude faster than Python regular expressions:

- **GPT-2 Tokenizer** (`gpt2`): Vocabulary size $V = 50{,}257$ (composed of 256 base byte tokens +50,000 learned BPE merges +1 special `<|endoftext|>` token).
- **GPT-4 Tokenizer** (`cl100k_base`): Vocabulary size $V = 100{,}277$. Expanding vocabulary from 50k to 100k reduces the total token count of natural language and code by $15 - 20\%$, directly reducing sequence length L, decreasing GPU memory footprint, and slashing API inference costs.

```python
import tiktoken

# Instantiate the GPT-2 tokenizer (vocab size 50,257)
tokenizer = tiktoken.get_encoding("gpt2")

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of
    someunknownPlace."
# Disallow special tokens check or explicitly allow specific ones
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print("Token IDs:", integers)

# Zero-shot handling of unknown words:
# 'someunknownPlace' is decomposed into subwords: ['some', 'unknown', 'Place']
for token_id in integers:
    print(f"{token_id:>5} -> {tokenizer.decode([token_id])!r}")

# Lossless roundtrip reconstruction
decoded_text = tokenizer.decode(integers)
assert decoded_text == text, "Decoded text must match original string identically!"
print("Lossless decoding confirmed!")
```

Listing 4.2: Byte-level BPE tokenization using OpenAI's `tiktoken` library [37].

Transition to continuous vector representations. Having established how Byte-Level BPE eliminates the out-of-vocabulary barrier and deterministically encodes raw text streams into discrete integer token IDs $\mathbf{T} = [t_0, t_1, \ldots, t_{L-1}] \in \mathbb{Z}^L$, we now confront the final representational bridge: how do discrete numerical indices enter the continuous, differentiable computational graph of modern neural networks? In the following subsections, we first establish the high-level architecture of the language model input pipeline, and then embark on an exhaustive investigation into positional representations.

4.11.5 From token IDs to composite inputs: high-level architecture of the foundation model input pipeline

Having transformed raw text into sequences of discrete token IDs $\mathbf{T} \in \mathbb{Z}^L$ (batched into mini-batch integer tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$), modern foundation models execute a unified input preparation pipeline that maps discrete token indices into continuous, differentiable vector representations equipped with sequential order prior to Transformer ingestion.

1. Delimiter and architectural control tokens. Before projecting indices into continuous embedding space, language models inject dedicated **special control tokens** into the sequence to regulate generation boundaries and downstream task execution:

- **Sequence demarcation tokens**:
  - ⟨BOS⟩ (*Beginning of Sequence*): Prepended to every sequence to prime autoregressive decoders, signaling the onset of generation.
  - ⟨EOS⟩ (*End of Sequence*): Appended to target text; during inference, emitting this token instructs the generation loop to terminate.
- **Architectural and task-specific delimiter tokens**:
  - [CLS] (*Classification token*): Introduced in bidirectional encoder architectures such as BERT. Prepend-ed at sequence index 0, its final contextual representation aggregates whole-sequence semantics for sentence classification or regression tasks.
  - [SEP] (*Separator token*): Delimits distinct segments within a single combined input sequence (for example, demarcating the boundary between a Question and an Answer, or between a code docstring and its method body), enabling the model to learn segment-specific interaction patterns.

2. The token embedding layer. In PyTorch, continuous token representations are instantiated via an embedding matrix:

```
token_embedding_layer = torch.nn.Embedding(num_embeddings=vocab_size, embedding_dim=d)
```

Passing batch tensor $\mathbf{X} \in \mathbb{Z}^{B \times L}$ retrieves the corresponding continuous embedding vectors:

$$\mathbf{E}_{\text{tok}} = \texttt{Embedding}(\mathbf{X}) \in \mathbb{R}^{B \times L \times d}, \tag{4.93}$$

where slice $(\mathbf{E}_{\text{tok}})_{b,t,:} = \mathbf{w}_{X_{b,t}} \in \mathbb{R}^d$ is the d-dimensional embedding vector for the token at position t in batch item b.

3. High-level additive synthesis of the composite Transformer input. To endow the model with sequential awareness, the input pipeline combines the lexical token representation $\mathbf{E}_{\text{tok}}$ and a dedicated positional tensor $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1\times L\times d}$ via **elementwise addition** rather than concatenation:

$$\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{B\times L\times d}, \tag{4.94}$$

where for each batch sample $b \in \{0, \dots, B-1\}$ and temporal coordinate $pos \in \{0, \dots, L-1\}$:

$$(\mathbf{X}_{\text{in}})_{b,pos,:} = \mathbf{w}_{X_{b,pos}} + \mathbf{p}_{pos} \in \mathbb{R}^{d}. \tag{4.95}$$

Listing 4.3 demonstrates how this composite synthesis is executed concretely in PyTorch using tensor broadcasting.

```python
import torch

vocab_size = 50257     # GPT-2 BPE vocabulary size
output_dim = 256       # Latent embedding dimension d
context_length = 4     # Sequence length L

# 1. Allocate trainable token embedding layer
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# 2. Allocate positional embedding layer (e.g., learned or precomputed sinusoidal)
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# Example batch of token IDs: [batch_size=2, context_length=4]
inputs = torch.tensor([[40, 367, 2885, 1464],
                       [1807, 3619, 402, 271]])

# Extract token embeddings: [2, 4, 256]
token_embeddings = token_embedding_layer(inputs)

# Generate position indices: tensor([0, 1, 2, 3]) -> [4, 256]
pos_indices = torch.arange(context_length)
pos_embeddings = pos_embedding_layer(pos_indices)

# 3. Additive composite synthesis with PyTorch tensor broadcasting
# token_embeddings: [2, 4, 256] + pos_embeddings: [4, 256] -> [2, 4, 256]
input_embeddings = token_embeddings + pos_embeddings

print("Token Embeddings Shape:", token_embeddings.shape)  # torch.Size([2, 4, 256])
print("Pos Embeddings Shape:  ", pos_embeddings.shape)    # torch.Size([4, 256])
print("Final Composite Shape: ", input_embeddings.shape)  # torch.Size([2, 4, 256])
```

Listing 4.3: PyTorch synthesis of token embeddings and positional embeddings [37].

Causal motivation for a deep dive into positional representations. While Equation (4.94) and Listing 4.3 summarize the high-level computational contract of the LM input pipeline, they raise foundational questions: *Why is the positional tensor* $\mathbf{E}_{pos}$ *strictly mandatory? Why do naive indexing schemes fail catastrophically? And what mathematical principles govern its construction?* We now dedicate the next subsection to an exhaustive investigation into positional representations.

4.11.6 Positional representations: intuition, mathematical formulation, and implementations

1. The permutation invariance dilemma: why positional encoding is mandatory. In classical Recurrent Neural Networks (RNNs), sequential order is implicitly baked into the architecture: the

hidden state transitions recurrently via $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, guaranteeing that token $\mathbf{x}_t$ is processed strictly after token $\mathbf{x}_{t-1}$. In Convolutional Neural Networks (CNNs), sequential order is preserved through localized spatial receptive fields.

In contrast, the cornerstone mathematical operator of the Transformer—**Self-Attention**—is strictly **permutation-equivariant**:

Theorem 4.5: *Permutation equivariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L\times d}$ be an input sequence of L vectors, and let $\mathbf{P} \in \{0,1\}^{L\times L}$ be an arbitrary permutation matrix (a square matrix having exactly one entry 1 per row and column, satisfying $\mathbf{P}^\top = \mathbf{P}^{-1}$). Under standard dot-product self-attention:

$$\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V, \tag{4.96}$$

the operator is equivariant to input permutation:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{4.97}$$

Proof. Let $\mathbf{X}' = \mathbf{P}\mathbf{X}$. The query and key projections evaluate to $\mathbf{Q}' = \mathbf{P}\mathbf{X}\mathbf{W}_Q$ and $\mathbf{K}' = \mathbf{P}\mathbf{X}\mathbf{W}_K$. The unscaled attention score matrix becomes:

$$\mathbf{Q}'(\mathbf{K}')^\top = (\mathbf{P}\mathbf{X}\mathbf{W}_Q)(\mathbf{P}\mathbf{X}\mathbf{W}_K)^\top = \mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top. \tag{4.98}$$

Because the Softmax function is evaluated independently along each row, applying a permutation matrix $\mathbf{P}$ on the left permutes the rows, while $\mathbf{P}^\top$ on the right permutes the columns:

$$\text{softmax}\left(\frac{\mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top. \tag{4.99}$$

Finally, multiplying by the value projection $\mathbf{V}' = \mathbf{P}\mathbf{X}\mathbf{W}_V$:

$$\begin{aligned}\texttt{Attention}(\mathbf{P}\mathbf{X}) &= \left[\mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top\right](\mathbf{P}\mathbf{X}\mathbf{W}_V)\\ &= \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)(\mathbf{P}^\top\mathbf{P})\mathbf{X}\mathbf{W}_V\\ &= \mathbf{P}\,\texttt{Attention}(\mathbf{X}),\end{aligned} \tag{4.100}$$

where the identity $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ collapses the internal permutation. ■ □

The unordered multiset collapse. Theorem 4.5 establishes that if we permute the input tokens, the output vectors undergo the exact same permutation with *identical values*. The attention mechanism treats the input sequence as an **unordered multiset** (a "bag of vectors") with zero intrinsic knowledge of temporal order!

To understand the catastrophic consequence of this property, consider two contrasting sentence pairs:

$$\begin{aligned}&\textbf{Sentence 1}\text{: } \textit{"I ate an apple"} \quad \Longrightarrow \quad \{\mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}, \mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}\}\\ &\textbf{Sentence 2}\text{: } \textit{"An apple, I ate"} \quad \Longrightarrow \quad \{\mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}, \mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}\}\end{aligned}$$

Or similarly:

$$\begin{aligned} &\textbf{Sentence A}: \textit{“The dog bit the man”} \quad \Longrightarrow \quad \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{dog}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{man}}\} \\ &\textbf{Sentence B}: \textit{“The man bit the dog”} \quad \Longrightarrow \quad \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{man}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{dog}}\} \end{aligned}$$

In both pairs, the sentences express fundamentally different or opposite semantic realities. Yet, because each pair comprises the exact same multiset of token embeddings, an unaugmented Transformer evaluates identical attention weights. To the model, subject, verb, and object become entirely conflated. Sequential order must therefore be injected into the representations explicitly.

Why naive positional representations fail. Before arriving at the modern formulation, it is vital to understand why naive indexing schemes break down:

1. **Naive Integer Addition ($pos \in \{0, 1, 2, \ldots, L-1\}$)**: If we add scalar indices pos to token embeddings, early tokens receive tiny perturbations (+0, +1), while token 2048 receives +2048. Because embedding vectors are initialized with mean 0 and unit variance, large coordinates overwhelm the semantic representations completely, and models fail to generalize to sequence lengths beyond training.

2. **Normalized Coordinates ($pos/L \in [0, 1]$)**: Normalizing positions by sequence length bounds values within $[0, 1]$. However, the physical meaning of a step size becomes variable: in a 10-token sentence, an offset of $\Delta = 0.1$ corresponds to an adjacent word, whereas in a 1000-token document, $\Delta = 0.1$ spans 100 words. The model cannot learn consistent relative distances.

3. **Coordinate Concatenation ($[\mathbf{w}_t; \mathbf{p}_t]$)**: Concatenating positional coordinates expands embedding dimensionality to $d + d_{\text{pos}}$, increasing parameter counts across all subsequent projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and preventing seamless parameter sharing.

The geometric clock intuition (the “Clock Way of Thought”). How can we construct an encoding that remains strictly bounded within $[-1, 1]$, assigns a unique signature to every position, and preserves consistent geometric notions of distance?
From binary counting to continuous multi-scale clocks.
To build geometric intuition, consider how a digital computer represents numbers using bounded bits $\{0, 1\}$. In a 3-bit binary counter:

- The lowest bit (Bit 0) alternates every single step: $0, 1, 0, 1, 0, 1 \ldots$ (high-frequency wave).
- The middle bit (Bit 1) alternates every two steps: $0, 0, 1, 1, 0, 0 \ldots$ (medium-frequency wave).
- The highest bit (Bit 2) alternates every four steps: $0, 0, 0, 0, 1, 1 \ldots$ (low-frequency wave).

Every integer receives an unambiguous signature, even though individual bits are bounded in $\{0, 1\}$. However, discrete step functions are non-differentiable. Sinusoidal positional encoding operates as the smooth, continuous, differentiable generalization of binary counting.

Rather than binary square waves, we assign continuous trigonometric waves oscillating at geometrically decreasing frequencies across embedding channels:

- **Fast Clocks (Seconds Hand)**: Oscillate rapidly across low dimension indices, capturing fine-grained local syntax and adjacent word order.
- **Medium Clocks (Minutes Hand)**: Oscillate at intermediate frequencies, capturing phrase and clause boundaries.
- **Slow Clocks (Hours Hand)**: Oscillate extremely slowly across high dimension indices, capturing document-level flow and paragraphs.

The clock hand problem: why sine alone fails. Why can we not simply use sine waves with varying frequencies? Consider an analog clock where you can only observe the vertical height of the hand ($Y = \sin\theta$). Because $\sin(\theta) = \sin(\pi - \theta)$, the hand is at the exact same vertical height at 2:00 and 10:00! Furthermore, $\sin(0) = \sin(\pi) = 0$. A single 1D wave suffers from severe directional ambiguity (aliasing).

To establish an unambiguous position, we need both coordinates of the clock face:

$$(X, Y) = (\cos\theta, \sin\theta). \tag{4.101}$$

Together, $(\sin\theta, \cos\theta)$ traces a point along the unit circle S^1. On a 2D circle, every angular orientation $\theta \in [0, 2\pi)$ has an unambiguous coordinate! This is why Transformer positional encodings alternate sin and cos across dimension pairs.

Figure 4.8 illustrates this multi-frequency clock mechanism and demonstrates how pairing sine and cosine eliminates 1D wave aliasing.

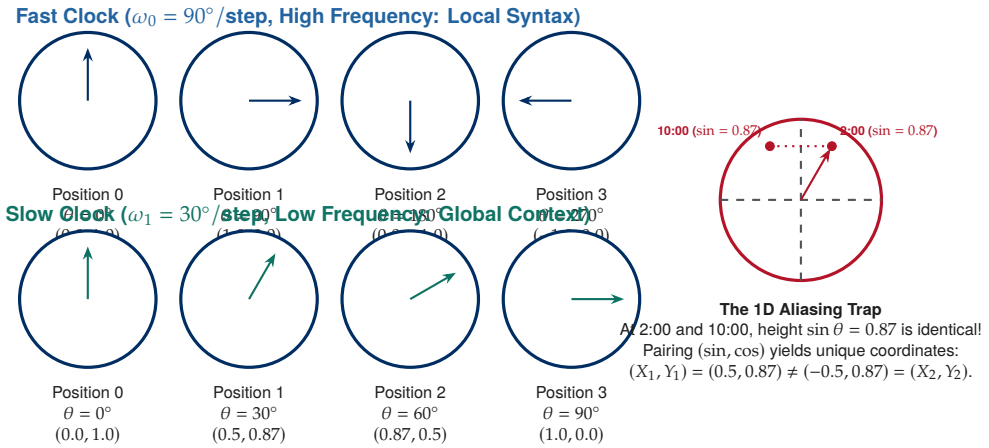


Figure 4.8: The geometric clock intuition behind positional encodings. Each dimension pair acts as an analog clock hand rotating at an angular frequency ω_i. Fast clocks disambiguate immediate neighbors (local syntax), while slow clocks preserve long-range sequence context. Pairing sin and cos prevents 1D wave aliasing by establishing a unique 2D coordinate on the unit circle S^1.

3. Taxonomy of positional representations in language models. Modern architectures implement positional awareness through three primary paradigms:

1. **Fixed Sinusoidal Positional Encodings [1]**: Fully deterministic, parameter-free functions computed from geometric progressions of trigonometric frequencies.

2. **Learned Absolute Positional Embeddings [12, 52, 53]**: Trainable parameter weights stored in an embedding matrix $\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ optimized via backpropagation (used in GPT-2, GPT-3, and BERT).

3. **Relative and Rotary Positional Embeddings (RoPE / ALiBi) [54–56]**: Encodings that inject positional information directly into attention scores as a function of relative distance $|i - j|$. Notably, Rotary Position Embedding (RoPE) [55] directly generalizes the 2D rotation matrix property of sinusoidal encodings into query and key representations.

4. Sinusoidal positional encodings: step-by-step mathematical derivation. In the seminal architecture introduced in [1], positional coordinates are generated deterministically:

Definition 4.6: *Sinusoidal positional encoding*

For a token at temporal sequence coordinate $pos \in \{0, 1, \ldots, L-1\}$ and channel index $i \in \{0, 1, \ldots, \frac{d}{2} - 1\}$, the components of the positional vector $\mathbf{p}_{pos} = \mathsf{PE}_{(pos,:)} \in \mathbb{R}^d$ are defined by:

$$\mathsf{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \tag{4.102}$$

$$\mathsf{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right). \tag{4.103}$$

Let us examine every term in this formulation:

- pos: The integer position of the token within the sequence $(0, 1, 2, \ldots, L-1)$.
- i: The frequency channel index $(0 \leq i < d/2)$.
- $2i$ and $2i+1$: Even indices receive sine values, odd indices receive cosine values.
- Angular frequency ω_i: Defined as $\omega_i = \frac{1}{10000^{2i/d}} = 10000^{-2i/d}$.
- Wavelength λ_i: The wavelength $\lambda_i = \frac{2\pi}{\omega_i} = 2\pi \cdot 10000^{2i/d}$ forms a geometric progression:

$$\text{At } i = 0: \quad \lambda_0 = 2\pi \cdot 10000^0 = 2\pi \approx 6.28 \text{ tokens}, \tag{4.104}$$

$$\text{At } i = \frac{d}{2} - 1: \quad \lambda_{\max} \approx 2\pi \cdot 10000^1 \approx 62{,}832 \text{ tokens}. \tag{4.105}$$

- **Why the base 10000?** As highlighted by the professor, 10,000 guarantees that within typical context lengths (up to $\approx 60{,}000$ tokens), the slowest frequency does not complete a single cycle. This guarantees that every position vector in the operational horizon is globally unique.

Mathematical properties and proofs. Sinusoidal positional encodings satisfy three pivotal theoretical properties:

1. **Positional uniqueness**: Because each channel i oscillates at a distinct frequency ω_i, no two positional vectors are identical. The vectors remain distinct for at least $2\pi \cdot 10000 \approx 62{,}832$ positions before periodic recurrence can occur.
2. **The linear relative shift property**: For any fixed spatial offset $k \in \mathbb{Z}$, the positional vector at position $pos + k$ can be computed as a direct linear transformation of the vector at position pos:

Theorem 4.6: *Linear relative transformation property*

For any frequency $\omega_i = 10000^{-2i/d}$ and spatial offset $k \in \mathbb{Z}$, there exists an orthogonal rotation matrix $\mathbf{M}_{i,k} \in \mathbb{R}^{2\times 2}$ independent of pos such that:

$$\begin{bmatrix} \mathsf{PE}_{(pos+k,2i)} \\ \mathsf{PE}_{(pos+k,2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \mathsf{PE}_{(pos,2i)} \\ \mathsf{PE}_{(pos,2i+1)} \end{bmatrix}. \tag{4.106}$$

Proof. Applying the standard trigonometric angle addition identities:

$$\sin(\omega_i(pos + k)) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k), \tag{4.107}$$
$$\cos(\omega_i(pos + k)) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k). \tag{4.108}$$

Expressing this system in matrix notation yields Equation (4.106). Because the rotation matrix $\mathbf{M}_{i,k}$ depends strictly on offset k and is independent of absolute position pos, linear attention projection layers ($\mathbf{W}_Q, \mathbf{W}_K$) can easily learn to attend to relative positions. ■ □

3. **Metric similarity and distance decay**: In self-attention, query and key vectors interact via inner products. The dot product between the 2D positional components of two tokens at positions p_1 and p_2 evaluates to:

$$\mathbf{p}_{p_1}^{(i)} \cdot \mathbf{p}_{p_2}^{(i)} = \sin(\omega_i p_1)\sin(\omega_i p_2) + \cos(\omega_i p_1)\cos(\omega_i p_2) = \cos(\omega_i(p_1 - p_2)). \tag{4.109}$$

The dot product depends **strictly on the relative distance** $\Delta p = p_1 - p_2$:

- When $p_1 = p_2$, $\cos(0) = 1$ (maximum correlation along the diagonal).
- As $|p_1 - p_2|$ increases, the similarity smoothly decays.

Computing the complete pairwise cosine similarity matrix across 500 positions demonstrates this property empirically:

- The 2D Gram matrix exhibits a bright, luminous main diagonal (cos = 1).
- A horizontal slice along row $pos = 100$ reveals a sharp symmetric peak at index 100, followed by a smooth, monotonic decay as distance $|pos - 100|$ expands.

Concrete numerical calculation of sinusoidal coordinates. To illustrate this formulation concretely, consider a canonical numerical configuration: sequence position $pos = 1$ (the second token, under 0-indexed coordinates) within a standard latent embedding space of dimension $d = 768$:

1. **Lowest channel pair ($i = 0$, high-frequency regime)**:

$$2i = 0 \implies \mathrm{PE}_{(1,0)} = \sin\left(\frac{1}{10000^{0/768}}\right) = \sin(1) \approx 0.8415, \tag{4.110}$$
$$2i + 1 = 1 \implies \mathrm{PE}_{(1,1)} = \cos\left(\frac{1}{10000^{0/768}}\right) = \cos(1) \approx 0.5403. \tag{4.111}$$

2. **Subsequent even channel ($i = 2$, channel index $2i = 4$)**:

$$2i = 4 \implies \mathrm{PE}_{(1,4)} = \sin\left(\frac{1}{10000^{4/768}}\right) = \sin\left(\frac{1}{10000^{0.005208}}\right)$$
$$\approx \sin\left(\frac{1}{1.0490}\right) = \sin(0.9533) \approx 0.8153. \tag{4.112}$$

These exact values (0.8415, 0.5403, and 0.8153) illustrate how trigonometric arguments shift smoothly across embedding indices, providing each positional coordinate with an unambiguous, deterministic signature.

Table 4.3: Architectural comparison: sinusoidal positional encodings vs. learned positional embeddings.

Property	Sinusoidal encodings [1]	Learned embeddings [12]
Parameters	0 trainable weights (fully deterministic)	$L_{\max} \times d$ trainable parameters
Extrapolation	Can theoretically extrapolate to sequence lengths $> L_{\max}$	Strictly confined to context length $L \leq L_{\max}$
Relative shift	Exact orthogonal rotation $\mathbf{M}_{i,k}$ provable analytically	Must be approximated empirically from data
Empirical similarity	Pure smooth monotonic distance decay around diagonal	Smooth decay near diagonal with data-driven periodic peaks
Empirical performance	Comparable translation BLEU in [1]	Standard in modern autoregressive decoders

Learned positional embeddings (GPT convention). An alternative approach, adopted by architectures such as GPT-2 and GPT-3 [2, 12], treats positional encodings as trainable parameter weights rather than fixed sinusoidal functions:

$$\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}. \tag{4.113}$$

Each integer coordinate $p \in \{0, 1, \ldots, L_{\max} - 1\}$ indexes a row vector $\mathbf{p}_p = (\mathbf{W}_{\text{pos}})_{p,:} \in \mathbb{R}^d$ optimized via gradient descent alongside token embeddings.

Computing the cosine similarity matrix of learned GPT-2 embeddings reveals high values along the main diagonal, accompanied by periodic secondary peaks at larger distances. These secondary peaks reflect statistical regularities in the training corpus (such as paragraph transitions, section breaks, or dialogue conventions).

Final clarification walkthrough: disambiguating opposite sentences. To crystallize all of these principles into an indelible mental picture, let us trace how a mini 2-clock system disambiguates our contrasting sentences:

Sentence 1: *"The dog bit the man."* vs. **Sentence 2**: *"The man bit the dog."*

Let $d = 4$ (consisting of 2 clocks):

- **Clock 1 (Fast Clock, $\omega_0 = 90°$/step)**: Rotates 90° clockwise per step.
- **Clock 2 (Slow Clock, $\omega_1 = 30°$/step)**: Rotates 30° clockwise per step.

Evaluating the positional vectors $\mathbf{PE}(pos) = [\sin(\omega_0 pos), \cos(\omega_0 pos), \sin(\omega_1 pos), \cos(\omega_1 pos)]$ yields:

Table 4.4: Exact numerical positional vectors for positions 0 through 4 under the 2-clock system.

Position (pos)	Clock 1 ($\omega_0 = 90° \cdot pos$)	Clock 2 ($\omega_1 = 30° \cdot pos$)	Positional Vector $\mathbf{p}_{pos}$
0	$\theta = 0° \implies [0.0, 1.0]$	$\theta = 0° \implies [0.0, 1.0]$	$[0.0, 1.0, 0.0, 1.0]$
1	$\theta = 90° \implies [1.0, 0.0]$	$\theta = 30° \implies [0.5, 0.87]$	$[1.0, 0.0, 0.5, 0.87]$
2	$\theta = 180° \implies [0.0, -1.0]$	$\theta = 60° \implies [0.87, 0.5]$	$[0.0, -1.0, 0.87, 0.5]$
3	$\theta = 270° \implies [-1.0, 0.0]$	$\theta = 90° \implies [1.0, 0.0]$	$[-1.0, 0.0, 1.0, 0.0]$
4	$\theta = 360° \implies [0.0, 1.0]$	$\theta = 120° \implies [0.87, -0.5]$	$[0.0, 1.0, 0.87, -0.5]$

Now consider the additive inputs:

- In **Sentence 1**: $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_4$.
- In **Sentence 2**: $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_4$.

How attention identifies the subject. In English syntax, the grammatical subject of the active verb "bit" resides 1 step prior (relative offset $k = -1$). At position $pos = 2$, Clock 1 points South (180°, vector $[0.0, -1.0]$). To identify the doer of the action, an attention head querying offset $k = -1$ rotates 90° counterclockwise:

$$180^\circ - 90^\circ = 90^\circ \implies [1.0, 0.0]. \tag{4.114}$$

The attention query searches for the token whose Clock 1 points East ($[1.0, 0.0]$):

- In **Sentence 1**, the token at position 1 with East-pointing clock is **"dog"**. The attention mechanism routes information from "dog" to "bit", resolving the dog as the subject.
- In **Sentence 2**, the token at position 1 with East-pointing clock is **"man"**. The attention mechanism routes information from "man" to "bit", resolving the man as the subject.

The ambiguity is completely resolved: through sinusoidal clock rotations, the Transformer effortlessly deciphers who bit whom!

Chapter 5

The Transformer architecture and attention mechanisms

In Chapter 4, we established the continuous mathematical foundation of distributed token representations, demonstrating how subword tokenizers and input embedding projections translate discrete alphanumeric sequences into dense vectors augmented with positional signatures. In Chapter 3, we analyzed sequential processing through recurrent neural networks, observing how hidden state recurrence passes information across time steps.

Yet, as established at the conclusion of our recurrent analysis, recurrence imposes a fundamental dual impasse: the strictly sequential dependency chain $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ forces modern parallel accelerator hardware to idle across sequence lengths, while fixed-size bottleneck representations suffer from exponential memory dissipation over extended context windows. Natural language and software source code require an architecture capable of processing sequences without recurrent serialization, establishing unconstrained communication highways between arbitrary tokens regardless of their relative distance.

In 2017, Vaswani et al. [1] broke this impasse with the **Transformer**, discarding recurrence entirely in favor of multi-head attention mechanisms. In this chapter, we develop the complete mathematical, structural, and algorithmic formulation of the Transformer. We begin by diagnosing the recurrent bottleneck, derive scaled dot-product attention from first principles as a variance-stabilized differentiable dictionary lookup, and dissect the operational triad of attention mechanisms: bidirectional encoder self-attention, causal masked decoder self-attention, and cross-attention memory routing. We then formalize the mechanics of multi-head projection subspaces, residual skip connections, Pre-LN vs. Post-LN Layer Normalization, and relative positional representations. Finally, we evaluate the foundational architectural paradigms branching from this blueprint—the Encoder-Decoder (T5), the Encoder-Only (BERT), and the Decoder-Only autoregressive foundations—and unpack the discrete sampling strategies governing sequence generation.

5.1 The architectural impasse of recurrent sequence models

For nearly a decade, sequence modeling was dominated by Recurrent Neural Networks (RNNs) and their gated variants, Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs). As established in Chapter 3, recurrent models represent a sequence of tokens by recursively updating an internal hidden state vector:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad t \in \{1, 2, \dots, L\}, \tag{5.1}$$

where $\mathbf{h}_t \in \mathbb{R}^{d_h}$ encapsulates the historical context observed up to time step t, and $\mathbf{x}_t \in \mathbb{R}^d$ is the input vector representation at time t.

While gating mechanisms and Constant Error Carousels mitigated the vanishing gradient problem over moderate temporal spans, recurrence inherently imposes two severe, insurmountable bottlenecks that halted further scaling in language modeling:

1. The sequential execution wall. The transition equation $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ enforces an unbroken, strictly sequential causal dependency chain: computation of $\mathbf{h}_t$ cannot commence until the physical memory write of $\mathbf{h}_{t-1}$ is completed. Consequently, an input sequence of L tokens mandates $O(L)$ sequential computational operations. Modern deep learning accelerators (GPUs and TPUs) derive their performance from massive data parallelism, executing thousands of arithmetic matrix operations concurrently across high-bandwidth tensor cores. Under recurrent processing, hardware parallelism across the temporal dimension is impossible: the accelerator is forced to idle between consecutive time steps, creating a computational wall that restricts training across massive datasets.

2. The information compression bottleneck of Seq2Seq encoders. In classical sequence-to-sequence architectures, an encoder compresses an input sequence of arbitrary length L into a single, fixed-size vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$ (or a tuple $(\mathbf{h}_L, \mathbf{c}_L)$ in LSTMs). This single vector is subsequently handed over to the decoder to initiate target sequence generation. As the input length L expands—such as in multi-turn dialogues, technical documentation, or software engineering source code files spanning thousands of tokens—compressing all semantic, syntactic, and structural relationships into a finite-dimensional vector becomes physically impossible. Information from early time steps undergoes continuous dissipation, inducing severe contextual amnesia.

The architectural objective. These limitations raised a fundamental question: *Can we design a neural architecture that completely eliminates recurrence, operates across all tokens in parallel with $O(1)$ sequential operations, and allows every token to directly communicate with every other token in the sequence without a fixed-size information bottleneck?* In 2017, this question was answered by the introduction of the **Transformer** [1], an architecture built entirely upon multi-head attention mechanisms.

5.2 The complete Transformer architecture: encoder-decoder breakdown

The foundational Transformer proposed in [1] adopts an **Encoder-Decoder** paradigm, structured to eliminate the recurrent compression bottleneck while enabling end-to-end parallel training. Figure 5.1 reproduces the canonical architectural diagram from the landmark paper [1].

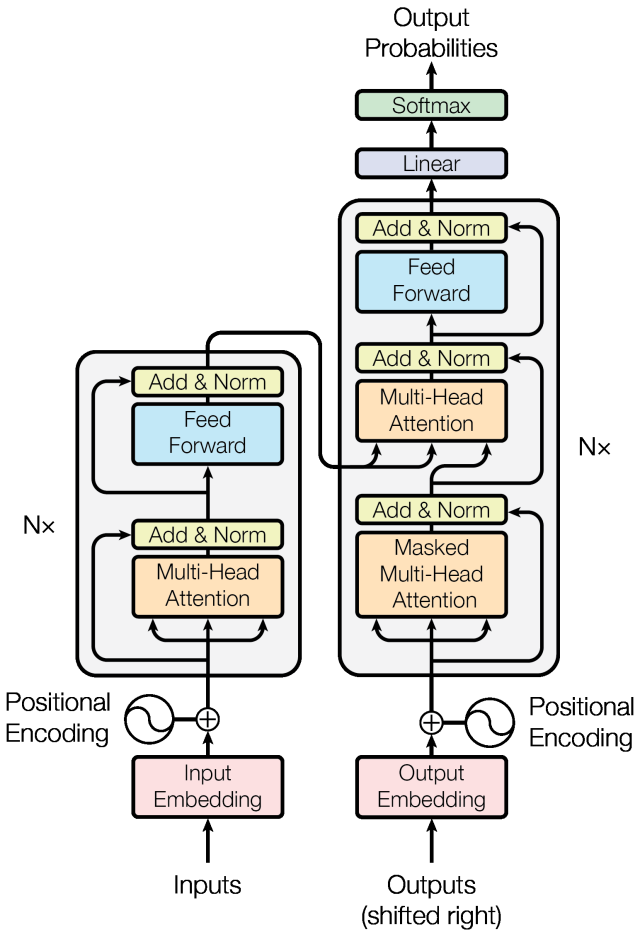


Figure 5.1: The original Transformer model architecture as published in [1]. The left stack corresponds to the Encoder ($N = 6$ identical layers), and the right stack corresponds to the Decoder ($N = 6$ identical layers), equipped with causal masked self-attention and cross-attention over the encoder representations.

1. The Encoder stack: uncompressed contextual representations. The Encoder consists of a stack of N identical layers (typically $N = 6$ in the base model). Crucially, the Encoder does *not* compress the input sequence into a single summary vector. Instead, it takes the input sequence of position-augmented embeddings $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ and transforms it into an equally sized sequence of contextual representations:

$$\mathbf{H}_{\text{enc}} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_{L_{\text{enc}}}] \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}. \tag{5.2}$$

Every output vector $\mathbf{h}_i$ possesses a global receptive field that incorporates information from every other token in the sequence through bidirectional self-attention, preserving fine-grained token-level semantics without information loss.

2. The Decoder stack: autoregressive generation and cross-attention. The Decoder also consists of a stack of N identical layers, operating autoregressively at inference time:

1. **Autoregressive sequence initialization**: Generation is primed by feeding the special beginning-of-sequence delimiter ⟨BOS⟩ as the initial token.
2. **Causal step-by-step prediction**: The Decoder processes the tokens generated so far, attends to the encoder representations $\mathbf{H}_{\text{enc}}$, and emits logits over vocabulary $\mathcal{V}$. The highest-scoring token (or sampled token) y_1 is generated.
3. **Prefix feedback loop**: The newly emitted token is appended to the decoder input, forming the context $[\langle\text{BOS}\rangle, y_1]$ to predict y_2. This loop repeats until the model outputs the end-of-sequence delimiter ⟨EOS⟩, terminating generation.

3. The training phase: Teacher Forcing and sequence parallelization. At inference, the decoder must generate tokens sequentially. However, during training, the entire target sequence $[y_0, y_1, \ldots, y_{L_{\text{dec}}-1}]$ is already known. By employing **Teacher Forcing** combined with causal attention masking (which prevents tokens from attending to subsequent target tokens), all L_{dec} generation steps are computed simultaneously in a single forward pass. The decoder does not unroll an iterative loop during training: the loss across all output positions is evaluated concurrently on the GPU.

5.3 Attention from first principles: differentiable soft dictionary retrieval

5.3.1 Why we need the attention mechanism: from RNN bottlenecks to dynamic interaction

To understand the conceptual necessity of the attention mechanism, we must examine the architectural bottleneck that afflicted sequential models prior to the Transformer, as formalized by Sebastian Raschka [37].

When translating natural language text (e.g., from German or Italian to English) or performing code generation from formal specifications, literal word-by-word substitution is fundamentally impossible. Grammatical structures, morphological agreements, and clause ordering diverge substantially between linguistic systems. Consequently, sequence transduction requires a deep model consisting of two interacting submodules: an **Encoder** that ingests and understands the source sequence, and a **Decoder** that synthesizes the target sequence.

Prior to Transformers, the dominant paradigm for sequence-to-sequence modeling was the recurrent neural network (RNN, Chapter 3). In an encoder-decoder RNN:

1. The encoder ingests source tokens $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_L$ sequentially, updating its recurrent hidden state at each time step via $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$.

2. At the conclusion of the source sequence, the encoder attempts to compress the entire semantic and syntactic content of the input into its final hidden state vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$.

3. This single vector $\mathbf{h}_L$ is handed over to the decoder as its initial context vector $\mathbf{c} = \mathbf{h}_L$, from which the decoder must generate the translated output token by token.

This architecture suffers from a catastrophic mathematical limitation: the **information compression bottleneck** [37]. Forcing an arbitrary-length sentence—spanning tens, hundreds, or thousands of tokens—into a single fixed-dimensional embedding vector creates an information-theoretic bottleneck. As sequence length expands:

- The decoder possesses *zero direct access* to intermediate representations $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_{L-1}$ from earlier encoder steps.
- Gradients backpropagating through time undergo exponential decay or explosion across recurrent chain products $\prod_{k=t+1}^{L} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$.
- Information from early tokens undergoes continuous dissipative dilution, causing severe context amnesia on long sequences.

To solve this dilemma, Bahdanau et al. [36] introduced the attention mechanism for RNNs, providing the decoder with dynamic, direct connections to *all* intermediate encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_L$ at every generation step. Three years later, Vaswani et al. [1] realized that recurrence itself was entirely superfluous. By introducing **Self-Attention**, the model allows every position within a sequence to directly attend to and dynamically interact with every other position in the sequence across an $O(1)$ sequential path length, computing rich contextual embeddings where each word's representation is dynamically updated based on the relevant context of the entire sentence [37].

5.3.2 The associative memory analogy: queries, keys, and values

To understand how attention operates mathematically from first principles, it is instructive to draw an analogy to associative retrieval structures in computer science [5, 57].

1. Classical dictionary lookup (discrete hash table). In computer systems, a dictionary or associative array stores information as a collection of m discrete key-value pairs:

$$\mathcal{D} = \{(\kappa_1, v_1), (\kappa_2, v_2), \ldots, (\kappa_m, v_m)\}. \tag{5.3}$$

When a discrete search query q is issued, the dictionary performs an exact binary match against the keys:

$$\texttt{Lookup}(q, \mathcal{D}) = \begin{cases} v_k & \text{if } \exists\, k \text{ such that } q = \kappa_k, \\ \texttt{None} & \text{otherwise.} \end{cases} \tag{5.4}$$

In this classical paradigm, retrieval is binary (0 or 1) and non-differentiable: if query q is slightly perturbed, the lookup either matches completely or fails catastrophically.

2. Soft, differentiable key-value retrieval. Attention generalizes dictionary lookup to continuous metric vector spaces $\mathbb{R}^d$:

- A **Query** vector $\mathbf{q} \in \mathbb{R}^{d_k}$ represents the information need: *"What linguistic, syntactic, or semantic attributes am I searching for?"*
- A set of m **Key** vectors $\mathbf{k}_j \in \mathbb{R}^{d_k}$ represent content identifiers or addresses: *"What attributes do I offer?"*
- A set of m **Value** vectors $\mathbf{v}_j \in \mathbb{R}^{d_v}$ represent the informative content or payload: *"What information do I communicate if selected?"*

Instead of checking exact equality $q = \kappa_k$, we evaluate a continuous compatibility score $s(\mathbf{q}, \mathbf{k}_j) \in \mathbb{R}$ using vector inner products:

$$s_j = \mathbf{q}^\top \mathbf{k}_j. \tag{5.5}$$

The raw alignment scores $[s_1, s_2, \ldots, s_m]$ are mapped onto the probability simplex via the softmax operator, yielding non-negative attention weights that sum to unity:

$$\alpha_j = \frac{\exp(s_j)}{\sum_{l=1}^{m} \exp(s_l)}, \quad \sum_{j=1}^{m} \alpha_j = 1, \quad \alpha_j \geq 0. \tag{5.6}$$

Finally, the retrieved representation is computed as the **convex combination** (weighted sum) of all value vectors:

$$\mathbf{o} = \sum_{j=1}^{m} \alpha_j \mathbf{v}_j \in \mathbb{R}^{d_v}. \tag{5.7}$$

Because inner products, exponentials, and summations are smooth operations, the entire lookup pipeline is fully differentiable with well-defined gradients $\frac{\partial \mathbf{o}}{\partial \mathbf{q}}$, $\frac{\partial \mathbf{o}}{\partial \mathbf{k}_j}$, and $\frac{\partial \mathbf{o}}{\partial \mathbf{v}_j}$.

Illustrative example: contextual displacement of a fluffy blue creature. To understand how soft key-value retrieval transforms static representations into rich contextual embeddings in high-dimensional vector spaces, consider the visual intuition formalized by Grant Sanderson [57] and illustrated in Figure 5.2:

1. **The static embedding**: Initially, each word in the phrase *"A fluffy blue creature roamed the verdant forest"* begins as an isolated unigram embedding vector $\mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$. The static vector for *"creature"* merely occupies a generic semantic coordinate indicating an animal or organism, completely devoid of context regarding its visual appearance.

2. **Queries as informational requests**: When passed into an attention block, linear projection $\mathbf{W}_Q$ maps the vector for *"creature"* into a Query vector $\mathbf{q}_{\text{creature}}$. Geometrically, this query acts as a directional probe asking: *"What descriptive adjectives precede me?"*

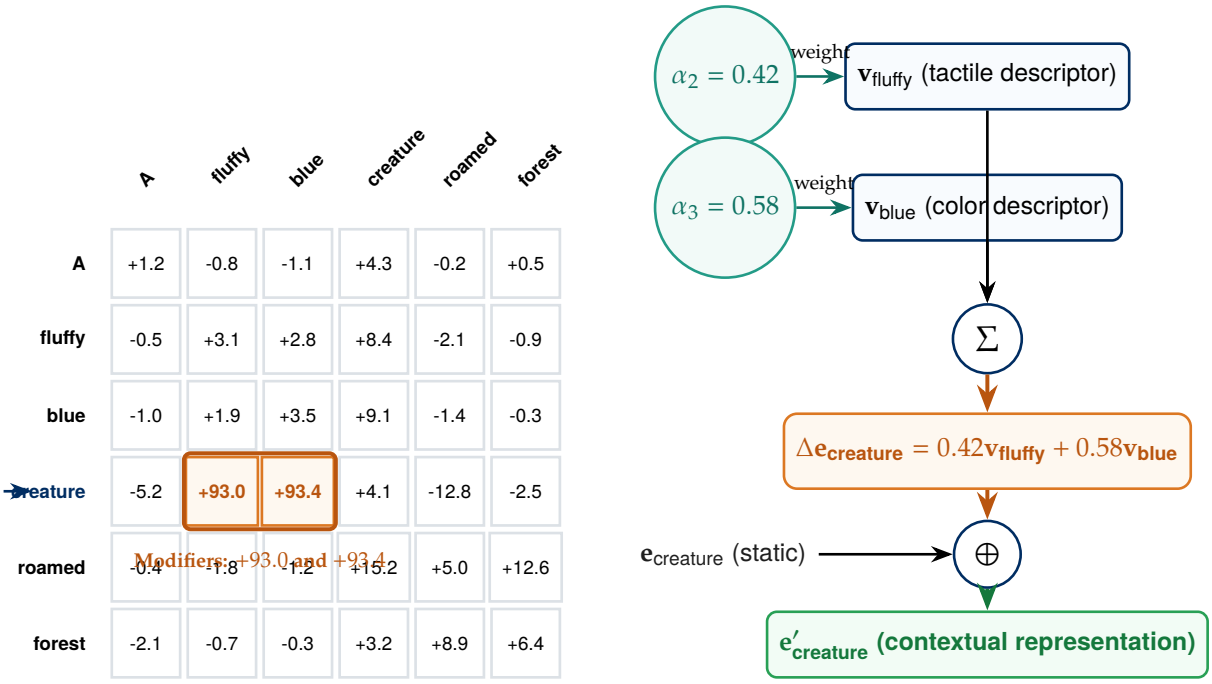
3. **Keys as feature advertisements**: Concurrently, projection matrix $\mathbf{W}_K$ maps all tokens into Key vectors. The preceding adjectives *"fluffy"* and *"blue"* produce keys $\mathbf{k}_{\text{fluffy}}$ and $\mathbf{k}_{\text{blue}}$ advertising: *"I am a tactile descriptor"* and *"I am a visual color descriptor"*.

4. **Inner-product compatibility**: Computing inner products $\mathbf{q}_{\text{creature}}^{\top}\mathbf{k}_j$ evaluates geometric alignment (Figure 5.2a). Because the query explicitly seeks adjective descriptors, it achieves high positive inner products with $\mathbf{k}_{\text{fluffy}}$ (+93.0) and $\mathbf{k}_{\text{blue}}$ (+93.4), but low or negative alignment with irrelevant tokens like *"roamed"* or *"forest"*.

5. **Value routing and displacement**: Linear projection $\mathbf{W}_V$ extracts the informative payload: Value vectors $\mathbf{v}_{\text{fluffy}}$ and $\mathbf{v}_{\text{blue}}$ encode specific sensory features. Taking their weighted sum produces a contextual displacement vector (Figure 5.2b):

$$\Delta\mathbf{e}_{\text{creature}} = \alpha_{\text{fluffy}}\mathbf{v}_{\text{fluffy}} + \alpha_{\text{blue}}\mathbf{v}_{\text{blue}}. \tag{5.8}$$

When added to the initial representation via residual connection, $\mathbf{e}_{\text{creature}} + \Delta\mathbf{e}_{\text{creature}}$ physically moves the point in embedding space into the specialized manifold region corresponding to a *fluffy, blue creature*!



(a) Query-Key compatibility matrix ($\mathbf{QK}^{\top}$) (b) Value matrix weighting and vector update ($\Delta\mathbf{e}$)

Figure 5.2: Geometric attention dynamics and matrix transformations in 3Blue1Brown's pedagogical breakdown [57]. (a) Pairwise Query-Key dot products evaluate compatibility across all token pairs in *"A fluffy blue creature roamed the verdant forest"*; the Query for *"creature"* strongly matches the Keys for descriptive modifiers *"fluffy"* (+93.0) and *"blue"* (+93.4). (b) The attention table routes the corresponding Value vectors: after softmax normalization (0.42 and 0.58), the weighted sum produces a displacement vector $\Delta\mathbf{E}_4 = 0.42\mathbf{V}_2 + 0.58\mathbf{V}_3$. Adding $\Delta\mathbf{E}_4$ to the generic creature vector $\mathbf{E}_4$ displaces the representation into the specialized manifold coordinates of a fluffy blue creature.

5.4 Scaled dot-product attention and variance stabilization

5.4.1 Definition of the scaled attention mechanism

When computing alignment scores between queries and keys, standard dot product $s = \mathbf{q}^\top \mathbf{k}$ suffers from a critical numerical failure mode as vector dimensionality increases. To resolve this, the landmark formulation in [1] introduced **Scaled Dot-Product Attention**.

Definition 5.1: *Scaled dot-product attention*

Let $\mathbf{Q} \in \mathbb{R}^{L_Q \times d_k}$, $\mathbf{K} \in \mathbb{R}^{L_K \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{L_K \times d_v}$ denote Query, Key, and Value matrices. The Scaled Dot-Product Attention operator evaluates:

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{5.9}$$

where $\mathbf{M} \in \{0, -\infty\}^{L_Q \times L_K}$ is an optional masking matrix, and softmax is applied row-wise.

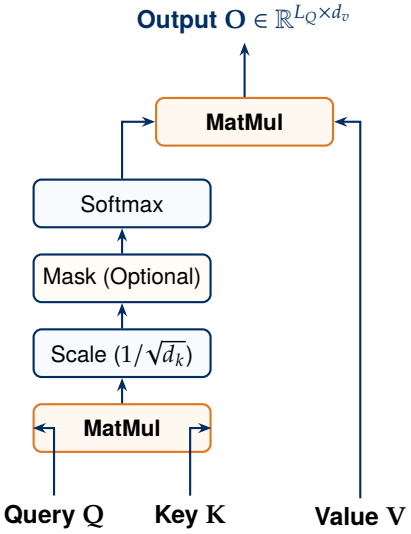


Figure 5.3: Computational dataflow of Scaled Dot-Product Attention [1]. The dot-product matrix is scaled by $1/\sqrt{d_k}$ to prevent gradient saturation before row-wise softmax normalization and value multiplication.

5.4.2 Why linear projections are mandatory

In self-attention, input tokens start as embeddings $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$. If we computed dot products directly between raw token embeddings $\mathbf{x}_i^\top \mathbf{x}_j$, two critical limitations would cripple model expressivity:

1. **Symmetry trap**: Because the dot product is commutative ($\mathbf{x}_i^\top \mathbf{x}_j = \mathbf{x}_j^\top \mathbf{x}_i$), the unprojected affinity between token i and token j would be strictly symmetric. A token could not express an asymmetric relationship, such as a verb actively seeking its direct object without the direct object reciprocating with identical strength.

2. **Role collapse**: A token vector would be forced to play three fundamentally distinct roles

simultaneously: formulating a search request (Query), advertising its semantic contents (Key), and providing the contextual payload (Value).

To break this symmetry and enable functional specialization, the model projects each input vector into three distinct latent subspaces using learned weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{5.10}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

5.4.3 Complete numerical walkthrough: reproducing the lecture calculations

To illustrate these matrix projections and attention operations concretely, we walk through step-by-step the complete numerical worked example, tracking intermediate values down to four decimal places.

1. Initial setup. Consider an input sequence of $L = 3$ tokens in a 2-dimensional latent embedding space ($d_{\text{model}} = 2$), with projection dimensions $d_k = d_v = 2$:

$$\mathbf{X} = \begin{bmatrix} 0.1 & 0.5 \\ 0.1 & 0.2 \\ 0.9 & 0.9 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{W}_Q = \begin{bmatrix} 0.9 & 0.9 \\ 0.5 & 0.3 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 1.0 & 0.9 \\ 1.0 & 0.6 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 0.8 & 0.9 \\ 0.9 & 1.0 \end{bmatrix}. \tag{5.11}$$

2. Linear projection into Queries, Keys, and Values. Multiplying $\mathbf{X}$ by each projection matrix yields:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q = \begin{bmatrix} 0.1(0.9) + 0.5(0.5) & 0.1(0.9) + 0.5(0.3) \\ 0.1(0.9) + 0.2(0.5) & 0.1(0.9) + 0.2(0.3) \\ 0.9(0.9) + 0.9(0.5) & 0.9(0.9) + 0.9(0.3) \end{bmatrix} = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix}, \tag{5.12}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K = \begin{bmatrix} 0.1(1.0) + 0.5(1.0) & 0.1(0.9) + 0.5(0.6) \\ 0.1(1.0) + 0.2(1.0) & 0.1(0.9) + 0.2(0.6) \\ 0.9(1.0) + 0.9(1.0) & 0.9(0.9) + 0.9(0.6) \end{bmatrix} = \begin{bmatrix} 0.60 & 0.39 \\ 0.30 & 0.21 \\ 1.80 & 1.35 \end{bmatrix}, \tag{5.13}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V = \begin{bmatrix} 0.1(0.8) + 0.5(0.9) & 0.1(0.9) + 0.5(1.0) \\ 0.1(0.8) + 0.2(0.9) & 0.1(0.9) + 0.2(1.0) \\ 0.9(0.8) + 0.9(0.9) & 0.9(0.9) + 0.9(1.0) \end{bmatrix} = \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix}. \tag{5.14}$$

3. Directly computing raw dot products $\mathbf{QK}^\top$ and scaled logits $\mathbf{S} = \frac{\mathbf{QK}^\top}{\sqrt{d_k}}$. Evaluating the full matrix multiplication $\mathbf{QK}^\top \in \mathbb{R}^{3\times 3}$ directly:

$$\mathbf{QK}^\top = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix} \begin{bmatrix} 0.60 & 0.30 & 1.80 \\ 0.39 & 0.21 & 1.35 \end{bmatrix} = \begin{bmatrix} 0.2976 & 0.1524 & 0.9360 \\ 0.1725 & 0.0885 & 0.5445 \\ 1.1772 & 0.6048 & 3.7260 \end{bmatrix}. \tag{5.15}$$

Alongside this matrix computation, examining row 1 reveals the individual scalar dot-product compatibility scores for the first token ($\mathbf{q}_1 = [0.34, 0.24]$):

$$s_{1,1} = \mathbf{q}_1^\top \mathbf{k}_1 = 0.34(0.60) + 0.24(0.39) = 0.2040 + 0.0936 = 0.2976, \tag{5.16}$$

$$s_{1,2} = \mathbf{q}_1^\top \mathbf{k}_2 = 0.34(0.30) + 0.24(0.21) = 0.1020 + 0.0504 = 0.1524, \tag{5.17}$$

$$s_{1,3} = \mathbf{q}_1^\top \mathbf{k}_3 = 0.34(1.80) + 0.24(1.35) = 0.6120 + 0.3240 = 0.9360. \tag{5.18}$$

Next, we scale the complete matrix by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.7071$:

$$\mathbf{S} = \frac{\mathbf{QK}^\top}{\sqrt{2}} = \begin{bmatrix} \frac{0.2976}{\sqrt{2}} & \frac{0.1524}{\sqrt{2}} & \frac{0.9360}{\sqrt{2}} \\ \frac{0.1725}{\sqrt{2}} & \frac{0.0885}{\sqrt{2}} & \frac{0.5445}{\sqrt{2}} \\ \frac{1.1772}{\sqrt{2}} & \frac{0.6048}{\sqrt{2}} & \frac{3.7260}{\sqrt{2}} \end{bmatrix} \approx \begin{bmatrix} 0.2104 & 0.1078 & 0.6619 \\ 0.1220 & 0.0626 & 0.3850 \\ 0.8324 & 0.4277 & 2.6347 \end{bmatrix}. \tag{5.19}$$

4. Matrix softmax normalization onto the probability simplex. Applying the row-wise softmax operator $\mathbf{A} = \text{softmax}(\mathbf{S})$ exponentiates each row and normalizes by its partition sum. For row 1:

$$\exp(0.2104) \approx 1.2342, \quad \exp(0.1078) \approx 1.1138, \quad \exp(0.6619) \approx 1.9385, \tag{5.20}$$

$$\sum_{j=1}^{3} \exp(\tilde{s}_{1,j}) = 1.2342 + 1.1138 + 1.9385 = 4.2865, \tag{5.21}$$

$$\alpha_{1,1} = \frac{1.2342}{4.2865} \approx 0.2879, \quad \alpha_{1,2} = \frac{1.1138}{4.2865} \approx 0.2598, \quad \alpha_{1,3} = \frac{1.9385}{4.2865} \approx 0.4522. \tag{5.22}$$

Evaluating rows 2 and 3 identically yields the full row-stochastic attention weight matrix:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix}, \tag{5.23}$$

where each row sums precisely to 1.0000.

5. Matrix value aggregation: computing the full output matrix $\mathbf{O} = \mathbf{AV}$. The full sequence output matrix is obtained by matrix multiplication $\mathbf{O} = \mathbf{AV} \in \mathbb{R}^{3\times 2}$:

$$\mathbf{O} = \mathbf{AV} = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix} \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 & 1.02 \\ 0.85 & 0.95 \\ 1.29 & 1.44 \end{bmatrix}. \tag{5.24}$$

For token 1, this matrix product directly computes the linear combination of value rows:

$$\mathbf{o}_1 = 0.2879 \begin{bmatrix} 0.53 \\ 0.59 \end{bmatrix} + 0.2598 \begin{bmatrix} 0.26 \\ 0.29 \end{bmatrix} + 0.4522 \begin{bmatrix} 1.53 \\ 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 \\ 1.02 \end{bmatrix}. \tag{5.25}$$

This reproduces identically the exact tensor results displayed in the calculations.

5.4.4 Dimensional constraints across projection matrices

The dimensional compatibility of the projection matrices is governed by three fundamental algebraic constraints:

- **Queries and Keys must share identical dimensionality (d_k)**: The dot product $\mathbf{q}^\top\mathbf{k}$ is an inner product between two vectors. An inner product is mathematically undefined unless both vectors live in the identical Euclidean vector space $\mathbb{R}^{d_k}$. Therefore, $\mathbf{W}_Q$ and $\mathbf{W}_K$ must always share identical column dimension d_k.
- **Values can possess independent dimensionality (d_v)**: Value vectors are never multiplied with queries or keys. They are only scaled by scalar attention weights $\alpha_j \in [0, 1]$ and summed together. Consequently, d_v could mathematically differ from d_k. In standard architectures (including [1]), practitioners choose $d_k = d_v = d_{\text{model}}/h$ purely for architectural symmetry and implementation simplicity.
- **Sequence lengths in self-attention vs. cross-attention**: In self-attention, $L_Q = L_K = L_V = L$. In cross-attention, queries stem from the decoder ($L_Q = L_{\text{dec}}$), while keys and values stem from the encoder ($L_K = L_V = L_{\text{enc}}$). The dot-product matrix $\mathbf{QK}^\top$ has shape $L_{\text{dec}} \times L_{\text{enc}}$, and multiplying by $\mathbf{V} \in \mathbb{R}^{L_{\text{enc}}\times d_v}$ yields the required decoder output shape $L_{\text{dec}} \times d_v$.

5.4.5 Mathematical derivation: theoretical necessity of the $\frac{1}{\sqrt{d_k}}$ scaling factor

Why must the dot product $\mathbf{q}^\top \mathbf{k}$ be scaled specifically by $\frac{1}{\sqrt{d_k}}$? To understand the mathematical necessity of this normalization, consider the variance behavior of unscaled dot products as dimensionality expands.

Proof of dot-product variance expansion. Assume that the components of query vector $\mathbf{q} \in \mathbb{R}^{d_k}$ and key vector $\mathbf{k} \in \mathbb{R}^{d_k}$ are independent, identically distributed (i.i.d.) random variables with zero mean and unit variance:

$$\mathbb{E}[q_i] = 0, \quad \mathbb{V}\mathrm{ar}(q_i) = 1, \quad \mathbb{E}[k_i] = 0, \quad \mathbb{V}\mathrm{ar}(k_i) = 1, \quad \forall i \in \{1, 2, \ldots, d_k\}. \tag{5.26}$$

The unscaled dot product is the random variable $z = \mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. By linearity of expectation and independence:

$$\mathbb{E}[z] = \mathbb{E}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\,\mathbb{E}[k_i] = \sum_{i=1}^{d_k} (0 \cdot 0) = 0. \tag{5.27}$$

To compute the variance of z, we expand the definition $\mathbb{V}\mathrm{ar}(z) = \sum_{i=1}^{d_k} \mathbb{V}\mathrm{ar}(q_i k_i)$. Using identity $\mathbb{V}\mathrm{ar}(XY) = \mathbb{E}[X^2]\,\mathbb{E}[Y^2] - (\mathbb{E}[X]\,\mathbb{E}[Y])^2$:

$$\begin{aligned} \mathbb{V}\mathrm{ar}(q_i k_i) &= \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] - (\mathbb{E}[q_i]\,\mathbb{E}[k_i])^2 \\ &= \left(\mathbb{V}\mathrm{ar}(q_i) + (\mathbb{E}[q_i])^2\right)\left(\mathbb{V}\mathrm{ar}(k_i) + (\mathbb{E}[k_i])^2\right) - 0 \\ &= (1+0)(1+0) = 1. \end{aligned} \tag{5.28}$$

Summing over all d_k independent dimensions:

$$\mathbb{V}\mathrm{ar}(z) = \sum_{i=1}^{d_k} \mathbb{V}\mathrm{ar}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k. \tag{5.29}$$

Thus, the standard deviation of the dot product is $\sigma_z = \sqrt{d_k}$.

The gradient saturation consequence. For large dimensions (e.g., $d_k = 64$ or 128), the variance d_k is substantial, causing dot products to take values with magnitude $|z| \gg 1$. Recall that the derivative of the softmax function with respect to logit z_i is given by:

$$\frac{\partial\,\mathrm{softmax}(z)_i}{\partial z_j} = \mathrm{softmax}(z)_i \left(\delta_{ij} - \mathrm{softmax}(z)_j\right). \tag{5.30}$$

When logits z have large absolute variance, the exponential $\exp(z_m)$ of the maximal logit completely dominates the sum. The softmax output degenerates into a nearly one-hot indicator distribution ($\alpha_m \approx 1$, and $\alpha_{j \neq m} \approx 0$). In this regime:

$$\frac{\partial\,\mathrm{softmax}(z)_i}{\partial z_j} \approx 0, \quad \forall i, j. \tag{5.31}$$

The gradients vanish, arresting backpropagation through the attention module. By scaling z by scalar factor $\frac{1}{\sqrt{d_k}}$, we renormalize the variance back to unity:

$$\mathbb{V}\mathrm{ar}\left(\frac{z}{\sqrt{d_k}}\right) = \frac{1}{(\sqrt{d_k})^2}\,\mathbb{V}\mathrm{ar}(z) = \frac{1}{d_k} \cdot d_k = 1. \tag{5.32}$$

Renormalizing the variance preserves healthy gradient dynamics across arbitrarily deep networks.

5.5 The triad of attention mechanisms

Within the complete Transformer architecture, attention is deployed across three distinct operational settings [1], summarized in Figure 5.4.

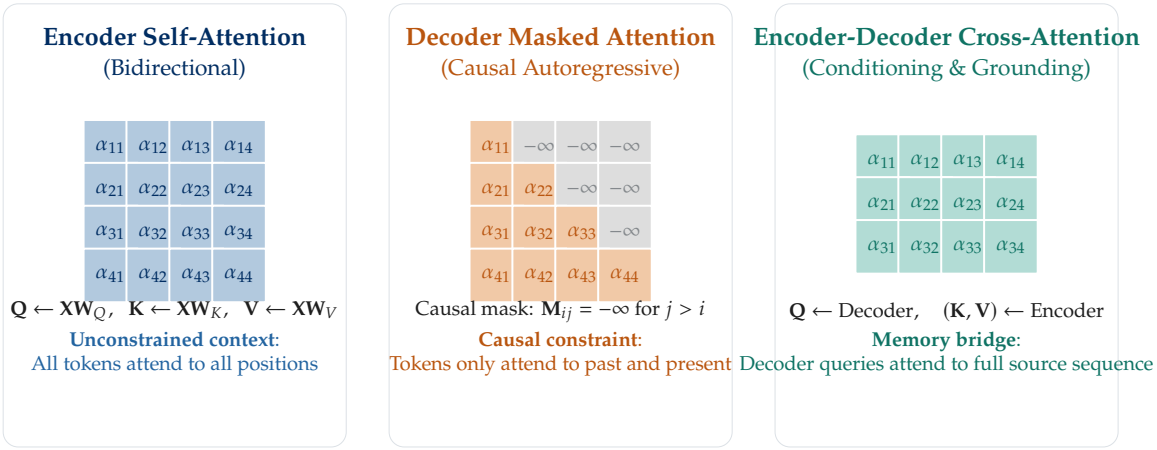


Figure 5.4: The triad of attention mechanisms across the Transformer architecture. (a) Encoder self-attention processes inputs bidirectionally with an unconstrained attention matrix. (b) Decoder masked self-attention applies a lower-triangular causal mask ($\mathbf{M}_{ij} = -\infty$ for $j > i$) to enforce autoregressive generation without future information leakage. (c) Encoder-decoder cross-attention allows decoder queries to attend across all encoder key-value pairs, bridging representations across different sequence lengths.

5.5.1 Encoder self-attention: unconstrained bidirectional contextualization

Why unconstrained bidirectional visibility is mandatory. In natural language understanding, the meaning of a word is profoundly shaped by tokens that appear both *before* and *after* it. For example, consider syntactic ambiguity, pronoun reference, or polysemy: in the sentence *"The bank of the river overflowed"*, the polysemous noun *"bank"* cannot be resolved into a geological landform rather than a financial institution without inspecting the downstream word *"river"*. Because the Encoder's sole objective is representation learning rather than autoregressive word generation, there is no risk of label leakage. Therefore, the Encoder must have access to the entire sentence simultaneously: the attention matrix $\mathbf{A} \in \mathbb{R}^{L_{\text{enc}} \times L_{\text{enc}}}$ is completely unconstrained ($\mathbf{M} = \mathbf{0}$), granting every token an instantaneous $O(1)$ global receptive field across all L_{enc} positions.

In the Encoder, queries, keys, and values are all linearly projected from the preceding encoder layer's representations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{5.33}$$

where $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$, $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. No masking matrix is applied ($\mathbf{M} = \mathbf{0}$). Every token i can attend freely to all other tokens $j \in \{1, 2, \ldots, L_{\text{enc}}\}$, producing unconstrained bidirectional representations.

Step-by-step mathematical example: bidirectional polysemy disambiguation. Consider an English input sentence of $L_{\text{enc}} = 3$ tokens:

$$X = [\text{"bank"}_1, \text{"of"}_2, \text{"river"}_3]. \tag{5.34}$$

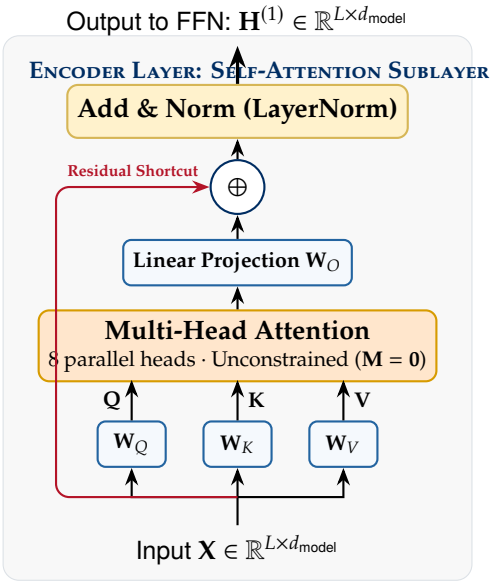


Figure 5.5: Architectural schematic of the Encoder Self-Attention sublayer [1]. Input tokens $\mathbf{X}$ are projected into queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$. Unconstrained multi-head attention evaluates all-to-all token visibility without masking ($\mathbf{M} = \mathbf{0}$), followed by linear output projection $\mathbf{W}_O$, identity residual addition, and Layer Normalization.

The token *"bank"* is polysemous. Without inspecting the future token *"river"*, disambiguation is impossible. Let $d_k = d_v = 2$. Suppose the linear projections $\mathbf{X}\mathbf{W}_Q$, $\mathbf{X}\mathbf{W}_K$, and $\mathbf{X}\mathbf{W}_V$ yield:

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.5 \\ 0.2 & 0.8 \\ 0.9 & 1.2 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.8 & 0.4 \\ 0.1 & 0.9 \\ 1.0 & 1.1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1.5 & 0.2 \\ 0.1 & 0.8 \\ 0.4 & 2.0 \end{bmatrix}. \tag{5.35}$$

We execute the attention mechanism step by step:

1. **Raw dot-product computation**: Evaluating unscaled inner products $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{3\times 3}$:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} (1.0)(0.8) + (0.5)(0.4) & (1.0)(0.1) + (0.5)(0.9) & (1.0)(1.0) + (0.5)(1.1) \\ (0.2)(0.8) + (0.8)(0.4) & (0.2)(0.1) + (0.8)(0.9) & (0.2)(1.0) + (0.8)(1.1) \\ (0.9)(0.8) + (1.2)(0.4) & (0.9)(0.1) + (1.2)(0.9) & (0.9)(1.0) + (1.2)(1.1) \end{bmatrix}. \tag{5.36}$$

   Evaluating each scalar entry yields:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 1.00 & 0.55 & 1.55 \\ 0.48 & 0.74 & 1.08 \\ 1.20 & 1.17 & 2.22 \end{bmatrix}. \tag{5.37}$$

2. **Scaling by $\sqrt{d_k} = \sqrt{2} \approx 1.414$**:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.707 & 0.389 & 1.096 \\ 0.339 & 0.523 & 0.764 \\ 0.849 & 0.827 & 1.570 \end{bmatrix}. \tag{5.38}$$

3. **Bidirectional softmax normalization**: Since no mask is present ($\mathbf{M} = \mathbf{0}$), we compute row-wise softmax directly across all columns: For row 1 (corresponding to *"bank"*):

$$\exp(0.707) \approx 2.028, \quad \exp(0.389) \approx 1.475, \quad \exp(1.096) \approx 2.992, \tag{5.39}$$

$$Z_1 = 2.028 + 1.475 + 2.992 = 6.495, \tag{5.40}$$

$$\alpha_{11} = \frac{2.028}{6.495} \approx 0.312, \quad \alpha_{12} = \frac{1.475}{6.495} \approx 0.227, \quad \alpha_{13} = \frac{2.992}{6.495} \approx 0.461. \tag{5.41}$$

Applying this across all rows yields the dense attention matrix:

$$\mathbf{A}_{\text{enc}} = \begin{bmatrix} 0.312 & 0.227 & 0.461 \\ 0.268 & 0.322 & 0.410 \\ 0.248 & 0.243 & 0.510 \end{bmatrix}. \tag{5.42}$$

4. **Contextual value aggregation**: Multiplying $\mathbf{A}_{\text{enc}}$ by $\mathbf{V} \in \mathbb{R}^{3\times 2}$ yields the updated contextual representations:

$$\mathbf{O}_{\text{enc}} = \mathbf{A}_{\text{enc}}\mathbf{V} = \begin{bmatrix} 0.312\mathbf{v}_1 + 0.227\mathbf{v}_2 + 0.461\mathbf{v}_3 \\ 0.268\mathbf{v}_1 + 0.322\mathbf{v}_2 + 0.410\mathbf{v}_3 \\ 0.248\mathbf{v}_1 + 0.243\mathbf{v}_2 + 0.510\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.675 & 1.165 \\ 0.598 & 1.131 \\ 0.600 & 1.263 \end{bmatrix}. \tag{5.43}$$

Key architectural takeaway. Notice that token 1 (*"bank"*) allocates its largest attention weight ($\alpha_{13} = 0.461$) to token 3 (*"river"*), looking *ahead* into future context. This unconstrained bidirectional information exchange allows *"bank"* to instantly absorb the geological semantics of *"river"* into its contextual output $\mathbf{o}_1$.

5.5.2 Decoder masked self-attention: causal autoregressive constraint

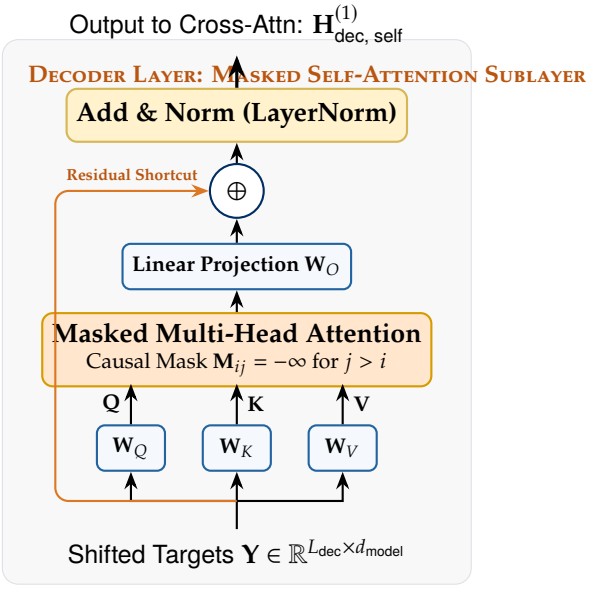


Figure 5.6: Architectural schematic of the Decoder Masked Self-Attention sublayer [1]. Causal masking enforces autoregressive conditioning by blinding all future positions with $-\infty$, ensuring query i attends exclusively to preceding tokens $j \leq i$.

Why the causal mask is mandatory for autoregressive decoding. Unlike the Encoder, the Decoder is tasked with sequential language generation: producing the next token y_t conditioned strictly on the prefix of tokens emitted so far $[y_1, y_2, \ldots, y_{t-1}]$. At test time, future tokens $y_{t+1}, y_{t+2}, \ldots$ do not exist. During training, however, the entire target sequence is supplied to the model simultaneously to enable parallel execution via Teacher Forcing. If unconstrained self-attention were permitted in the decoder, a token at position t could directly attend to position $t+1$ and cheat by copying the ground-truth answer. The model would suffer from catastrophic information leakage, failing to learn any predictive distribution over future tokens. To enforce

the arrow of time while preserving parallel tensor execution on hardware, a **causal mask M** is added to the attention logits prior to softmax, strictly setting all future attention weights to zero.

To preserve causality while retaining parallel training under Teacher Forcing, a **causal mask** $\mathbf{M} \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{dec}}}$ is added to the scaled dot products prior to the softmax operator:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases} \tag{5.44}$$

In matrix form for a sequence of length 3:

$$\mathbf{M} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}. \tag{5.45}$$

Because $\exp(-\infty) = 0$, evaluating the row-wise softmax guarantees:

$$\alpha_{i,j} = 0 \quad \forall j > i. \tag{5.46}$$

From a hardware efficiency standpoint, computing the full dense GEMM operation $\mathbf{QK}^\top$ followed by a tensor addition with $\mathbf{M}$ fully saturates GPU tensor cores and memory buses, outperforming jagged triangular loops.

Step-by-step mathematical example: autoregressive generation under causal masking. Now consider the Decoder generating the Italian translation prefix of length $L_{\text{dec}} = 3$:

$$Y = [y_1 = \langle \text{BOS} \rangle,\ y_2 = \text{“riva”},\ y_3 = \text{“del”}]. \tag{5.47}$$

All queries, keys, and values are projected from the decoder's target token embeddings ($d_k = d_v = 2$):

$$\mathbf{Q} = \begin{bmatrix} 0.5 & 0.5 \\ 1.2 & 0.3 \\ 0.8 & 1.0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.6 & 0.4 \\ 1.0 & 0.2 \\ 0.7 & 0.9 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 0.2 & 0.5 \\ 1.8 & 0.4 \\ 0.6 & 1.4 \end{bmatrix}. \tag{5.48}$$

Let us trace the computation step by step:

1. **Raw scaled dot products (unmasked)**: Computing $\mathbf{S} = \frac{\mathbf{QK}^\top}{\sqrt{2}}$:

$$\mathbf{S} = \frac{1}{\sqrt{2}} \begin{bmatrix} 0.50 & 0.60 & 0.80 \\ 0.84 & 1.26 & 1.11 \\ 0.88 & 1.00 & 1.46 \end{bmatrix} \approx \begin{bmatrix} 0.354 & 0.424 & 0.566 \\ 0.594 & 0.891 & 0.785 \\ 0.622 & 0.707 & 1.032 \end{bmatrix}. \tag{5.49}$$

   Notice that unmasked score $S_{13} = 0.566$ is the highest in the first row: without masking, ⟨BOS⟩ would strongly attend to future token *“del”*!

2. **Applying the causal mask M**: We add $\mathbf{M}$ to $\mathbf{S}$:

$$\mathbf{S}_{\text{masked}} = \mathbf{S} + \mathbf{M} = \begin{bmatrix} 0.354 & -\infty & -\infty \\ 0.594 & 0.891 & -\infty \\ 0.622 & 0.707 & 1.032 \end{bmatrix}. \tag{5.50}$$

3. **Row-wise softmax under masking**:

   - **Row 1 ($y_1 = \langle \text{BOS} \rangle$)**: $\exp(0.354) \approx 1.425$, while $\exp(-\infty) = 0$:

$$\alpha_{11} = \frac{1.425}{1.425 + 0 + 0} = 1.000, \quad \alpha_{12} = 0.000, \quad \alpha_{13} = 0.000. \tag{5.51}$$

     Position 1 can *only* attend to itself!

- **Row 2 (y_2 = "riva")**: $\exp(0.594) \approx 1.811$, $\exp(0.891) \approx 2.438$, $\exp(-\infty) = 0$:

$$Z_2 = 1.811 + 2.438 = 4.249, \tag{5.52}$$

$$\alpha_{21} = \frac{1.811}{4.249} \approx 0.426, \quad \alpha_{22} = \frac{2.438}{4.249} \approx 0.574, \quad \alpha_{23} = 0.000. \tag{5.53}$$

  Position 2 attends strictly to the prefix $\{y_1, y_2\}$; future token y_3 is zeroed out.

- **Row 3 (y_3 = "del")**: $\exp(0.622) \approx 1.863$, $\exp(0.707) \approx 2.028$, $\exp(1.032) \approx 2.807$:

$$Z_3 = 1.863 + 2.028 + 2.807 = 6.698 \implies \alpha_{31} \approx 0.278, \quad \alpha_{32} \approx 0.303, \quad \alpha_{33} \approx 0.419. \tag{5.54}$$

This produces the strictly lower-triangular attention matrix:

$$\mathbf{A}_{\text{dec}} = \begin{bmatrix} 1.000 & 0.000 & 0.000 \\ 0.426 & 0.574 & 0.000 \\ 0.278 & 0.303 & 0.419 \end{bmatrix}. \tag{5.55}$$

4. **Causal value aggregation**: Multiplying by $\mathbf{V} \in \mathbb{R}^{3\times 2}$:

$$\mathbf{O}_{\text{dec}} = \mathbf{A}_{\text{dec}}\mathbf{V} = \begin{bmatrix} 1.000\mathbf{v}_1 \\ 0.426\mathbf{v}_1 + 0.574\mathbf{v}_2 \\ 0.278\mathbf{v}_1 + 0.303\mathbf{v}_2 + 0.419\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.200 & 0.500 \\ 1.118 & 0.443 \\ 0.852 & 0.847 \end{bmatrix}. \tag{5.56}$$

Key architectural takeaway. Contrast this directly with Encoder self-attention: despite $S_{13} = 0.566$ having high compatibility, the causal mask completely extinguishes its influence ($\alpha_{13} = 0$). Output $\mathbf{o}_1$ is computed strictly from $\mathbf{v}_1$, preserving the causality required for autoregressive sequence modeling.

5.5.3 Encoder-decoder cross-attention: dynamic source grounding

Why queries originate from the decoder while keys and values originate from the encoder. In sequence-to-sequence translation, the decoder is generating tokens in the target language, while the encoder holds the semantic context of the source sentence. Recall the fundamental roles of the attention triad:

- A **Query** asks: *"What information am I currently searching for to generate my next word?"* Because the entity generating the next word is the decoder, the queries $\mathbf{Q}$ must be projected from the decoder's current representation.
- A **Key** advertises: *"What features do I describe in the source sentence?"*
- A **Value** provides: *"What source meaning should be retrieved?"*

Because the source content is held exclusively in the encoder, the keys $\mathbf{K}$ and values $\mathbf{V}$ must originate from the encoder's output representations. If queries were taken from the encoder instead, the source sentence would be searching the target words, which completely contradicts the directional objective of translating source meaning into target text. By projecting queries from the decoder and keys/values from the encoder, cross-attention acts as a dynamic search engine allowing each generated target token to query the entire uncompressed source sentence.

In cross-attention, the model bridges the target sequence with the source sequence. The queries originate from the decoder's intermediate representations $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{L_{\text{dec}}\times d_{\text{model}}}$, while keys and values originate from the uncompressed output of the encoder stack $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{L_{\text{enc}}\times d_{\text{model}}}$:

$$\mathbf{Q} = \mathbf{H}_{\text{dec}}\mathbf{W}_Q^{\text{cross}} \in \mathbb{R}^{L_{\text{dec}}\times d_k}, \quad \mathbf{K} = \mathbf{H}_{\text{enc}}\mathbf{W}_K^{\text{cross}} \in \mathbb{R}^{L_{\text{enc}}\times d_k}, \quad \mathbf{V} = \mathbf{H}_{\text{enc}}\mathbf{W}_V^{\text{cross}} \in \mathbb{R}^{L_{\text{enc}}\times d_v}. \tag{5.57}$$

Crucially:

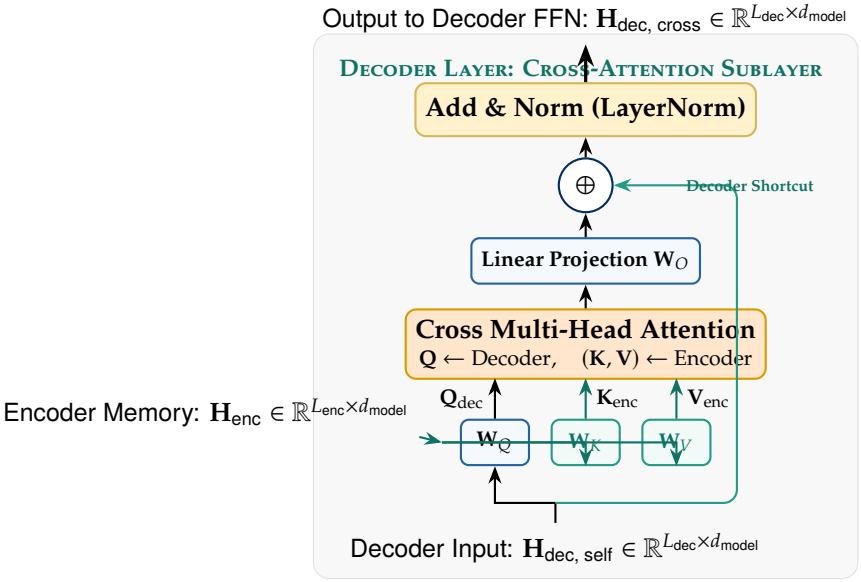


Figure 5.7: Architectural schematic of the Encoder-Decoder Cross-Attention sublayer [1]. The decoder generates Query vectors $\mathbf{Q}$ from its self-attended states, while Key $\mathbf{K}$ and Value $\mathbf{V}$ vectors are projected from the final encoder representations, allowing the decoder to dynamically attend across the entire source sequence.

- **Asymmetric sequence lengths**: The number of queries L_{dec} does not need to equal the number of keys/values L_{enc}. In translation, sentences in different languages rarely have identical token counts.
- **No causal mask on the source**: The source sentence was fully processed by the encoder before decoding began. Thus, every decoder query position can inspect *all* L_{enc} source tokens ($\mathbf{M} = \mathbf{0}$).

Step-by-step mathematical example: cross-lingual alignment with asymmetric lengths. Suppose our decoder has generated a target prefix of length $L_{\text{dec}} = 2$ ([⟨BOS⟩, "riva"]), and cross-attends to our earlier English source sentence of length $L_{\text{enc}} = 3$ (["bank", "of", "river"]). The decoder provides $L_{\text{dec}} = 2$ queries $\mathbf{Q} \in \mathbb{R}^{2\times 2}$ (taken from the first two rows of (5.48)), while the encoder provides $L_{\text{enc}} = 3$ keys and values $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{3\times 2}$ (from (5.35)):

$$\mathbf{Q} = \begin{bmatrix} 0.5 & 0.5 \\ 1.2 & 0.3 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.8 & 0.4 \\ 0.1 & 0.9 \\ 1.0 & 1.1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1.5 & 0.2 \\ 0.1 & 0.8 \\ 0.4 & 2.0 \end{bmatrix}. \tag{5.58}$$

Let us trace the rectangular cross-attention computation step by step:

1. **Rectangular dot-product matrix ($L_{\text{dec}} \times L_{\text{enc}} = 2 \times 3$):**

$$\mathbf{QK}^\top = \begin{bmatrix} (0.5)(0.8) + (0.5)(0.4) & (0.5)(0.1) + (0.5)(0.9) & (0.5)(1.0) + (0.5)(1.1) \\ (1.2)(0.8) + (0.3)(0.4) & (1.2)(0.1) + (0.3)(0.9) & (1.2)(1.0) + (0.3)(1.1) \end{bmatrix}. \tag{5.59}$$

   Evaluating the pairwise inner products yields:

$$\mathbf{QK}^\top = \begin{bmatrix} 0.60 & 0.50 & 1.05 \\ 1.08 & 0.39 & 1.53 \end{bmatrix}. \tag{5.60}$$

2. **Scaling by $\sqrt{d_k} = \sqrt{2}$:**

$$\mathbf{S}_{\text{cross}} = \frac{\mathbf{QK}^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.424 & 0.354 & 0.742 \\ 0.764 & 0.276 & 1.082 \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{5.61}$$

3. **Row-wise softmax normalization (over $L_{\mathbf{enc}} = 3$ source keys)**:

   - **Row 1 ($\mathbf{q}_{\langle \mathbf{BOS} \rangle}$)**: $\exp(0.424) \approx 1.528$, $\exp(0.354) \approx 1.425$, $\exp(0.742) \approx 2.100$:

   $$Z_1 = 1.528 + 1.425 + 2.100 = 5.053 \implies \mathbf{A}_{1,:} \approx [0.302,\ 0.282,\ 0.416]. \tag{5.62}$$

   - **Row 2 ($\mathbf{q}_{\text{“riva”}}$)**: $\exp(0.764) \approx 2.147$, $\exp(0.276) \approx 1.318$, $\exp(1.082) \approx 2.951$:

   $$Z_2 = 2.147 + 1.318 + 2.951 = 6.416 \implies \mathbf{A}_{2,:} \approx [0.335,\ 0.205,\ 0.460]. \tag{5.63}$$

   This yields the rectangular cross-attention matrix:

   $$\mathbf{A}_{\text{cross}} = \begin{bmatrix} 0.302 & 0.282 & 0.416 \\ 0.335 & 0.205 & 0.460 \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{5.64}$$

4. **Cross-attention context retrieval**: Multiplying $\mathbf{A}_{\text{cross}} \in \mathbb{R}^{2\times 3}$ by encoder values $\mathbf{V} \in \mathbb{R}^{3\times 2}$ yields the grounded representations $\mathbf{O}_{\text{cross}} \in \mathbb{R}^{2\times 2}$:

$$\mathbf{O}_{\text{cross}} = \mathbf{A}_{\text{cross}}\mathbf{V} = \begin{bmatrix} 0.302\mathbf{v}_1 + 0.282\mathbf{v}_2 + 0.416\mathbf{v}_3 \\ 0.335\mathbf{v}_1 + 0.205\mathbf{v}_2 + 0.460\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.648 & 1.117 \\ 0.706 & 1.151 \end{bmatrix}. \tag{5.65}$$

Key architectural takeaway. Query $\mathbf{q}_{\text{“riva”}}$ strongly aligns with source key $\mathbf{k}_{\text{“river”}}$ ($\alpha_{23} = 0.460$) and source key $\mathbf{k}_{\text{“bank”}}$ ($\alpha_{21} = 0.335$). The decoder directly pulls the exact semantic coordinates of the geological shore from the source representation without compressing the source into a single bottleneck vector!

5.6 Multi-head attention (MHA)

The limitation of single-head attention. In single-head attention, the model computes a single set of attention weights $\alpha_{i,j}$, averaging all values into a single weighted sum. However, a single token often participates in multiple simultaneous linguistic relationships:

- A verb must track both its grammatical subject (often located earlier in the sentence) and its direct object (often located downstream).
- In programming languages, an identifier must link to its type declaration, its scope boundary, and its mutating assignments.

A single attention head cannot focus simultaneously on disparate positions with different semantic intents: the softmax distribution is forced to average across them, diluting specific relational signals.

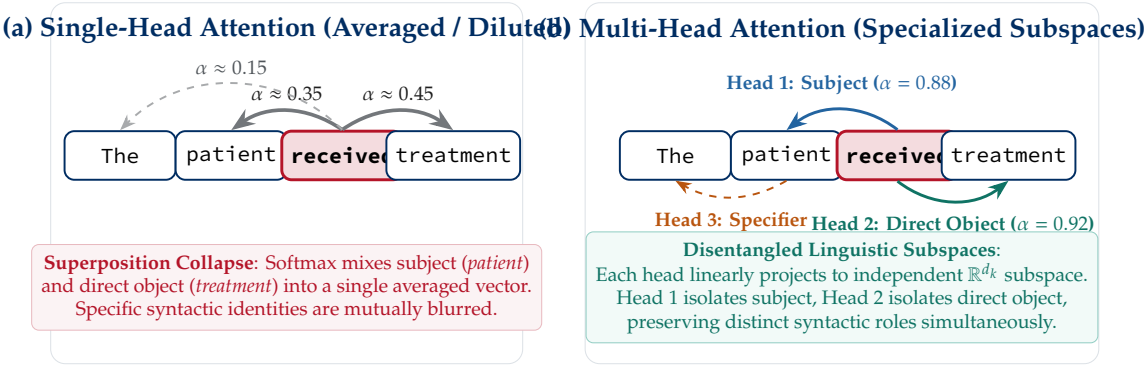


Figure 5.8: Linguistic relationship modeling: Single-Head vs. Multi-Head Attention. In Single-Head Attention (a), a token involved in multiple syntactic dependencies must compromise its single softmax distribution, averaging disparate features and diluting specific relational roles. In Multi-Head Attention (b), projection into h distinct representation subspaces enables separate heads to concurrently isolate orthogonal linguistic relationships (such as grammatical subject tracking and direct object binding) without mutual interference.

Mathematical formulation. Multi-Head Attention (MHA) resolves this by linearly projecting queries, keys, and values h times with different, learned parameter matrices:

Definition 5.2: *Multi-head attention*

Let h denote the number of attention heads. Multi-Head Attention projects $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into h distinct representation subspaces of dimension $d_k = d_{\text{model}}/h$ and $d_v = d_{\text{model}}/h$:

$$\texttt{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1 \parallel \text{head}_2 \parallel \cdots \parallel \text{head}_h]\,\mathbf{W}^O, \tag{5.66}$$

where each individual head is evaluated via:

$$\text{head}_i = \texttt{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), \quad i \in \{1, 2, \ldots, h\}, \tag{5.67}$$

with parameter projection matrices $\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and output projection matrix $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

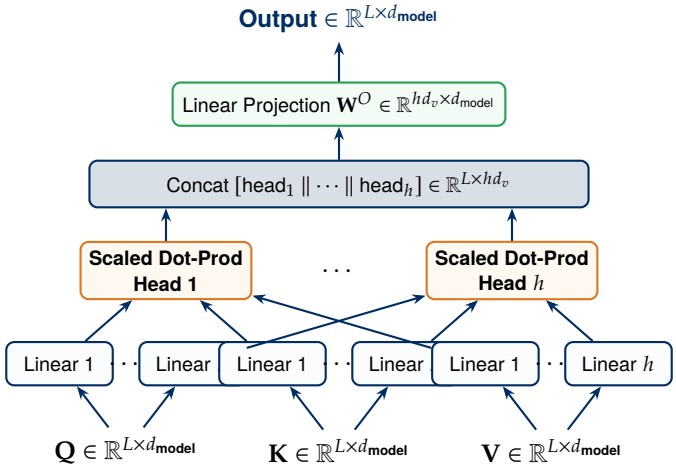


Figure 5.9: Architecture of Multi-Head Attention (MHA) [1]. Projections map inputs into h parallel representation subspaces of reduced dimension $d_k = d_v = d_{\text{model}}/h$, whose outputs are concatenated and linearly projected back to d_{model}.

Parameter equivalence and computational efficiency. Due to the reduced dimension of each head ($d_k = d_v = d_{\text{model}}/h$), the total computational cost of multi-head attention is identical to that of single-head attention with full dimensionality d_{model}:

$$h \times \left(d_{\text{model}} \times \frac{d_{\text{model}}}{h} \right) = d_{\text{model}}^2 \text{ operations for projections.} \tag{5.68}$$

In standard configurations (e.g., $d_{\text{model}} = 512, h = 8 \implies d_k = d_v = 64$, or $d_{\text{model}} = 768, h = 12 \implies d_k = d_v = 64$), parallel heads learn specialized sub-behaviors: some heads attend to local syntactic structure, others to long-range dependencies, and others to delimiter tokens.

5.7 Sublayer mechanics: residual connections and Layer Normalization

In deep Transformer architectures comprising dozens of layers, stacking attention blocks and non-linear feed-forward networks directly would lead to optimization failure due to exploding or vanishing gradients. To ensure stable gradient propagation, each Transformer sublayer is wrapped in a residual connection followed by Layer Normalization:

$$\mathbf{y} = \texttt{LayerNorm}(\mathbf{x} + \texttt{Sublayer}(\mathbf{x})). \tag{5.69}$$

1. Residual skip connections (gradient superhighways). First introduced in [58] for deep residual networks (ResNets), identity shortcut connections pass the unmodified input $\mathbf{x}$ around the sublayer (Figure 5.10):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}), \tag{5.70}$$

where $\mathcal{F}(\mathbf{x})$ represents the multi-head attention or feed-forward transformation.

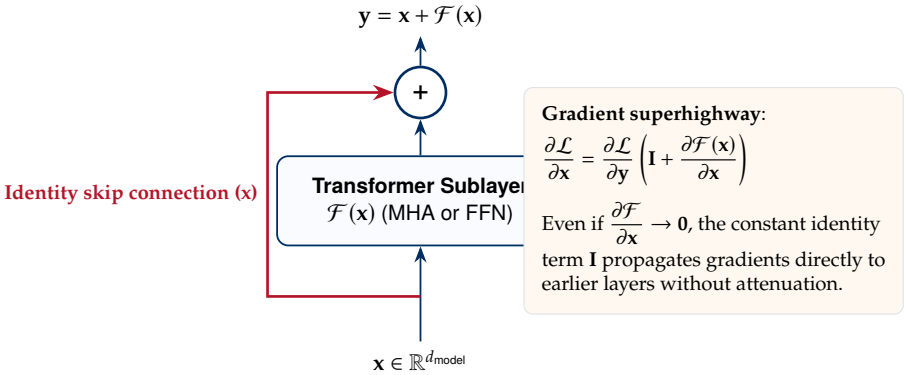


Figure 5.10: Residual skip connection and gradient propagation dynamics. The input $\mathbf{x}$ bypasses the non-linear sublayer $\mathcal{F}(\mathbf{x})$ via an identity shortcut. During backpropagation, the derivative $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + \frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ ensures that the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ flows unimpeded across arbitrary layer depths via the identity term $\mathbf{I}$.

During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to input $\mathbf{x}$ expands via the chain rule to:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\mathbf{I} + \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}. \tag{5.71}$$

The presence of the identity matrix $\mathbf{I}$ guarantees an unobstructed **gradient superhighway**: even if the Jacobian $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanishes (for instance, when weights are initialized small or activations saturate), term $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ flows directly back to earlier layers in a single step, completely preventing gradient extinction.

2. Layer Normalization. Unlike Batch Normalization (which computes statistics across the mini-batch dimension, making it sensitive to batch size and variable sequence lengths), **Layer Normalization** [59] computes normalization statistics across the feature dimensions for each individual token independently (Figure 5.11):

Definition 5.3: *Layer Normalization*

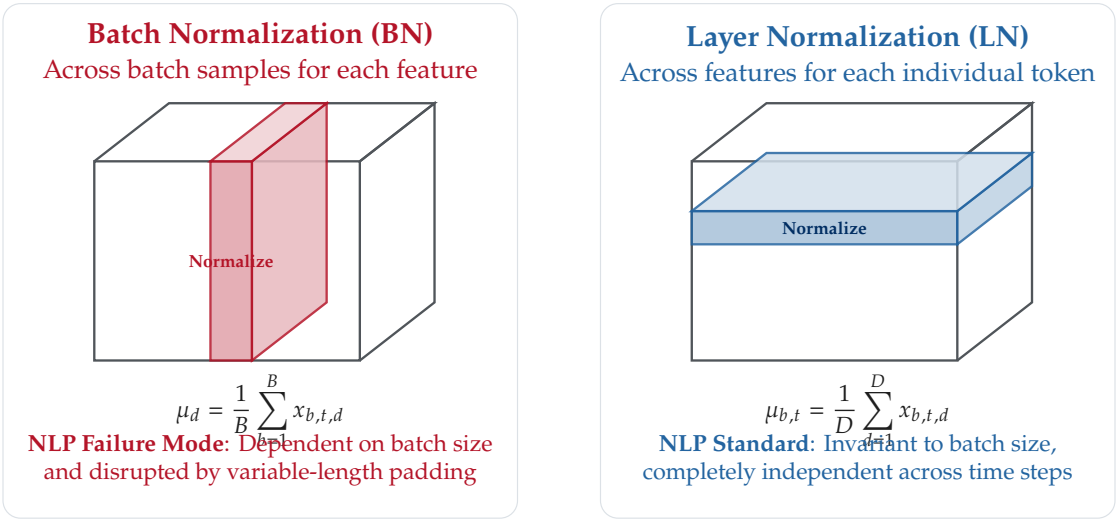


Figure 5.11: Comparison of normalization geometries in 3D activation tensor space $\mathcal{X} \in \mathbb{R}^{B\times T\times D}$. (Left) Batch Normalization computes statistics across all batch instances B for each feature channel d, creating inter-sample dependencies and breaking down under variable sequence lengths and sequence padding. (Right) Layer Normalization computes statistics across all feature channels D for each token vector $\mathbf{x}_{b,t}$ independently, ensuring strict batch-size invariance and identical execution during training and autoregressive inference.

For a token representation $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top \in \mathbb{R}^d$, Layer Normalization evaluates:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2, \tag{5.72}$$

$$\texttt{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \tag{5.73}$$

where $\epsilon > 0$ is a small numerical stabilization constant, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ are learnable affine gain and bias parameters initialized to $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$.

Because Layer Normalization operates on a per-token basis without batch-level pooling, its behavior is identical during training and evaluation, providing exceptional stability across variable-length sequences.

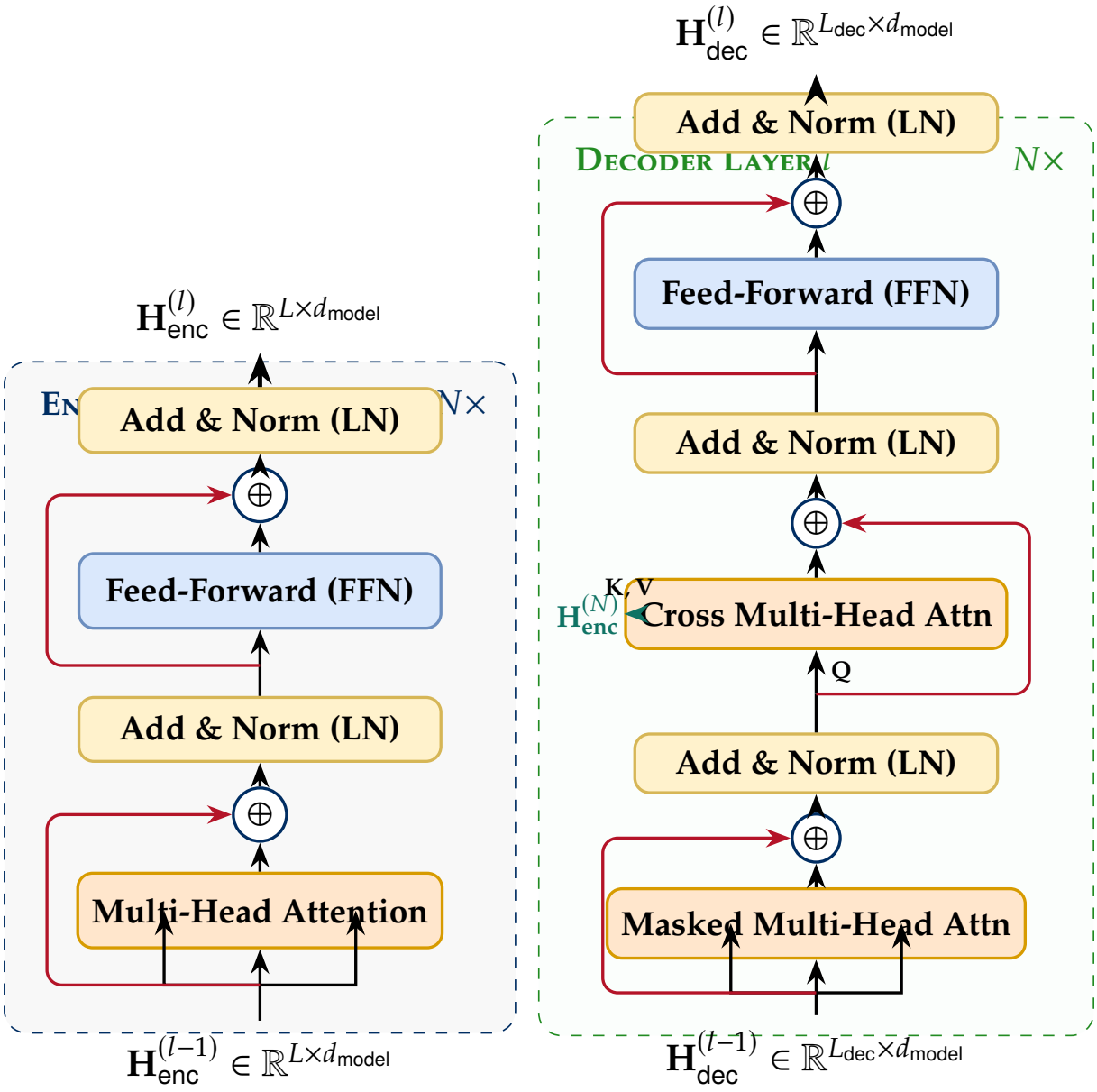
3. Position-wise Feed-Forward Networks (FFN). In addition to attention sublayers, each layer in the encoder and decoder contains a fully connected **Feed-Forward Network** applied to each position separately and identically:

$$\texttt{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2, \tag{5.74}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}}\times d_{\text{ff}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d_{\text{model}}}$, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$. Typically, the inner hidden dimension is expanded by a factor of 4 ($d_{\text{ff}} = 4d_{\text{model}}$, e.g., $d_{\text{ff}} = 2048$ for $d_{\text{model}} = 512$). While attention routes information across different temporal positions, the FFN provides non-linear feature transformation within each individual position.

5.7.1 End-to-end sublayer integration: the complete mathematical pipeline

Having analyzed Multi-Head Attention, residual skip connections, Layer Normalization, and Feed-Forward Networks individually, we now formalize how these mathematical operators chain together within complete Transformer layers, tracing their exact locations in the canonical architecture (Figure 5.12).



(a) Encoder layer stack ($N\times$)

(b) Decoder layer stack ($N\times$)

Figure 5.12: End-to-end sublayer connectivity within the canonical Transformer architecture [1]. Every attention and feed-forward block is coupled with an identity residual shortcut and an "Add & Norm" operator.

1. Complete Encoder layer pipeline. An encoder layer $l \in \{1, \ldots, N\}$ receives tensor $\mathbf{H}_{\text{enc}}^{(l-1)} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ (with $\mathbf{H}_{\text{enc}}^{(0)} = \mathbf{X}_{\text{enc}}$) and applies two composite sublayers:

1. **Sublayer 1 (Bidirectional Multi-Head Self-Attention)**:

$$\mathbf{Z}^{(l)} = \texttt{MultiHead}\left(\mathbf{H}_{\text{enc}}^{(l-1)}, \mathbf{H}_{\text{enc}}^{(l-1)}, \mathbf{H}_{\text{enc}}^{(l-1)}\right), \tag{5.75}$$

$$\tilde{\mathbf{H}}_{\text{enc}}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{enc}}^{(l-1)} + \mathbf{Z}^{(l)}\right). \tag{5.76}$$

2. **Sublayer 2 (Position-wise Feed-Forward Network)**:

$$\mathbf{F}^{(l)} = \texttt{FFN}\left(\tilde{\mathbf{H}}_{\text{enc}}^{(l)}\right), \tag{5.77}$$

$$\mathbf{H}_{\text{enc}}^{(l)} = \texttt{LayerNorm}\left(\tilde{\mathbf{H}}_{\text{enc}}^{(l)} + \mathbf{F}^{(l)}\right). \tag{5.78}$$

2. Complete Decoder layer pipeline. A decoder layer $l \in \{1, \ldots, N\}$ receives target representation $\mathbf{H}_{\text{dec}}^{(l-1)} \in \mathbb{R}^{L_{\text{dec}} \times d_{\text{model}}}$ along with the terminal encoder output $\mathbf{H}_{\text{enc}}^{(N)} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$, executing three consecutive sublayers:

1. **Sublayer 1 (Causal Masked Multi-Head Self-Attention)**:

$$\mathbf{Z}_{\text{self}}^{(l)} = \texttt{MaskedMultiHead}\left(\mathbf{H}_{\text{dec}}^{(l-1)}, \mathbf{H}_{\text{dec}}^{(l-1)}, \mathbf{H}_{\text{dec}}^{(l-1)}\right), \tag{5.79}$$

$$\mathbf{H}_{\text{dec},1}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec}}^{(l-1)} + \mathbf{Z}_{\text{self}}^{(l)}\right). \tag{5.80}$$

2. **Sublayer 2 (Encoder-Decoder Multi-Head Cross-Attention)**:

$$\mathbf{Z}_{\text{cross}}^{(l)} = \texttt{MultiHead}\left(\mathbf{H}_{\text{dec},1}^{(l)}, \mathbf{H}_{\text{enc}}^{(N)}, \mathbf{H}_{\text{enc}}^{(N)}\right), \tag{5.81}$$

$$\mathbf{H}_{\text{dec},2}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec},1}^{(l)} + \mathbf{Z}_{\text{cross}}^{(l)}\right). \tag{5.82}$$

   Here, queries are projected from $\mathbf{H}_{\text{dec},1}^{(l)}$, while keys and values are projected from $\mathbf{H}_{\text{enc}}^{(N)}$.

3. **Sublayer 3 (Position-wise Feed-Forward Network)**:

$$\mathbf{F}_{\text{dec}}^{(l)} = \texttt{FFN}\left(\mathbf{H}_{\text{dec},2}^{(l)}\right), \tag{5.83}$$

$$\mathbf{H}_{\text{dec}}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec},2}^{(l)} + \mathbf{F}_{\text{dec}}^{(l)}\right). \tag{5.84}$$

Post-LN vs. Pre-LN parameter stabilization. The formulation above represents the original **Post-LN** architecture of [1], where normalization is placed on the residual sum: $\mathbf{y} = \texttt{LN}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$. While mathematically elegant, deep Post-LN models require delicate learning rate warm-up schedules because gradients passing through the unnormalized residual additions can destabilize early training. Modern implementations (such as GPT-2, LLaMA, and Mistral) predominantly adopt the **Pre-LN** formulation:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\left(\texttt{LayerNorm}(\mathbf{x})\right), \tag{5.85}$$

which preserves an unbroken, completely unnormalized identity shortcut throughout the entire depth of the network, enabling stable optimization without learning rate warm-up.

5.8 Relative positional representations

In Chapter 3, we introduced absolute positional encodings—both fixed sinusoidal functions [1] and learned absolute embedding vectors. Under absolute encoding, each index in the sequence ($pos = 0, 1, 2, \dots$) is assigned a dedicated vector added directly to the token embedding before entering the Transformer.

Motivation for relative representations. In natural language and software engineering source code, the precise absolute coordinate of a token is frequently less important than its **relative displacement** with respect to other tokens. For example, whether a verb appears at index 12 or index 84, its grammatical subject is typically found 2 or 3 tokens earlier. To incorporate this inductive bias directly into the attention mechanism, Shaw et al. [54] formulated **Relative Positional Representations**.

Mathematical formulation. Instead of adding absolute vectors to the input, relative positional embeddings inject pairwise distance information directly into the scaled dot-product attention logits :

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top + \mathbf{S}_{\text{rel}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{5.86}$$

where $\mathbf{S}_{\text{rel}} \in \mathbb{R}^{L\times L}$ is a relative position bias matrix. For query position i and key position j, the entry is defined by:

$$S_{\text{rel}}[i, j] = \mathbf{q}_i^\top \mathbf{a}^K_{\text{clip}(i-j,k)}, \tag{5.87}$$

where $\mathbf{a}^K$ is a learned embedding representing the relative offset $i - j$, and $\text{clip}(x, k) = \max(-k, \min(k, x))$ truncates relative distances beyond a maximum clipping horizon k.

Asymmetry of relative positional embeddings: self-attention vs. cross-attention. Relative positional embeddings make sense exclusively for self-attention, and cannot be naturally defined for cross-attention:

- **Self-attention**: Both queries and keys originate from the same sequence. The scalar distance $i - j$ represents a physical, well-defined offset within that sequence (e.g., token j occurs 2 steps before token i).
- **Cross-attention**: Queries stem from the target sequence (e.g., generated English words in machine translation), while keys stem from the source sequence (e.g., input Italian words). Because source and target sentences possess different lengths, word orders, and grammatical alignments, there is no shared coordinate frame: asking whether an English target word is "three positions ahead" of an Italian source word is mathematically meaningless. Consequently, cross-attention cannot leverage relative positional biases.

5.9 Parallelization advantages and computational complexity

To appreciate why the Transformer superseded Recurrent Neural Networks, we analyze three foundational computational metrics: computational complexity per layer, sequential operations, and maximum signal path length [1], summarized in Table 5.1.

Table 5.1: Computational complexity and operational parallelism across sequence modeling layers [1].

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-Attention	$O(L^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN/LSTM)	$O(L \cdot d^2)$	$O(L)$	$O(L)$
Convolutional	$O(k \cdot L \cdot d^2)$	$O(1)$	$O(\log_k(L))$

1. Sequential operations: shattering the recurrence barrier. In an RNN, computing the representation at step L requires traversing an unbroken chain of L sequential state updates ($O(L)$ sequential operations). In Self-Attention, computing all pairwise dot products $\mathbf{QK}^\top$ and value weighted sums involves dense matrix multiplications that execute in a single parallel step ($O(1)$ sequential operations) across GPU tensor cores.

2. Maximum path length: constant-time gradient propagation. In recurrent networks, information and backpropagation gradients traveling between token 1 and token L must traverse L intermediate state transitions. As proven by the Jacobian product $\prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$ in Chapter 3, long path lengths cause exponential gradient decay. In Self-Attention, the distance between any arbitrary pair of tokens (i, j) is strictly $O(1)$: the attention weight $\alpha_{i,j}$ directly links token i to token j in a single operation. This constant path length completely eliminates topological distance as an obstacle to learning long-range dependencies.

3. Asymptotic trade-off: sequence length vs. latent dimension. Self-attention incurs a quadratic time and memory complexity $O(L^2 \cdot d)$ with respect to sequence length L. When sequence length L is smaller than embedding dimension d (as was typical in early machine translation benchmarks where $L \approx 50$ and $d = 512$), $O(L^2 \cdot d)$ is strictly faster than the recurrent cost $O(L \cdot d^2)$. While long context windows ($L > 8192$) eventually encounter the $O(L^2)$ memory wall, the massive parallel speedup during training outweighs this cost for standard operational contexts.

5.10 The full Encoder-Decoder paradigm: T5

Following the introduction of the unified Encoder-Decoder Transformer in [1], the machine learning community recognized that many tasks do not require both components simultaneously. Maintaining both stacks incurs unnecessary parameter bloat and memory overhead when the target problem involves either pure sequence transduction with distinct input and output vocabularies, or pure feature representation and bidirectional sequence understanding. Consequently, the architecture bifurcated into specialized paradigms: full Encoder-Decoder architectures and pure Encoder-Only architectures. (We will address pure autoregressive Decoder-Only architectures in the subsequent chapter dedicated to large-scale autoregressive pre-training and scaling laws).

5.10.1 The unified text-to-text framework

The full Encoder-Decoder architecture was revitalized by Raffel et al. [60] through the **Text-to-Text Transfer Transformer (T5)** (Figure 5.13).

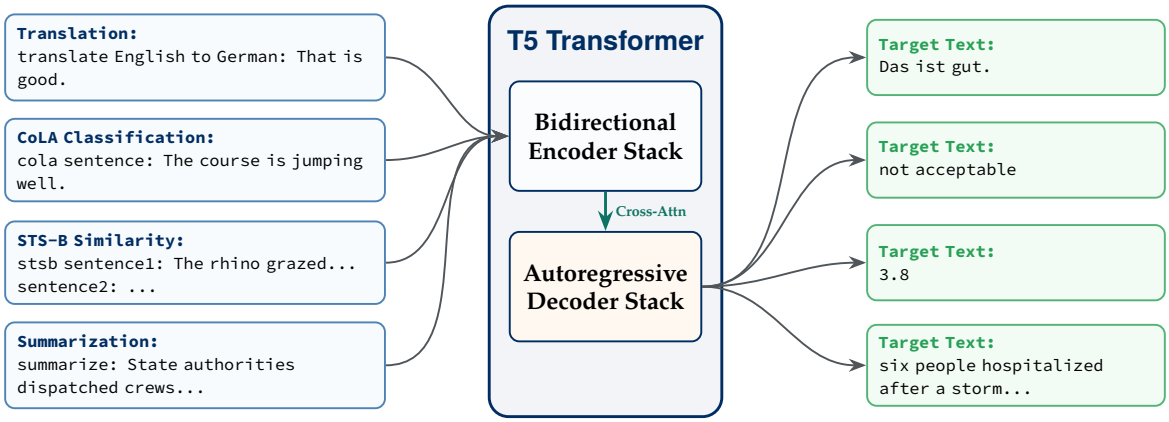


Figure 5.13: The T5 unified text-to-text framework [60]. Every natural language problem—including machine translation, linguistic acceptability (CoLA), sentence similarity scoring (STS-B), and document summarization—is framed as receiving a task-conditioned text prompt with a fixed prefix and autoregressively generating a target text string.

1. The unified text-to-text framing. Rather than designing separate network architectures for different NLP tasks (e.g., classification heads for sentiment analysis, sequence taggers for named entity recognition, and autoregressive decoders for summarization), T5 casts **every** problem into an identical format: text-to-text sequence generation. The input string encodes both the task directive and the input data via a prepended **task prefix**:

- **Machine translation**: `"translate English to German: That is good."` ⟶ Output: `"Das ist gut."`
- **Summarization**: `"summarize: State authorities dispatched emergency crews..."` ⟶ Output: `"Crews responded to flood alerts."`
- **Linguistic acceptability (CoLA)**: `"cola sentence: The course is great."` ⟶ Output: `"acceptable"`

2. Critical distinction: task conditioning vs. instruction tuning. While T5's prefix-based behavior superficially resembles modern conversational assistants, **T5 is not instruction-tuned**.

T5 was trained strictly on fixed, rigid task prefixes. The model learns to recognize these specific keywords and route its internal representations accordingly. However, it exhibits no zero-shot generalization to unseen instructions or altered formulations:

- A slightly verbose variation, such as `"Can you translate English to German the following sentence: That is good"`, causes the model to break down and emit ungrammatical gibberish.
- Prompts requesting novel reasoning, such as `"Compute 2 + 2"`, fail completely because arithmetic was not included in its pre-training task templates.

5.10.2 Trade-offs of full encoder-decoder architectures

While the unified encoder-decoder architecture provides exceptional flexibility for arbitrary sequence transduction, maintaining both an encoder and an autoregressive decoder introduces significant computational and representational overhead when the downstream task does not involve sequence generation. In sequence classification, token tagging, or representation retrieval, running an autoregressive decoder step-by-step is completely wasteful. This motivates isolating the encoder stack into a standalone, deeply bidirectional architecture.

5.11 The Encoder-Only paradigm: BERT (Bidirectional Encoder Representations from Transformers)

When the objective is sequence understanding, document classification, or semantic code search rather than autoregressive text generation, the decoder stack is completely superfluous. Devlin et al. [53] isolated the bidirectional encoder stack to create **BERT** (Figure 5.14).

To master BERT from first principles, it is essential to systematically decompose and analyze each of its core architectural and training components:

1. **Input representation**: Why we must analyze the tripartite embedding construction (Token, Segment, Position) and special tokens (`[CLS]`, `[SEP]`), which enables a single unified Transformer stack to ingest either isolated sentences or structured sentence pairs without architectural modifications.
2. **Masked Language Model (MLM)**: Why we must analyze bidirectional Cloze pre-training, resolving the fundamental paradox that standard autoregressive next-token prediction causes trivial label leakage in bidirectional self-attention networks.
3. **Next Sentence Prediction (NSP)**: Why intra-sentence bidirectional token conditioning alone is insufficient for discourse-level relational reasoning, requiring explicit cross-sentence relationship pre-training.
4. **Attention map anatomy and emergent linguistic patterns**: Why we must inspect the empirical behavior across the 144 attention heads, discovering how self-attention spontaneously organizes into syntactic dependency trackers, coreference resolvers, and delimiter registers without supervised parse trees.
5. **Downstream fine-tuning mechanisms**: Why we must formalize the adaptation mechanics (sequence-level pooling vs. token-level classification heads), demonstrating how pre-trained bidirectional representations transfer to diverse task topologies with minimal task-specific parameters.

5.11.1 Input representation and special tokens

Motivation for analyzing input representation. A central challenge in designing a universal sequence understanding backbone is handling diverse downstream input topologies: single text sequences (e.g., sentiment analysis, spam detection), paired sequences with asymmetric roles (e.g., Question Answering premise-hypothesis pairs), and span-level entities. Rather than designing disparate input pipelines for each task type, BERT unifies all inputs into a single continuous stream. Analyzing this construction reveals how lexical identity, sentence membership, and ordinal sequence position are simultaneously communicated to permutation-invariant self-attention layers.

BERT supports pairs of sentences (A, B) packed into a single sequence (Figure 5.15). Every input token representation is constructed by summing three distinct embedding vectors:

$$\mathbf{x}_i = \texttt{TokenEmbedding}(w_i) + \texttt{SegmentEmbedding}(s_i) + \texttt{PositionEmbedding}(i), \tag{5.88}$$

where:

- **Token embeddings**: Subword tokens derived from WordPiece tokenization (with vocabulary cardinality $|\mathcal{V}| = 30{,}522$).
- **Segment embeddings**: Distinguishes whether the token belongs to Sentence A (E_A) or Sentence B (E_B). This allows self-attention queries to identify cross-sentence relationships and apply sentence-specific relational transformations.

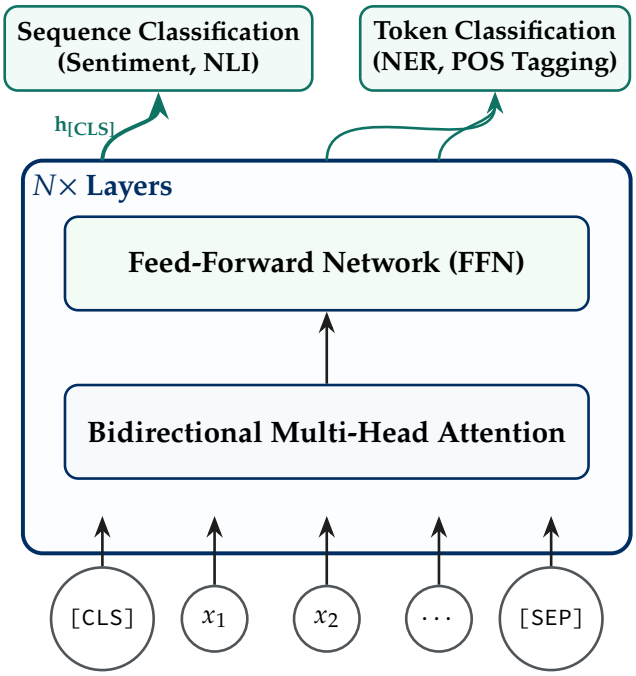


Figure 5.14: The Encoder-only architecture (BERT) [53]. The autoregressive decoder is discarded. Stacked bidirectional Transformer encoder layers process the full context simultaneously. Downstream tasks branch directly off the contextual representations: sequence-level tasks utilize the pooled $\mathbf{h}_{[\text{CLS}]}$ representation, while token-level tasks attach classification heads to individual token outputs $\mathbf{h}_i$.

- **Position embeddings**: Learned absolute positional vectors $(E_0, \ldots, E_{511})$ that inject spatial sequence ordering.

BERT introduces two canonical special tokens:

- `[CLS]` (**Classification**): Always prepended as the very first token ($i = 0$). Because self-attention is bidirectional, the final hidden state $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$ aggregates context from the entire sequence and serves as the pooled representation for sentence-level classification.
- `[SEP]` (**Separator**): Appended to mark the boundary between Sentence A and Sentence B, as well as the termination of the sequence.

5.11.2 Pre-training task 1: Masked Language Model (MLM)

Motivation for analyzing the MLM objective. Analyzing the MLM objective is crucial because bidirectional representations cannot be pre-trained using standard autoregressive next-token prediction. In a bidirectional self-attention model, token i can attend to all tokens $j > i$. If tasked with predicting x_{i+1}, the network would simply route attention to the next position and trivial identity-copy the answer, collapsing gradient learning. The MLM task formulates a non-trivial, label-leakage-free self-supervised objective inspired by the Cloze procedure [53].

In each input sequence, a subset $\mathcal{M} \subset \{1, 2, \ldots, L\}$ representing approximately 15% of all input tokens is uniformly selected to provide pre-training supervision. However, how should these selected positions be presented to the network?

The general motivation: resolving the pre-training vs. fine-tuning mismatch. The naive Cloze procedure would simply replace every selected token in $\mathcal{M}$ with the artificial marker

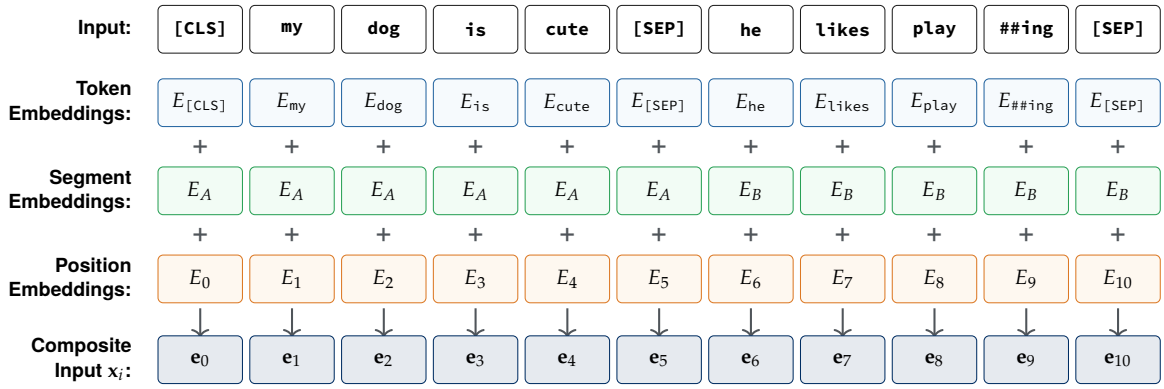


Figure 5.15: BERT input representation structure [53]. Each input vector $\mathbf{e}_i$ is constructed by the element-wise sum of its Token Embedding (WordPiece vocabulary), Segment Embedding (E_A for Sentence A, E_B for Sentence B), and Position Embedding ($E_0, \ldots, E_{10}$). Sequences begin with `[CLS]` and separate sentences with `[SEP]`. Distinct learned segment embeddings allow the self-attention mechanism to discriminate between paired sentences.

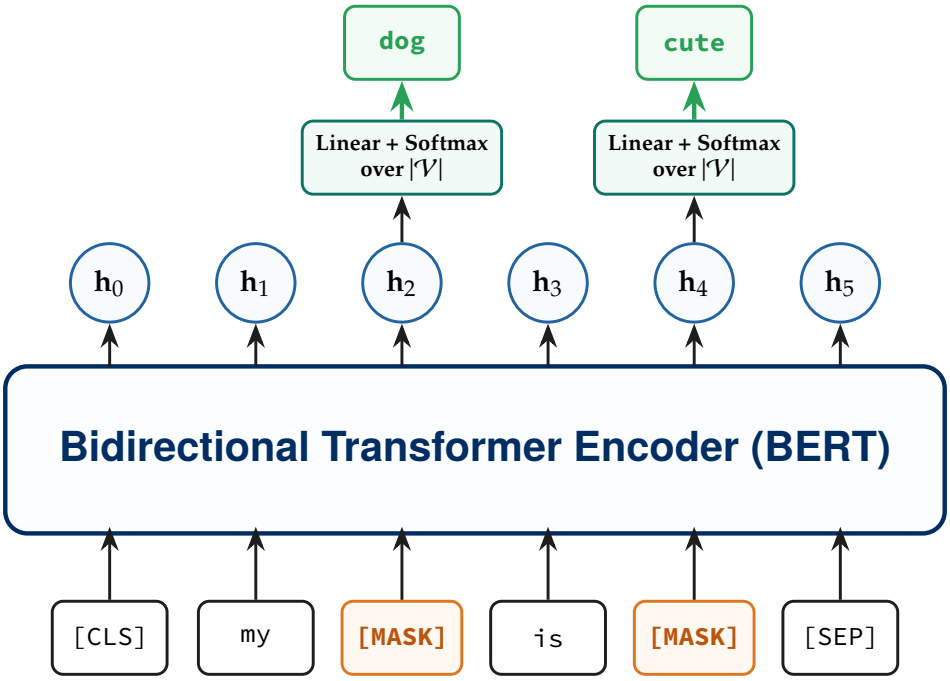


Figure 5.16: The Masked Language Model (MLM) pre-training task in BERT. 15% of input tokens are corrupted using the 80/10/10 masking rule. The model passes corrupted sequences through bidirectional self-attention and trains linear classification heads over the full vocabulary $\mathcal{V}$ specifically on hidden representations $\mathbf{h}_i$ at masked positions to reconstruct the original ground-truth tokens.

[MASK]. While conceptually straightforward, this naive strategy introduces a fatal structural flaw known as the **pre-training vs. fine-tuning discrepancy** (or domain shift):

1. **Absence of [MASK] in downstream tasks**: When BERT is subsequently fine-tuned on real-world applications (e.g., sequence classification, sentiment analysis, named entity recognition, or question answering), the input text consists exclusively of genuine natural language tokens. The artificial delimiter [MASK] *never appears during inference*.

2. **Pathological representation specialization**: If the encoder were exposed to [MASK] whenever a prediction was required, its internal self-attention heads would learn a degenerative heuristic: treat all regular words as unquestioned, static context, and only activate deep predictive feature extraction when encountering the explicit [MASK] token. Consequently, during fine-tuning, the network would fail to generate optimal contextual representations for normal tokens.

3. **Neglect of the observed token**: If target positions are always overwritten with a mask symbol, the model never learns to evaluate or preserve the identity of the token physically occupying position i.

To force the model to build high-quality contextual representations for *all* tokens—while keeping the model uncertain about which positions are being graded—Devlin et al. [53] formulated the **80/10/10 corruption policy**. Once the 15% target positions $\mathcal{M}$ are chosen, they are perturbed according to three distinct operations:

1. **The 80% replacement with [MASK] (Cloze semantic reconstruction)**: In 80% of the selected positions (12% of total sequence tokens), the original token is replaced with [MASK]:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my [MASK] is cute.}$$

   Motivation: This forms the core Cloze engine. Because the ground-truth token is physically removed from the input stream, the model cannot copy it. It is compelled to synthesize bidirectional semantic and syntactic clues from the entire left and right contexts (e.g., using "my" and "is cute" to infer a domestic pet like "dog").

2. **The 10% replacement with a random token (Denoising and contextual validation)**: In 10% of the selected positions (1.5% of total sequence tokens), the target token is replaced with an arbitrary token drawn uniformly from the vocabulary $w_{\text{rand}} \sim \mathcal{V}$:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my apple is cute.}$$

   Motivation: If the model only ever saw [MASK] or correct words, it would assume that every non-masked word in the sentence is guaranteed to be 100% truthful and contextually valid. By injecting random corrupted words, self-attention heads are forced to act as an active **anomaly detector** and **denoiser**. When the encoder encounters "apple" framed by "my" and "is cute", it must detect that "apple" creates a semantic contradiction, discount the corrupted surface form, and reconstruct the true underlying concept ("dog"). This prevents the network from blindly trusting unmasked tokens.

3. **The 10% retention of the original token (Grounding and representation anchoring)**: In the remaining 10% of selected positions (1.5% of total sequence tokens), the token is kept strictly unchanged:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my dog is cute.}$$

   Motivation: If target positions were *always* altered (either masked or replaced with random noise), the encoder would learn that the input symbol at target position i is never the right

answer, incentivizing it to completely discard the token's own embedding. By leaving the correct word intact 10% of the time, the model is taught that the observed word may well be genuine. This forces the final hidden representation $\mathbf{h}_i$ to remain faithfully anchored to the actual lexical token observed at that position, directly bridging the gap to downstream fine-tuning where all input tokens are real.

The global awareness effect. Because of this tri-fold distribution, when BERT computes self-attention across any token x_i, it operates under total uncertainty:

- The network does not know whether token x_i is one of the supervised targets being evaluated by cross-entropy loss.
- If x_i is a target, the network does not know whether it represents an uncorrupted word, an anomalous impostor, or an explicit mask.

This deliberate ambiguity eliminates any possibility of lazy shortcut heuristics, compelling the Transformer encoder to maintain an expressive, contextually verified representation $\mathbf{h}_i$ for **every single token** across the entire sequence.

For each selected position $i \in \mathcal{M}$, the topmost encoder output $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ is projected onto the full vocabulary $\mathcal{V}$ through a linear classification layer parameterized by $\mathbf{W}_{\text{MLM}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ (whose weights are tied with the input token embedding matrix $\mathbf{E}_{\text{tok}}$) and bias $\mathbf{b}_{\text{MLM}} \in \mathbb{R}^{|\mathcal{V}|}$:

$$P(w \mid \tilde{\mathbf{X}}; \theta) = \frac{\exp(\mathbf{w}_w^\top \mathbf{h}_i + b_w)}{\sum_{v \in \mathcal{V}} \exp(\mathbf{w}_v^\top \mathbf{h}_i + b_v)}, \quad \forall w \in \mathcal{V}, \tag{5.89}$$

where $\tilde{\mathbf{X}}$ denotes the corrupted input sequence. The MLM pre-training objective minimizes the cross-entropy loss aggregated across all masked positions:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log P(w_i^* \mid \tilde{\mathbf{X}}; \theta), \tag{5.90}$$

where w_i^* is the original ground-truth token at position i.

5.11.3 Pre-training task 2: Next Sentence Prediction (NSP)

Motivation for analyzing the NSP objective. While the Masked Language Model objective forces the encoder to capture deep intra-sentence lexical and syntactic relationships, many critical downstream tasks—such as Question Answering (QA), Natural Language Inference (NLI), and code-comment retrieval—depend fundamentally on understanding logical coherence across *pairs* of sentences. Analyzing NSP explains how the network develops cross-sentence relational awareness, and motivates why subsequent architectures (e.g., RoBERTa) later audited whether NSP was truly necessary or suboptimal.

During corpus pre-training, input sequences are constructed by sampling pairs of sentences (A, B) concatenated with segment markers (Figure 5.17):

- **50% of the time (Label $y = 1$, IsNext)**: Sentence B is the actual contiguous sentence immediately succeeding Sentence A in the source document.
- **50% of the time (Label $y = 0$, NotNext)**: Sentence B is a random sentence sampled independently from an entirely unrelated document in the pre-training corpus.

To classify the sentence pair without introducing an ad-hoc pooling mechanism, BERT leverages the special leading token representation $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$. Because self-attention is

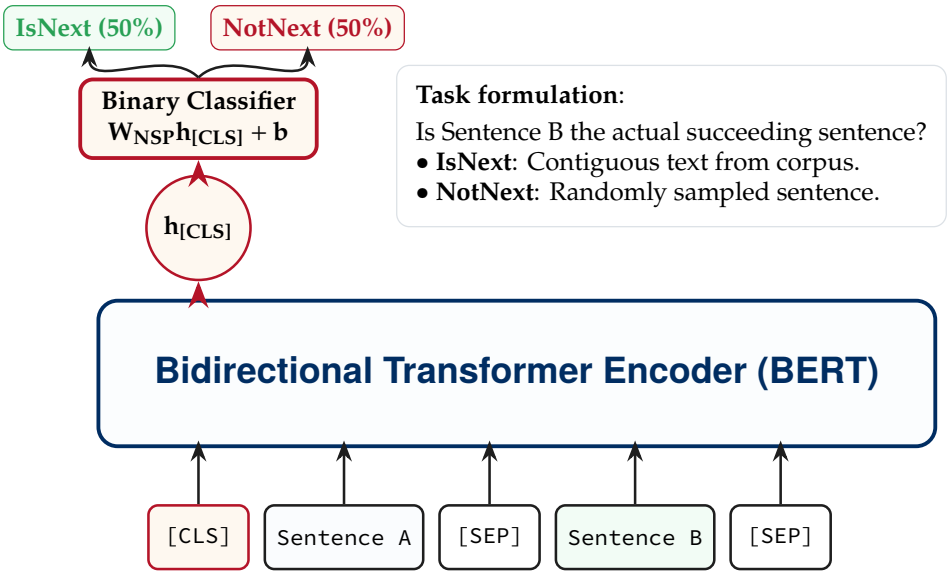


Figure 5.17: The Next Sentence Prediction (NSP) pre-training task in BERT. Two sentences are packed with delimiters `[CLS]` Sentence A `[SEP]` Sentence B `[SEP]`. The pooled final representation $\mathbf{h}_{[\text{CLS}]}$ is projected through a binary classification head $\mathbf{W}_{\text{NSP}}$ to predict whether Sentence B logically follows Sentence A.

bidirectional, $\mathbf{h}_{[\text{CLS}]}$ computes attention over every token in both Sentence A and Sentence B. This pooled vector is projected through a non-linear dense transformation:

$$\mathbf{h}_{\text{pool}} = \tanh(\mathbf{W}_{\text{pool}}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{pool}}), \quad \mathbf{W}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}, \ \mathbf{b}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}}}. \tag{5.91}$$

The binary prediction probability is emitted by a logistic regression head:

$$\hat{y}_{\text{NSP}} = \sigma(\mathbf{w}_{\text{NSP}}^{\top}\mathbf{h}_{\text{pool}} + b_{\text{NSP}}), \quad \mathbf{w}_{\text{NSP}} \in \mathbb{R}^{d_{\text{model}}}, \ b_{\text{NSP}} \in \mathbb{R}, \tag{5.92}$$

and evaluated under the binary cross-entropy loss:

$$\mathcal{L}_{\text{NSP}}(\theta) = -y \log \hat{y}_{\text{NSP}} - (1 - y) \log(1 - \hat{y}_{\text{NSP}}). \tag{5.93}$$

The unified pre-training objective jointly optimizes the composite loss across all sampled sequences:

$$\mathcal{L}_{\text{BERT}}(\theta) = \mathcal{L}_{\text{MLM}}(\theta) + \mathcal{L}_{\text{NSP}}(\theta). \tag{5.94}$$

5.11.4 Attention map anatomy and emergent linguistic structures

Motivation for analyzing attention maps. Analyzing internal attention maps demystifies the "black box" of deep Transformer encoders. It allows researchers to verify whether the statistical optimizations of MLM and NSP genuinely induce human-like linguistic and syntactic competencies, or whether the model merely relies on shallow statistical surface correlations. By visualizing the attention matrices $\mathbf{A}^{(l,h)}$, we observe how individual attention heads autonomously specialize into interpretable linguistic operators.

A standard BERT-Base model comprises $L = 12$ stacked encoder layers with $h = 12$ attention heads per layer, operating across a hidden dimension of $d_{\text{model}} = 768$ ($d_k = d_{\text{model}}/h = 64$). Consequently, for any input sequence, the network computes:

$$12 \times 12 = 144 \text{ distinct attention heads (and corresponding attention maps).} \tag{5.95}$$

For an input sequence of length N, each attention head generates an $N \times N$ row-stochastic attention weight matrix $\mathbf{A}^{(l,h)} \in [0, 1]^{N \times N}$ satisfying $\sum_{j=1}^{N} A_{ij}^{(l,h)} = 1$.

Consider the canonical linguistic probe evaluated in [61]:

`[CLS] The dog ate the food because it was hungry [SEP]` ($N = 11$ tokens).

Across the 144 attention maps, empirical probing reveals that self-attention layers autonomously partition into highly specialized linguistic registers without explicit syntactic supervision (Figure 5.18):

1. **Coreference and anaphora resolution (intermediate layers, e.g., Layer 5, Head 4)**: Specific attention heads in the middle layers specialize in resolving pronominal anaphora across clausal boundaries. In the running example, when computing attention from the pronoun query token `"it"`, the head routes over 82% of its total attention mass directly to the antecedent noun key `"dog"` ($\alpha = 0.82$). This allows the contextual representation of `"it"` to absorb the semantic properties of the entity it replaces.

2. **Syntactic dependency tracking (early-to-intermediate layers)**: Distinct heads capture grammatical relations: verb-object heads route strong attention from transitive verbs to their direct objects (e.g., `"ate"` $\rightarrow$ `"food"` with $\alpha = 0.74$), determiner heads link articles to head nouns (`"The"` $\rightarrow$ `"dog"`, `"the"` $\rightarrow$ `"food"`), and predicate heads connect adjectives to sentence subjects (`"hungry"` $\rightarrow$ `"dog"`).

3. **Surface sequential tracking (early layers)**: In early layers, several heads attend predominantly to adjacent tokens ($i \rightarrow i-1$ or $i \rightarrow i+1$), preserving localized n-gram ordering within the permutation-invariant self-attention matrix.

4. **The delimiter and neutral register phenomenon (deep layers, e.g., Layer 10, Head 8)**: In deeper layers, Clark et al. [61] and Michel et al. [62] observed that multiple attention heads allocate between 50% and 90% of their total attention weight directly to the special delimiter token `[SEP]`. This behavior arises from a fundamental mathematical constraint: the softmax operator strictly enforces $\sum_{j=1}^{N} A_{ij} = 1$. When a specialized attention head (such as a coreference head) processes a token that does not participate in that specific linguistic relation (e.g., the conjunction `"because"` or punctuation tokens), the head must still distribute a probability mass of 1.0. Distributing this mass across arbitrary semantic words would inject irrelevant noise and distort their representations. Therefore, the network designates `[SEP]` as a **neutral default register / "no-op" target**: unassigned attention is dumped onto `[SEP]`, leaving the contextual representation of the source token virtually unaltered ($\Delta \mathbf{h} \approx \mathbf{0}$).

Empirical layer-wise emergence: from surface adjacency to syntax and delimiter registers. To deeply understand how representations transform across the encoder depth, we inspect the actual attention heatmaps across the entire network hierarchy (Figures 5.19a and 5.20). This reveals a pronounced, progressive stratification of linguistic functions across depth:

The emergent patterns across the layers decompose into three canonical phases:

1. **Early layers (Layers 1–3: Surface and Local N-gram Geometry)**: As shown in Figure 5.19a, attention weights are concentrated heavily along the main diagonal ($i \rightarrow i$) and adjacent off-diagonals ($i \rightarrow i-1$ and $i \rightarrow i+1$). The model first reconstructs sequential positional adjacency, functioning much like convolutional filters with small receptive fields. Furthermore, basic lexical dependencies begin forming (e.g., `"ate"` linking with `"food"`).

2. **Middle layers (Layers 4–8: Syntactic Trees and Anaphora Resolution)**: In the intermediate layers, attention patterns become markedly sparse and structured (Figure 5.19b). The heads no longer attend to adjacent words; instead, they bridge long-range syntactic gaps. Dedicated heads track verb-object dependencies, noun-modifier bindings, and pronoun coreferences across subordinate conjunctions (`"it"` $\rightarrow$ `"dog"`). This proves that the multi-head mechanism divides labor: different heads track different syntactic relations concurrently.

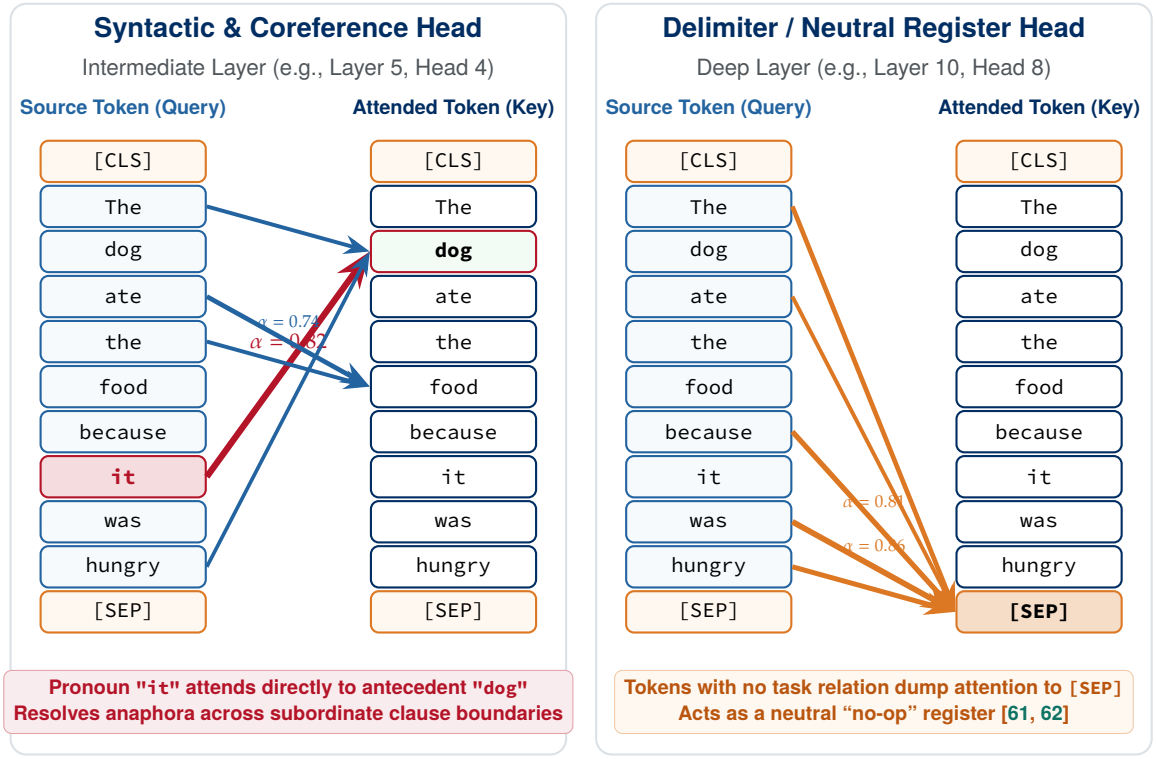
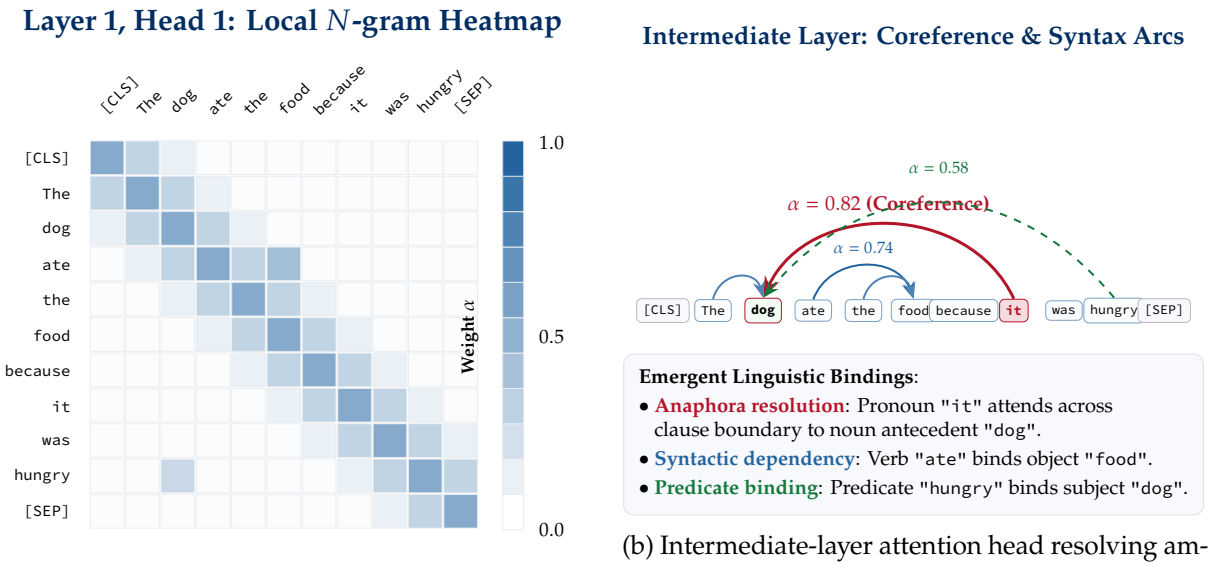


Figure 5.18: Emergent linguistic specialization and delimiter behavior across BERT attention heads for the input sequence "[CLS] The dog ate the food because it was hungry [SEP]" (11 tokens, evaluated across $12 \times 12 = 144$ attention maps). Left panel: An intermediate-layer head (Layer 5, Head 4) resolves anaphoric coreference by routing strong attention from pronoun "it" to antecedent "dog" ($\alpha = 0.82$), alongside verb-object dependencies ("ate" $\rightarrow$ "food"). Right panel: A deep-layer head (Layer 10, Head 8) exhibits delimiter pooling where unassigned tokens dump over 80% of their attention mass onto [SEP], functioning as a neutral no-op register that leaves vector representations undisturbed [61, 62].



(a) Attention heatmap for Layer 1, Head 1. Attention mass concentrates tightly along the diagonal ($i \to i, i \pm 1$), functioning like local convolutional filters to preserve immediate sequential n-grams.

(b) Intermediate-layer attention head resolving ambiguous pronominal coreference. Query token `"it"` allocates massive attention mass ($\alpha = 0.82$) to antecedent `"dog"`, allowing BERT to accurately bind pronominal entities across clausal boundaries.

Figure 5.19: Emergent internal representation geometry across BERT attention heads. Early layers form localized n-gram filters, while intermediate layers synthesize non-local syntactic and pronominal coreference dependencies without supervised parse trees.

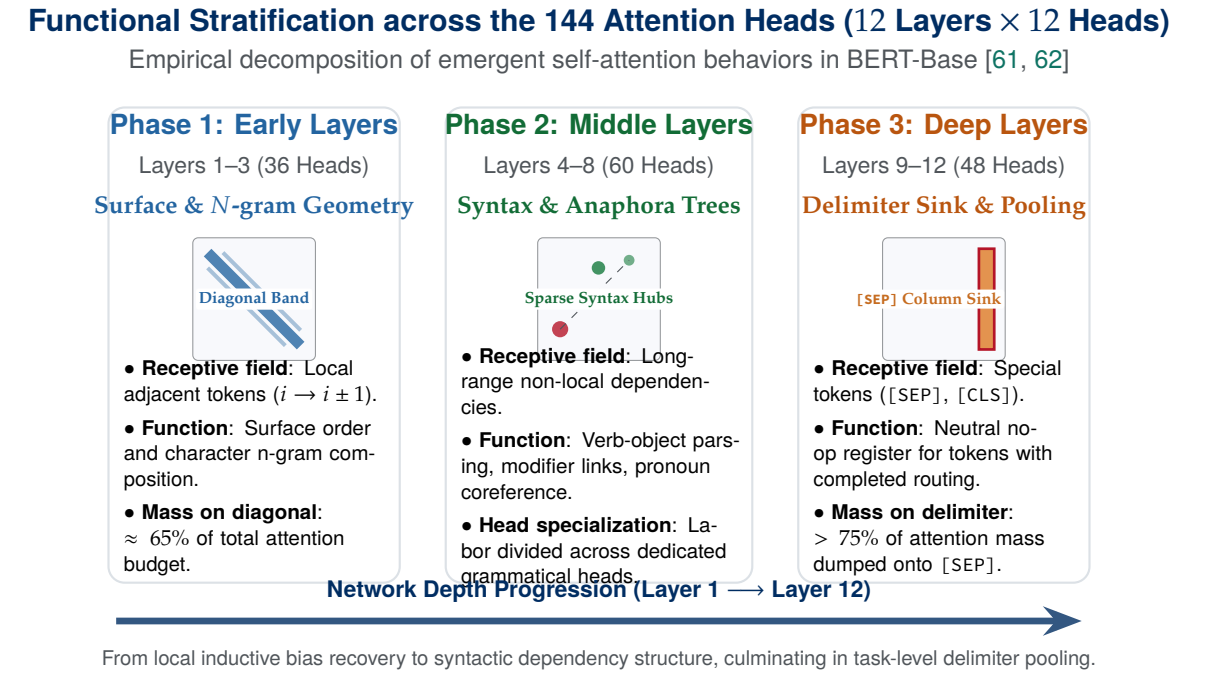


Figure 5.20: The emergent landscape of $12 \times 12 = 144$ attention maps across BERT-Base. Rather than an uninterpretable black box, self-attention exhibits clear layer-wise functional stratification: early layers act as local n-gram filters (diagonal band); middle layers construct sparse syntactic trees and resolve pronominal coreference; final layers pool inactive representations into the `[SEP]` delimiter register [61, 62].

3. **Deep layers (Layers 9–12: High-Level Task Abstraction and Delimiter Pooling)**: In the final layers, attention distributions shift again. Many heads show broad, diffuse patterns, while a significant fraction dump 80%–90% of their attention mass onto the special token `[SEP]`. Because the final layers synthesize task-level features, tokens that have already aggregated their required syntactic context utilize `[SEP]` as an inert sink. This preserves their accumulated representation vectors without absorbing unwanted semantic drift in the topmost layers before classification or MLM loss computation.

5.11.5 Downstream fine-tuning mechanisms

Motivation for analyzing fine-tuning mechanisms. A pre-trained foundation model is only as valuable as its adaptability to real-world tasks. Analyzing BERT's downstream adaptation demonstrates the paradigm shift from task-specific feature engineering to task-agnostic representation fine-tuning. By modifying only the topmost prediction layer while allowing gradients to backpropagate through all 110 million parameters, BERT achieves state-of-the-art performance across diverse natural language tasks with minimal training time.

Following pre-training on MLM and NSP, the generic pre-training classification layers are discarded. The pre-trained Transformer weights are retained, and minimal task-specific projection heads are appended to adapt the model to downstream NLP applications with end-to-end gradient updates:

1. **Sequence-level classification** (e.g., sentiment analysis, code defect prediction, natural language inference): The pooled sentence representation $\mathbf{h}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}}}$ from Equation (5.91) is projected through a task-specific linear head:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{seq}}\mathbf{h}_{\text{pool}} + \mathbf{b}_{\text{seq}}), \quad \mathbf{W}_{\text{seq}} \in \mathbb{R}^{C \times d_{\text{model}}}, \ \mathbf{b}_{\text{seq}} \in \mathbb{R}^{C}, \tag{5.96}$$

   where C denotes the number of target classes. All parameters (both $\mathbf{W}_{\text{seq}}$ and the encoder backbone) are fine-tuned via cross-entropy loss.

2. **Token-level sequence labeling** (e.g., Named Entity Recognition NER, Part-of-Speech tagging, code token vulnerability tagging): Instead of pooling, the final contextual representation $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ of each individual token $i \in \{1, 2, \ldots, L\}$ is mapped to a discrete label distribution:

$$\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_{\text{tok}}\mathbf{h}_i + \mathbf{b}_{\text{tok}}), \quad \mathbf{W}_{\text{tok}} \in \mathbb{R}^{K \times d_{\text{model}}}, \ \mathbf{b}_{\text{tok}} \in \mathbb{R}^{K}, \tag{5.97}$$

   where K is the number of entity or syntactic tags.

3. **Extractive Question Answering** (e.g., SQuAD): Conditioned on a passage and a question concatenated as Sentence A and Sentence B, the model identifies the continuous answer span by learning a start vector $\mathbf{w}_{\text{start}} \in \mathbb{R}^{d_{\text{model}}}$ and an end vector $\mathbf{w}_{\text{end}} \in \mathbb{R}^{d_{\text{model}}}$. The probability that token i is the start of the answer span is:

$$P_{\text{start}}(i) = \frac{\exp(\mathbf{w}_{\text{start}}^{\top}\mathbf{h}_i)}{\sum_{j=1}^{L} \exp(\mathbf{w}_{\text{start}}^{\top}\mathbf{h}_j)}, \tag{5.98}$$

   with an identical formulation for $P_{\text{end}}(i)$.

Architectural transition: from bidirectional representations to autoregressive generation. While the Encoder-Decoder paradigm addresses sequence transduction and the Encoder-Only paradigm excels at bidirectional representation learning and sequence classification, pure autoregressive generative tasks (such as open-ended text synthesis and conversational reasoning)

rely on Decoder-Only architectures. We defer the comprehensive architectural analysis of large-scale Decoder-Only models to Chapter 6, where scaling laws and autoregressive pre-training are developed in full detail.

However, regardless of whether generation occurs within the decoder stack of an Encoder-Decoder model or a dedicated autoregressive language decoder, the fundamental problem of selecting discrete tokens from continuous output distributions remains mathematically invariant. In the subsequent section, we formalize the probabilistic mechanics governing autoregressive decoding.

5.12 Autoregressive decoding and sampling strategies

At each step t of autoregressive sequence generation, an autoregressive language decoder projects its topmost contextual hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ through an unembedding linear transformation $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ parameterized with vocabulary bias $\mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}$. This maps the continuous latent manifold into raw unnormalized affinities over discrete token space:

$$\mathbf{z}_t = \mathbf{W}_U \mathbf{h}_t + \mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}, \tag{5.99}$$

where $\mathbf{z}_t = [z_{t,1}, z_{t,2}, \ldots, z_{t,|\mathcal{V}|}]^\top \in \mathbb{R}^{|\mathcal{V}|}$ denotes the unnormalized log-affinity (logit) vector across the vocabulary $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$. Passing $\mathbf{z}_t$ through the canonical softmax operator $\text{softmax} : \mathbb{R}^{|\mathcal{V}|} \to \Delta^{|\mathcal{V}|-1}$ maps the continuous unbounded logits onto the standard $(|\mathcal{V}| - 1)$-dimensional probability simplex $\Delta^{|\mathcal{V}|-1} = \{\mathbf{p} \in \mathbb{R}^{|\mathcal{V}|} : p_i \geq 0, \sum_{i=1}^{|\mathcal{V}|} p_i = 1\}$.

This defines the discrete categorical conditional probability mass function governing the next-token random variable $X_t \sim \text{Categorical}(\mathbf{p}_t)$:

$$\begin{aligned} P(X_t = v_i \mid w_{<t}) &= [\text{softmax}(\mathbf{z}_t)]_i \\ &= \frac{\exp(z_{t,i})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j})} \\ &= \frac{\exp\left(\mathbf{w}_{U,i}^\top \mathbf{h}_t + b_{U,i}\right)}{\sum_{j=1}^{|\mathcal{V}|} \exp\left(\mathbf{w}_{U,j}^\top \mathbf{h}_t + b_{U,j}\right)}, \quad \forall v_i \in \mathcal{V}, \end{aligned} \tag{5.100}$$

where $\mathbf{w}_{U,i}^\top \in \mathbb{R}^{1 \times d_{\text{model}}}$ denotes the i-th row vector of the unembedding matrix $\mathbf{W}_U$ (often tied with the input embedding matrix $\mathbf{E}_{\text{tok}}^\top$ in weight-tied models), and $b_{U,i} \in \mathbb{R}$ represents the corresponding vocabulary bias term. By construction, the probabilities satisfy the Kolmogorov axioms:

$$P(X_t = v_i \mid w_{<t}) \in (0, 1) \quad \text{and} \quad \sum_{i=1}^{|\mathcal{V}|} P(X_t = v_i \mid w_{<t}) = 1. \tag{5.101}$$

In modern language models, the vocabulary cardinality $|\mathcal{V}|$ spans tens of thousands of subword units (for example, $|\mathcal{V}| = 30{,}522$ in BERT's WordPiece, $|\mathcal{V}| = 50{,}257$ in GPT-2's Byte-Pair Encoding). Consequently, $\mathbf{p}_t$ represents a massive, highly non-uniform probability distribution over tens of thousands of discrete outcomes.

Once this dense probability distribution $P(w_i \mid w_{<t})$ is obtained, the fundamental challenge of autoregressive inference arises: **how do we select the discrete token $\hat{w}_t$ to append to the generated sequence?** The choice of decoding policy governs the trade-off between deterministic precision and stochastic diversity, directly deciding whether the model produces factual truth, creative narrative, generic clichés, or ungrammatical gibberish.

To establish an identical, controlled empirical testbed across all decoding policies studied in this section, we condition each strategy on the opening prompt drawn from Arthur Conan Doyle's *The Red-Headed League* (as examined in lecture):

> *"I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair. With an apology for my intrusion, I was about to withdraw when Holmes pulled me abruptly into the room and closed the door behind me."*

In the following subsections, we analyze the six canonical decoding paradigms in strict linear progression: from deterministic greedy selection and beam optimization to stochastic sampling, entropy modulation, and dynamic truncation filters.

5.12.1 Greedy decoding (argmax) and the degeneration trap

Mathematical formulation. The simplest and computationally most direct decoding policy is **Greedy Search**. At each temporal step t, the algorithm deterministically selects the single token assigned the highest instantaneous conditional probability:

$$\hat{w}_t = \text{argmax}_{w \in \mathcal{V}} P(w \mid w_{<t}). \tag{5.102}$$

Greedy decoding requires zero hyperparameters and incurs minimal computational overhead: it performs a single $O(|\mathcal{V}|)$ reduction across the logit vector at each time step.

Concrete numerical example. Suppose after processing the prompt, the model emits the normalized probabilities across a representative candidate subset shown in Table 5.2.

Table 5.2: Worked probability distribution for next-token selection at step $t = 1$.

Token Candidate w	`"I`	`"He`	`"The`	`"Mr.`	`"What`	... [Remaining 50,252 tokens]
Logit z	14.20	13.50	12.80	11.90	10.10	< 8.00
Softmax $P(w \mid w_{<t})$	**0.521**	0.260	0.130	0.053	0.009	$\sum = 0.027$

Under greedy search, the algorithm evaluates $\text{argmax}_w P(w \mid w_{<t})$ and deterministically emits token $\hat{w}_t =$ `"I`. All remaining hypotheses are discarded permanently.

The failure mode: mathematical myopia and cyclic repetition. Despite its superficial appeal, greedy decoding exhibits two catastrophic failure modes in open-ended language generation:

1. **Mathematical Myopia (Local Greediness ≠ Global Sequence Optimality)**: Sequence likelihood factorizes via the probabilistic chain rule as $P(w_{1:T}) = \prod_{t=1}^{T} P(w_t \mid w_{<t})$. A greedy local maximum at step t frequently leads into a barren probability subspace at subsequent steps:

$$\text{argmax}_{w_1,\ldots,w_T} \prod_{t=1}^{T} P(w_t \mid w_{<t}) \neq \prod_{t=1}^{T} \text{argmax}_{w_t} P(w_t \mid w_{<t}). \tag{5.103}$$

   As illustrated in Figure 5.21, committing to token A because $P(A) = 0.60 > P(B) = 0.40$ traps the generator if A can only be followed by low-probability tokens ($P(C \mid A) = 0.10$, yielding path likelihood 0.06), whereas selecting the locally sub-optimal token B uncovers a highly fluent sequence ($P(D \mid B) = 0.80$, yielding path likelihood 0.32).

2. **Neural Text Degeneration**: Holtzman et al. [63] demonstrated that greedy decoding on neural language models triggers self-reinforcing attractor loops. Common syntactic markers (e.g., pronouns, auxiliary verbs) dominate local argmax selections. Once a repetitive n-gram is appended to the context, self-attention attending back to those identical tokens artificially elevates their conditional probabilities even higher at subsequent steps, locking the model into an inescapable loop of cyclic degeneration:

```
"I am sorry, Mr. Holmes," I said. "I am sorry, Mr. Holmes, for having
been so rude to you. I am sorry, Mr. Holmes, for having been so rude
to you. I am sorry, Mr. Holmes, for having been so rude to you..."
```

Causal bridge to Beam Search. Because greedy search suffers from irrecoverable path pruning, we require an algorithm capable of looking ahead and exploring multiple alternative sequences concurrently. This directly motivates **Beam Search**.

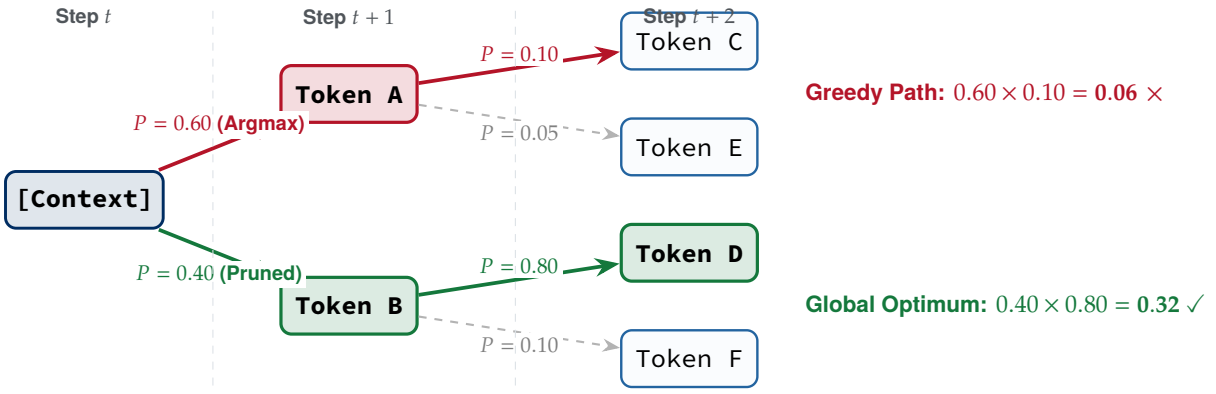


Figure 5.21: The failure mechanism of greedy decoding. By greedily selecting `Token A` ($P = 0.60$) over `Token B` ($P = 0.40$) at step $t+1$, the search space collapses into low-probability continuations ($P = 0.06$). The global optimum sequence through `Token B` ($P = 0.32$) is permanently pruned.

5.12.2 Beam search: exploring parallel hypotheses

Mathematical formulation. **Beam Search** avoids the strict myopia of greedy decoding by maintaining a fixed set of B top candidate sequences (termed the *beam width* B, or k in literature) at each time step.

Let $\mathcal{Y}_t = \{\mathbf{y}_t^{(1)}, \mathbf{y}_t^{(2)}, \ldots, \mathbf{y}_t^{(B)}\}$ denote the set of active partial hypotheses of length t. At generation step $t+1$:

1. Every hypothesis $\mathbf{y}_t^{(b)} \in \mathcal{Y}_t$ is expanded across all $|\mathcal{V}|$ vocabulary candidates, forming $B \times |\mathcal{V}|$ child candidate sequences $\mathbf{y}_{t+1} = (\mathbf{y}_t^{(b)}, w)$.

2. The cumulative score of each sequence is evaluated in logarithmic space to prevent numerical underflow:
$$\text{Score}(\mathbf{y}_{1:t+1}) = \sum_{i=1}^{t+1} \log P(y_i \mid \mathbf{y}_{<i}). \tag{5.104}$$

3. The expanded set of $B \cdot |\mathcal{V}|$ candidates is sorted, and only the top-B candidates with the highest cumulative log-probabilities are preserved:
$$\mathcal{Y}_{t+1} = \text{argtop-B}_{\mathbf{y} \in \mathcal{Y}_t \times \mathcal{V}} \text{Score}(\mathbf{y}). \tag{5.105}$$
All other $B \cdot (|\mathcal{V}| - 1)$ hypotheses are permanently pruned.

Length penalty normalization. Because $\log P(y_i \mid \mathbf{y}_{<i}) \leq 0$ for all valid probabilities, the objective in Equation (5.104) is strictly non-increasing with sequence length. Consequently, unconstrained beam search inherently penalizes longer generations, favoring unnaturally short, truncated outputs. To counteract this bias, standard implementations incorporate a length normalization heuristic:

$$\text{Score}_{\text{norm}}(Y) = \frac{\sum_{i=1}^{|Y|} \log P(y_i \mid \mathbf{y}_{<i})}{\text{LP}(Y)}, \quad \text{where} \quad \text{LP}(Y) = \frac{(5 + |Y|)^\alpha}{(5+1)^\alpha}, \tag{5.106}$$

with length penalty parameter $\alpha \in [0.6, 1.0]$. When $\alpha = 0$, no length normalization occurs; when $\alpha = 1.0$, the score is normalized by length.

Worked numerical example. Consider beam search with beam width $B = 2$ expanding from the root token `"The"`, exactly replicating the lecture derivation:

1. **Step** $t = 1$: From `"The"`, candidate probabilities are:

$$P(\texttt{"nice"} \mid \texttt{"The"}) = 0.50, \quad P(\texttt{"dog"} \mid \texttt{"The"}) = 0.40, \quad P(\texttt{"car"} \mid \texttt{"The"}) = 0.10. \quad (5.107)$$

   The top-2 beams retained are $\mathcal{Y}_1 = \{(\texttt{"The"}, \texttt{"nice"}), (\texttt{"The"}, \texttt{"dog"})\}$. The branch `"car"` is pruned.

2. **Step** $t = 2$: Each of the 2 active beams expands its candidates:

   - From `"The dog"`: `"has"` ($P = 0.90 \implies$ joint $0.40 \times 0.90 = 0.36$), `"and"` ($P = 0.05 \implies$ joint 0.02), `"run"` ($P = 0.05 \implies$ joint 0.02).
   - From `"The nice"`: `"woman"` ($P = 0.40 \implies$ joint $0.50 \times 0.40 = 0.20$), `"house"` ($P = 0.30 \implies$ joint 0.15), `"guy"` ($P = 0.30 \implies$ joint 0.15).

   Across all 6 expanded paths, the top-2 global sequences are `"The dog has"` ($P = 0.36$) and `"The nice woman"` ($P = 0.20$). All other 4 paths are pruned.

Figure 5.22 illustrates this exact operational trellis.

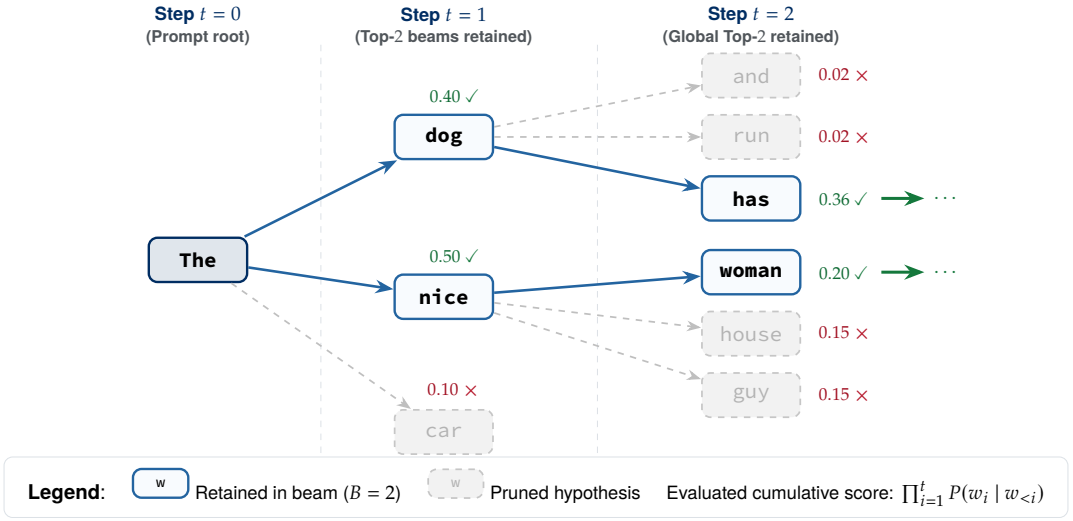


Figure 5.22: Beam search decoding with beam width $B = 2$. At each generation step, all B candidate sequences are expanded across vocabulary candidates. The algorithm prunes lower-probability branches, maintaining only the top-B cumulative probability hypotheses (here, `"The dog has"` with $P = 0.36$ and `"The nice woman"` with $P = 0.20$).

The failure mode of beam search in open-ended generation. While beam search is the gold standard for closed, goal-oriented tasks with narrow target spaces (such as neural machine translation and formal code generation), it fails strikingly in open-ended creative text generation:

- **Deterministic Blandness and Clichés**: Human communication does not optimize for maximum likelihood at every step. Natural human language contains an alternating rhythm of predictable syntactic connectors (low information entropy) and surprising, domain-specific words (high information entropy). Maximizing cumulative likelihood forces the model into high-probability clichés and generic sentences.
- **Repetitive Stagnation**: Holtzman et al. [63] demonstrated that increasing the beam width B in open-ended language generation actually *degrades* perceived quality: as $B \to \infty$, the search uncovers pathological repeating loops that accumulate high probability mass over extended horizons.

Causal bridge to Random Sampling. To escape deterministic clichés and inject human-like entropy, we must transition from deterministic maximization to **stochastic generation**. This leads directly to **Pure Random Sampling**.

5.12.3 Pure random sampling and the catastrophic tail risk

Mathematical formulation. Instead of deterministically tracking top-ranking candidates, **Pure Random Sampling** (Multinomial Sampling) draws the next token directly from the categorical probability distribution generated by the softmax unembedding:

$$w_t \sim \text{Categorical}\left(P(w_1 \mid w_{<t}), P(w_2 \mid w_{<t}), \ldots, P(w_{|\mathcal{V}|} \mid w_{<t})\right). \tag{5.108}$$

Sampling is implemented computationally via Inverse Transform Sampling: given a uniform pseudo-random draw $u \sim \mathcal{U}[0,1)$, select index $\hat{i}$ satisfying:

$$\sum_{j=1}^{\hat{i}-1} P(w_j \mid w_{<t}) \leq u < \sum_{j=1}^{\hat{i}} P(w_j \mid w_{<t}). \tag{5.109}$$

Worked numerical example. Returning to the distribution from Table 5.2:

- Under greedy search, `"I` is selected with 100% probability.
- Under random sampling:

$$P(w_t = \texttt{"I"}) = 52.1\%, \quad P(w_t = \texttt{"He"}) = 26.0\%, \quad P(w_t = \texttt{"The"}) = 13.0\%, \quad \ldots \tag{5.110}$$

  If the pseudo-random variable evaluates to $u = 0.650$, since $0.521 \leq 0.650 < (0.521 + 0.260 = 0.781)$, the generator selects token $\hat{w}_t =$ `"He"`. The generated continuation gains immediate narrative variety.

The failure mode: the unreliable long tail and hallucination cascades. While random sampling completely eliminates repetitive loops, it introduces a severe vulnerability: **the catastrophic tail hazard**. In language models with $|\mathcal{V}| \approx 50{,}000$, although individual tokens in the tail have infinitesimal probabilities (e.g., $P(w) < 10^{-4}$), the *cumulative aggregate mass* of the thousands of tail tokens commonly sums to 15% to 30% of the entire probability simplex (Figure 5.23).

Consequently, when generating a multi-token sequence, the probability of sampling *at least one* erratic tail token over N steps is $1 - (1 - P_{\text{tail}})^N \approx 1$. Once an aberrant token is sampled into the context, self-attention attends to it, irrevocably corrupting the topic representation and causing severe semantic collapse:

```
"...and my amazement owled wild-it seems an impossibility-and while it
was gone in an stupor and good hearing from me I pretended to know his
name, assuming some personal information which I would..."
```

Causal bridge to Temperature Scaling. Before resorting to hard truncation of the vocabulary, can we smoothly reshape the probability landscape itself, suppressing the tail while accentuating dominant modes? This leads to **Temperature Scaling**.

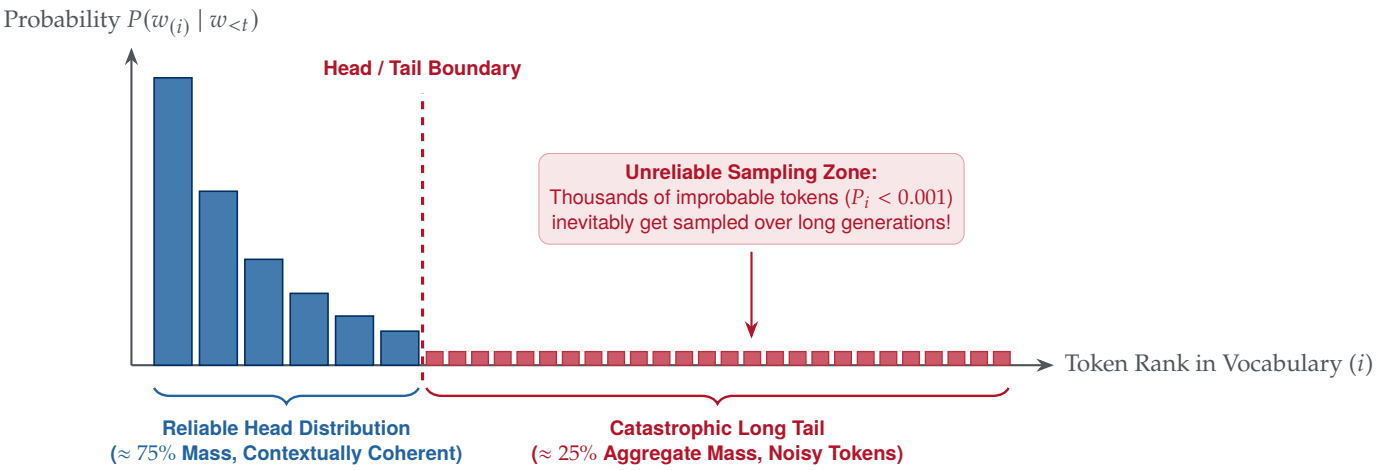


Figure 5.23: The long-tail vulnerability of pure multinomial random sampling. Although individual low-probability tokens have negligible probabilities, their cumulative probability mass (≈ 25%) guarantees that over a horizon of dozens of tokens, the generator will sample an erratic, nonsensical token from the tail, corrupting the subsequent context history.

5.12.4 Temperature scaling: thermodynamic entropy modulation

Mathematical formulation. **Temperature Scaling** modifies the sharpness of the categorical probability distribution by introducing a continuous thermodynamic parameter $T > 0$ that scales raw logits prior to exponentiation:

$$P(w_i; T) = \frac{\exp(z_i/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j/T)}, \quad \forall w_i \in \mathcal{V}. \tag{5.111}$$

The parameter T acts as a continuous entropy dial:

- **Low Temperature ($T < 1.0$)**: Amplifies relative logit differences $z_i - z_j$, concentrating probability mass on the highest-scoring tokens (reducing Shannon entropy).
- **Standard Temperature ($T = 1.0$)**: Preserves the original, calibrated categorical distribution learned during maximum-likelihood pre-training.
- **High Temperature ($T > 1.0$)**: Dampens logit disparities, flattening the distribution toward maximum-entropy uniformity.

Concrete numerical example. Consider a model evaluating three candidate tokens with raw logits $\mathbf{z} = [10.0, 8.0, 5.0]^\top$. Table 5.3 computes the resulting probabilities across four temperature regimes.

Table 5.3: Behavior of probabilities under temperature scaling for logits $\mathbf{z} = [10.0, 8.0, 5.0]^\top$.

Regime	Scaled Logits $\mathbf{z}/T$	$\exp(\mathbf{z}/T)$	Probabilities $P(w;T)$	Shannon Ent
$T = 0.5$ (Low / Sharp)	$[20.0, 16.0, 10.0]$	$[4.85 \cdot 10^8, 8.89 \cdot 10^6, 2.20 \cdot 10^4]$	$[\mathbf{0.9820}, 0.0180, 0.00004]$	0.092 na
$T = 1.0$ (Default)	$[10.0, 8.0, 5.0]$	$[22026, 2981, 148]$	$[\mathbf{0.8756}, 0.1185, 0.0059]$	0.388 na
$T = 2.0$ (High / Soft)	$[5.0, 4.0, 2.5]$	$[148.4, 54.6, 12.2]$	$[\mathbf{0.6896}, 0.2537, 0.0567]$	0.784 na
$T = 5.0$ (Extreme)	$[2.0, 1.6, 1.0]$	$[7.39, 4.95, 2.72]$	$[\mathbf{0.4907}, 0.3287, 0.1806]$	1.042 na

As demonstrated, decreasing T from 1.0 to 0.5 drives the top token's probability from 87.6% to 98.2%, virtually suppressing all tail tokens. Conversely, increasing T to 5.0 flattens the distribution toward equal weighting ($1/3 \approx 0.333$).

Figure 5.24 visualizes this transformation across the vocabulary spectrum.

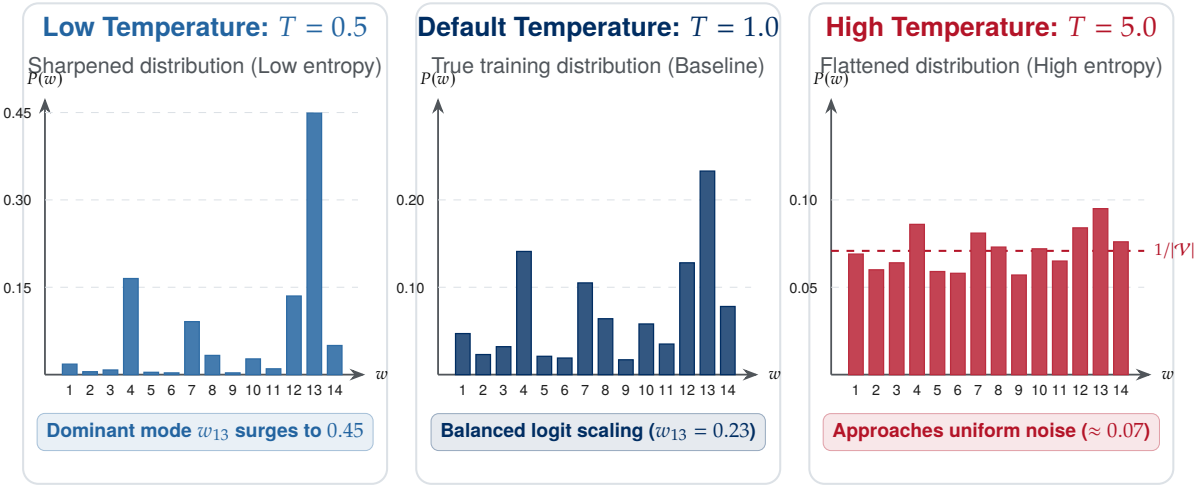


Figure 5.24: Temperature scaling of the softmax distribution. Scaling logits by temperature T modulates distribution entropy: $T < 1.0$ (left) exponentially sharpens differences toward the dominant mode (w_{13}), approaching deterministic greedy decoding as $T \to 0$; $T = 1.0$ (center) preserves the model's true training logits; $T > 1.0$ (right) flattens logit disparities toward a uniform distribution, producing erratic and ungrammatical token sequences.

Analytical limits and mathematical proofs. The behavior of temperature scaling is bounded by two rigorous analytical limits:

1. **Low-Temperature Limit ($T \to 0^+$, Greedy Convergence)**: Let $z^* = \max_j z_j$ denote the unique maximal logit. Factoring out $\exp(z^*/T)$ from the numerator and denominator of Equation (5.111):

$$\lim_{T\to 0^+} P(w_i;T) = \lim_{T\to 0^+} \frac{\exp((z_i - z^*)/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp((z_j - z^*)/T)}. \tag{5.112}$$

For the maximal token where $z_i = z^*$, $(z_i - z^*)/T = 0$, so $\exp(0) = 1$. For any sub-optimal token k where $z_k < z^*$, the numerator satisfies $z_k - z^* = -\delta < 0$ ($\delta > 0$). As $T \to 0^+$, $-\delta/T \to -\infty$, yielding $\exp(-\delta/T) \to 0$. Therefore:

$$\lim_{T\to 0^+} P(w_i;T) = \begin{cases} 1 & \text{if } z_i = \max_j z_j, \\ 0 & \text{if } z_i < \max_j z_j. \end{cases} \tag{5.113}$$

Temperature sampling converges continuously to deterministic **Greedy decoding**.

2. **High-Temperature Limit ($T \to \infty$, Uniform Collapse)**: As $T \to \infty$, for every finite logit z_i, the argument $z_i/T \to 0$, implying $\exp(z_i/T) \to 1$. Consequently:

$$\lim_{T\to\infty} P(w_i;T) = \frac{1}{\sum_{j=1}^{|\mathcal{V}|} 1} = \frac{1}{|\mathcal{V}|}. \tag{5.114}$$

The distribution collapses into a maximal-entropy uniform distribution across all $|\mathcal{V}|$ vocabulary tokens.

3. **Monotonicity of Shannon Entropy**: The Shannon entropy of the distribution is defined as $H(P;T) = -\sum_{i=1}^{|\mathcal{V}|} P_i(T) \log P_i(T)$. Differentiating with respect to T yields:

$$\frac{\partial H(P;T)}{\partial T} = \frac{1}{T^3} \mathrm{Var}_P(z) = \frac{1}{T^3} \left(\sum_{i=1}^{|\mathcal{V}|} P_i(T) z_i^2 - \left(\sum_{i=1}^{|\mathcal{V}|} P_i(T) z_i \right)^2 \right) \geq 0. \tag{5.115}$$

Because variance is strictly non-negative ($\mathrm{Var}_P(z) \geq 0$), entropy increases strictly monotonically with T whenever logits are non-identical.

Limitation of temperature scaling. Although temperature scaling compresses or flattens the distribution, **it never zeros out the probability of any token**: every token in the vocabulary retains $P(w_i; T) > 0$. In low-entropy contexts where only a single token is correct, even a lowered temperature leaves non-zero probability mass in the tail, still leaving room for improbable hallucinations. Conversely, lowering temperature globally to combat tail noise also suppresses legitimate synonyms in creative passages.

Causal bridge to Top-k Sampling. To guarantee that low-probability tokens cannot be sampled under any circumstances, we must apply explicit **truncation** to the support of the distribution. This motivates **Top-k Sampling**.

5.12.5 Top-k sampling: hard cardinality truncation

Mathematical formulation. Introduced by Fan et al. [64], **Top-k Sampling** bounds tail risk by strictly restricting the sampling candidate set to the k most probable tokens at each generation step.

Definition 5.4: *Top-k sampling*

Let $\mathcal{V}^{(k)} \subset \mathcal{V}$ denote the subset of size $|\mathcal{V}^{(k)}| = k$ containing the tokens with the highest probabilities under $P(w \mid w_{<t})$:

$$\sum_{w \in \mathcal{V}^{(k)}} P(w \mid w_{<t}) \quad \text{is maximized subject to } |\mathcal{V}^{(k)}| = k. \tag{5.116}$$

All tokens outside $\mathcal{V}^{(k)}$ have their probabilities truncated to zero. The remaining k probabilities are renormalized by dividing by their cumulative mass $S_k = \sum_{w_j \in \mathcal{V}^{(k)}} P(w_j)$:

$$P'(w_i) = \begin{cases} \frac{P(w_i)}{\sum_{w_j \in \mathcal{V}^{(k)}} P(w_j)} & \text{if } w_i \in \mathcal{V}^{(k)}, \\ 0 & \text{otherwise.} \end{cases} \tag{5.117}$$

The next token is sampled from this truncated categorical distribution: $w_t \sim \text{Categorical}(P')$.

Worked numerical example. Suppose a model evaluates the token distribution in Table 5.4 under a top-k policy with $k = 3$.

Table 5.4: Worked example of Top-k truncation ($k = 3$) and probability renormalization.

Token Candidate w	`"dog"`	`"cat"`	`"bird"`	`"fish"`	`"horse"`	`...` [Tail]
Original $P(w)$	0.40	0.30	0.15	0.08	0.04	$\sum = 0.03$
In Top-3 Set $\mathcal{V}^{(k)}$**?**	✓	✓	✓	×	×	×
Truncated Mass	0.40	0.30	0.15	0.00	0.00	0.00
Renormalized $P'(w)$	$\frac{0.40}{0.85} = \mathbf{0.471}$	$\frac{0.30}{0.85} = \mathbf{0.353}$	$\frac{0.15}{0.85} = \mathbf{0.176}$	0.000	0.000	0.000

The cumulative mass of the top-3 candidates is $S_3 = 0.40 + 0.30 + 0.15 = 0.85$. Dividing each retained candidate by 0.85 rescales the probabilities so that $P'(\texttt{"dog"}) + P'(\texttt{"cat"}) + P'(\texttt{"bird"}) =$

1.000. All tail candidates (accounting for 15% of original probability mass) are strictly excluded.

Figure 5.25 illustrates this truncation and renormalization geometry.

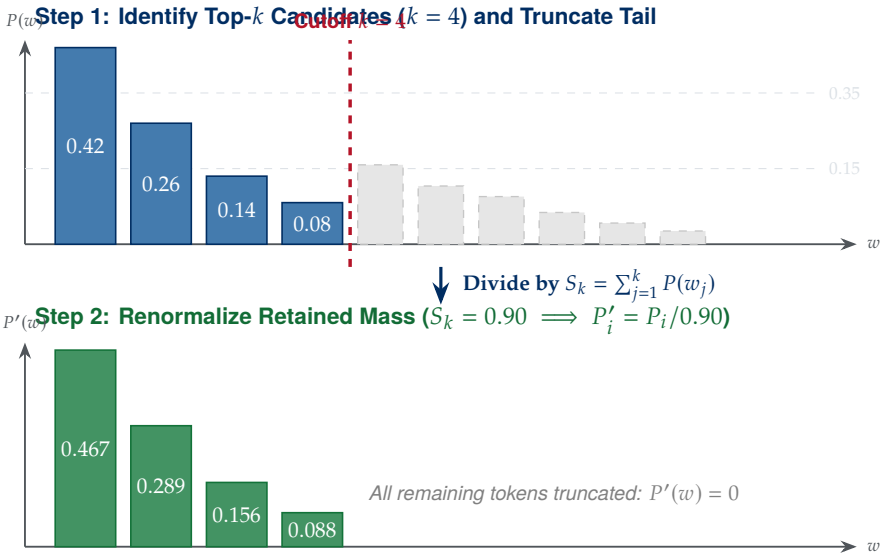


Figure 5.25: The mechanics of Top-k sampling ($k = 4$). The algorithm isolates the top-k tokens, strictly zeroing out the remaining probability mass. Renormalizing by the sum of retained mass rescales the top-k probabilities so they sum to 1.0, shielding the generator from tail errors.

Evaluating top-k sampling with $k = 50$ on the Sherlock Holmes prompt eliminates bizarre tail tokens and produces coherent dialogue:

```
"'You will think, Doctor, that a man of my age would be so much of a menace
to me, if I were to refuse a pass for him.  I should rather get a few things
away from him than any which might injure his reputation...'"
```

The failure mode: the structural rigidity of a static k. Despite its widespread adoption, top-k sampling suffers from an inherent architectural rigidity: **a static threshold k cannot adapt to varying distribution entropy across different contexts**:

1. **Low-Entropy (Peaked / Factual Context)**: Suppose the prompt is `"The capital of France is"`. The model assigns $P(\texttt{"Paris"}) = 0.98$, while all other tokens are factual errors. If $k = 50$, top-k sampling is forced to retain 49 improbable, factually erroneous tokens. Because their collective mass is renormalized, the chance of sampling an error is artificially elevated.

2. **High-Entropy (Flat / Creative Context)**: Suppose the prompt is `"She looked out into the vast horizon and saw"`. Hundreds of valid English nouns and verbs (e.g., `"mountains"`, `"ships"`, `"stars"`, `"glaciers"`) share roughly equal probabilities (~ 0.005). If $k = 10$ or $k = 20$, top-k prematurely truncates dozens of perfectly fluent, highly creative candidate tokens.

Causal bridge to Top-p (Nucleus) Sampling. To overcome the inflexibility of a fixed cardinality k, the candidate pool size must become **dynamic**, expanding in high-entropy situations and contracting in low-entropy situations. This motivates **Top-p (Nucleus) Sampling**.

5.12.6 Top-p (nucleus) sampling: dynamic cumulative mass truncation

Mathematical formulation. To eliminate the rigid cardinality constraint of top-k, Holtzman et al. [63] proposed **Top-p (Nucleus) Sampling**. Instead of fixing the number of tokens k, nucleus sampling fixes the **cumulative probability mass threshold** $p \in (0, 1]$.

Definition 5.5: *Top-p (Nucleus) sampling*

Let $p \in (0, 1]$ be a cumulative probability threshold. Let $\mathcal{V}^{(p)} \subset \mathcal{V}$ denote the smallest subset of vocabulary tokens whose cumulative probability mass exceeds threshold p:

$$\mathcal{V}^{(p)} = \text{argmin}_{V' \subseteq \mathcal{V}} |V'| \quad \text{subject to} \quad \sum_{w \in V'} P(w \mid w_{<t}) \geq p. \tag{5.118}$$

Equivalently, sorting tokens in descending order of probability $P(w_{(1)}) \geq P(w_{(2)}) \geq \cdots \geq P(w_{(|\mathcal{V}|)})$, the nucleus cutoff index $k^*(p)$ is:

$$k^*(p) = \min \left\{ k \in \{1, \ldots, |\mathcal{V}|\} \;\middle|\; \sum_{i=1}^{k} P(w_{(i)} \mid w_{<t}) \geq p \right\}. \tag{5.119}$$

All tokens with rank $i > k^*(p)$ are zeroed out, and the probabilities within $\mathcal{V}^{(p)}$ are renormalized:

$$P'(w_i) = \begin{cases} \frac{P(w_i)}{\sum_{w_j \in \mathcal{V}^{(p)}} P(w_j)} & \text{if } w_i \in \mathcal{V}^{(p)}, \\ 0 & \text{otherwise.} \end{cases} \tag{5.120}$$

The next token is sampled from P'.

Worked numerical example: adaptive candidate cardinality. To demonstrate the dynamic adaptivity of nucleus sampling, consider setting threshold $p = 0.90$ across two different contexts:

- **Context A (Peaked / Low Entropy)**: Prompt: `"The capital of France is"`.

$$P(\texttt{"Paris"}) = 0.92, \quad P(\texttt{"the"}) = 0.03, \quad P(\texttt{"a"}) = 0.01, \quad \ldots$$
$$\text{Cumulative sum}: \quad P(\texttt{"Paris"}) = 0.92 \geq 0.90.$$

  The nucleus contains **only 1 token**: $\mathcal{V}^{(p)} = \{\texttt{"Paris"}\}$. The candidate pool automatically collapses to greedy behavior, preventing hallucinations.

- **Context B (Flat / High Entropy)**: Prompt: `"She looked through the telescope and saw"`. Suppose tokens $w_{(1)}$ through $w_{(15)}$ each carry $P \approx 0.06$. To accumulate $\sum P \geq 0.90$, the nucleus retains $k^*(0.90) = 15$ tokens. The candidate pool dynamically expands, allowing rich lexical variation.

Figure 5.26 demonstrates this dynamic expansion and contraction across high-entropy and low-entropy distributions.

Evaluating nucleus sampling with $p = 0.90$ on the Sherlock Holmes prompt produces fluent, highly natural prose that maintains the atmospheric literary tone of Arthur Conan Doyle without degenerate loops or tail gibberish:

```
"Mr.  Holmes had been talking for awhile and seemed very engrossed in the
conversation.  He wished to conduct his last spoke through the eyes of
another, and by that he meant to tell me the whole of the publick story
that had been passed from one of them into the other..."
```

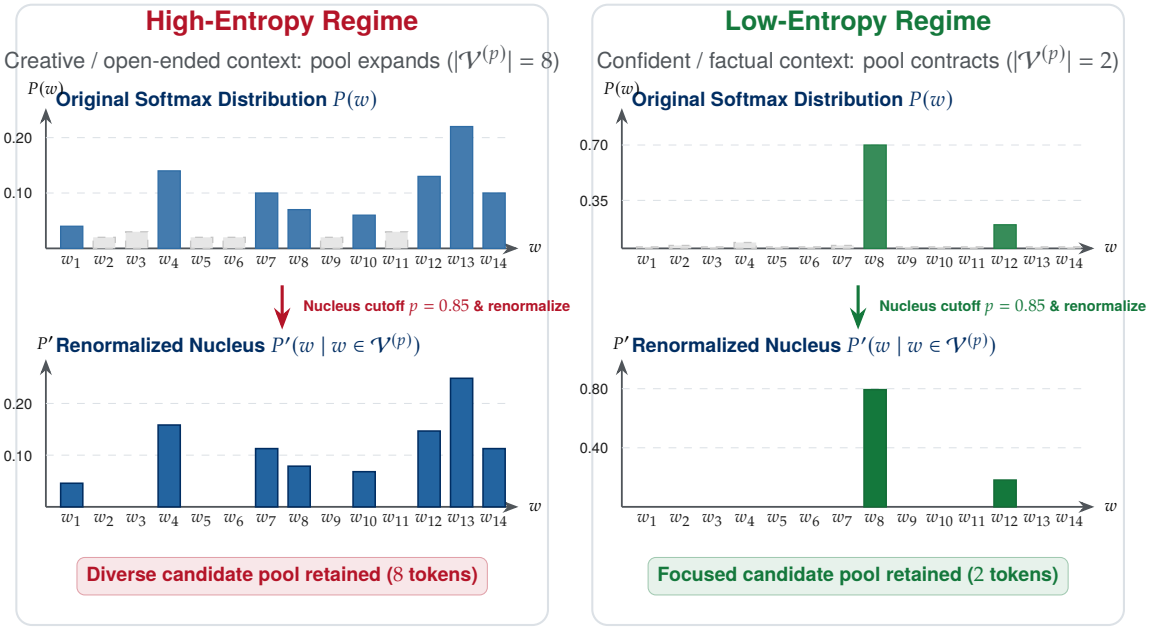


Figure 5.26: Top-p (Nucleus) sampling mechanism. Instead of a fixed k, the candidate pool dynamically expands or contracts to cover cumulative probability mass p. In high-entropy regimes (left), the pool expands to preserve lexical diversity (8 tokens); in low-entropy regimes (right), the pool automatically contracts to dominant tokens (2 tokens), preventing tail hallucinations [63].

5.12.7 Empirical synthesis: the Sherlock Holmes spectrum and decoding selection

The Sherlock Holmes generation spectrum. To directly observe how temperature interacts with sampling mechanics, Table 5.5 summarizes the empirical outputs obtained across temperature and truncation regimes conditioned on the identical Sherlock Holmes prefix.

Algorithmic selection guide for software engineering and NLP tasks. In practical production deployments, no single decoding policy dominates all use cases. The optimal strategy depends strictly on whether the downstream task prioritizes **invariance to randomness** (factual precision, exact syntax) or **stochastic diversity** (creative exploration, synthetic data generation):

1. **Deterministic Tasks (Code Generation, Unit Test Synthesis, Mathematical Reasoning, Machine Translation)**: Require strict reproducibility and syntactic validity. Standard practice employs **Greedy Decoding** ($T = 0$) or **Beam Search** with small beam width ($B \in [2, 5]$) and length penalty $\alpha \in [0.6, 0.8]$.

2. **Open-Ended Creative Tasks (Conversational Agents, Story Generation, Synthetic Documentation)**: Require human-like cadence and vocabulary richness. Standard practice employs **Top-p (Nucleus) Sampling** ($p \in [0.90, 0.95]$) coupled with moderate temperature ($T \in [0.6, 0.8]$).

3. **Hybrid Pipeline: Combining Temperature, Top-k, and Top-p**: Modern inference engines (e.g., vLLM, Hugging Face 'transformers') execute decoding as a multi-stage sequential filter pipeline:

$$\mathbf{z} \xrightarrow{\text{Temperature } T} \frac{\mathbf{z}}{T} \xrightarrow{\text{Top-}k\text{ Truncation}} \mathcal{V}^{(k)} \xrightarrow{\text{Top-}p\text{ Nucleus}} \mathcal{V}^{(p)} \xrightarrow{\text{Softmax \& Categorical Draw}} \hat{w}_t. \tag{5.121}$$

Table 5.5: Empirical generation spectrum across decoding policies on the Sherlock Holmes testbed.

Decoding Policy	Configuration	Observed Output Behavior
Greedy (Argmax)	Deterministic ($T \to 0$)	Collapses into cyclic repetition: *"I am sorry, Mr. Holmes... I am sorry, Mr. Holmes..."*
Beam Search	Width $B = 4$, $\alpha = 0.8$	Grammatically pristine but highly generic and formulaic; lacks human stylistic variance.
Pure Random	Multinomial ($T = 1.0$)	Catastrophic tail contamination: emits ungrammatical fragments (*"owled wild–it seems..."*).
Temperature	$T = 0.1$ (Near Greedy)	Repetition loop persists identically to greedy decoding.
Temperature	$T = 0.5$ (Balanced Low)	Highly coherent, narrative continuity: *"It was a very sad moment, but a true one..."*
Temperature	$T = 1.1$ (High Entropy)	Stylistic drift; bizarre metaphorical imagery (*"grim gulags..."*).
Temperature	$T = 1.5$ (Extreme Noise)	Complete entropy collapse into gibberish subwords (*"IMLEG it off... doctorm Hiraghbene"*).
Top-k	$k = 50$, $T = 0.8$	Fluent dialogue, eliminates bizarre tails, but risks hallucinating if context is strictly factual.
Top-p (Nucleus)	$p = 0.90$, $T = 0.7$	Optimal human-like balance; dynamic candidate pool preserves narrative voice while preventing collapse.

Scaling logits by T first modulates relative entropy; filtering by Top-k removes extreme vocabulary outliers (e.g., $k = 50$); filtering by Top-p dynamically adapts candidate size for the remaining distribution; and sampling from the renormalized subset provides controlled, diverse outputs.

5.13 Comparative paradigm synthesis and bridge to scaling laws

Table 5.6 synthesizes the architectural trade-offs across the three Transformer families studied in this chapter.

Table 5.6: Comprehensive comparison of Transformer architectural paradigms.

Feature	Encoder-Decoder (e.g., T5)	Encoder-Only (e.g., BERT)	Decoder-Only (Preview: Chapter 6)
Attention types	Self-Attention, Masked Self-Attention, Cross-Attention	Bidirectional Self-Attention only	Causal Masked Self-Attention only
Pre-training loss	Sequence-to-Sequence span corruption	Masked LM (15%) + Next Sentence Prediction	Autoregressive Next-Token Prediction
Receptive field	Bidirectional (Enc) + Causal (Dec)	Fully Bidirectional across all tokens	Strictly Causal (preceding tokens only)
Best suited for	Translation, summarization, structured text-to-text	Sentence classification, NER, semantic code search	Open-ended text generation, dialogue, few-shot prompting
Inference style	Autoregressive decoding conditioned on encoder	Single forward pass (non-autoregressive)	Autoregressive generation loop with sampling

The causal bridge to Scaling Laws. While encoder-only models dominated semantic representation, syntax parsing, and discriminative classification, decoder-only models introduced a profound architectural unification: their pre-training objective (autoregressive next-token prediction) is identical to their inference task. By pairing decoder-only transformers with stochastic sampling strategies—such as nucleus sampling ($p \in [0.9, 0.95]$) and temperature scaling ($T \in [0.6, 0.8]$)—researchers uncovered an extraordinary emergent capability: as model parameter count, training token volume, and computational budget scale upward, language models exhibit few-shot and zero-shot reasoning without task-specific parameter fine-tuning. In the subsequent chapter, we formalize the empirical principles and mathematical boundaries governing this regime: **Scaling Laws and Large-Scale Pre-training**.

Chapter 6

LLM pre-training, scaling laws, and open models

In Chapter 5, we established the fundamental architectural mechanics of the Transformer: multi-head self-attention, feed-forward sub-layers, residual connections, and normalization layers. We demonstrated that removing sequential recurrence unblocks massive hardware acceleration across tensor cores while enabling tokens to attend directly across arbitrary sequence lengths. However, an uninitialized Transformer with random weights produces complete syntactic and semantic chaos. The decisive breakthrough of modern natural language processing lies in the **pre-training paradigm**: training high-capacity neural architectures on vast, unannotated text corpora to acquire general-purpose linguistic, factual, and reasoning representations prior to downstream task adaptation.

This chapter investigates the foundational trajectory that transformed sequence modeling from task-specific fine-tuning into the era of generalist foundation models. We begin by examining the architectural fork between encoder-only, encoder-decoder, and decoder-only models, tracing why causal autoregressive generation became the dominant paradigm, synthesized into a unified chronological timeline of foundation model milestones (2017–2024). We then dissect the evolutionary milestones of the Generative Pre-trained Transformer series—GPT-1, GPT-2, and GPT-3—showing how unsupervised next-token prediction unlocked emergent in-context learning.

Next, we establish a first-principles mathematical exploration of **scaling laws**. We first analyze the empirical power laws formalized by Kaplan et al. [15], proving why early research concluded that parameter growth should outpace dataset size ($N \propto C^{0.73}, D \propto C^{0.27}$). We then diagnose the methodological flaws of this first paradigm, exposing the crisis of colossal yet severely undertrained half-trillion-parameter models. This motivates our deep focus on DeepMind's Chinchilla framework (Hoffmann et al. [3]), where we derive via three independent analytical approaches that compute optimality demands equal scaling of parameters and data ($N \propto \sqrt{C}, D \propto \sqrt{C}$). Finally, we trace the modern frontier of inference-optimal training and the democratization of open-weights models.

6.1 Architectural paradigms and pre-training objectives

Prior to the advent of large-scale pre-training, deep learning for natural language processing was dominated by supervised, task-specific architectures. Practitioners collected human-annotated datasets for individual tasks—such as sentiment analysis (SST-2), natural language inference (SNLI, MNLI), named entity recognition (CoNLL-2003), or question answering (SQuAD)—and trained specialized neural networks from scratch or initialized word vectors using static lookup tables such as Word2Vec or GloVe.

This classical paradigm suffered from three catastrophic structural bottlenecks:

1. **The annotation bottleneck**: Supervised datasets require expensive, human-in-the-loop labeling. Real-world labeled corpora typically span only 10^4 to 10^5 samples. Such small

training volumes severely constrain the parameter capacity of neural models, inducing severe overfitting when scaling beyond several million parameters.

2. **Static representation collapse**: Static embeddings map each word to a single spatial coordinate $\mathbf{w} \in \mathbb{R}^d$, completely ignoring polysemy and local syntactic context (e.g., the word "bank" in "river bank" versus "investment bank").

3. **Architectural siloing**: Every distinct NLP task required custom output layers, specialized pooling operations, or task-specific bi-directional recurrent architectures, preventing positive transfer across related domains.

The solution to these bottlenecks emerged with **self-supervised learning**, wherein the supervisory signal is derived autonomously from raw, unannotated text without human labels. By exploiting the inherent sequential structure of natural language, models can be pre-trained on billions or trillions of tokens, learning grammatical syntax, world knowledge, semantic coherence, and logical reasoning patterns.

6.1.1 The tripartite architectural fork

Following the introduction of the Transformer by Vaswani et al. [1], the NLP community underwent a major architectural fork based on how the self-attention mechanism is masked and connected:

- **Encoder-only architectures (e.g., BERT [53], RoBERTa)**: Encoder-only models utilize full bidirectional self-attention across the sequence: every token attends to all previous, current, and subsequent tokens simultaneously. The attention mask $\mathbf{M} \in \mathbb{R}^{T \times T}$ is all zeros (no masking). Pre-training relies on the **Masked Language Modeling (MLM)** objective: a random subset (typically 15%) of tokens is corrupted (replaced by a special $\langle \text{MASK} \rangle$ token, random words, or kept unchanged), and the model must reconstruct the original tokens:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log \mathbb{P}_\theta(w_i \mid \mathbf{w}_{\setminus \mathcal{M}}), \tag{6.1}$$

  where $\mathcal{M}$ is the set of masked indices and $\mathbf{w}_{\setminus \mathcal{M}}$ denotes the unmasked context. While encoder-only models excel at discriminative understanding tasks (sentence classification, extractive question answering, token classification), they suffer from a fatal drawback: *they cannot generate text autoregressively in a single forward pass*. Generating a sequence of length L requires L separate forward iterations, and predicting a masked token conditional on future masked tokens violates probabilistic chain rules.

- **Encoder-decoder architectures (e.g., T5 [60], BART)**: Encoder-decoder models retain the original Vaswani et al. [1] duality. A bidirectional encoder processes the input prefix $\mathbf{x} = [x_1, \ldots, x_{T_{\text{in}}}]$, and an autoregressive decoder generates the target sequence $\mathbf{y} = [y_1, \ldots, y_{T_{\text{out}}}]$ token by token using causal self-attention and cross-attention over the encoder's top hidden states $\mathbf{H}_{\text{enc}}$. Pre-training typically uses a span corruption objective (e.g., in T5, replacing arbitrary spans of text with sentinel tokens $\langle X \rangle, \langle Y \rangle$ and training the decoder to emit the masked spans). This provides an elegant, unified "text-to-text" framework for sequence transduction (machine translation, document summarization). However, for pure open-ended text generation, maintaining two separate stacks doubles parameter memory overhead and requires executing cross-attention at every decoding step.

- **Decoder-only architectures (e.g., GPT [65], GPT-2 [12], GPT-3 [2])**: Decoder-only models eliminate the encoder entirely, consisting of a single, stacked Transformer decoder equipped

with **causal (lower-triangular) self-attention masking**:

$$\mathbf{M}_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i. \end{cases} \tag{6.2}$$

The causal mask ensures that the representation of token t depends strictly on tokens at positions $\leq t$. Pre-training optimizes the **Causal Language Modeling (CLM)** objective, predicting the next token w_t conditioned on historical prefix $w_{<t}$:

$$\mathcal{L}_{\text{CLM}}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log \mathbb{P}_\theta(w_t \mid w_1, w_2, \ldots, w_{t-1}). \tag{6.3}$$

6.1.2 Convergence on the causal decoder-only paradigm

Although encoder-only and encoder-decoder models initially held state-of-the-art results across standard NLU benchmarks (such as GLUE and SuperGLUE), the generative AI revolution converged almost exclusively on the **decoder-only** architecture. This convergence stems from fundamental computational and algorithmic advantages:

1. **Unified training and generative alignment**: Causal next-token prediction matches the physical inference process. Every training token serves as both input context and ground-truth target, maximizing data efficiency without requiring artificial mask tokens that induce train-test distribution mismatches.

2. **Simplified memory and KV caching**: In decoder-only models, key and value tensors for prior tokens are simply appended to a single, contiguous cache tensor ($\mathbf{K}, \mathbf{V} \in \mathbb{R}^{B\times H\times T\times d_k}$), avoiding the dual-cache synchronization required in encoder-decoder cross-attention.

3. **Emergence of in-context learning**: As model scale expands, the causal objective forces the model to encode implicit reasoning, syntactic parsing, and world knowledge into its left-to-right activation trajectory, enabling zero-shot and few-shot task execution without gradient updates.

Having established why the decoder-only causal framework is uniquely suited for general-purpose sequence modeling, Figure 6.1 charts the complete chronological timeline of foundation model milestones from 2017 to 2024 with monthly granularity, establishing the historical roadmap across pre-training foundations, in-context emergence, compute scaling laws, and modern open foundation models. We now trace each milestone in detail, beginning with OpenAI's Generative Pre-trained Transformer series.

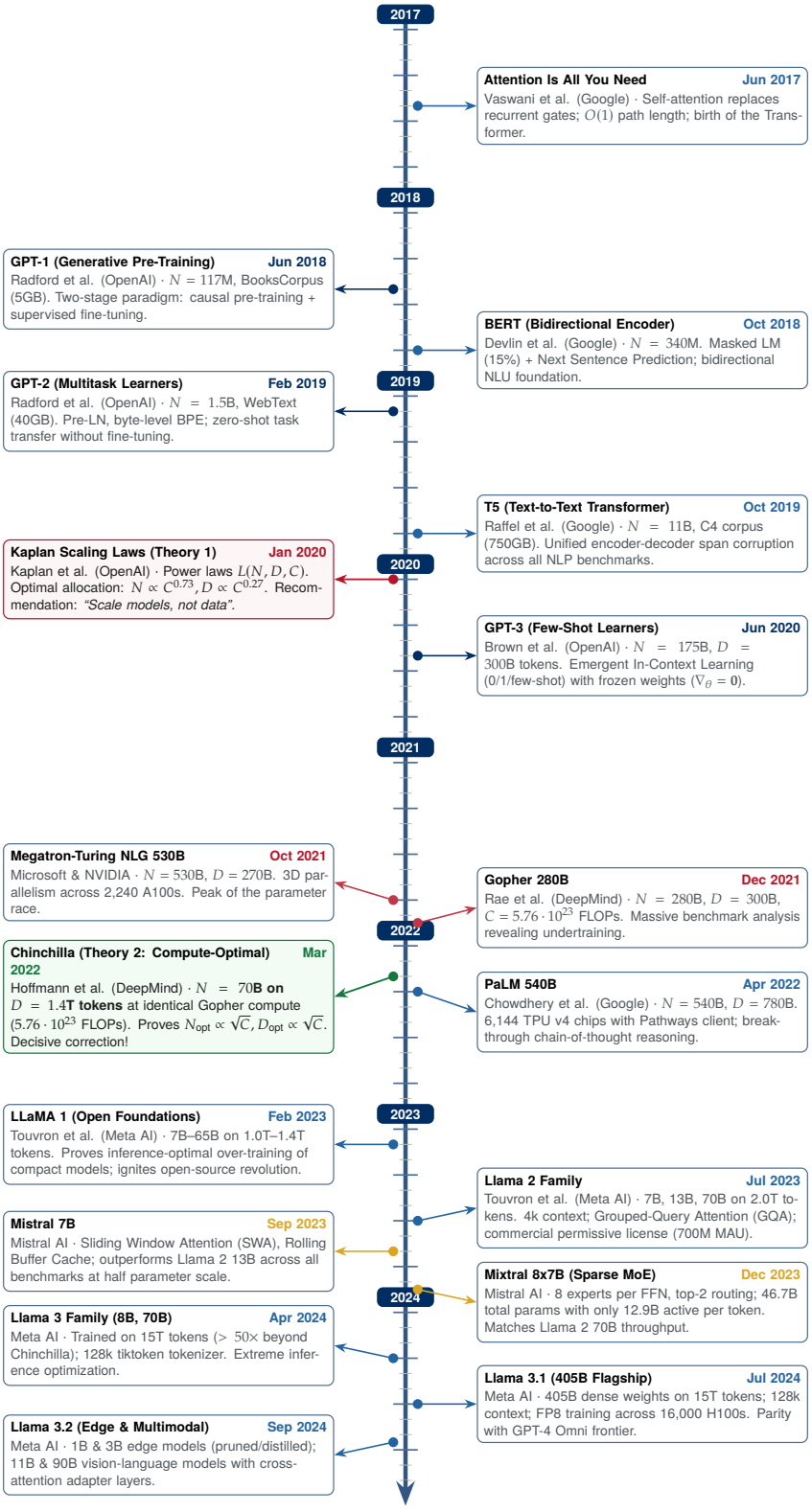


Figure 6.1: Chronological timeline of large language model milestones (2017–2024) with monthly granularity. Each year is partitioned into a 12-month axis where milestones are positioned according to their exact publication month (e.g., Kaplan scaling laws in January at 1/12 of 2020, Chinchilla in March at 3/12 of 2022). The historical trajectory charts the convergence on causal decoder-only models and the transition from parameter scale to compute optimality and inference-optimal open weights.

6.2 GPT-1: Generative pre-training

In 2018, Radford et al. [65] published "Improving Language Understanding by Generative Pre-Training", introducing the original **GPT-1**. The central thesis of GPT-1 was a two-stage semi-supervised framework:

1. **Stage 1: Unsupervised pre-training**: Train a high-capacity 12-layer decoder-only Transformer on a massive, unannotated corpus of contiguous text using causal next-token prediction to acquire general linguistic representations.
2. **Stage 2: Supervised fine-tuning**: Adapt the pre-trained model to specific downstream discriminative tasks by appending a lightweight task-specific linear classification head and fine-tuning all model weights on labeled data.

This canonical two-stage architecture and decoder-only block topology are illustrated in Figure 6.2.

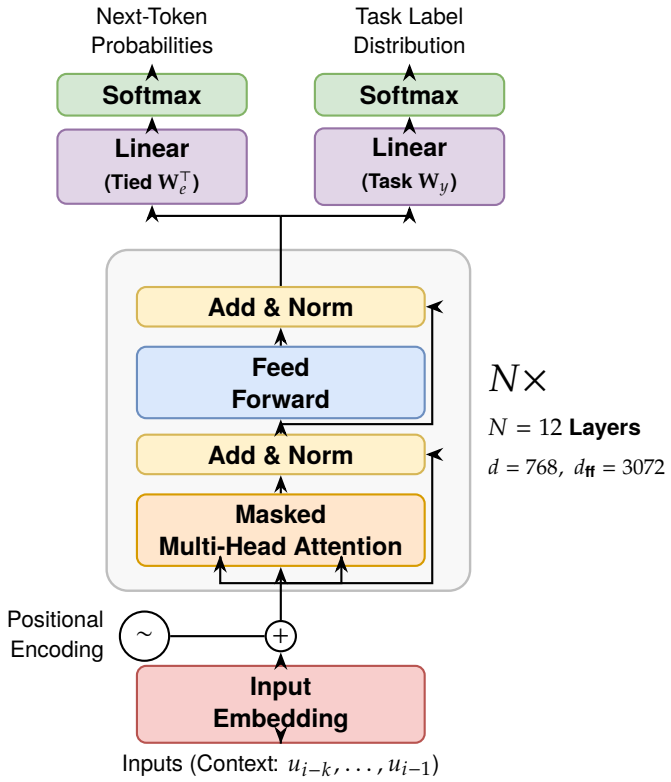


Figure 6.2: The GPT-1 decoder-only Transformer architecture and two-stage adaptation framework [65]. Input tokens and learned positional embeddings are summed and propagated through a stack of $N = 12$ Post-LN decoder blocks. Each block features causal Masked Multi-Head Attention and a position-wise Feed-Forward Network, coupled with identity residual shortcuts and post-layer normalization (`Add & Norm`). In Stage 1 (unsupervised pre-training), output activations pass through tied linear projection $\mathbf{W}_e^\top$ and softmax to predict next tokens. In Stage 2 (supervised fine-tuning), the final token activation passes through task-specific linear classifier $\mathbf{W}_y$ and softmax.

6.2.1 Unsupervised pre-training mechanics

Let $\mathcal{U} = \{u_1, u_2, \ldots, u_N\}$ denote an unlabeled corpus of discrete tokens. The unsupervised pre-training objective minimizes the negative log-likelihood over a sliding context window of

size k:

$$\mathcal{L}_1(\mathcal{U}) = -\sum_{i=1}^{N} \log \mathbb{P}(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta), \tag{6.4}$$

where Θ represents the complete parameter set of the network.

Mathematical formulation and layer pass. Given an input sequence of token indices $\mathbf{u} = [u_{-k}, \ldots, u_{-1}]$, the initial activation matrix $\mathbf{H}_0 \in \mathbb{R}^{k \times d}$ is constructed by summing the token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times d}$ and the learned positional embedding matrix $\mathbf{W}_p \in \mathbb{R}^{k \times d}$:

$$\mathbf{H}_0 = \mathbf{U}\mathbf{W}_e + \mathbf{W}_p, \tag{6.5}$$

where $\mathbf{U} \in \{0, 1\}^{k \times |\mathcal{V}|}$ is the one-hot indicator matrix of the input prefix.

The activation tensor is then sequentially propagated through a stack of L identical Transformer decoder blocks:

$$\mathbf{H}_l = \text{TransformerBlock}(\mathbf{H}_{l-1}), \quad \forall l \in \{1, 2, \ldots, L\}. \tag{6.6}$$

Each GPT-1 block employs the original post-layer-normalization formulation:

$$\mathbf{H}'_l = \text{LayerNorm}\left(\mathbf{H}_{l-1} + \text{MultiHead}\left(\mathbf{H}_{l-1}, \mathbf{H}_{l-1}, \mathbf{H}_{l-1}\right)\right), \tag{6.7}$$

$$\mathbf{H}_l = \text{LayerNorm}\left(\mathbf{H}'_l + \text{FFN}(\mathbf{H}'_l)\right), \tag{6.8}$$

where the Multi-Head Attention (MultiHead $(\cdot)$) enforces causal masking $\mathbf{M}_{i,j} = -\infty$ for $j > i$, and the position-wise feed-forward network uses the Gaussian Error Linear Unit (GELU) activation:

$$\text{FFN}(\mathbf{X}) = \text{GELU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad \mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \ \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}. \tag{6.9}$$

Finally, the categorical probability distribution over the vocabulary $\mathcal{V}$ for predicting the next token u_i is computed by projecting the top hidden state $\mathbf{H}_L[k] \in \mathbb{R}^d$ through the transpose of the embedding matrix (weight tying):

$$\mathbb{P}(u_i \mid u_{i-k}, \ldots, u_{i-1}) = \text{softmax}\left(\mathbf{H}_L[k]\mathbf{W}_e^\top\right). \tag{6.10}$$

6.2.2 Supervised task adaptation and input formatting

In the second stage, the pre-trained Transformer is adapted to a labeled dataset C, where each instance consists of an input sequence of tokens $\mathbf{x} = [x_1, \ldots, x_m]$ and a categorical target label $y \in \mathcal{Y}$. The sequence $\mathbf{x}$ is passed through the pre-trained Transformer to obtain the final activation of the last token, $\mathbf{h}_L^m \in \mathbb{R}^d$. A linear projection matrix $\mathbf{W}_y \in \mathbb{R}^{d \times |\mathcal{Y}|}$ maps this hidden state to class logits:

$$\mathbb{P}(y \mid x_1, \ldots, x_m) = \text{softmax}\left(\mathbf{h}_L^m \mathbf{W}_y\right). \tag{6.11}$$

The supervised objective minimizes cross-entropy loss over C:

$$\mathcal{L}_2(C) = -\sum_{(\mathbf{x}, y) \in C} \log \mathbb{P}(y \mid x_1, \ldots, x_m). \tag{6.12}$$

Crucially, Radford et al. [65] discovered that including an **auxiliary language modeling objective** during fine-tuning substantially improves generalization and prevents catastrophic forgetting of pre-trained syntactic representations:

$$\mathcal{L}_3(C) = \mathcal{L}_2(C) + \lambda \mathcal{L}_1(C), \tag{6.13}$$

where λ is a balancing hyperparameter (typically $\lambda = 0.5$).

A key architectural innovation of GPT-1 was the avoidance of custom architectural modifications for different downstream NLP tasks. Instead of designing specialized pooling or recurrent layers, all structured inputs were flattened into a single linear sequence using delimiter tokens (Start $\langle s \rangle$, Delimiter $\langle \text{delim} \rangle$, and Extract $\langle e \rangle$), as depicted in Figure 6.3:

- **Text Classification**: The input text is flanked by start and end delimiters: $[\langle s \rangle, \text{Sequence}, \langle e \rangle]$. The final state at $\langle e \rangle$ is fed into the classifier.
- **Natural Language Inference (NLI)**: The premise and hypothesis are concatenated with a delimiter token: $[\langle s \rangle, \text{Premise}, \langle \text{delim} \rangle, \text{Hypothesis}, \langle e \rangle]$.
- **Semantic Similarity / Paraphrasing**: Because similarity is symmetric, the input order should not matter. Two sequences are passed in parallel: $[\langle s \rangle, \text{Text 1}, \langle \text{delim} \rangle, \text{Text 2}, \langle e \rangle]$ and $[\langle s \rangle, \text{Text 2}, \langle \text{delim} \rangle, \text{Text 1}, \langle e \rangle]$. Their final extract representations are element-wise summed prior to classification.
- **Multiple-Choice Question Answering**: For context C, question Q, and candidate answers $\{A_1, \ldots, A_k\}$, k separate sequences are formed: $[\langle s \rangle, C, \langle \text{delim} \rangle, Q, \langle \text{delim} \rangle, A_k, \langle e \rangle]$. Each sequence produces an extract representation projected to a scalar logit, normalized across all candidates via Softmax.

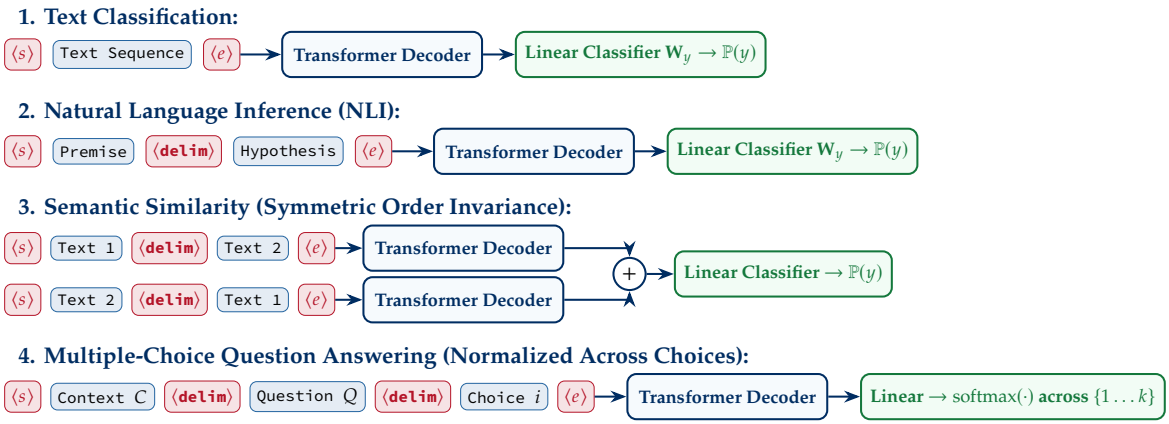


Figure 6.3: Downstream task input transformations with delimiter tokens in GPT-1 [65]. By linearizing structured inputs with special delimiter tokens ($\langle s \rangle$, $\langle \text{delim} \rangle$, $\langle e \rangle$), diverse NLP tasks—classification, entailment (NLI), symmetric semantic similarity, and multiple-choice question answering—are processed through the pre-trained Transformer decoder without any task-specific architectural modifications.

The physical configuration of GPT-1 was established as follows:

- **Model Dimensions**: $L = 12$ layers, $H = 12$ attention heads, hidden embedding dimension $d = 768$, and intermediate feed-forward dimension $d_{\text{ff}} = 3072$, totaling approximately 117×10^6 parameters (117M).
- **Context and Positional Embeddings**: Maximum context length $T = 512$ tokens, using learned absolute positional embeddings $\mathbf{W}_p \in \mathbb{R}^{512 \times 768}$.
- **Tokenization**: Byte-Pair Encoding (BPE) vocabulary with 40,000 merge operations.
- **Pre-training Corpus**: The **BooksCorpus** dataset, containing over 7,000 unpublished fiction books totaling approximately 5 GB of raw text. Crucially, BooksCorpus provided long, contiguous passages of prose, enabling the model to condition on extended historical context and learn long-range semantic dependencies.

While GPT-1 achieved major improvements over state-of-the-art recurrent models across 9 of 12 evaluated NLP benchmarks, it still required collecting task-specific annotated datasets and updating model parameters via gradient descent ($\nabla_\theta \neq \mathbf{0}$) for every single target task. This bottleneck prompted researchers to ask: *can a sufficiently large language model perform tasks directly from raw natural language instructions, without requiring any weight updates?* This question directly motivated GPT-2.

6.3 GPT-2: Zero-shot multitask learning

In 2019, Radford et al. [12] released GPT-2 with the landmark paper "Language Models are Unsupervised Multitask Learners". GPT-2 introduced a radical paradigm shift: **zero fine-tuning**. The authors hypothesized that sufficiently high-capacity language models trained on diverse, naturalistic text corpora naturally learn to perform multiple downstream tasks *without any parameter updates* ($\nabla_\theta = \mathbf{0}$).

Mathematically, supervised learning estimates the conditional distribution $\mathbb{P}(\text{target} \mid \text{input})$. Multitask learning conditions additionally on the task identifier: $\mathbb{P}(\text{target} \mid \text{input}, \text{task})$. Radford et al. posited that in natural language, both the input and the task description can be framed as a single unified sequence of tokens. For example:

- **Machine Translation**: $\mathbf{x}$ = "Translate English to French: The book is on the table =>"
- **Summarization**: $\mathbf{x}$ = "[Article text] TL;DR:"
- **Reading Comprehension**: $\mathbf{x}$ = "[Passage] Q: What color was the car? A:"

By simply continuing the sequence autoregressively, the language model solves the specified task without explicit fine-tuning.

This unified zero-shot generative decoder architecture is illustrated in Figure 6.4.

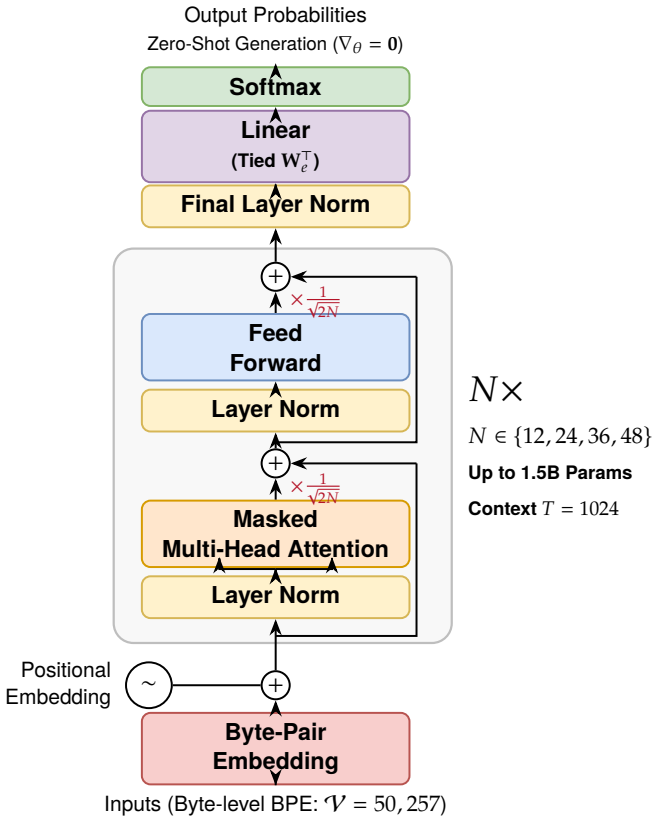


Figure 6.4: The GPT-2 zero-shot multitask decoder architecture [12]. Built upon the Pre-Layer Normalization (Pre-LN) paradigm, Layer Normalization precedes each sublayer while an uninterrupted identity residual shortcut preserves unperturbed gradient highways. Output projection layers are scaled at initialization by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ to bound residual variance growth across deep stacks ($N \in \{12, 24, 36, 48\}$). An additional final LayerNorm stabilizes activations prior to the tied linear projection and softmax language modeling head.

6.3.1 Pre-training corpus curation: the WebText heuristic

Pre-training GPT-1 on BooksCorpus limited the model's exposure to diverse writing styles, non-fiction reasoning, code, and dialogue. To support generalist multitask emergence, GPT-2 required a radically larger and more diverse corpus. However, naive web scraping (such as raw Common Crawl) contains vast amounts of low-quality text, repetitive programmatic spam, duplicate marketing copy, and machine-translated gibberish. Radford et al. engineered an elegant, human-curated filtering heuristic:

- They scraped all outbound web links submitted to the social news platform **Reddit**.
- They filtered exclusively for links originating from posts that received at least **3 karma upvotes**.
- The karma threshold served as an automated human curation heuristic, selecting content that human users found interesting, informative, or well-written.
- The resulting dataset, named **WebText**, encompassed text from over 45 million links, totaling approximately 40 GB of high-quality text (an 8× expansion over BooksCorpus).
- Importantly, Wikipedia articles were explicitly filtered out of WebText to test whether the model could generalize to unseen encyclopedic domains during evaluation.

6.3.2 Architectural stabilization: Pre-LN and byte-level BPE

While preserving the core causal decoder-only Transformer backbone, GPT-2 introduced critical architectural refinements to stabilize training dynamics across deep networks and resolve out-of-vocabulary representation failures.

Pre-layer normalization (Pre-LN).

In GPT-1 (and the original Transformer [1]), **Post-LN** placed layer normalization on the residual path strictly after the addition:

$$\mathbf{x}_l = \text{LayerNorm}(\mathbf{x}_{l-1} + \text{SubLayer}(\mathbf{x}_{l-1})). \tag{6.14}$$

Under Post-LN, backpropagating gradients must traverse the derivative of the LayerNorm operator at every single layer. For deep architectures ($L \geq 24$), the gradient magnitude through the unnormalized skip connections decays exponentially or oscillates wildly, necessitating delicate learning-rate warmup schedules to avoid catastrophic early divergence.

To resolve this instability, GPT-2 adopted **Pre-Layer Normalization (Pre-LN)**, applying normalization strictly to the input of each sub-layer:

$$\mathbf{x}'_l = \mathbf{x}_{l-1} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x}_{l-1})). \tag{6.15}$$

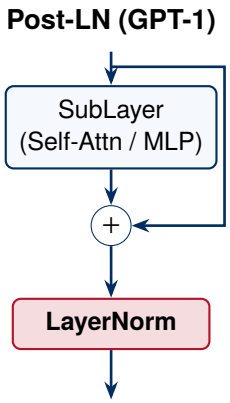
Because $\mathbf{x}_{l-1}$ enters the addition unmodified, the residual stream forms a continuous identity highway:

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} \text{SubLayer}(\text{LayerNorm}(\mathbf{x}_{l-1})). \tag{6.16}$$

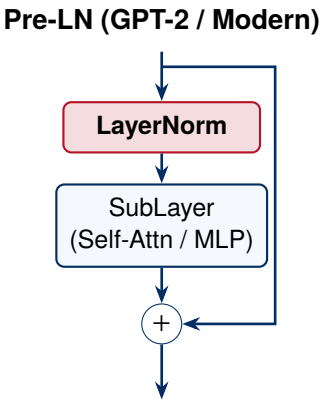
Differentiating with respect to the input embedding $\mathbf{x}_0$ yields an uninterrupted identity matrix $\mathbf{I}$, allowing error gradients to propagate unattenuated throughout all layers. An additional final layer normalization $\text{LayerNorm}_{\text{final}}$ is appended after the final layer to normalize the accumulated residual sum prior to output projection.

Residual projection scaling.

While Pre-LN ensures clean gradient propagation, the unnormalized additive summation across



(a) Post-LN (GPT-1): Normalization follows residual addition.



(b) Pre-LN (GPT-2): Normalization precedes sub-layer computation.

Figure 6.5: Structural comparison between Post-LN and Pre-LN Transformer blocks. Pre-LN maintains a clean identity residual highway, preventing vanishing and exploding gradients in deep networks.

depth causes the variance of the residual stream to grow monotonically with the number of layers:

$$\mathbb{V}\mathrm{ar}(\mathbf{x}_l) \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_{l-1}) + \mathbb{V}\mathrm{ar}\big(\mathrm{SubLayer}(\mathrm{LayerNorm}(\mathbf{x}_{l-1}))\big). \tag{6.17}$$

If each sub-layer output has variance $\sigma^2 \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_{l-1})$, after M residual additions the variance at layer L expands linearly: $\mathbb{V}\mathrm{ar}(\mathbf{x}_L) \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_0) + M\sigma^2$. Because each Transformer decoder block contains *two* distinct residual branches—the causal Multi-Head Attention and the position-wise Feed-Forward Network—the total number of residual additions across the network is:

$$M = 2N_{\text{layer}}. \tag{6.18}$$

In deep models such as GPT-2 Extra Large ($N_{\text{layer}} = 48$), the cumulative variance would reach $96\sigma^2$, leading to extreme activation explosion and numerical instability in half-precision representations.

To bound the variance across the entire residual stream so that $\mathbb{V}\mathrm{ar}(\mathbf{x}_L) \approx \mathcal{O}(1)$, Radford et al. [12] introduced **residual projection scaling**. At initialization, the weights of the final linear projection layer in each residual branch are scaled down by:

$$\gamma = \frac{1}{\sqrt{2N_{\text{layer}}}}. \tag{6.19}$$

Exact structural location in the decoder: As illustrated in Figure 6.6, this scaling factor is applied precisely to two weight matrices inside every decoder layer:

1. **Attention Output Projection $\mathbf{W}^O$**: The linear projection matrix mapping concatenated attention heads back to the hidden dimension d:

$$\mathbf{W}^O \leftarrow \frac{1}{\sqrt{2N_{\text{layer}}}}\mathbf{W}^O, \quad \mathbf{W}^O \in \mathbb{R}^{d\times d}. \tag{6.20}$$

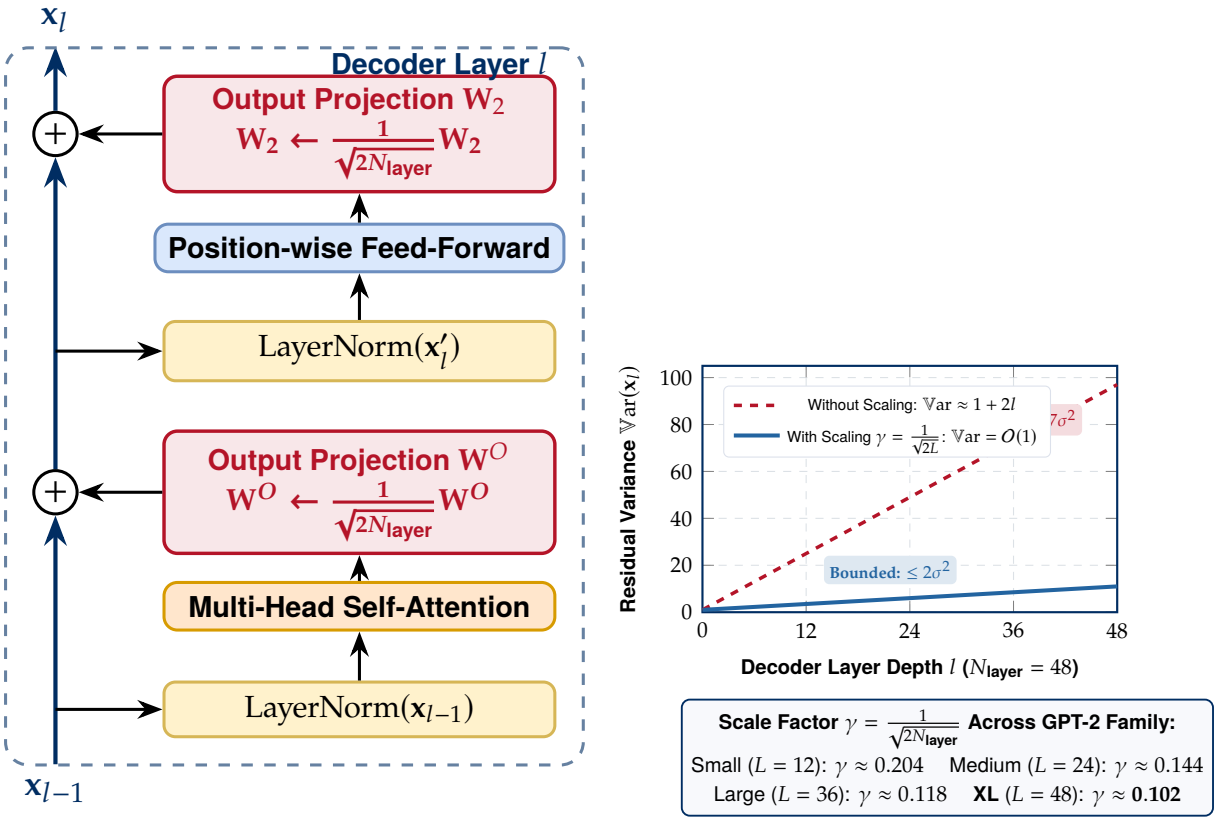
2. **Feed-Forward Output Projection $\mathbf{W}_2$**: The second linear projection matrix mapping the intermediate expanded representation back to dimension d:

$$\mathbf{W}_2 \leftarrow \frac{1}{\sqrt{2N_{\text{layer}}}}\mathbf{W}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d}. \tag{6.21}$$

Because output variance is quadratic in weight magnitude ($\mathbb{V}\text{ar}(\mathbf{W}\mathbf{x}) = \|\mathbf{W}\|_F^2\, \mathbb{V}\text{ar}(\mathbf{x})$), multiplying weights by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ scales the variance of each residual branch by $\frac{1}{2N_{\text{layer}}}$. Summing across all $2N_{\text{layer}}$ residual branches yields:

$$\mathbb{V}\text{ar}(\mathbf{x}_L) \approx \mathbb{V}\text{ar}(\mathbf{x}_0) + \sum_{m=1}^{2N_{\text{layer}}} \frac{\sigma^2}{2N_{\text{layer}}} = \mathbb{V}\text{ar}(\mathbf{x}_0) + \sigma^2 \approx \mathcal{O}(1), \tag{6.22}$$

maintaining stable variance across arbitrary network depths.



(a) Exact structural location within decoder block (b) Variance accumulation dynamics across depth

Figure 6.6: Residual projection scaling mechanism in GPT-2 [12]. (a) Exact structural location within the decoder block: scaling by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ is applied strictly to the output linear projection matrices ($\mathbf{W}^O$ in Multi-Head Attention and $\mathbf{W}_2$ in the Feed-Forward Network) right before the residual summation nodes ($\bigoplus$). (b) Variance accumulation dynamics across network depth $l \in [0, 48]$: without scaling, residual addition causes activation variance to explode linearly ($\mathbb{V}\text{ar} \propto 2l$), reaching nearly 100× in GPT-2 XL. With residual projection scaling, cumulative variance remains strictly bounded ($\mathcal{O}(1)$), ensuring numerical stability during training.

Byte-level byte-pair encoding (BPE).
GPT-1 used character-level BPE with 40,000 merges, which frequently encountered out-of-vocabulary (⟨UNK⟩) tokens when processing foreign characters, unicode symbols, or source code tokens. GPT-2 introduced **byte-level BPE** operating over raw UTF-8 bytes with a vocabulary of 50,257 tokens (comprising 256 raw byte values, special tokens, and 50,000 learned merges). Because any text sequence decomposes into valid bytes, byte-level BPE guarantees zero out-of-vocabulary tokens on any arbitrary string. Additionally, regex rules prevented BPE from merging across distinct character classes (e.g., preventing punctuation from fusing into words).
Expanded context and batch capacity.
GPT-2 doubled maximum context length from $T = 512$ to $T = 1024$ tokens, enabling the

capture of broader contextual discourse. The training batch size was increased to 512 sequences (totaling 524,288 tokens per training step), stabilizing stochastic gradient descent across massive distributed clusters.

6.3.3 Model family scaling and staged release governance

GPT-2 was released across four distinct model scales, demonstrating continuous, predictable performance scaling as parameter count grew:

- **GPT-2 Small**: 117M parameters ($L = 12, d = 768, H = 12$) — identical in size to GPT-1.
- **GPT-2 Medium**: 345M parameters ($L = 24, d = 1024, H = 16$).
- **GPT-2 Large**: 762M parameters ($L = 36, d = 1280, H = 20$).
- **GPT-2 Extra Large (1.5B)**: 1542M parameters ($L = 48, d = 1600, H = 25$).

Empirically, Radford et al. [12] observed that *perplexity decreased log-linearly with model capacity across all evaluation benchmarks*. Furthermore, without any task-specific fine-tuning, on language modeling benchmarks (WikiText-103, LAMBADA), GPT-2 established new state-of-the-art perplexities in zero-shot settings, beating supervised models. On reading comprehension (CoQA), the 1.5B model achieved an F1 score of 55, matching or outperforming baseline supervised models. On French-to-English translation, prompting with simple natural language exemplars yielded 11.5 BLEU, outperforming several unsupervised translation baselines despite having almost no French in WebText.

In February 2019, OpenAI announced GPT-2 alongside a controversial decision: they declined to release the full 1.5B parameter weights, citing concerns over dual-use technology and potential malicious applications, such as the automated generation of deceptive political disinformation, spam, and synthetic abusive content. They initially published only the 117M model and technical paper. Over the subsequent nine months, OpenAI conducted staged incremental releases: the 345M model in May 2019, the 762M model in August 2019, and finally the complete 1.5B model in November 2019. While this staged release drew sharp initial criticism from open-source advocates, it marked the historical inception of AI safety discourse, model governance protocols, and modern alignment concerns surrounding high-capacity foundation models.

GPT-2 established that increasing parameter count from 117M to 1.5B dramatically improved language modeling loss and unlocked emergent zero-shot capabilities. However, scaling remained an intuitive, trial-and-error craft. Researchers had no mathematical formula to answer: *If we have a fixed budget of compute, how big should our model be? How many tokens should we collect? Does network depth matter more than width?* To answer these questions systematically, researchers turned to empirical physics-inspired scaling laws.

6.4 Kaplan scaling laws

Prior to 2020, scaling neural networks was largely an empirical art driven by intuition and trial-and-error. Researchers lacked quantitative formulas to answer fundamental engineering questions: *If compute budget increases by* 100×*, how much should we expand model parameters versus training dataset size? Does network depth matter more than width? Is training to full convergence optimal?*

In January 2020, Jared Kaplan and colleagues at OpenAI [15] published the landmark paper "Scaling Laws for Neural Language Models". By training dozens of autoregressive Transformer models spanning over four orders of magnitude in compute, three orders of magnitude in dataset size, and three orders of magnitude in parameter count, the authors demonstrated that **language model performance follows precise, predictable power laws** over immense dynamic ranges.

6.4.1 Empirical power-law formulations

When model performance is not bottlenecked by any single resource constraint (such as data exhaustion or inadequate parameter capacity), the test cross-entropy loss $\mathcal{L}$ adheres to power-law scaling relationships:

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \text{for unlimited data } D \text{ and compute } C, \tag{6.23}$$

$$\mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \text{for unlimited parameters } N \text{ and compute } C, \tag{6.24}$$

$$\mathcal{L}(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \quad \text{for optimal allocation of } N \text{ and } D, \tag{6.25}$$

where:

- N is the number of non-embedding parameters.
- D is the number of training tokens.
- C is the total training compute expressed in floating-point operations (FLOPs).
- N_c, D_c, C_c are normalization constants, and $\alpha_N, \alpha_D, \alpha_C$ are scaling exponents.

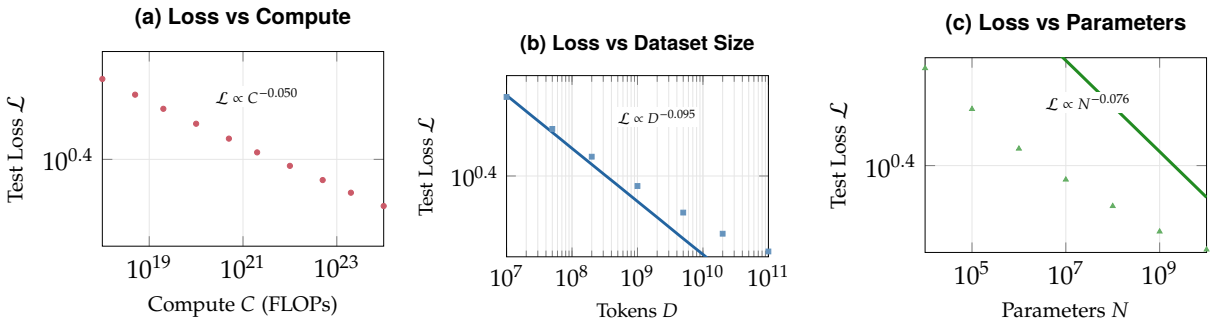
Kaplan et al. empirically estimated the power-law exponents for autoregressive language modeling as:

$$\alpha_N \approx 0.076, \quad \alpha_D \approx 0.095, \quad \alpha_C \approx 0.050. \tag{6.26}$$

A striking finding of Kaplan et al. was that performance depends **very weakly** on detailed architectural hyperparameters:

- **Depth versus width**: For a fixed parameter count N, varying the ratio between the number of layers n_{layer} and the hidden embedding dimension d_{model} over a wide range has negligible impact on cross-entropy loss.
- **Attention heads**: Varying the number of attention heads H (or per-head dimension d_k) produces virtually identical loss trajectories.
- **Feed-forward dimension**: Altering the feed-forward multiplier $d_{\text{ff}}/d_{\text{model}}$ produces minimal deviation.

The fundamental implication is that **scale dominates shape**: macroscopic resource allocation (N, D, C) dictates model performance, while microscopic architectural tuning yields only minor second-order effects.



(a) Compute scaling (b) Dataset size scaling (c) Parameter scaling

Figure 6.7: Empirical scaling laws for language model performance from Kaplan et al. [15]. Cross-entropy test loss demonstrates smooth, predictable power-law scaling spanning multiple orders of magnitude across compute C (FLOPs), dataset size D (tokens), and parameter count N (non-embedding parameters).

6.4.2 Floating-point arithmetic and compute allocation

To analyze compute-optimal allocation, we must establish the physical relationship between floating-point operations, model parameters, and training tokens.

Theorem 6.1: *Transformer compute budget*

For a standard Transformer decoder consisting of N non-embedding parameters trained on D tokens, the total training compute C (in FLOPs) is given by:

$$C \approx 6ND. \tag{6.27}$$

Proof. Consider a general linear matrix multiplication $\mathbf{Y} = \mathbf{X}\mathbf{W}$, where input activations $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$ and weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. Computing each scalar entry of $\mathbf{Y}$ requires d_{in} multiplications and d_{in} additions, totaling $2d_{\text{in}}$ floating-point operations. The matrix has $d_{\text{in}} \times d_{\text{out}}$ parameters and produces $B \times d_{\text{out}}$ outputs. Thus, processing 1 token through $\mathbf{W}$ requires:

$$\text{FLOPs}_{\text{forward}} = 2 \times (\text{number of parameters}). \tag{6.28}$$

Summing across all linear projections in the multi-head attention blocks ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and feed-forward networks ($\mathbf{W}_1, \mathbf{W}_2$), the forward pass requires:

$$C_{\text{forward}} \approx 2N \text{ FLOPs per token.} \tag{6.29}$$

In the backward pass (backpropagation), gradients must be computed with respect to both the activations (to propagate error backward) and the weight parameters (to update weights):

- Gradient with respect to activations: $\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \mathbf{W}^\top \implies 2N$ FLOPs per token.
- Gradient with respect to weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \implies 2N$ FLOPs per token.

Thus, the backward pass requires twice the compute of the forward pass:

$$C_{\text{backward}} \approx 4N \text{ FLOPs per token.} \tag{6.30}$$

Summing forward and backward passes yields the standard total:

$$C_{\text{total}} = C_{\text{forward}} + C_{\text{backward}} \approx 2ND + 4ND = 6ND. \tag{6.31}$$

□

Derivation of the Kaplan allocation exponents. Given a fixed compute budget $C \approx 6ND$, how should resources be divided between model size N and token count D? Kaplan et al. derived the following empirical optimal scaling relationships:

$$N_{\text{opt}} \propto C^{0.73}, \quad B_{\text{opt}} \propto C^{0.24}, \quad S_{\text{opt}} \propto C^{0.03}, \tag{6.32}$$

where B is the critical batch size and S is the number of optimization steps. Since dataset size is the product of batch size and steps, $D = B \times S$:

$$D_{\text{opt}} = B_{\text{opt}} \times S_{\text{opt}} \propto C^{0.24+0.03} = C^{0.27}. \tag{6.33}$$

The exponents $N \propto C^{0.73}$ and $D \propto C^{0.27}$ deliver a startling engineering conclusion: If the computational budget C is scaled by a factor of 10×:

- Model parameters should expand by: $10^{0.73} \approx 5.37\times$.
- Training tokens should expand by only: $10^{0.27} \approx 1.86\times$.
- Optimization steps increase by: $10^{0.03} \approx 1.07\times$.

Kaplan et al. explicitly advised the field: *"As computational budget C increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size."*

This paper became the gospel of the AI industry between 2020 and 2022. It directly catalyzed the global race to build massive models with hundreds of billions of parameters, while keeping training datasets relatively static at roughly 300 billion tokens. The immediate first fruit of this philosophy was GPT-3.

6.5 GPT-3 and in-context learning

Armed with Kaplan's scaling laws, OpenAI developed and released **GPT-3** in June 2020 with the monumental paper "Language Models are Few-Shot Learners" by Brown et al. [2]. GPT-3 scaled the autoregressive Transformer architecture by an order of magnitude:

- **Parameters**: 175 billion (175×10^9) parameters—over 100× larger than GPT-2 Extra Large (1.5B) and nearly 1,500× larger than the original GPT-1 (117M).
- **Architecture**: $L = 96$ layers, $H = 96$ attention heads ($d_{\text{head}} = 128$), hidden state dimension $d = 12,288$, intermediate feed-forward dimension $d_{\text{ff}} = 49,152$, and context window $T = 2048$ tokens.
- **Alternating Dense and Locally Banded Sparse Attention**: Similar to the Sparse Transformer, GPT-3 alternated between standard dense multi-head attention and locally banded sparse attention patterns to accelerate long-sequence processing.

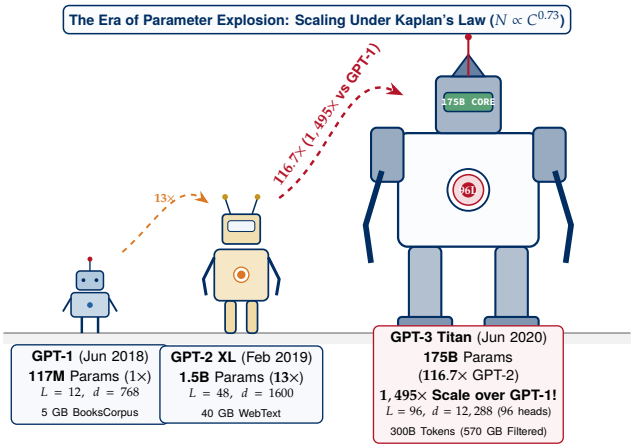


Figure 6.8: Visual parameter scale comparison across the first three generations of OpenAI's Generative Pre-trained Transformers (GPT-1, GPT-2 XL, and GPT-3). Driven by the Kaplan scaling law ($N \propto C^{0.73}$), AI research institutions poured virtually all available compute into expanding model parameters from 117 million to 175 billion ($\approx 1,500\times$ increase in two years), while training data scale remained comparatively constrained (5 GB to 300B tokens, barely 60×).

The physical layout and signal flow of the 175-billion-parameter GPT-3 architecture are illustrated in Figure 6.9.

To train a 175B model without catastrophic overfitting, OpenAI curated a pre-training corpus spanning hundreds of billions of tokens. Recognizing that raw web crawls contain overwhelming quantities of low-quality text, the authors built an aggressive multi-stage filtering pipeline:

1. **Quality Classifier Filtering**: A high-quality reference corpus (WebText) was used to train a logistic regression classifier with word and sentence-level features to distinguish high-quality prose from low-quality web garbage. Common Crawl was filtered through this classifier, discarding over 98% of raw pages (compressing 45 TB of raw text into 570 GB of filtered text).
2. **Fuzzy MinHash Deduplication**: MinHash LSH was applied to remove duplicate and near-duplicate web documents.
3. **High-Quality Dataset Blending**: The final pre-training mixture combined Filtered Common Crawl (410B tokens), WebText2 (19B tokens), Books1 (12B tokens), Books2 (55B tokens), and English Wikipedia (3B tokens), totaling approximately 300 billion training

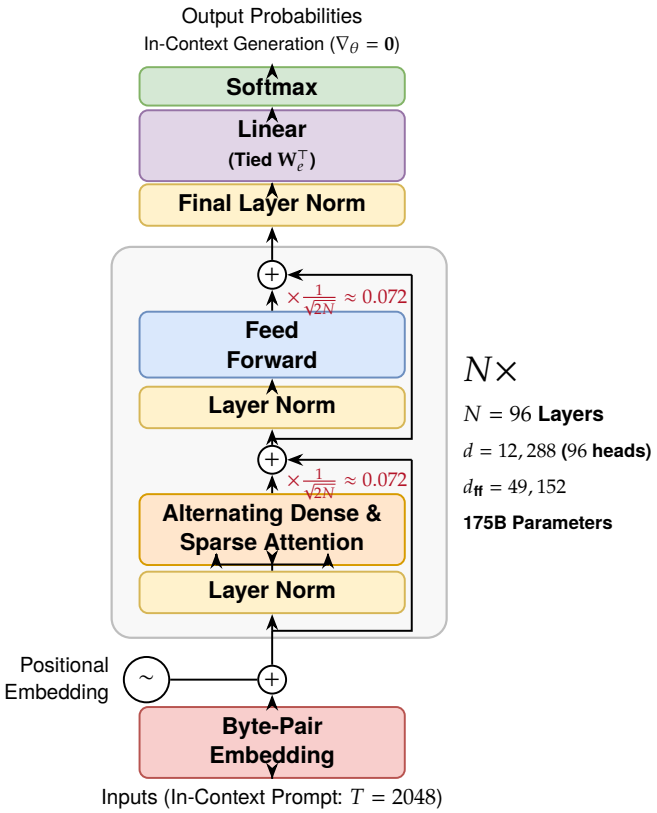


Figure 6.9: The GPT-3 autoregressive Transformer decoder architecture [2]. Scaling to 175 billion parameters ($N = 96$ layers, $d = 12,288$, 96 attention heads), GPT-3 preserves the Pre-LN residual backbone and output projection variance scaling ($\frac{1}{\sqrt{2N_{\text{layer}}}} \approx 0.072$) while alternating dense and locally banded sparse attention patterns across context windows of $T = 2048$ tokens. All downstream tasks are performed through frozen in-context generation ($\nabla_\theta = \mathbf{0}$) without parameter updates.

tokens. Sources were sampled according to perceived quality rather than raw volume (e.g., Wikipedia was sampled 3.4× per epoch, whereas Common Crawl was sampled 0.44×).

In-context learning modalities.
The central discovery of GPT-3 was the triumph of **In-Context Learning (ICL)**. Prior to GPT-3, applying a pre-trained model to a task required fine-tuning:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\text{task}}(\theta_t), \tag{6.34}$$

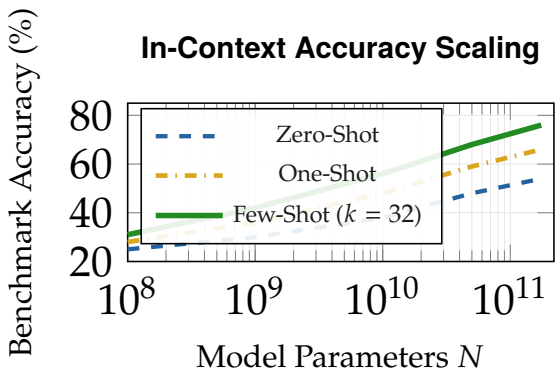
updating millions of parameters via gradient descent on task-specific training data. In contrast, in-context learning leaves the model parameters **completely frozen** ($\nabla_\theta = \mathbf{0}$). The model is prompted with a natural language task description and zero, one, or several demonstration exemplars entirely within the input prompt context $\mathbf{x}_{\text{prompt}}$. Depending on the number of conditioning exemplars provided prior to the target query, in-context learning encompasses three distinct operational regimes:

- **Zero-Shot**: Conditioning exclusively on the task instruction without demonstrations (e.g., `"Translate English to French: cheese => fromage"`). The model relies entirely on semantic priors acquired during pre-training.
- **One-Shot**: Providing exactly one worked input-output demonstration (e.g., `"sea otter => loutre de mer"`) to illustrate syntax and expected output structure before presenting the target query.
- **Few-Shot**: Providing k demonstration pairs (typically $10 \leq k \leq 100$, bounded by the 2048-token context window). This conditioning activates in-context meta-learning, drastically resolving task ambiguities without executing any backward pass.

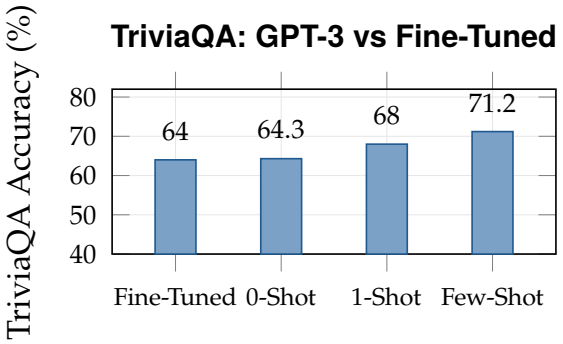
Whereas fine-tuning creates separate specialized model copies by altering network weights ($\Delta\theta \neq \mathbf{0}$), in-context learning employs a single universal frozen model ($\Delta\theta = \mathbf{0}$) that adapts dynamically through attention routing over prompt activations.

Empirical capabilities and benchmark evaluations.
The empirical findings of Brown et al. [2] yielded three profound takeaways that fundamentally transformed natural language processing:

1. **The power of few-shot prompting**: Across virtually every benchmark, the performance gap between zero-shot and one-shot was massive, with few-shot prompting ($k = 32$) achieving substantial further gains (Figure 6.10a). Providing even a single demonstration exemplar dramatically clarifies the expected output format and task semantics.
2. **Rivaling fine-tuned state of the art**: On demanding closed-book QA benchmarks such as TriviaQA (64.3% accuracy) and WebQuestions, GPT-3 175B in the few-shot setting **matched or exceeded state-of-the-art models that had been explicitly fine-tuned on the task training set** (e.g., T5-11B), as illustrated in Figure 6.10b. Scaling parameters eliminated the absolute necessity of task-specific training sets.
3. **Emergent reasoning abilities**: While smaller models (under 10B parameters) achieved near-zero accuracy on complex multi-digit arithmetic (e.g., 3-digit addition, subtraction), anagram unscrambling, and synthetic word manipulation, GPT-3 175B demonstrated sudden, qualitative leaps in accuracy.



(a) Accuracy scaling across prompt regimes.



(b) Few-shot GPT-3 surpasses fine-tuned SOTA without updates.

Figure 6.10: Empirical scaling and performance of in-context learning in GPT-3 [2]. (a) Scaling curves showing prompt effectiveness. (b) Performance on TriviaQA demonstrating that scaling eliminates task-specific fine-tuning.

6.6 The model undertraining crisis

Following the release of GPT-3 and the widespread adoption of Kaplan's scaling laws ($N \propto C^{0.73}, D \propto C^{0.27}$), the global AI research community entered a period of frantic model expansion. Research institutions concluded that the primary path to superior intelligence was expanding parameter count while holding training tokens roughly constant at approximately 300 billion tokens.

Between 2020 and early 2022, several colossal models were trained:

- **Jurassic-1 (AI21 Labs, 2021) [66]**: A 178-billion-parameter autoregressive model featuring an expanded vocabulary of 250,000 subword tokens (including multi-word expressions) and trained on 300 billion tokens.
- **Gopher (DeepMind, Dec 2021) [67]**: A 280-billion-parameter model ($L = 80, d = 16,384, H = 128$) trained on the MassiveText corpus (300 billion tokens) using 5.76×10^{23} FLOPs. The Gopher publication spanned an unprecedented 120 pages: pages 1–7 presented the architecture, while pages 8–120 documented an exhaustive analysis of capabilities, hallucinations, bias, and toxic text generation, establishing the modern format for foundation model technical reports.
- **Megatron-Turing NLG (Microsoft & NVIDIA, Oct 2021) [14]**: A massive 530-billion-parameter model ($L = 105, d = 20,480, H = 128$) trained on 270 billion tokens. Training MT-NLG pushed distributed hardware to its limits, requiring 3D parallelism: combining 8-way Megatron tensor parallelism, 35-way pipeline parallelism across DeepSpeed, and 8-way data parallelism across 2,240 NVIDIA A100 GPUs.
- **PaLM (Google Pathways, April 2022) [68]**: A 540-billion-parameter model ($L = 118, d = 18,432, H = 48$) trained on 780 billion tokens across 6,144 TPU v4 chips using the Pathways system. PaLM introduced SwiGLU activations, parallel attention-MLP layers, and Rotary Position Embeddings (RoPE), achieving breakthrough performance on mathematical reasoning (GSM8K) and code generation.

Table 6.1: The race to scale: Landmark dense foundation models trained between 2020 and 2022.

Model	Organization	Release	Params (N)	Tokens (D)	Ratio (N/D)	Compute (C FLOPs)
GPT-3	OpenAI	Jun 2020	175B	300B	0.58	3.15×10^{23}
Jurassic-1	AI21 Labs	Sep 2021	178B	300B	0.59	3.20×10^{23}
Megatron-Turing	NVIDIA / Microsoft	Oct 2021	530B	270B	**1.96**	8.58×10^{23}
Gopher	DeepMind	Dec 2021	280B	300B	0.93	5.76×10^{23}
PaLM	Google	Apr 2022	540B	780B	0.69	2.53×10^{24}

6.6.1 Pathologies of oversized models

Notice the profound structural imbalance revealed in Table 6.1. Between GPT-1 (117M) and PaLM (540B), model parameters had exploded by over $4,600\times$. Yet during this exact same period, training corpus volume expanded from 5 GB to barely 300–780 billion tokens—a factor of scarcely 60×!

Every major AI laboratory was strictly executing Kaplan's prescription: spend virtually all available compute scaling parameters ($N \propto C^{0.73}$), while treating training token volume as a secondary consideration ($D \propto C^{0.27}$). In Megatron-Turing NLG (530B), the parameter-to-data ratio reached an astounding:

$$\frac{N}{D} = \frac{530 \times 10^9}{270 \times 10^9} \approx 1.96 \text{ parameters per token.} \tag{6.35}$$

In other words, the model possessed nearly **two parameters for every single token seen during its entire training lifetime**! Each parameter received on average only ≈ 0.51 tokens of gradient signal. This severe asymmetry induced multiple catastrophic operational and theoretical pathologies:

1. **Capacity under-utilization and generalization brittleness**: Statistical learning theory dictates that overparameterized models require rich empirical data distributions to regularize high-dimensional hypothesis spaces. When a 530B-parameter model is trained on only 270B tokens, vast sub-circuits and feed-forward weight blocks remain severely under-trained, having received minuscule gradient updates. While the model achieves high memorization of superficial syntactic patterns, its deeper semantic representations remain brittle. This manifested in practice as extreme prompt fragility (where substituting minor synonyms or altering punctuation drastically altered outputs), frequent multi-step reasoning breakdowns, and elevated rates of fluent hallucinations and toxic generations. In the 120-page Gopher technical report [67], DeepMind demonstrated that scaling parameter count from 44M to 280B at fixed $D = 300$B yielded diminishing returns on factual accuracy while providing no protection against hallucinatory confabulations.

2. **Inference memory-bandwidth saturation**: While pre-training compute is amortized once across the model's development cycle, inference costs scale linearly with every generated token across millions of daily user interactions. Crucially, autoregressive language model inference is fundamentally **memory-bandwidth bound**, rather than compute-bound. During token-by-token autoregressive decoding, each forward pass generates exactly one new token per sequence ($T = 1$). Computing that single token requires loading **every single weight of the model from High-Bandwidth Memory (HBM) into the GPU streaming multiprocessors (SMs)**, yielding an operational arithmetic intensity of approximately ≈ 1 FLOP per byte of memory loaded.

   For a 530B-parameter model stored in 16-bit floating point precision (FP16 or BF16, at 2 bytes per parameter), the static weight footprint alone is:

   $$M_{\text{weights}} = 530 \times 10^9 \times 2 \text{ bytes} \approx 1,060 \text{ GB} = 1.06 \text{ TB}. \tag{6.36}$$

   On standard NVIDIA A100 GPUs (equipped with 80 GB HBM2e memory), storing merely the static weights requires a strict theoretical minimum of $\lceil 1060/80 \rceil = 14$ GPUs, realistically requiring a full server pod of 16 A100 GPUs. Furthermore, serving concurrent user requests requires allocating high-speed memory for the dynamic Key-Value (KV) cache across all layers and attention heads:

   $$M_{\text{KV}} = 2 \times B \times L \times d \times T_{\text{ctx}} \times 2 \text{ bytes}, \tag{6.37}$$

   where B is batch size, $L = 105$ layers, $d = 20,480$, and T_{ctx} is context length. For a modest batch size of $B = 16$ at $T_{\text{ctx}} = 2048$, the KV cache requires an additional ≈ 280 GB of VRAM, pushing total memory consumption beyond 1.34 TB.

3. **Multi-node cluster communication latency and MFU collapse**: Distributing a 530B model across 16 GPUs requires combining Tensor Parallelism ($TP = 8$) across NVLink within a node and Pipeline Parallelism ($PP = 2$) across nodes via InfiniBand. At every single Transformer layer, tensor parallelism performs an `All-Reduce` collective communication primitive across the high-speed crossbar to synchronize activation vectors. Because the arithmetic execution for a single token completes in microseconds, the GPU compute cores spend the vast majority of execution time stalled waiting for inter-GPU communication and memory bus transfers. Consequently, Model FLOPs Utilization (MFU) during interactive serving collapses to under 10–15%, causing per-token generation latencies to exceed 200–500 milliseconds per token.

4. **The economic fallacy of "cheap pre-training, ruinous inference"**: Kaplan's scaling laws optimized exclusively for training compute efficiency ($C = 6ND$), implicitly treating inference costs as zero. In industrial software engineering and commercial software ecosystems, however, inference queries outnumber pre-training passes by many orders of magnitude. An oversized, undertrained model like MT-NLG (530B) or Gopher (280B) represents an economic disaster: it was relatively inexpensive to pre-train (because it saw only 270B–300B tokens), but catastrophically expensive to host and execute indefinitely. Conversely, a compute-optimal model (e.g., 70B parameters trained on 1.4 trillion tokens) achieves identical or superior validation loss while occupying $\frac{1}{7.5}$ of the memory footprint, fitting comfortably onto a single 8-GPU node (or a quantized 2-GPU setup), and delivering $\approx 7\times$ higher inference throughput at a fraction of the electricity and cloud operational expenditure (OpEx).

6.6.2 Methodological diagnosis of Kaplan scaling laws

How could a landmark empirical study conducted by OpenAI reach scaling laws that led the entire field into a multi-year undertraining cul-de-sac? When researchers at DeepMind rigorously audited the experimental protocol of Kaplan et al. [15], they uncovered four fundamental methodological flaws:

1. **Suboptimal learning rate schedules and absence of cosine cooldown**: In modern Transformer pre-training, the learning rate η_t follows a cosine decay schedule with initial linear warmup:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\text{total}}}\pi\right)\right), \tag{6.38}$$

   where T_{total} represents the predetermined total step horizon, decaying to $\eta_{\min} = 0.1\eta_{\max}$. A well-documented phenomenon in deep optimization is that **a disproportionately large fraction of total loss reduction occurs during the final cosine cooldown phase** (the last 10–20% of steps where the learning rate anneals to its minimum). Kaplan et al. did not re-tune T_{total} independently for every model run. Instead, they held the schedule length fixed to a constant step count or evaluated intermediate checkpoints along a single extended trajectory (e.g., evaluating a run at step 20,000 when $T_{\text{total}} = 100,000$). At step 20,000, the learning rate was still hovering near $\eta_{\max}$, meaning the model had never experienced the critical loss-reducing cooldown! Because smaller models reached lower compute budgets earlier in the trajectory, they were systematically evaluated in an under-converged, un-cooled state. This severely depressed the apparent sample efficiency of smaller models, creating the false mathematical illusion that increasing tokens yields diminishing returns compared to expanding parameters. When DeepMind properly trained every model with a cosine schedule matched exactly to its training duration, smaller models exhibited dramatically steeper loss drops.

2. **Exclusion of vocabulary and embedding parameters from compute accounting**: Kaplan et al. defined model size strictly as $N_{\text{non-emb}}$, explicitly discarding the parameters of the input embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}|\times d}$ and output unembedding layer $\mathbf{W}_u \in \mathbb{R}^{d\times|\mathcal{V}|}$ from the compute formula $C \approx 6N_{\text{non-emb}}D$. Their justification was that embedding weights do not participate in token-to-token attention operations. However, for a standard Byte-Pair Encoding vocabulary of $|\mathcal{V}| = 50,257$, the embedding matrix comprises $50,257 \times d$ parameters. For smaller models with $d = 768$ (such as GPT-1), $\mathbf{W}_e$ accounts for ≈ 38.6M parameters—nearly **33% of the entire network**! Excluding embeddings introduced a substantial non-linear distortion at small and medium model scales ($10^7 \leq N \leq 10^9$), corrupting the parameter-to-token trade-off and biasing the fitted regression toward expanding depth and width rather than training data.

3. **Evaluation on sequence tail tokens rather than whole-sequence loss**: Kaplan et al. evaluated validation cross-entropy loss exclusively on the final tokens of each sequence (the sequence tail, e.g., the last 10% of tokens in context $T = 1024$), rather than averaging loss across the entire context window. While intended to measure performance under maximal historical context, this evaluation protocol created a structural bias in favor of massive parameter counts: high-capacity models possess superior associative memory for long prefixes, but this masked the severe degradation that undertrained models suffer when predicting tokens across early and middle context positions. Averaging loss over the entire sequence reveals that well-trained smaller models generalize vastly better than undertrained giants.

4. **Restricted token dynamic range and regression multicollinearity**: In Kaplan et al.'s experimental runs, model parameters N were varied across nearly six orders of magnitude (10^3 to 10^9), but training tokens D were varied across a significantly narrower dynamic range, often held constant across entire sub-experiments. When fitting a bivariate power-law loss surface:
$$\mathcal{L}(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} + \mathcal{L}_0, \tag{6.39}$$
the lack of an independently perturbed, balanced two-dimensional grid of (N, D) runs induced severe **multicollinearity** in the log-linear regression. The optimization algorithm attributed the majority of compute-driven loss reduction to N ($\alpha_N \approx 0.076 \implies a = 0.73$), while severely dampening the sensitivity to D ($\alpha_D \approx 0.057 \implies b = 0.27$).

This critical realization proved that the industry was building models that were **oversized and significantly undertrained**. To discover the true compute-optimal allocation, DeepMind launched an exhaustive empirical investigation that resulted in the Chinchilla scaling laws.

6.7 Compute-optimal scaling: Chinchilla

In March 2022, Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, and colleagues at DeepMind published one of the most influential papers in modern artificial intelligence: **"Training Compute-Optimal Large Language Models"** [3].

The central claim of the Chinchilla paper shattered the prevailing industry consensus:

> *"We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. . . For every doubling of model size, the number of training tokens should also be doubled."*

DeepMind proved that the conclusions of Kaplan et al. [15] gave vastly too much weight to model size. While OpenAI advocated $N \propto C^{0.73}$ and $D \propto C^{0.27}$, DeepMind demonstrated that the true compute-optimal frontier requires **equal, balanced scaling**:

$$N_{\text{opt}} \propto \sqrt{C}, \quad D_{\text{opt}} \propto \sqrt{C}. \tag{6.40}$$

To establish this result incontrovertibly, DeepMind trained over 400 language models across varying scales and evaluated the data through **three independent analytical approaches**.

6.7.1 Approach 1: minimum loss envelopes

In the first approach, the authors trained a family of models with fixed parameter sizes $N \in \{70\text{M}, \ldots, 16\text{B}\}$ across varying training token durations.

For each individual model size N, training traces out a smooth curve of cross-entropy loss as a function of consumed compute $C = 6ND$:

1. For any specified compute budget C_0 (e.g., $C_0 = 10^{20}$ FLOPs), we inspect all model curves intersecting this budget.
2. We identify the specific model parameter count N that achieves the **global minimum loss** $\min \mathcal{L}$ at C_0.
3. We record the pair (N_{opt}, C_0) and the corresponding token count $D_{\text{opt}} = C_0/(6N_{\text{opt}})$.
4. Repeating this across all compute budgets from 10^{18} to 10^{22} FLOPs, we fit power-law relationships:
$$N_{\text{opt}} = GC^a, \quad D_{\text{opt}} = HC^b. \tag{6.41}$$

Empirically, Approach 1 yielded:

$$a = 0.50, \quad b = 0.50. \tag{6.42}$$

When projected to the computational budget of Gopher ($C = 5.76 \times 10^{23}$ FLOPs), Approach 1 indicated that the optimal model size was **67 billion parameters**, requiring **1.43 trillion tokens**—not 280 billion parameters trained on 300 billion tokens! Figure 6.11 displays the empirical envelope across model sizes.

6.7.2 Approach 2: isoflop profiles

In the second approach, DeepMind fixed the total compute budget $C \in \{10^{18}, 10^{19}, 10^{20}, 10^{21}, 10^{22}\}$ FLOPs and systematically varied the trade-off between parameter count N and dataset size $D = C/(6N)$.

For each IsoFLOP slice, the final training loss was plotted against parameter count N on a logarithmic scale. Empirically, the data forms an exact **parabola** (a convex bowl):

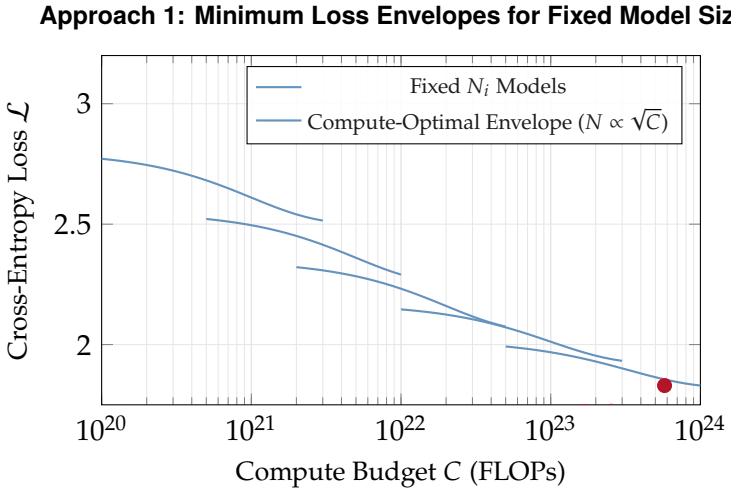


Figure 6.11: Chinchilla Approach 1 (fixed-size model envelopes) [3]. By evaluating training runs across varying parameter counts, the lower convex envelope identifies the compute-optimal parameter count for each compute budget ($a = 0.50, b = 0.50$), proving Gopher should have had 67B parameters on 1.43T tokens.

- If N is too small (left side of parabola), the model has insufficient capacity to absorb the information in D tokens, causing high loss due to under-parameterization.
- If N is too large (right side of parabola), the model is severely undertrained because $D = C/(6N)$ is too small, causing high loss due to data starvation.

The loss within an IsoFLOP slice is modeled by fitting a quadratic polynomial in $\log N$:

$$\log \mathcal{L}(N) = c_0 + c_1 \log N + c_2 (\log N)^2, \quad \text{with } c_2 > 0. \tag{6.43}$$

The optimal model size corresponds precisely to the **vertex (minimum)** of the parabola:

$$\frac{\partial \log \mathcal{L}}{\partial \log N} = c_1 + 2c_2 \log N = 0 \implies \log N_{\text{opt}} = -\frac{c_1}{2c_2} \implies N_{\text{opt}} = \exp\left(-\frac{c_1}{2c_2}\right). \tag{6.44}$$

By extracting N_{opt} for each IsoFLOP curve and fitting power-law relations $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$, Approach 2 yielded:

$$a = 0.49, \quad b = 0.51. \tag{6.45}$$

For Gopher's compute budget, Approach 2 determined that the compute-optimal model size was **63 billion parameters** trained on **1.53 trillion tokens**. Figure 6.12 displays the parabolic loss profiles across IsoFLOP slices.

6.7.3 Approach 3: parametric loss surface

The third approach formulated an analytical parametric model of the loss surface $\mathcal{L}(N, D)$ over all training runs simultaneously.

> **Theorem 6.2:** *Parametric scaling law formulation*
>
> The expected cross-entropy loss $\mathcal{L}(N, D)$ for an autoregressive language model with N parameters trained on D tokens can be decomposed into an irreducible linguistic entropy

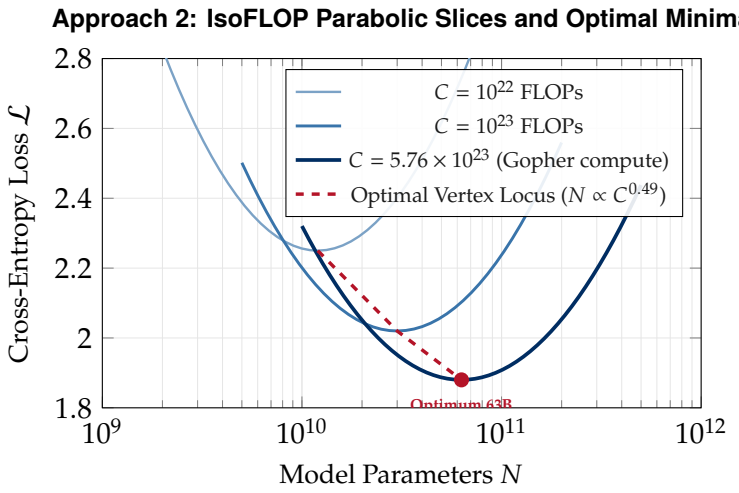


Figure 6.12: Chinchilla Approach 2 (IsoFLOP profiles) [3]. For any fixed compute budget C, the loss follows a parabolic curve whose vertex identifies the optimal parameter count ($a = 0.49, b = 0.51$), demonstrating that Gopher should have been 63B parameters.

and power-law penalties for finite capacity and finite data:

$$\hat{\mathcal{L}}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \tag{6.46}$$

where:

- E is the **irreducible entropy** of natural text (the theoretical floor of cross-entropy loss).
- $\frac{A}{N^{\alpha}}$ represents the risk penalty due to **finite model capacity**.
- $\frac{B}{D^{\beta}}$ represents the risk penalty due to **finite training dataset size**.
- A, B, E, α, β are positive constants.

Fitting this functional form over all 400+ experimental runs using Huber loss minimization yielded:

$$E \approx 1.69, \quad A \approx 406.4, \quad B \approx 410.7, \quad \alpha \approx 0.34, \quad \beta \approx 0.28. \tag{6.47}$$

We can now mathematically derive the compute-optimal allocation by solving a **constrained optimization problem**:

$$\min_{N,D} \hat{\mathcal{L}}(N, D) = E + AN^{-\alpha} + BD^{-\beta} \quad \text{subject to } 6ND = C. \tag{6.48}$$

Proof. Formulate the Lagrangian function with multiplier λ:

$$\mathcal{L}_{\text{Lagrange}}(N, D, \lambda) = E + AN^{-\alpha} + BD^{-\beta} + \lambda(6ND - C). \tag{6.49}$$

Setting the first-order partial derivatives with respect to N and D to zero:

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial N} = -\alpha AN^{-\alpha-1} + 6\lambda D = 0 \implies 6\lambda ND = \alpha AN^{-\alpha}, \tag{6.50}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial D} = -\beta BD^{-\beta-1} + 6\lambda N = 0 \implies 6\lambda ND = \beta BD^{-\beta}. \tag{6.51}$$

Equating the two expressions for $6\lambda ND$:

$$\alpha A N^{-\alpha} = \beta B D^{-\beta}. \tag{6.52}$$

Equation (6.52) establishes the **fundamental economic optimality condition**: at the compute-optimal frontier, the marginal reduction in loss obtained by spending compute on model parameters must exactly balance the marginal reduction in loss obtained by spending compute on additional tokens.

To express N_{opt} and D_{opt} purely in terms of total compute C, substitute $D = \frac{C}{6N}$ into Equation (6.52):

$$\alpha A N^{-\alpha} = \beta B \left(\frac{C}{6N}\right)^{-\beta} = \beta B 6^{\beta} C^{-\beta} N^{\beta}. \tag{6.53}$$

Grouping powers of N:

$$N^{\alpha+\beta} = \frac{\alpha A}{\beta B 6^{\beta}} C^{\beta} \implies N_{\text{opt}}(C) = \left(\frac{\alpha A}{\beta B 6^{\beta}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\beta}{\alpha+\beta}}. \tag{6.54}$$

By symmetry, substituting $N = \frac{C}{6D}$ yields the optimal dataset size:

$$D_{\text{opt}}(C) = \left(\frac{\beta B}{\alpha A 6^{\alpha}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\alpha}{\alpha+\beta}}. \tag{6.55}$$

Substituting the empirically fitted exponents $\alpha \approx 0.34$ and $\beta \approx 0.28$:

$$a = \frac{\beta}{\alpha+\beta} = \frac{0.28}{0.34+0.28} = \frac{0.28}{0.62} \approx 0.45, \tag{6.56}$$

$$b = \frac{\alpha}{\alpha+\beta} = \frac{0.34}{0.34+0.28} = \frac{0.34}{0.62} \approx 0.55. \tag{6.57}$$

□

Figure 6.13a and Figure 6.13b display the contour geometry of this parametric loss surface and contrast the resulting compute-optimal trajectory against Kaplan's undertrained scaling regime.

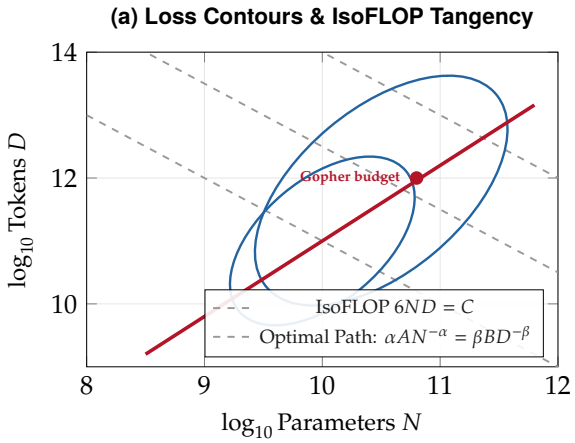
6.7.4 Empirical validation: Chinchilla vs. Gopher

To empirically validate their theoretical derivations, DeepMind trained **Chinchilla**:

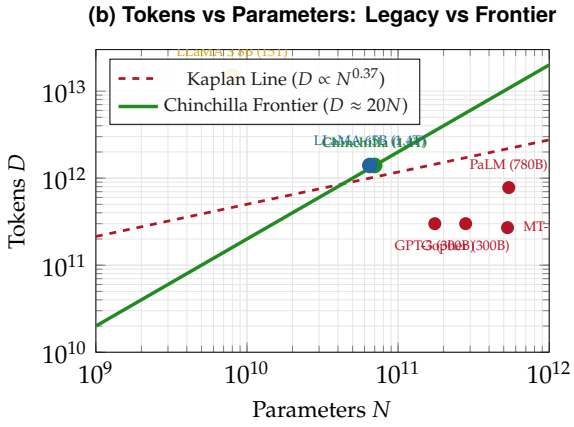
- **Identical Compute Budget**: Chinchilla was allocated exactly the same compute budget as Gopher (5.76×10^{23} FLOPs).
- **Parameters**: 70 billion parameters ($L = 80, d = 8192, H = 64$)—exactly **one-quarter the size of Gopher** (280B).
- **Training Tokens**: 1.4 trillion (1.4×10^{12}) tokens—over **four times more data than Gopher** (300B).

The empirical evaluation results were decisive:

- Chinchilla uniformly **outperformed Gopher (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B)** across virtually every downstream evaluation benchmark.
- On the Massive Multitask Language Understanding (MMLU) benchmark, Chinchilla achieved an average score of 67.5%, outperforming Gopher (60.0%) by 7.5 percentage points.

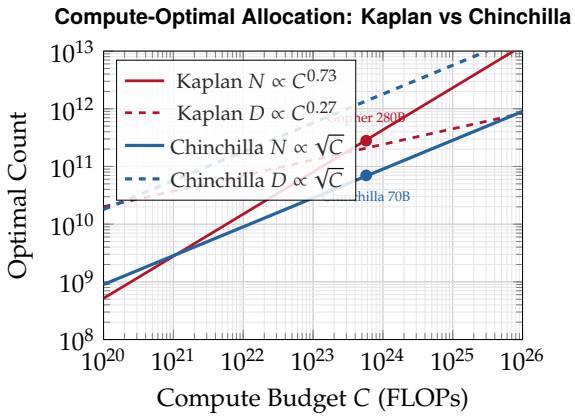


(a) Parametric loss contour surface: IsoFLOP slices and compute-optimal tangency.

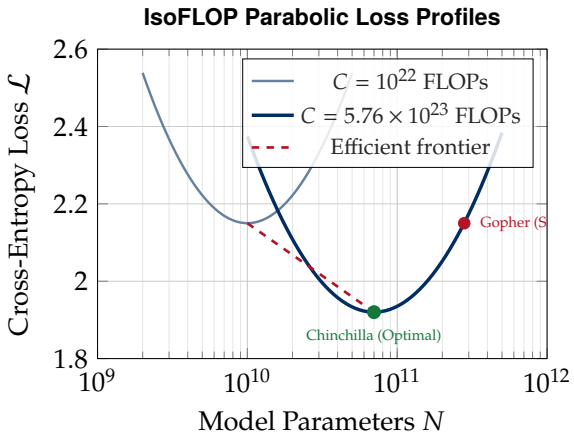


(b) Model comparison: legacy half-trillion giants were severely undertrained.

Figure 6.13: Chinchilla Approach 3 parametric loss surface modeling and empirical model comparison [3]. (a) Optimal allocation occurs at the tangency of IsoFLOP constraint lines and loss contours. (b) Dataset size versus parameter count, proving earlier giants were starved for tokens.



(a) Scaling exponents: Kaplan favors massive parameters ($a = 0.73$), while Chinchilla proves balanced scaling ($a \approx b \approx 0.50$).



(b) IsoFLOP profile at Gopher budget: Gopher sits on the undertrained right wing; Chinchilla sits at the parabolic minimum.

Figure 6.14: Analytical derivations of compute-optimal language modeling from DeepMind's Chinchilla study [3].

Table 6.2: Direct architectural and performance showdown: Gopher (280B) versus Chinchilla (70B) at identical compute.

Attribute	Gopher (Kaplan Baseline)	Chinchilla (Compute-Optimal)
Parameters (N)	280 Billion (280×10^9)	**70 Billion** (70×10^9) [$\frac{1}{4}\times$ size]
Training Tokens (D)	300 Billion (300×10^9)	**1.4 Trillion** (1.4×10^{12}) [4.7× tokens]
Compute Budget (C)	5.76×10^{23} FLOPs	5.76×10^{23} FLOPs (**Identical**)
MMLU Benchmark	60.0% average accuracy	**67.5%** average accuracy (+7.5%)
BIG-bench Tasks	Baseline	Outperforms Gopher on > 65% of tasks
Inference Memory (FP16)	≈ 560 GB VRAM (min 8 A100s)	≈ **140 GB VRAM** (fits on 2 A100s)
Inference Latency	High (∝ 280B)	**Cut by** ≈ 75% (∝ 70B)

- On commonsense reasoning, reading comprehension, and math benchmarks (GSM8K), Chinchilla established comprehensive state-of-the-art results.
- Crucially, because Chinchilla had only 70B parameters, its **inference latency was dramatically lower and memory consumption was cut by 75%**, requiring only two NVIDIA A100 GPUs instead of eight.

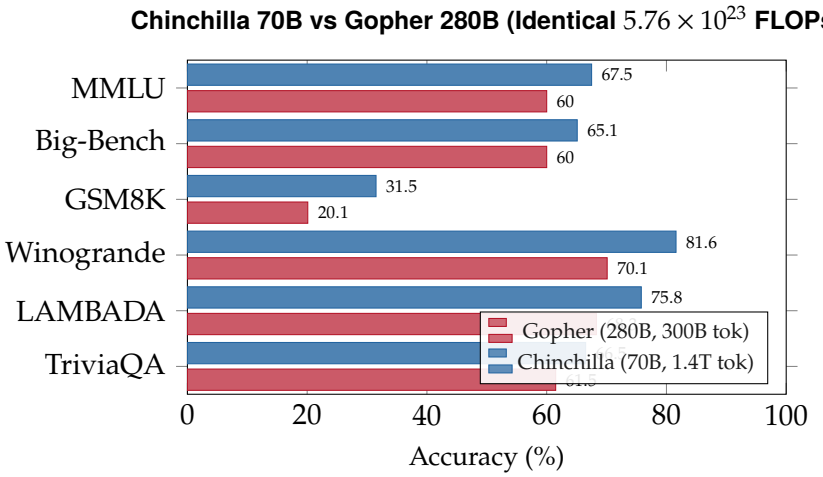


Figure 6.15: Empirical validation of compute-optimality: Chinchilla 70B versus Gopher 280B across major benchmarks [3]. With one-quarter the parameter count and four times more tokens at identical training compute (5.76×10^{23} FLOPs), Chinchilla comprehensively outperforms Gopher while drastically slashing inference serving overhead.

6.8 Open foundation models

The transition from the Kaplan scaling regime to Chinchilla compute-optimality fundamentally restructured the economics and accessibility of artificial intelligence. To appreciate why compute-optimal scaling catalyzed the explosive rise of **open-weights foundation models**, we must examine the stark contrast between the hardware barriers of the 2020–2022 parameter race and the deployment realities of compact, highly trained architectures.

6.8.1 The economic and hardware drivers of the open-weight revolution

Under Kaplan et al. [15], leading laboratories allocated over 70% of scaling compute to parameter expansion ($N \propto C^{0.73}$). This doctrine culminated in half-trillion-parameter monoliths: GPT-3 (175B), Gopher (280B), and Megatron-Turing NLG (530B). These mega-models introduced an insurmountable hardware barrier:

- **Colossal Memory Requirements**: Storing 175B to 530B parameters in 16-bit precision requires between 350 GB and 1,060 GB of VRAM strictly for model weights. Serving interactive generation requires allocating an additional 200 GB to 400 GB for the dynamic Key-Value (KV) cache.
- **Cluster-Scale Hardware Monopolies**: Serving a single instance of a 175B+ model necessitated high-end multi-GPU server nodes equipped with 8 to 16 NVIDIA A100 (80 GB) GPUs connected via high-speed NVLink and InfiniBand fabrics. The acquisition, electrical power, and cooling infrastructure required to run such clusters cost hundreds of thousands of dollars, confining practical deployment exclusively to trillion-dollar tech conglomerates.
- **Centralized API Enclosures**: Because individual researchers, universities, and small enterprises could neither run nor fine-tune these models locally, frontier AI was locked behind proprietary commercial web APIs (such as OpenAI's completions endpoint). Users were reduced to passive consumers, unable to audit model internals, inspect training data contamination, or fine-tune weights for privacy-sensitive corporate workflows.

The Chinchilla revelation (Hoffmann et al. [3]) exposed the fundamental flaw of this paradigm: monster models were severely under-trained, wasting immense parameter capacity on small data volumes. A compute-optimal 70B model trained on 1.4 trillion tokens achieved parity with a 280B model trained on 300 billion tokens while consuming identical pre-training compute.

Crucially, in industrial deployment and software engineering, pre-training compute is amortized once, whereas inference serving costs scale perpetually across billions of operational queries ($Q \gg 1$). The **lifecycle computational cost** of a model is given by:

$$C_{\text{lifecycle}} = C_{\text{train}} + Q \times C_{\text{inference}} \approx C_{\text{train}} + Q \times \left(2NL_{\text{gen}}\right), \tag{6.58}$$

where L_{gen} is the average sequence length generated per query. Because per-token inference compute scales strictly linearly with parameter count N, serving a 175B model is approximately 25× more expensive per query than serving a 7B model.

This realization gave birth to **inference-optimal overtraining**: investing surplus compute during pre-training to train compact models ($N \in \{7\text{B}, 8\text{B}, 13\text{B}\}$) far past the Chinchilla point on multi-trillion-token corpora (1.0T to 15T tokens). While sub-optimal for pre-training FLOP efficiency alone, this strategy produces compact models with capabilities rivaling legacy mega-models, while drastically slashing lifetime serving costs.

The consumer hardware democratization threshold.

The breakthrough for the open-source community was physical:

1. A 7-billion-parameter model in 16-bit precision requires only ≈ 14 GB of memory.

2. Applying state-of-the-art 4-bit Post-Training Quantization (PTQ) techniques (such as AWQ or GPTQ) compresses the entire model footprint to **under 4.5 GB of memory** with negligible perplexity degradation.

For the first time in history, a frontier-quality foundation model could be loaded into memory and executed locally on commodity consumer hardware: a standard mid-range gaming laptop (e.g., an NVIDIA RTX 3060 or 4060 with 6–8 GB VRAM), an Apple Silicon Mac with unified memory, or a single commodity cloud GPU.

This collapse of the hardware barrier shattered corporate monopolies, sparking a decentralized explosion of open research. Practitioners could now execute private code generation in local IDEs without leaking intellectual property, perform parameter-efficient fine-tuning (LoRA, QLoRA) on single workstations, and rigorously audit safety, alignment, and algorithmic bias.

6.8.2 The LLaMA family and modern open architectures

Starting in February 2023, Meta AI introduced the **LLaMA** (Large Language Model Meta AI) family, establishing the architectural gold standard for open-weights models:

- **LLaMA 1 (Feb 2023) [13]**: Trained across four sizes: 7B, 13B, 33B, and 65B on 1.0T to 1.4T tokens. LLaMA demonstrated that training exclusively on publicly available datasets (Common Crawl, GitHub, Wikipedia, ArXiv, StackExchange) could match or exceed proprietary closed models like GPT-3. LLaMA modernized the Transformer architecture with three critical innovations:

  1. **RMSNorm (Root Mean Square Normalization)**: Replacing standard LayerNorm to eliminate mean-centering, saving 7% of memory bandwidth:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}. \tag{6.59}$$

  2. **SwiGLU Activation Function**: Replacing standard ReLU or GELU with Swish-Gated Linear Units [69]:

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W} \odot \text{SiLU}(\mathbf{x}\mathbf{V}))\mathbf{W}_2, \quad \text{SiLU}(z) = z \cdot \sigma(z). \tag{6.60}$$

  3. **Rotary Position Embeddings (RoPE)** [55]: Injecting positional information by rotating query and key vectors in complex 2D subspaces, enabling natural length extrapolation.

- **Llama 2 (July 2023) [70]**: Scaled training tokens to 2.0 trillion tokens across 7B, 13B, and 70B sizes. Llama 2 doubled the context window to 4096 tokens and introduced **Grouped-Query Attention (GQA)** on the 70B model, sharing single key and value heads across groups of query heads to drastically compress KV-cache memory during autoregressive generation. Meta also released official instruction-tuned models (Llama-2-Chat) aligned with Reinforcement Learning from Human Feedback (RLHF).

- **Llama 3 (April 2024) and Llama 3.1 (July 2024) [71]**: Pushed inference-optimal overtraining to the absolute extreme:

  – The 8B and 70B models were trained on over **15 trillion tokens**—overtrained by more than 50× beyond the Chinchilla compute-optimal frontier!

  – Implemented a massive 128,256-token tiktoken tokenizer, boosting compression efficiency on code and multilingual text by 15%.

  – Llama 3.1 introduced the frontier **405B flagship model** trained on 16 trillion tokens across 16,000 NVIDIA H100 GPUs, supporting a native 128k context window and rivaling top proprietary models (GPT-4, Claude 3.5 Sonnet).

- **Llama 3.2 (September 2024)**: Expanded the family into edge deployment with lightweight 1B and 3B models optimized for on-device mobile hardware, alongside multimodal vision foundation models (11B and 90B).
- **Specialized Variants (e.g., Code Llama)**: Fine-tuned specifically on 500B tokens of source code, supporting the Fill-in-the-Middle (FIM) objective for real-time code infilling and completion in software engineering IDEs.

Licensing and commercial barriers.
Meta released the LLaMA weights under a bespoke permissive license that catalyzed academic and industrial adoption while establishing a strategic defensive moat: usage is entirely free for research and commercial deployment, *provided the licensee's product serves fewer than 700 million monthly active users* (set at 100 million in earlier drafts). As highlighted in lecture, this threshold served as an explicit barrier against massive hyperscale competitors (such as Google and Amazon) utilizing LLaMA to power their commercial search ecosystems without reciprocal licensing, while ensuring universal free access for startups, universities, and independent practitioners.

6.8.3 Mistral AI and sparse mixture-of-experts

Founded in Paris in 2023, Mistral AI introduced architectural innovations focusing on maximum compute efficiency:

- **Mistral 7B (October 2023)** [72]: Outperformed Llama 2 13B across all benchmarks despite having half the parameter count. Key mechanisms included:
  1. **Sliding Window Attention (SWA)**: Each token attends only to the preceding $W = 4096$ tokens at each layer. Across L layers, information propagates through an effective receptive field of $L \times W$, achieving linear $O(T \times W)$ computational and cache memory complexity.
  2. **Rolling Buffer Cache**: Storing keys and values in a fixed-size circular cache of capacity W, overwriting slots modulo W to eliminate memory growth on infinite generation streams.
  3. **Grouped-Query Attention (GQA)**: Standard across all model sizes (8 query heads per KV head).
- **Mixtral 8x7B (December 2023)**: Pioneered open-weights **Sparse Mixture-of-Experts (MoE)**. While the model contains 46.7 billion total parameters, a learned router routes each token to only 2 out of 8 feed-forward experts per layer. Consequently, only **12.9 billion parameters are active per token**, delivering the inference latency and compute cost of a 13B model while matching the performance of a 70B dense model.
- **Pixtral**: A 12B multimodal model supporting native image and video understanding alongside text.

Beyond Meta and Mistral, other prominent open foundation families made decisive contributions:

- **EleutherAI (GPT-Neo, GPT-J, GPT-NeoX-20B)**: Early non-profit open-source pioneers that reproduced GPT-3-scale architectures, creating the Pile dataset (825 GB) and establishing open benchmarks to accelerate transparent AI alignment research.
- **Falcon Series (Technology Innovation Institute, UAE)** [73]: Released 7B, 40B, and 180B parameter models trained on **RefinedWeb**, demonstrating that meticulously filtered web text alone could produce state-of-the-art reasoning without proprietary curation. Crucially,

Falcon addressed the acute tokenization penalty suffered by non-English languages (specifically Arabic). In standard English-centric BPE tokenizers, Arabic morphological structures fragment into excessive byte sequences (often 4 to 8 tokens per word), drastically compressing the effective context window and inflating generation costs. Falcon engineered a custom balanced tokenizer to overcome this bottleneck. Furthermore, the institute pioneered hybrid architectures combining **State Space Models (Mamba)** with attention layers, exploring sub-quadratic alternatives to standard Transformers.

- **GLM (General Language Model, Zhipu AI & Tsinghua University)**: An autoregressive blank infilling architecture trained on massive bilingual corpora, achieving top-tier proficiency across both English and Chinese.

The historical evolution from the original 2017 Transformer thus demonstrates a definitive convergence: while encoder-only and encoder-decoder models established specialized benchmarks, the autoregressive decoder-only architecture—fortified by compute-optimal pre-training and inference-optimal overtraining—has emerged as the universal foundation for modern artificial intelligence.

CHAPTER 7

Language model evaluation, metrics, and benchmarks

The pre-training paradigm detailed in Chapter 6 established that scaling parameter count, dataset volume, and training compute yields continuous, predictable reductions in next-token cross-entropy loss. By optimizing this autoregressive objective across trillions of tokens, modern foundation models acquire rich linguistic representations, syntactic abstractions, and emergent in-context learning capabilities. However, a fundamental discrepancy exists between pre-training optimization and practical deployment: minimizing the empirical cross-entropy loss over a corpus merely measures the statistical predictability of tokens under teacher forcing. It does not directly quantify whether an autoregressively generated sequence is coherent, factually truthful, stylistically appropriate, or capable of solving complex multi-step reasoning tasks.

Evaluating natural language generation is fundamentally challenging because open-ended text does not conform to the closed-world assumptions of classical machine learning. In standard classification or regression tasks, the output space is bounded and discrete labels provide unambiguous ground truth. In contrast, generative language modeling is combinatorial: a single semantic concept, question prompt, or code specification can be articulated through an astronomical number of valid, syntactically diverse surface formulations. Conversely, a generated string may match reference vocabulary with high lexical overlap while completely inverting factual truth through the addition of a single negation token. Qualitative inspection—relying on human observers to informally assess whether sampled text appears coherent or degraded—is unscalable, subjective, and unable to establish statistically sound comparisons between competing architectures or training checkpoints.

To establish rigorous engineering standards, the natural language processing community has developed an evaluation hierarchy that spans multiple levels of abstraction. At the lowest level lie intrinsic probabilistic metrics, such as cross-entropy and perplexity, which evaluate the model's internal likelihood assignments directly on held-out text without requiring generated outputs or human task labels. At the intermediate level lie reference-based lexical metrics, such as BLEU and ROUGE, which compare model-generated candidate sequences against human reference strings using n-gram overlap and length penalties. At higher levels of the hierarchy, evaluation shifts toward contextual embedding similarity (such as BERTScore), model-based evaluators, and standardized task benchmarks (ranging from core natural language understanding suites like GLUE to comprehensive multi-task and code-generation suites like MMLU and HumanEval).

This chapter formalizes the principles of language model evaluation. We begin by examining the core mathematical properties required for rigorous evaluation metrics, focusing on the distinction between additive and multiplicative averaging, before conducting an exhaustive investigation of intrinsic uncertainty estimation via perplexity and surface-string reference matching via BLEU.

7.1 Evaluation metrics for language models

Evaluation metrics in natural language processing can be partitioned into two distinct paradigms: intrinsic likelihood metrics and extrinsic generation metrics. Intrinsic metrics evaluate the model's mathematical representation of language by computing the probability distribution it assigns to unseen test corpora under teacher forcing. They require no sample generation, no decoding algorithms, and no reference texts; they assess the predictive confidence of the probability model itself. Extrinsic metrics, by contrast, operate on the discrete sequences emitted during autoregressive inference, comparing the generated hypothesis against one or more human gold-standard references.

The geometric mean and multiplicative error penalization.

Before formalizing specific metrics, we must establish the mathematical foundation of sequence-level score aggregation. When evaluating models across multiple tokens, sentences, or n-gram orders, individual scores must be aggregated into a single scalar summary. While the standard arithmetic mean is familiar, it possesses structural properties that render it unsuitable for probability spaces and multi-criteria quality evaluation.

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n_{>0}$ denote a collection of n positive evaluation values (such as token probabilities or n-gram precision scores). The arithmetic mean (AM) and geometric mean (GM) are defined as:

$$\mathrm{AM}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i, \tag{7.1}$$

$$\mathrm{GM}(\mathbf{x}) = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}. \tag{7.2}$$

The critical distinction between these two operators lies in how they penalize low values. The arithmetic mean exerts an *additive effect*: a severely low score in one component can be easily masked by high scores in other components. For example, consider three evaluation scores $\mathbf{x} = (0.1, 0.9, 0.9)$. The arithmetic mean yields:

$$\mathrm{AM}(0.1, 0.9, 0.9) = \frac{0.1 + 0.9 + 0.9}{3} = \frac{1.9}{3} \approx 0.6333, \tag{7.3}$$

producing a deceptively favorable aggregate score despite catastrophic failure on the first criterion. In contrast, the geometric mean exerts a *multiplicative effect*:

$$\mathrm{GM}(0.1, 0.9, 0.9) = (0.1 \times 0.9 \times 0.9)^{\frac{1}{3}} = (0.081)^{\frac{1}{3}} \approx 0.4327. \tag{7.4}$$

Because the terms multiply, any degradation in an individual component disproportionately suppresses the entire product. Crucially, if any single term reaches zero ($x_i = 0$), the geometric mean collapses to zero:

$$\exists i \text{ such that } x_i = 0 \implies \mathrm{GM}(\mathbf{x}) = 0. \tag{7.5}$$

This property is indispensable in evaluation: a model must achieve reasonable competence across all evaluated dimensions to attain a high overall score. This behavior directly parallels the harmonic mean used in the F_1-score to balance precision and recall.

Numerical stability via the logarithmic formulation.

In practical computation, evaluating long sequences or high-order n-grams involves multiplying dozens or hundreds of terms bounded in $(0, 1]$. In finite-precision floating-point arithmetic (IEEE 754 float32 or float64), direct repeated multiplication inevitably causes arithmetic underflow, rounding the running product to absolute zero.

To guarantee numerical stability, the geometric mean is computed in the logarithmic domain. Utilizing the monotonic identity $\ln\left(\prod x_i\right) = \sum \ln x_i$, the n-th root is transformed into an arithmetic average of logarithms:

$$\mathrm{GM}(\mathbf{x}) = \exp\left(\ln\left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}\right) = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln x_i\right). \tag{7.6}$$

As emphasized in lecture, the base of the logarithm must strictly match the base of the exponentiation operator. Standard scientific implementations employ the natural logarithm ln (base e) and natural exponential $\exp(\cdot)$ so that $\exp(\ln(z)) = z$ is an exact inverse mapping. If base-2 logarithms ($\log_2$) are adopted, the outer transformation must evaluate as $2^{\frac{1}{n}\sum \log_2 x_i}$.

7.1.1 Perplexity

The primary intrinsic metric for evaluating autoregressive language models is **perplexity** (PPL). Rooted in Claude Shannon's mathematical theory of communication and information entropy [4], perplexity quantifies the degree of uncertainty a probabilistic model exhibits when predicting a sequence of tokens.

Conceptual intuition and the effective branching factor.

Rather than assessing whether a model generates fluent answers, perplexity measures how surprised the model is when presented with human-written, grammatically correct text. If a model has acquired an accurate statistical representation of English, it will assign high probability to legitimate sentences from held-out corpora. Conversely, if the model is poorly trained or faces out-of-domain text, it will assign low probability to valid tokens, indicating substantial confusion.

Perplexity provides an intuitive geometric interpretation as the **effective branching factor** of the language model. Suppose that at a specific step in a sentence, the model assigns probability $P(w_i \mid w_{<i})$ to the ground-truth token. The reciprocal:

$$K_i = \frac{1}{P(w_i \mid w_{<i})} \tag{7.7}$$

represents the number of equally probable alternative words among which the model was undecided.

Consider a thought experiment where the model's vocabulary contains 50,000 discrete tokens:

- If the model assigns probability $P = 0.25$ to the correct token, $\frac{1}{0.25} = 4$. Even though the vocabulary contains 50,000 candidates, the model's uncertainty is equivalent to rolling a fair 4-sided die of equally likely tokens.
- If the model assigns probability $P = 0.90$, $\frac{1}{0.90} \approx 1.11$. The model is near-certain of the true token, undecided between approximately 1.1 alternatives.
- If the model assigns probability $P = 1.0$, $\frac{1}{1.0} = 1.0$. The model has zero uncertainty.
- If the model assigns probability $P = 0.01$, $\frac{1}{0.01} = 100$. The model is as confused as choosing uniformly among 100 distinct words.

Across an entire sequence of N words, perplexity represents the geometric average of these individual branching factors. A perplexity of 50 indicates that, on average across each position in the sentence, the model was as uncertain as choosing uniformly among 50 equally probable words. Lower perplexity reflects greater predictive confidence and superior alignment with the underlying data distribution.

Formal mathematical derivation and the negative log-likelihood.
Let $W = (w_1, w_2, \ldots, w_N)$ denote a sequence of N tokens. By the probability chain rule, the joint probability assigned to sequence W by an autoregressive language model parameterized by θ is:

$$P(W) = \prod_{i=1}^{N} P(w_i \mid w_1, \ldots, w_{i-1}; \theta) = \prod_{i=1}^{N} P(w_i \mid w_{<i}). \tag{7.8}$$

The perplexity of the model over the sequence W is formally defined as the reciprocal of the geometric mean of the token conditional probabilities:

$$\mathrm{PPL}(W) = P(W)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} P(w_i \mid w_{<i}) \right)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{<i})} \right)^{\frac{1}{N}}. \tag{7.9}$$

Applying the logarithmic transformation in Equation (7.6), we rewrite perplexity in terms of the sum of token negative log-likelihoods (NLL):

$$\ln \mathrm{PPL}(W) = -\frac{1}{N} \ln P(W) = -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}), \tag{7.10}$$

$$\mathrm{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}) \right) = \exp\left(\frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \right), \tag{7.11}$$

where $\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i}) \geq 0$ is the negative log-likelihood of token w_i.

Equation (7.11) reveals the direct mathematical link between perplexity and the pre-training cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$:

$$\mathcal{L}_{\mathrm{CE}} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \implies \mathrm{PPL}(W) = e^{\mathcal{L}_{\mathrm{CE}}}. \tag{7.12}$$

Perplexity is simply the exponentiated cross-entropy loss.

Mathematical justification for the geometric mean. The selection of the geometric mean in Equation (7.9) is mathematically mandatory for three reasons:

1. **Consistency with probability factorization**: In probability theory, the likelihood of a sequence is formed by the product of conditional probabilities, not their sum. Normalizing a product of N terms by sequence length requires taking the N-th root, so that $\mathrm{PPL}(W)^N = \frac{1}{P(W)}$. Taking an arithmetic mean of probabilities ($\frac{1}{N} \sum P_i$) violates the product rule and lacks theoretical interpretation.

2. **Information-theoretic duality**: Taking the logarithm of the geometric mean yields the arithmetic mean of negative log-likelihoods (cross-entropy), establishing an exact duality with Shannon entropy. An arithmetic mean of probabilities cannot be converted into cross-entropy.

3. **Zero-frequency penalty**: If a model assigns zero probability to a single valid token ($P(w_k \mid w_{<k}) = 0$), the geometric mean product collapses to 0, which inverts to infinite perplexity: $\mathrm{PPL}(W) \to \infty$. This correctly identifies the model as broken, since it declared a real-world sentence impossible. An arithmetic mean would merely incur a negligible $\frac{1}{N}$ reduction.

The theoretical lower bound of perplexity is PPL = 1.0. This minimum occurs if and only if the model assigns probability $P(w_i \mid w_{<i}) = 1.0$ to every token in the sequence ($\mathrm{NLL}_i = 0 \implies e^0 = 1$),

representing absolute, deterministic certainty. In practice, PPL = 1.0 is observed only when a model severely overfits a training sequence.

Step-by-step worked examples: confident versus uncertain models.

To illustrate the mechanics of perplexity, we walk through the exact numerical evaluations presented in the lecture. Consider a held-out test sentence delimited by boundary markers:

$$W = \langle \text{bos} \rangle \, \text{The dog chased the cat} \, \langle \text{eos} \rangle.$$

The beginning-of-sequence token $\langle \text{bos} \rangle$ serves as initial conditioning prefix ($P = 1.0$). The sequence contains $N = 6$ evaluated transitions ($w_1 = \text{The}, \ldots, w_6 = \langle \text{eos} \rangle$).

Case 1: Confident, well-trained model. Suppose a model assigns high probability to each correct token, reflecting strong linguistic familiarity. The conditional probabilities, negative log-likelihoods, and equivalent branching numbers are listed in Table 7.1.

Table 7.1: Step-by-step perplexity computation for a confident model on $W = \langle \text{bos} \rangle$ The dog chased the cat $\langle \text{eos} \rangle$.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\text{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.90	$-\ln(0.90) \approx 0.1054$	1.1111
2	dog	0.85	$-\ln(0.85) \approx 0.1625$	1.1765
3	chased	0.88	$-\ln(0.88) \approx 0.1278$	1.1364
4	the	0.75	$-\ln(0.75) \approx 0.2877$	1.3333
5	cat	0.95	$-\ln(0.95) \approx 0.0513$	1.0526
6	$\langle \text{eos} \rangle$	0.80	$-\ln(0.80) \approx 0.2231$	1.2500
Sum / Mean	—	$\sum \text{NLL}_i = 0.9578 \implies \overline{\text{NLL}} = 0.1596$		$\prod K_i \approx 2.6074$

Evaluating the perplexity via the average negative log-likelihood:

$$\begin{aligned} \text{PPL}(W) &= \exp\left(\frac{1}{6}\sum_{i=1}^{6} \text{NLL}_i\right) \\ &= \exp\left(\frac{0.1054 + 0.1625 + 0.1278 + 0.2877 + 0.0513 + 0.2231}{6}\right) \\ &= e^{0.15963} \approx \mathbf{1.1731}. \end{aligned} \tag{7.13}$$

Equivalently, taking the 6-th root of the product of equivalent words:

$$\text{PPL}(W) = (1.1111 \times 1.1765 \times 1.1364 \times 1.3333 \times 1.0526 \times 1.2500)^{\frac{1}{6}} \approx (2.6074)^{\frac{1}{6}} \approx \mathbf{1.1731}. \tag{7.14}$$

The model's perplexity is 1.1731, indicating that on average, the model was undecided over approximately 1.17 equivalent words per position. It possessed near-complete certainty throughout the sequence.

Case 2: Uncertain, poorly trained model. Now suppose an under-trained or out-of-domain model evaluates the identical sentence, assigning substantially lower probabilities to the correct tokens, as detailed in Table 7.2.

Computing the geometric mean directly from the branching factors:

$$\prod_{i=1}^{6} K_i = 20 \times 10 \times 5 \times 100 \times 200 \times 1000 = 2 \times 10^{10}, \tag{7.15}$$

$$\text{PPL}(W) = \left(2 \times 10^{10}\right)^{\frac{1}{6}} \approx \mathbf{52.10}. \tag{7.16}$$

Table 7.2: Step-by-step perplexity computation for an uncertain model on $W = \langle \text{bos} \rangle$ The dog chased the cat $\langle \text{eos} \rangle$.

i	Token w_i	Probability $P(w_i \mid w_{<i})$	$\text{NLL}_i = -\ln P(w_i \mid w_{<i})$	Branching $K_i = \frac{1}{P_i}$
1	The	0.050	$-\ln(0.050) \approx 2.9957$	20
2	dog	0.100	$-\ln(0.100) \approx 2.3026$	10
3	chased	0.200	$-\ln(0.200) \approx 1.6094$	5
4	the	0.010	$-\ln(0.010) \approx 4.6052$	100
5	cat	0.005	$-\ln(0.005) \approx 5.2983$	200
6	$\langle \text{eos} \rangle$	0.001	$-\ln(0.001) \approx 6.9078$	1000
Sum / Product	—	$\sum \text{NLL}_i = 23.7190 \implies \overline{\text{NLL}} = 3.9532$		$\prod K_i = 2 \times 10^{10}$

Verifying via the average negative log-likelihood:

$$\text{PPL}(W) = \exp\left(\frac{23.7190}{6}\right) = \exp(3.95317) \approx \mathbf{52.10}. \tag{7.17}$$

Both formulations yield identical results: PPL = 52.10. On average, the uncertain model was confused across approximately 52 equally likely candidate tokens at every position.

Lecture insight: Perplexity as a historical and domain probe. As highlighted by the professor, perplexity serves as a diagnostic probe into the temporal and thematic distribution of a model's pre-training data. If an older model trained on corpora prior to 2016 is evaluated on the prompt "The President of the United States is [NAME]", evaluating "Barack Obama" yields low perplexity, whereas evaluating "Donald Trump" or "Joe Biden" produces high perplexity. The elevated perplexity does not imply that the model is architecturally defective; rather, it indicates statistical surprise relative to the pre-training data distribution.

Advantages and fundamental limitations of perplexity.

Perplexity possesses distinct methodological advantages that have cemented its status as standard reporting metric in pre-training literature:

- **Unsupervised and task-agnostic**: Computing perplexity requires only raw, unannotated text. It eliminates the need for expensive human reference translations, task labels, or subjective grading rubrics.
- **Zero inference compute overhead**: Because evaluation occurs under teacher forcing, perplexity is computed in a single forward pass over batches of tokens simultaneously with cross-entropy training loss, avoiding autoregressive generation.
- **Continuous and sensitive**: Perplexity is a smooth, continuous scalar that detects subtle improvements in language modeling capability long before downstream task accuracies show measurable gains.

Despite these strengths, perplexity suffers from fundamental limitations that prevent it from serving as a universal measure of model quality:

- **Vocabulary and tokenizer incomparability**: Perplexity is strictly incomparable across models that use different vocabularies or tokenization algorithms. Consider a byte-level tokenizer ($|\mathcal{V}| = 256$) versus a subword tokenizer ($|\mathcal{V}| = 100,000$). Because a byte-level model predicts tiny, predictable character fragments, its per-token perplexity will appear dramatically lower than the subword model, even though the subword model generates superior semantic text. Valid cross-model comparisons require normalizing perplexity by the number of words rather than subword tokens:

$$\text{PPL}_{\text{word}} = \exp\left(\frac{N_{\text{tokens}}}{N_{\text{words}}} \mathcal{L}_{\text{CE}}\right). \tag{7.18}$$

- **Disconnection from reasoning and factual truth**: A model can achieve outstanding (low) perplexity by mastering local grammatical syntax, common idioms, and surface transitions while remaining completely incapable of logical deduction, arithmetic calculation, or factual consistency. Perplexity rewards familiarity, not correctness.
- **Vulnerability to data contamination**: If portions of the evaluation test set are inadvertently present in the multi-terabyte pre-training scrape, the model will memorize the sequences and exhibit near-zero perplexity, creating a false impression of general capability.

7.1.2 Bilingual Evaluation Understudy (BLEU)

While perplexity evaluates the model's internal probability distribution over given text, practical deployment requires evaluating the actual text generated by the model during autonomous inference. In 2002, Papineni, Roukos, Ward, and Zhu [74] introduced the **Bilingual Evaluation Understudy** (BLEU) metric at IBM Research. Originally developed for machine translation, BLEU became the foundational reference-based evaluation metric across natural language processing.

Conceptual intuition and reference-based lexical matching.

The fundamental premise of BLEU is that the closer a machine-generated candidate translation resembles a human gold-standard translation, the higher its quality. Evaluating open-ended text against human references requires navigating two competing linguistic dimensions:

1. **Adequacy (Word Choice)**: Did the model translate the correct conceptual content? This is captured by measuring lexical overlap at the unigram level (1-grams).
2. **Fluency (Word Order)**: Did the model assemble the words in a natural, syntactically coherent sequence? This is captured by measuring contiguous overlap across higher-order n-grams (2-grams, 3-grams, 4-grams).

BLEU is an extrinsic metric: it requires running the model autoregressively to generate a hypothesis string G and comparing G against one or more reference strings $R = \{R_1, \ldots, R_m\}$. It is defined as a modified n-gram precision metric multiplied by an exponential brevity penalty.

Mathematical formulation and the brevity penalty.

To compute BLEU, we first extract all contiguous n-grams of size $i \in \{1, 2, \ldots, n\}$ from both the generated candidate sequence G and the reference sequence R.

Modified clipped n-gram precision. A naive precision metric would simply count the fraction of candidate n-grams that appear in the reference:

$$p_{\text{naive}} = \frac{\sum_{g \in G} \mathbb{I}(g \in R)}{|G|}.$$

However, naive precision can be exploited through pathological repetition. For example, if the reference is "The cat sat on the mat" and a defective model outputs the candidate "the the the the the the", every single generated unigram matches the reference word "the". Naive precision would reward this degenerative output with a perfect score: $p_{\text{naive}} = \frac{6}{6} = 1.0$.

To eliminate this vulnerability, Papineni et al. [74] introduced **clipped precision** (p_i). For each i-gram in the candidate sequence, the count is clipped so that it cannot exceed the maximum number of times that identical i-gram appears in any single reference sentence:

$$\text{Count}_{\text{clip}}(g_i) = \min\left(\text{Count}_G(g_i), \max_{R_j \in R} \text{Count}_{R_j}(g_i)\right). \tag{7.19}$$

The modified i-gram precision p_i is then computed as the sum of clipped counts divided by the total number of candidate i-grams generated:

$$p_i = \frac{\sum_{g_i \in G} \text{Count}_{\text{clip}}(g_i)}{\sum_{g_i \in G} \text{Count}_G(g_i)}. \tag{7.20}$$

In the pathological repetition example, the reference contains the unigram "the" twice ($\max \text{Count}_R(\text{"the"}) = 2$). The clipped count for the candidate is $\min(6, 2) = 2$, reducing the unigram precision to $\frac{2}{6} = 0.3333$.

Multi-order geometric mean aggregation. To synthesize adequacy and fluency, BLEU computes the geometric mean of modified precisions across n-gram orders up to maximum order n (typically $n = 2$ or $n = 4$):

$$\text{Score}_{\text{prec}} = \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right) = \left(\prod_{i=1}^{n} p_i\right)^{\frac{1}{n}}, \tag{7.21}$$

where uniform weights $w_i = \frac{1}{n}$ are standard. The geometric mean ensures that if a model fails completely at any n-gram order ($p_i = 0$), the entire score collapses to zero, enforcing that fluent text must exhibit both lexical accuracy and local phrase cohesion.

*The brevity penalty (*BP*).* While clipped precision prevents repetitive over-generation, precision alone introduces an opposite failure mode: **under-generation**.

Suppose the reference sentence contains 20 words, and the candidate model generates only two words: "The dog". Both words appear in the reference, so $p_1 = \frac{2}{2} = 1.0$. The bigram "The dog" also appears in the reference, so $p_2 = \frac{1}{1} = 1.0$. Without length regulation, the truncated output would achieve a flawless score: BLEU = 1.0, despite discarding 90% of the intended content.

Standard classification metrics resolve under-generation by computing recall. However, in machine translation and open-ended generation, defining recall is complicated because human translations exhibit natural length variance, and multiple reference translations often use different phrasing. Instead of explicit recall, BLEU introduces an exponential multiplicative **Brevity Penalty** (BP).

Let g denote the length of the generated candidate sequence, and let r denote the effective length of the reference sequence. The brevity penalty is defined as:

$$\text{BP} = \begin{cases} 1.0 & \text{if } g > r, \\ \exp\left(1 - \frac{r}{g}\right) & \text{if } g \leq r. \end{cases} \tag{7.22}$$

Figure 7.1 illustrates the mathematical behavior of the brevity penalty as a function of the length ratio $\frac{g}{r}$.

- When $g \geq r$, the candidate length equals or exceeds the reference length. In this regime, BP = 1.0: no brevity penalty is applied. Overly long generations are naturally penalized by the denominator of the precision terms in Equation (7.20).
- When $g < r$, the exponent $1 - \frac{r}{g}$ is strictly negative, causing BP to decay exponentially below 1.0. As the candidate length approaches zero ($g \to 0$), the ratio $\frac{r}{g} \to \infty$, and BP $\to 0$, aggressively driving the overall score to zero.

Synthesizing the brevity penalty and modified n-gram precisions yields the complete formulation of BLEU-n:

$$\text{BLEU-}n = \text{BP} \cdot \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right) = \text{BP} \cdot \left(\prod_{i=1}^{n} p_i\right)^{\frac{1}{n}}. \tag{7.23}$$

Step-by-step worked example: BLEU-2 calculation.
We now execute the step-by-step computation of BLEU-2 ($n = 2$, uniform weights $w_1 = w_2 = 0.5$) on the worked example from the lecture slides.

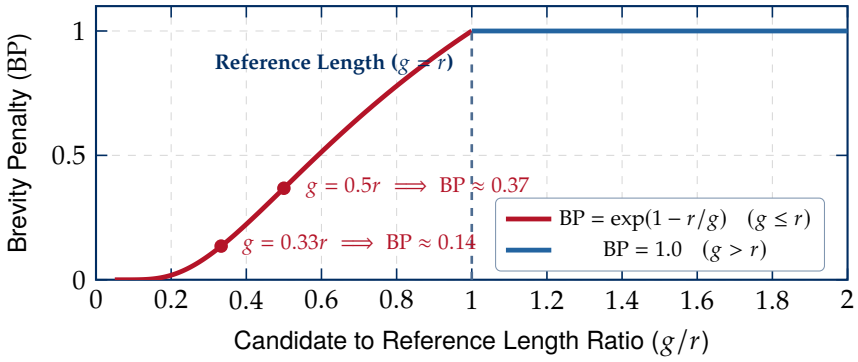


Figure 7.1: The BLEU Brevity Penalty (BP) as a function of the generated-to-reference length ratio g/r. For $g \geq r$, BP = 1.0 because over-generation is already penalized by precision denominators. For $g < r$, BP = $\exp(1 - r/g)$ decays exponentially toward zero, severely penalizing short or truncated generations to prevent precision gaming.

Let the reference and machine-generated candidate sequences be:

$$R : \text{“The dog chased the cat”} \quad (r = 5 \text{ tokens}),$$
$$G : \text{“The dog ran after the cat”} \quad (g = 6 \text{ tokens}).$$

Step 1: Unigram precision ($i = 1$). Extract all unigrams from both sentences:

$$\text{Reference unigrams } R_1 = \{\text{“The”, “dog”, “chased”, “the”, “cat”}\} \quad (|R_1| = 5),$$
$$\text{Generated unigrams } G_1 = \{\text{“The”, “dog”, “ran”, “after”, “the”, “cat”}\} \quad (|G_1| = 6).$$

We count matching unigrams between G and R:

- “The”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “dog”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “ran”: absent in R ($\text{Count}_{\text{clip}} = 0$)
- “after”: absent in R ($\text{Count}_{\text{clip}} = 0$)
- “the”: appears in R ($\text{Count}_{\text{clip}} = 1$)
- “cat”: appears in R ($\text{Count}_{\text{clip}} = 1$)

Total matching unigrams = 4. The total generated unigrams = 6. The modified unigram precision is:

$$p_1 = \frac{4}{6} \approx \mathbf{0.6667}. \tag{7.24}$$

Step 2: Bigram precision ($i = 2$). Extract all contiguous pairs of adjacent tokens (bigrams):

$$\text{Reference bigrams } R_2 = \{\text{“The dog”, “dog chased”, “chased the”, “the cat”}\} \quad (|R_2| = 4),$$
$$\text{Generated bigrams } G_2 = \{\text{“The dog”, “dog ran”, “ran after”, “after the”, “the cat”}\} \quad (|G_2| = 5).$$

We count matching bigrams:

- “The dog”: matches R_2 ($\text{Count}_{\text{clip}} = 1$)
- “dog ran”: absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- “ran after”: absent in R_2 ($\text{Count}_{\text{clip}} = 0$)

- "after the": absent in R_2 ($\text{Count}_{\text{clip}} = 0$)
- "the cat": matches R_2 ($\text{Count}_{\text{clip}} = 1$)

Total matching bigrams = 2. The total generated bigrams = 5. The modified bigram precision is:

$$p_2 = \frac{2}{5} = \mathbf{0.4000}. \tag{7.25}$$

Step 3: Geometric mean of precisions. Combining p_1 and p_2 via the geometric mean:

$$\begin{aligned} \text{GM} &= \exp\left(\frac{1}{2}\left(\ln(0.6667) + \ln(0.4000)\right)\right) = \sqrt{p_1 \times p_2} \\ &= \sqrt{\frac{4}{6} \times \frac{2}{5}} = \sqrt{\frac{8}{30}} = \sqrt{\frac{4}{15}} \approx \mathbf{0.5165}. \end{aligned} \tag{7.26}$$

Step 4: Brevity penalty calculation. We compare the candidate length $g = 6$ against the reference length $r = 5$:

$$g = 6 > r = 5 \implies \text{BP} = 1.0. \tag{7.27}$$

Because the candidate sequence is not shorter than the reference, no brevity penalty is incurred (BP = 1.0).

Step 5: Final BLEU-2 score. Multiplying the brevity penalty by the precision geometric mean yields:

$$\text{BLEU-2} = \text{BP} \times \text{GM} = 1.0 \times 0.5165 = \mathbf{0.5165} \quad (\text{or } 51.65\%). \tag{7.28}$$

Pedagogical analysis of the worked example. This calculation exposes the defining characteristics and underlying limitations of n-gram lexical matching. The candidate phrase "ran after" is semantically equivalent to the reference verb "chased". A human evaluator would judge "The dog ran after the cat" as a completely correct, fluent, and faithful translation. However, because BLEU enforces exact surface string equality, the bigrams "dog ran", "ran after", and "after the" were scored as total errors (0 matches). Consequently, p_2 dropped from an expected near-perfect score to 0.4000, driving the overall BLEU-2 score down to 0.5165.

Advantages and fundamental limitations of BLEU.

BLEU has served as the universal benchmark in machine translation and conditional generation for over two decades due to several compelling advantages:

- **Automated and cost-effective**: BLEU replaces slow, expensive human judges with an instantaneous, deterministic algorithmic script that costs nothing to evaluate.
- **Language-independent**: BLEU operates strictly on string matching over whitespace-separated tokens. It requires no morphological analyzers, syntactic dependency parsers, or semantic ontologies.
- **Empirical human correlation**: Across large corpora containing thousands of sentences, aggregate BLEU scores correlate reasonably well with human assessments of translation quality, effectively discriminating between high-performing and defective models.
- **Enforcement of phrasing structure**: By incorporating higher-order n-grams ($n \geq 2$), BLEU aggressively penalizes "word salad" outputs that contain relevant keywords in scrambled or ungrammatical orders.

Nonetheless, BLEU exhibits severe architectural limitations that have prompted the development of neural semantic evaluators:

- **Total semantic blindness**: BLEU recognizes only exact surface character matches. It is completely blind to synonymy, paraphrasing, morphological inflection, and active-to-passive voice transformations. As demonstrated in our worked example, using "ran after" instead of "chased" severely degrades the score despite preserving identical semantics.
- **Insensitivity to grammatical negation and factual correctness**: Surface overlap does not track truth value. If a reference states "The system is secure" and a model generates "The system is not secure", the candidate matches 4 out of 5 unigrams and achieves an excellent BLEU score, despite asserting the absolute opposite fact.
- **Brittle reference dependency**: BLEU evaluates alignment with a specific reference phrasing. If the human reference uses rare vocabulary or unusual syntax, a model generating natural, idiomatic text will be penalized simply because its stylistic choices diverge from the specific reference annotator.
- **Unsuitability for open-ended generation**: In tasks such as dialogue, creative writing, or long-form reasoning, there is no single "correct" reference string. Demanding exact n-gram overlap against a fixed reference penalizes valid divergent creativity.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[4] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[5] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall / Online Draft, 3rd draft edition, 2024. URL https://web.stanford.edu/~jurafsky/slp3/.

[6] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3-4):237–264, 1953.

[7] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[8] Slava M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(3):400–401, 1987.

[9] Reinhard Kneser and Hermann Ney. Improved backing-off for m-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[10] Gerasimos Potamianos and Frederick Jelinek. A study of n-gram and decision tree letter language modeling methods. *Speech Communication*, 24(2):171–192, 1998.

[11] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[13] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[14] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, Yuxiong He, et al. Using DeepSpeed and Megatron to train Megatron-Turing NLG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022. URL https://arxiv.org/abs/2201.11990.

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[16] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[17] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[18] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[19] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[20] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[21] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL https://udlbook.github.io/udlbook/.

[22] Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2924–2932, 2014. URL https://proceedings.neurips.cc/paper/2014/hash/1a5b1e4daae265b790965a275b53ae50-Abstract.html.

[23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

[24] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[25] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[26] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. URL https://www.bishopbook.com/.

[27] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

[28] Grant Sanderson. Visualizing attention, embeddings and neural network trajectories. 3Blue1Brown Educational Video Series, 2024. URL https://www.3blue1brown.com/lessons/gpt.

[29] Paul J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[30] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[31] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

[32] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015. URL https://karpathy.github.io/2015/05/21/rnn-effectiveness/.

[33] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[34] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3104–3112, 2014.

[36] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.

[37] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[38] Zellig S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[39] John R. Firth. A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, pages 1–32, 1957.

[40] Andrej Karpathy. Building makemore part 2: MLP language model. YouTube Video Tutorial and GitHub Repository, 2022. URL https://github.com/karpathy/makemore.

[41] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR) Workshop Track*, 2013.

[42] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[43] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, pages 246–252, 2005.

[44] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[45] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146, 2017.

[46] Enkk. Cosa sono gli embeddings: Geometria semantica, distanza coseno e vector databases. YouTube Educational Lecture, 2023. URL https://www.youtube.com/watch?v=5sYkC8aVv1k.

[47] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[48] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[49] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[50] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[51] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[52] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1243–1252, 2017. URL https://arxiv.org/abs/1705.03122.

[53] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[54] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 464–468, 2018. URL https://arxiv.org/abs/1803.02155.

[55] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. URL https://arxiv.org/abs/2104.09864.

[56] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2108.12409.

[57] Grant Sanderson. Attention in transformers, visually explained. 3Blue1Brown Video Series, 2024. URL https://www.youtube.com/watch?v=eMlx5fFNoYc.

[58] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[59] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[60] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL https://jmlr.org/papers/v21/19-460.html.

[61] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, 2019. URL https://arxiv.org/abs/1906.04341.

[62] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 14014–14024, 2019. URL https://arxiv.org/abs/1905.10650.

[63] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09751.

[64] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 889–898, 2018. URL https://arxiv.org/abs/1805.04833.

[65] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018. URL https://openai.com/research/language-unsupervised.

[66] Opher Lieber, Or Sharir, Barak Lenz, and Yoav Shoham. Jurassic-1: Technical details and evaluation. *AI21 Labs Whitepaper*, 2021. URL https://www.ai21.com/white-paper/jurassic-1.

[67] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2021. URL https://arxiv.org/abs/2112.11446.

[68] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with Pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. URL https://jmlr.org/papers/v24/22-1144.html.

[69] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

[70] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. URL https://arxiv.org/abs/2307.09288.

[71] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[72] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023. URL https://arxiv.org/abs/2310.06825.

[73] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. The Falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023. URL https://arxiv.org/abs/2311.16867.

[74] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. URL https://aclanthology.org/P02-1040/.